

C++ Foundations

Learning the Language, Programming Fundamentals, Memory, and Object-Oriented Design

Dr. Charles Dorner

2026

C++ Foundations

1. [About this expanded volume](#)
 1. [Audience and progression](#)
 2. [Technical scope and primary references](#)
2. [How to use this book](#)
 1. [Read for a decision, then trace the mechanism](#)
 2. [Build the supplied examples](#)
 3. [Source and output conventions](#)
 4. [Use failures deliberately](#)
 5. [Verification boundaries](#)
 6. [The central operating rule](#)
3. [Chapter 1 — Understanding C++ and How Programs Become Executables](#)
 1. [1.1 Understanding the Role of C++ in Modern Software](#)
 1. [1.1.1 Distinguishing Systems Programming from Application Programming](#)
 2. [1.1.2 Understanding Where C++ Is Used in Industry](#)
 2. [1.2 Understanding the C++ Compilation Pipeline](#)
 1. [1.2.1 Transforming Source Code Through Preprocessing and Compilation](#)
 2. [1.2.2 Combining Object Files Through Linking](#)
 3. [1.3 Understanding the C++ Standard and Compiler Implementations](#)
 1. [1.3.1 Understanding Language Standards and Standard-Library Versions](#)
 2. [1.3.2 Comparing GCC, Clang, and Microsoft Visual C++](#)
 4. [Worked example — Identify the language mode before trusting a build](#)
 1. [Requirement and simplest approach](#)
 2. [Establish the starting state](#)
 3. [Trace the implementation](#)
 4. [Complete worked source](#)
 5. [Run and verify](#)
 6. [Read the result at the correct level](#)
 7. [A misleading success: the stale executable](#)
 5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Build an evidence trail](#)
 3. [Review and solution reasoning](#)
4. [Chapter 2 — Creating a Complete C++ Development Environment](#)
 1. [2.1 Installing and Configuring a C++ Compiler](#)
 1. [2.1.1 Installing GCC or Clang on Linux](#)
 2. [2.1.2 Installing Microsoft Visual C++ on Windows](#)
 2. [2.2 Selecting and Configuring Development Tools](#)
 1. [2.2.1 Developing with Visual Studio and Visual Studio Code](#)
 2. [2.2.2 Developing Directly from the Command Line](#)
 3. [2.3 Compiling Programs Manually](#)
 1. [2.3.1 Selecting the C++ Language Standard](#)
 2. [2.3.2 Enabling Compiler Warnings and Debug Information](#)

4. [Worked example — Build an environment fingerprint](#)
 1. [Requirement and simplest approach](#)
 2. [Build the probe independently of the editor](#)
 3. [Trace the implementation](#)
 4. [Complete worked source](#)
 5. [Run and verify](#)
 6. [Environment facts and environment claims](#)
 7. [Failure clinic — Four similar-looking setup failures](#)
5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Reproduce the environment from a clean directory](#)
 3. [Review and solution reasoning](#)
5. [Chapter 3 — Writing, Compiling, and Running Your First C++ Programs](#)
 1. [3.1 Understanding the Structure of a C++ Program](#)
 1. [3.1.1 Understanding main\(\) as the Program Entry Point](#)
 2. [3.1.2 Understanding Statements, Expressions, and Semicolons](#)
 2. [3.2 Producing Console Output](#)
 1. [3.2.1 Writing Text with std::cout](#)
 2. [3.2.2 Formatting Output with Stream Operations](#)
 3. [3.3 Receiving User Input](#)
 1. [3.3.1 Reading Values with std::cin](#)
 2. [3.3.2 Recognizing and Handling Invalid Input](#)
 4. [Worked example — Validate a count before printing a report](#)
 1. [Requirement and simplest approach](#)
 2. [Trace the data, not just the lines](#)
 3. [Trace the implementation](#)
 4. [Complete worked source](#)
 5. [Run and verify](#)
 6. [Run the same program without typing interactively](#)
 7. [Failure clinic — The default value that hid a bad read](#)
 5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Exercise every input category](#)
 3. [Review and solution reasoning](#)
 6. [Chapter 4 — Representing Information with Variables and Fundamental Types](#)
 1. [4.1 Declaring and Initializing Variables](#)
 1. [4.1.1 Comparing Copy, Direct, and List Initialization](#)
 2. [4.1.2 Preventing Uninitialized Data](#)
 2. [4.2 Working with Numeric Types](#)
 1. [4.2.1 Understanding Signed and Unsigned Integers](#)
 2. [4.2.2 Understanding Floating-Point Representation](#)
 3. [4.3 Working with Characters, Booleans, and Constants](#)
 1. [4.3.1 Representing Character and Logical Data](#)
 2. [4.3.2 Using const and constexpr](#)
 4. [Worked example — Observe representation without assuming it](#)
 1. [Requirement and simplest approach](#)

2. [Trace value, type, and interpretation](#)
 3. [Trace the implementation](#)
 4. [Complete worked source](#)
 5. [Run and verify](#)
 6. [Failure clinic — A cast that changed the symptom](#)
 7. [Failure clinic — The invisible distinction between constants](#)
 8. [A useful counterexample](#)
5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Design a type sheet for the inventory project](#)
 3. [Review and solution reasoning](#)
7. [Chapter 5 — Building Expressions with Operators](#)
 1. [5.1 Performing Arithmetic Operations](#)
 1. [5.1.1 Using Addition, Subtraction, Multiplication, and Division](#)
 2. [5.1.2 Understanding Integer Division and Modulus](#)
 2. [5.2 Comparing and Combining Values](#)
 1. [5.2.1 Using Relational Operators](#)
 2. [5.2.2 Combining Conditions with Logical Operators](#)
 3. [5.3 Understanding Expression Evaluation](#)
 1. [5.3.1 Understanding Precedence and Associativity](#)
 2. [5.3.2 Avoiding Unexpected Side Effects](#)
 4. [Worked example — Calculate complete groups and a remainder safely](#)
 1. [Requirement and simplest approach](#)
 2. [A small proof before execution](#)
 3. [Trace the implementation](#)
 4. [Complete worked source](#)
 5. [Run and verify](#)
 6. [Evidence table — What each intermediate proves](#)
 7. [Failure clinic — Widening too late](#)
 8. [Failure clinic — A range check that always seems true](#)
 5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Generalize the packing calculation](#)
 3. [Review and solution reasoning](#)
8. [Chapter 6 — Controlling Program Decisions with Conditional Logic](#)
 1. [6.1 Making Decisions with if Statements](#)
 1. [6.1.1 Evaluating Boolean Conditions](#)
 2. [6.1.2 Building Multi-Branch if/else Logic](#)
 2. [6.2 Selecting Among Multiple Cases](#)
 1. [6.2.1 Building switch Statements](#)
 2. [6.2.2 Understanding Fallthrough and break](#)
 3. [6.3 Writing Clear Conditional Logic](#)
 1. [6.3.1 Simplifying Complex Boolean Expressions](#)
 2. [6.3.2 Avoiding Deeply Nested Decision Trees](#)
 4. [Worked example — Classify threshold boundaries without overlapping branches](#)
 1. [Requirement and simplest approach](#)

2. [Trace one boundary from each side](#)
3. [Trace the implementation](#)
4. [Complete worked source](#)
5. [Run and verify](#)
6. [Failure clinic — Valid syntax, wrong partition](#)
7. [Evidence beyond a single branch](#)
5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Turn a decision table into tests](#)
 3. [Review and solution reasoning](#)
9. [Chapter 7 — Repeating Operations with Loops](#)
 1. [7.1 Repeating While Conditions Remain True](#)
 1. [7.1.1 Building while Loops](#)
 2. [7.1.2 Building do-while Loops](#)
 2. [7.2 Performing Counted Iteration](#)
 1. [7.2.1 Building Traditional for Loops](#)
 2. [7.2.2 Using Range-Based for Loops](#)
 3. [7.3 Controlling Loop Execution](#)
 1. [7.3.1 Exiting Loops with break](#)
 2. [7.3.2 Skipping Iterations with continue](#)
 4. [Worked example — Process a bounded stream of readings](#)
 1. [Requirement and simplest approach](#)
 2. [Trace the invariant](#)
 3. [Trace the implementation](#)
 4. [Complete worked source](#)
 5. [Run and verify](#)
 6. [A trace table is a debugging tool](#)
 7. [Failure clinic — The loop that never reaches the next item](#)
 8. [Failure clinic — Averages and empty acceptance](#)
 5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Challenge the termination and acceptance rules](#)
 3. [Review and solution reasoning](#)
10. [Chapter 8 — Decomposing Programs with Functions](#)
 1. [8.1 Declaring and Defining Functions](#)
 1. [8.1.1 Understanding Function Signatures](#)
 2. [8.1.2 Returning Values from Functions](#)
 2. [8.2 Passing Information into Functions](#)
 1. [8.2.1 Passing Arguments by Value](#)
 2. [8.2.2 Passing Arguments by Reference](#)
 3. [8.3 Designing Flexible Functions](#)
 1. [8.3.1 Using Function Overloading](#)
 2. [8.3.2 Using Default Arguments](#)
 4. [Worked example — Separate validation, calculation, and caller-owned mutation](#)
 1. [Requirement and simplest approach](#)
 2. [Trace the call boundary](#)

- 3. [Trace the implementation](#)
 - 4. [Complete worked source](#)
 - 5. [Run and verify](#)
 - 6. [Failure clinic — The parameter that changed nothing](#)
 - 7. [Failure clinic — A sentinel treated as an ordinary answer](#)
 - 8. [Refactoring without changing behavior](#)
- 5. [Practice and implementation lab](#)
 - 1. [Starting exercise: solution notes](#)
 - 2. [Guided lab — Specify and test a function contract](#)
 - 3. [Review and solution reasoning](#)
- 11. [Chapter 9 — Understanding Scope, Lifetime, and Storage Duration](#)
 - 1. [9.1 Understanding Where Names Are Visible](#)
 - 1. [9.1.1 Working with Block and Function Scope](#)
 - 2. [9.1.2 Working with Namespace Scope](#)
 - 2. [9.2 Understanding How Long Objects Exist](#)
 - 1. [9.2.1 Understanding Automatic Storage Duration](#)
 - 2. [9.2.2 Understanding Static Storage Duration](#)
 - 3. [9.3 Separating Scope from Lifetime](#)
 - 1. [9.3.1 Recognizing Lifetime-Related Bugs](#)
 - 2. [9.3.2 Avoiding Unnecessary Global State](#)
 - 4. [Worked example — Compare persistent state with caller-owned state](#)
 - 1. [Requirement and simplest approach](#)
 - 2. [Build a time-ordered object map](#)
 - 3. [Trace the implementation](#)
 - 4. [Complete worked source](#)
 - 5. [Run and verify](#)
 - 6. [Failure clinic — Tests that depend on their order](#)
 - 7. [Failure clinic — The local value that seemed to survive](#)
 - 8. [Refactoring checkpoint](#)
 - 5. [Practice and implementation lab](#)
 - 1. [Starting exercise: solution notes](#)
 - 2. [Guided lab — Draw lifetimes before writing pointers](#)
 - 3. [Review and solution reasoning](#)
- 12. [Chapter 10 — Storing Fixed Collections with Arrays](#)
 - 1. [10.1 Creating and Accessing Arrays](#)
 - 1. [10.1.1 Declaring Fixed-Size Arrays](#)
 - 2. [10.1.2 Accessing Elements by Index](#)
 - 2. [10.2 Processing Collections of Array Elements](#)
 - 1. [10.2.1 Traversing Arrays Safely](#)
 - 2. [10.2.2 Processing Multidimensional Arrays](#)
 - 3. [10.3 Understanding the Limitations of Raw Arrays](#)
 - 1. [10.3.1 Recognizing Out-of-Bounds Access](#)
 - 2. [10.3.2 Comparing Raw Arrays with Standard Containers](#)
 - 4. [Worked example — Sum a fixed frame and a small matrix](#)
 - 1. [Requirement and simplest approach](#)
 - 2. [Trace the frame and the grid separately](#)

3. [Trace the implementation](#)
4. [Complete worked source](#)
5. [Run and verify](#)
6. [Failure clinic — The count mistaken for an index](#)
7. [Failure clinic — The array that became a pointer](#)
8. [A practical review technique](#)
5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Make shape changes safe](#)
 3. [Review and solution reasoning](#)
13. [Chapter 11 — Manipulating Text with std::string](#)
 1. [11.1 Creating and Modifying Strings](#)
 1. [11.1.1 Constructing and Assigning Strings](#)
 2. [11.1.2 Concatenating and Appending Text](#)
 2. [11.2 Searching and Extracting Text](#)
 1. [11.2.1 Finding Characters and Substrings](#)
 2. [11.2.2 Extracting Substrings](#)
 3. [11.3 Converting Between Strings and Other Types](#)
 1. [11.3.1 Parsing Numeric Values from Text](#)
 2. [11.3.2 Formatting Values as Strings](#)
 4. [Worked example — Parse a complete numeric field](#)
 1. [Requirement and simplest approach](#)
 2. [Read the supporting notation](#)
 3. [Trace the implementation](#)
 4. [Complete worked source](#)
 5. [Run and verify](#)
 6. [Trace rejection without corrupting prior state](#)
 7. [Failure clinic — Absence treated as position zero](#)
 8. [Failure clinic — Parsing succeeded, the record did not](#)
 9. [Diagnostic evidence for text](#)
 5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Build a strict field parser](#)
 3. [Review and solution reasoning](#)
14. [Chapter 12 — Managing Dynamic Collections with Standard Containers](#)
 1. [12.1 Using std::array for Fixed-Size Collections](#)
 1. [12.1.1 Creating Type-Safe Fixed Arrays](#)
 2. [12.1.2 Iterating Through std::array](#)
 2. [12.2 Using std::vector for Dynamic Collections](#)
 1. [12.2.1 Adding and Removing Elements](#)
 2. [12.2.2 Understanding Size and Capacity](#)
 3. [12.3 Choosing Between Basic Containers](#)
 1. [12.3.1 Comparing Fixed and Dynamic Storage](#)
 2. [12.3.2 Selecting Containers Based on Requirements](#)
 4. [Worked example — Distinguish reserved storage from live elements](#)
 1. [Requirement and simplest approach](#)

2. [Trace two different kinds of change](#)
3. [Trace the implementation](#)
4. [Complete worked source](#)
5. [Run and verify](#)
6. [Failure clinic — Reserved storage mistaken for elements](#)
7. [Failure clinic — A reference survives in syntax only](#)
8. [Evidence about capacity](#)
5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Model a bounded reading batch](#)
 3. [Review and solution reasoning](#)
15. [Chapter 13 — Sharing Existing Objects with References](#)
 1. [13.1 Understanding Reference Semantics](#)
 1. [13.1.1 Binding References to Existing Objects](#)
 2. [13.1.2 Understanding References as Aliases](#)
 2. [13.2 Using References in Functions](#)
 1. [13.2.1 Modifying Caller-Owned Data](#)
 2. [13.2.2 Avoiding Expensive Copies](#)
 3. [13.3 Protecting Referenced Data](#)
 1. [13.3.1 Using const References](#)
 2. [13.3.2 Choosing Value or Reference Parameters](#)
 4. [Worked example — Observe and update the same collection through different contracts](#)
 1. [Requirement and simplest approach](#)
 2. [Trace object identity through the calls](#)
 3. [Trace the implementation](#)
 4. [Complete worked source](#)
 5. [Run and verify](#)
 6. [Failure clinic — The loop that modified copies](#)
 7. [Failure clinic — A failed operation left half the data changed](#)
 8. [Borrowing boundary](#)
 5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Make mutation transactional](#)
 3. [Review and solution reasoning](#)
16. [Chapter 14 — Working Directly with Memory Through Pointers](#)
 1. [14.1 Understanding Memory Addresses](#)
 1. [14.1.1 Obtaining an Address with &](#)
 2. [14.1.2 Dereferencing an Address with *](#)
 2. [14.2 Representing Optional or Indirect Objects](#)
 1. [14.2.1 Initializing Pointers with nullptr](#)
 2. [14.2.2 Checking Pointer Validity Before Dereferencing](#)
 3. [14.3 Understanding Pointer Arithmetic](#)
 1. [14.3.1 Moving Between Contiguous Elements](#)
 2. [14.3.2 Recognizing Pointer Arithmetic Hazards](#)
 4. [Worked example — Use a nullable borrower and a half-open pointer range](#)
 1. [Requirement and simplest approach](#)

2. [Follow the two objects](#)
3. [Trace the implementation](#)
4. [Complete worked source](#)
5. [Run and verify](#)
6. [Failure clinic — Nonnull but expired](#)
7. [Failure clinic — The pointer moved instead of the value](#)
8. [Failure clinic — A missing optional-result branch](#)
5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Select without escaping the range](#)
 3. [Review and solution reasoning](#)
17. [Chapter 15 — Allocating and Releasing Dynamic Memory](#)
 1. [15.1 Understanding Stack and Heap Storage](#)
 1. [15.1.1 Comparing Automatic and Dynamic Storage](#)
 2. [15.1.2 Understanding Object Lifetime on the Heap](#)
 2. [15.2 Performing Manual Dynamic Allocation](#)
 1. [15.2.1 Allocating Objects with new](#)
 2. [15.2.2 Releasing Objects with delete](#)
 3. [15.3 Recognizing Memory-Management Failures](#)
 1. [15.3.1 Preventing Memory Leaks](#)
 2. [15.3.2 Preventing Dangling Pointers and Double Deletion](#)
 4. [Worked example — Compare manual ownership with container ownership](#)
 1. [Requirement and simplest approach](#)
 2. [Trace ownership separately from arithmetic](#)
 3. [Trace the implementation](#)
 4. [Complete worked source](#)
 5. [Run and verify](#)
 6. [Failure clinic — Returning before cleanup](#)
 7. [Failure clinic — Two aliases, one release](#)
 8. [Verification with instrumentation](#)
 5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Write an ownership ledger](#)
 3. [Review and solution reasoning](#)
18. [Chapter 16 — Modeling Structured Data with Structures and Enumerations](#)
 1. [16.1 Grouping Related Data with struct](#)
 1. [16.1.1 Declaring Data Members](#)
 2. [16.1.2 Initializing Aggregate Objects](#)
 2. [16.2 Representing Named Choices with Enumerations](#)
 1. [16.2.1 Using Traditional Enumerations](#)
 2. [16.2.2 Using Type-Safe enum class](#)
 3. [16.3 Designing Meaningful Domain Types](#)
 1. [16.3.1 Replacing Unrelated Variables with Structured Models](#)
 2. [16.3.2 Replacing Magic Numbers with Enumerations](#)
 4. [Worked example — Report device states without magic numeric codes](#)
 1. [Requirement and simplest approach](#)

2. [Trace data and behavior separately](#)
3. [Trace the implementation](#)
4. [Complete worked source](#)
5. [Run and verify](#)
6. [Failure clinic — The fields drifted apart](#)
7. [Failure clinic — A cast accepted an unsupported state](#)
8. [Failure clinic — A default that was mistaken for approval](#)
5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Model and validate a device record](#)
 3. [Review and solution reasoning](#)
19. [Chapter 17 — Designing Classes and Objects](#)
 1. [17.1 Defining Classes](#)
 1. [17.1.1 Declaring Data Members](#)
 2. [17.1.2 Implementing Member Functions](#)
 2. [17.2 Protecting Object State](#)
 1. [17.2.1 Understanding public, private, and protected](#)
 2. [17.2.2 Maintaining Class Invariants](#)
 3. [17.3 Understanding Object Behavior](#)
 1. [17.3.1 Using the this Pointer](#)
 2. [17.3.2 Separating Interface from Implementation](#)
 4. [Worked example — Enforce a bounded quantity invariant](#)
 1. [Requirement and simplest approach](#)
 2. [Trace the state transitions](#)
 3. [Trace the implementation](#)
 4. [Complete worked source](#)
 5. [Run and verify](#)
 6. [Failure clinic — Validation after assignment](#)
 7. [Failure clinic — Private storage with a public escape hatch](#)
 8. [Failure clinic — A class that owns too many jobs](#)
 5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — Extend an invariant without exposing the representation](#)
 3. [Review and solution reasoning](#)
20. [Chapter 18 — Controlling Object Construction and Destruction](#)
 1. [18.1 Constructing Valid Objects](#)
 1. [18.1.1 Creating Default and Parameterized Constructors](#)
 2. [18.1.2 Using Member Initializer Lists](#)
 2. [18.2 Simplifying Constructor Logic](#)
 1. [18.2.1 Delegating Between Constructors](#)
 2. [18.2.2 Providing Sensible Default Values](#)
 3. [18.3 Cleaning Up Objects](#)
 1. [18.3.1 Understanding Destructor Execution](#)
 2. [18.3.2 Connecting Destruction to Resource Management](#)
 4. [Worked example — Establish an invariant once and observe cleanup order](#)
 1. [Requirement and simplest approach](#)

2. [Construction and cleanup ledger](#)
3. [Trace the implementation](#)
4. [Complete worked source](#)
5. [Run and verify](#)
6. [Evidence and interpretation](#)
7. [Failure clinic — cleanup during an exception](#)
5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — audit an object's complete lifetime](#)
 3. [Review and solution notes](#)
21. [Chapter 19 — Organizing Programs Across Multiple Files](#)
 1. [19.1 Separating Declarations from Definitions](#)
 1. [19.1.1 Creating Header Files](#)
 2. [19.1.2 Creating Implementation Files](#)
 2. [19.2 Understanding Translation Units](#)
 1. [19.2.1 Understanding Separate Compilation](#)
 2. [19.2.2 Understanding the Linker's Role](#)
 3. [19.3 Preventing Header Problems](#)
 1. [19.3.1 Using Include Guards](#)
 2. [19.3.2 Using #pragma once](#)
 4. [Worked example — Compile and link a small library interface](#)
 1. [Requirement and simplest approach](#)
 2. [Build dependency view](#)
 3. [Small controlled change](#)
 4. [Trace the implementation](#)
 5. [Complete worked source](#)
 6. [Run and verify](#)
 7. [Evidence and interpretation](#)
 8. [Failure clinic — the stale object](#)
 5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — produce a reusable calculation boundary](#)
 3. [Review and solution notes](#)
22. [Chapter 20 — Building a Complete Multi-File C++ Application](#)
 1. [20.1 Defining Application Requirements](#)
 1. [20.1.1 Converting Requirements into Program Features](#)
 2. [20.1.2 Modeling Application Data](#)
 2. [20.2 Implementing the Application](#)
 1. [20.2.1 Dividing Responsibilities Across Classes and Functions](#)
 2. [20.2.2 Organizing Source and Header Files](#)
 3. [20.3 Testing and Improving the Finished Program](#)
 1. [20.3.1 Debugging Functional Defects](#)
 2. [20.3.2 Refactoring Repetitive or Fragile Code](#)
 4. [Worked example — Build HarborInventory as a tested multi-file application](#)
 1. [Requirement and simplest approach](#)
 2. [Contract ledger](#)

3. [Mechanism view](#)
4. [Prepare a reproducible session](#)
5. [Trace the implementation](#)
6. [Complete worked source](#)
7. [Run and verify](#)
8. [Run a complete command session](#)
9. [Evidence and interpretation](#)
10. [Failure clinic — acceptance reported after a partial parse](#)
11. [Recovery and application boundary](#)
5. [Practice and implementation lab](#)
 1. [Starting exercise: solution notes](#)
 2. [Guided lab — finish with an acceptance review](#)
 3. [Solution notes for the extension choices](#)
 4. [Review and solution notes](#)
23. [Appendix A — A practical diagnostic desk reference](#)
 1. [Start with the failed boundary](#)
 2. [A compact failure map](#)
 3. [Read a diagnostic without losing the first cause](#)
 4. [A test is a claim with a witness](#)
 5. [Close an investigation with evidence](#)
24. [Appendix B — A worked solution for transactional batch updates](#)
 1. [Requirement and design](#)
 2. [Trace the evidence](#)
 3. [Run and extend](#)
25. [Appendix C — A worked solution for construction failure and cleanup](#)
 1. [Requirement and design](#)
 2. [Interpret the result](#)
26. [Appendix D — Capstone acceptance worksheet](#)
27. [Glossary](#)
28. [Technical reference directory](#)
 1. [ARRAY — Fixed-size array container](#)
 2. [CHARCONV — Primitive numeric conversions](#)
 3. [CLANG — Clang C++ language support](#)
 4. [CLASS — Classes and special member functions](#)
 5. [CMAKE — CMake tutorial](#)
 6. [EXPR — Expressions and evaluation](#)
 7. [FUNC — Functions and declarations](#)
 8. [GCC — GCC C++ language support](#)
 9. [GUIDELINES — C++ Core Guidelines](#)
 10. [IO — Input/output library](#)
 11. [LANG — C++ working draft](#)
 12. [LIFETIME — Object lifetime](#)
 13. [MSVCINSTALL — Install Microsoft C++ tools](#)
 14. [MSVCSTD — Microsoft C++ /std options](#)
 15. [NUMERIC — Fundamental types and numeric limits](#)
 16. [ODR — One-definition rule](#)

17. [STMT — Statements and control flow](#)
18. [STRING — Basic string](#)
19. [VECTOR — Vector container](#)
20. [GCC: Overall Options](#)
21. [Microsoft: Compile a native C++ program from the command line](#)
22. [GCC: Warning Options](#)
23. [C++ working draft: vector capacity](#)
24. [C++ working draft: initializing bases and members](#)
25. [C++ working draft: destructors](#)
26. [Clang: AddressSanitizer](#)
27. [Clang: UndefinedBehaviorSanitizer](#)

About this expanded volume

C++ Foundations

Learning the Language, Programming Fundamentals, Memory, and Object-Oriented Design

C++ From Beginner to Expert · Book I

Dr. Charles Dorner

Expanded manuscript · 2026

A compiler can accept a program that computes the wrong answer. A test can pass because it never exercised the boundary that matters. A pointer can contain a nonzero address after the object it once referred to has ceased to exist. Learning C++ begins with syntax, but it becomes useful when syntax is connected to contracts, object lifetime, evidence, and responsibility.

This volume follows the fictional HarborWorks laboratory as it develops small utilities and eventually a local equipment application called HarborInventory. The records, measurements, scenarios, and limits are invented teaching examples. No employer's confidential source or infrastructure is used. The examples are deliberately small enough to trace, yet complete enough to compile and test.

The twenty chapters retain the full curriculum hierarchy: each chapter has three numbered sections, and each section has two numbered subsections. The expanded treatment provides explanation between the headings, not just a longer list of topics. Every chapter connects its concepts to a complete worked program, a controlled failure investigation, and a guided implementation lab with review and solution notes.

The editorial model is the companion Linux administration book: understand the requirement, inspect the mechanism, make a bounded change, verify the result, and identify the recovery boundary. For C++, that means distinguishing source from executable, a value from its representation, an owner from a borrow, and a program's exit status from proof that its application contract is satisfied.

Audience and progression

No previous C++ experience is assumed. You should be comfortable creating files, opening a terminal or development environment, and following a directory path. Chapter 2 establishes the compiler and build tools. Concepts appear in a deliberate order: ordinary values and control flow precede containers; containers precede detailed pointer and manual-allocation reasoning; those foundations precede classes and the multi-file capstone.

The common source baseline is C++20. Most of Book I uses an intentionally smaller subset. A few names, such as `std::string_view`, `std::move`, and deleted copy operations, appear where they clarify a local example. Their immediate meaning is explained without claiming that the entire modern-language topic has already been covered. Book II develops resource safety, copy and move semantics, templates, ranges, and the broader standard library.

The last application is complete for its stated classroom contract, not a production asset-management service. It has no durable storage, authentication, or concurrent-user support. Those exclusions are design

boundaries rather than hidden defects. A sound learning outcome is being able to explain exactly what a program does and does not promise.

Technical scope and primary references

The default companion requires a C++20-capable compiler and standard library, CMake 3.24 or later, Python 3.10 or later, and a supported native build tool. Its CPU examples have no third-party source-download requirement. Linux is the execution environment used for the verification recorded with this package. Windows setup instructions are provided as platform guidance; Windows execution is not established by the Linux test results.

The reference directory points to compiler documentation, standard-library and language clauses, and project guidance. The online C++ working draft is a living document, not a claim to reproduce the published C++20 standard. Newer draft wording or library additions should not silently change the C++20 baseline of an exercise. Consult documentation appropriate to the compiler and library actually installed.

The added explanatory scenarios, traces, exercises, and contract examples are authored teaching material. Technical references support underlying mechanisms; they do not endorse the fictional applications or certify the examples for a particular industry. The package's revision notes explain the comparison with the earlier Book I and the Linux reference.

How to use this book

Read for a decision, then trace the mechanism

Before running a program, state what question it answers. A compiler warning identifies a potential code issue; a runtime check compares an actual result with an expectation; a sanitizer observes certain executed defects. None of those tools replaces the other two. The useful habit is selecting evidence that addresses the specific claim you intend to make.

Read the numbered explanation first. At the worked example, write the input, expected output, state transitions, and failure conditions before compiling. Then run the unchanged source. Only after establishing that baseline should you modify the program for the lab. A change is easier to explain when the earlier behavior is known.

Keep a short engineering notebook. Record the chapter, compiler version, command, input, expected result, actual result, and next hypothesis. When something fails, preserve the first meaningful diagnostic and the smallest reproducer. Avoid changing several unrelated variables at once; doing so can hide the cause even when the program begins to work.

Build the supplied examples

Extract the complete package and open a terminal in its root directory, the directory containing **source**, **tools**, and this book's supporting files. Do not execute the code with administrator privileges. The normal build writes generated files to a separate tree:

```
cmake -S source -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build --parallel 4
ctest --test-dir build --parallel 4 --output-on-failure
```

A target such as **b01_c03** means Book I, Chapter 3. Executables are placed under the build directory's **bin** subdirectory. On Windows, use the corresponding **.exe** filename and, when using a multi-configuration generator, supply **--config Debug** to the build and **-C Debug** to CTest.

To configure, build, and test just one chapter through the supplied helper:

```
python3 tools/run_chapter.py 3
```

The command above takes one chapter number, not a book number followed by a chapter number. This expanded package is Book I only. The full-series package used a different helper interface; do not mix its commands with this package's layout.

Source and output conventions

Code blocks labeled `cpp` contain C++ source. Blocks labeled `bash` contain shell commands; configuration files and illustrative output are labeled separately. A line beginning with `#include` is a C++ preprocessor directive, not a shell command. A path preceding a listing identifies the companion file containing the authoritative executable source.

Complete chapter sources are included in the book and under `source/ch01` through `source/ch20`. Short snippets elsewhere illustrate a mechanism and may require their surrounding declarations or headers. Do not paste an entire chapter into one source file or shell session. Read the context that distinguishes a complete program from a local expression.

Expected output is an acceptance condition, not a claim that an untested platform produced the same text. Some examples print implementation-dependent observations such as type sizes or vector capacity. Their tests check the intended invariant rather than requiring one machine's exact capacity value everywhere.

Use failures deliberately

Intentional compiler errors and unsafe conceptual counterexamples belong in disposable scratch copies. They are not part of the default passing build. Never execute undefined behavior merely to see which answer it produces and then treat that observation as a language guarantee. Many important defects can be explained with an ownership or range argument before executing them.

When a lab asks for a broken build, preserve the working reference. Change one dependency or rule, record the failure stage, and restore the known-good version. When a lab asks for an invalid input, keep the data invented and bounded. The goal is evidence about a mechanism, not accidental damage to unrelated files or valuable records.

Verification boundaries

The package includes `TEST_REPORT.md`, actual compiler and test logs, and document-check results. Read that report for completed checks and exact environment information. A finite passing suite is not an exhaustive correctness proof, and a sanitizer pass is not a promise that no unexecuted path contains a defect.

The PDF is a 7 × 10-inch reading edition with page numbers and navigable contents. It is not automatically an approved print-on-demand interior. The EPUB is reflowable; a small screen may wrap a long source line. Copy source from the companion rather than reconstructing it from visual wrapping.

The publishing pipeline assembles the supplied authored material and source into the books. It does not contact an AI service, fetch missing prose, or generate chapter filler from headings. Editable authoring inputs and the assembled Markdown are both included, together with the exact build scripts used for this revision.

The central operating rule

Understand the contract. Trace the mechanism. Verify the result. A beginner who can explain why a small program is correct, what can invalidate its assumptions, and how its behavior was tested has a foundation that scales beyond memorized syntax.

Chapter 1 — Understanding C++ and How Programs Become Executables

HarborWorks wants a small program that can summarize equipment readings on an ordinary workstation and later become part of a server application. Before writing its calculations, you need to understand what will execute them. A source file is a description of behavior, not an executable instruction stream. The compiler, linker, standard library, and operating system each contribute something different to the finished program.

This chapter gives you a map of those boundaries. When a build fails, ask which stage failed before asking which line to change. A missing semicolon and an unresolved function name at link time are different categories of problem.

A beginner often treats the Build button as a single operation. That model works until the button reports a hundred lines of output and none seems related to the change just made. A better model separates the document you edit, the tools that translate it, and the process that eventually runs. You do not need to read machine instructions yet. You do need to know which artifact should exist after each successful stage.

Learning outcome. Given a small program and a build command, identify the source language, selected standard, output artifact, and failing stage. Explain why an executable left over from yesterday is not evidence that today's source compiled successfully.

1.1 Understanding the Role of C++ in Modern Software

1.1.1 Distinguishing Systems Programming from Application Programming

Systems programming deals directly with the facilities on which other software depends: memory allocation, file access, communication, scheduling, and hardware interfaces. Application programming expresses a user's task, such as editing a document or analyzing measurements. These categories overlap. A measurement application may contain a high-level report generator and a low-level acquisition component.

C++ lets both layers share a type system and executable. This does not mean every component should use raw memory or operating-system calls. Keep the application's decisions separate from platform mechanisms. A function that computes an average should not also decide how a device is opened or how a window is painted. That separation makes the calculation reusable and the platform-specific behavior easier to test.

Think of HarborWorks' equipment report as three responsibilities. One responsibility describes the business rule: add the quantities of accepted equipment records. Another obtains bytes from a terminal or file. A third arranges for the result to appear on a screen. The first can be expressed without knowing whether the machine runs Linux or Windows. The others eventually depend on the environment. The distinction is about responsibility, not the prestige or difficulty of the code.

A systems-oriented choice has consequences outside a single calculation. Selecting a buffer size can affect memory use. Keeping a file open can prevent another operation. Blocking for input can delay a user interface. An application-oriented rule has consequences too: counting a failed sensor as available can make a correct low-level implementation produce an incorrect report. Neither category replaces the other.

For this volume, most programs remain at the portable application layer. They use standard-library facilities instead of direct operating-system calls. This is intentional: understanding values, conditions, and lifetime gives you a stable foundation for the lower-level facilities in Book IV. When an example prints a value, distinguish the computation that produced it from the library operation that transports its textual representation. That habit will later help you separate an incorrect result from an input, output, or platform failure.

1.1.2 Understanding Where C++ Is Used in Industry

C++ is useful when software must combine abstraction with explicit control over representation, resource lifetime, and execution costs. Examples include native applications, game engines, embedded software, numerical libraries, storage systems, and network services. These examples describe workload families, not a promise that one language is automatically the right choice for every project.

Evaluate the requirement instead of choosing by reputation. A latency-sensitive parser may justify careful memory layout; a short administrative script may not justify a native build pipeline. Throughout this series, explain what C++ buys for a particular component: a predictable data structure, a reusable type, a controlled resource boundary, or an efficient numerical loop. The explanation matters more than calling the language fast.

The useful question is not “Is C++ fast?” but “Which requirements make its control valuable here?” A device application might need access to an existing C or C++ library. A numerical program might need a representation that avoids unnecessary data movement. A desktop application might need to reuse a mature native component. These are different reasons to choose the same language; none establishes that every program should be written in it.

Consider a proposed inventory utility. Its first version accepts a few dozen records and prints a total. Human-readable code and reliable validation matter more than elaborate allocation techniques. Later, a measurement component may process large buffers repeatedly. That component might justify closer attention to representation and resource costs. The language allows the design to evolve, but it does not require you to begin with the most complicated implementation.

In an engineering discussion, name the constraint before naming the feature. “The parser must reject an invalid quantity without changing inventory” is a useful constraint. “We should use pointers because this is C++” is not. At the end of the volume, you should be able to defend the simple design where it is sufficient and identify the evidence needed before adopting a more complex one. This is also how you resist premature optimization: you keep the actual workload, not an imagined future machine, at the center of the decision.

Value-Type Sensor Processing with Standard Containers

Purpose. The program combines low-level value representation with high-level library abstractions, which is a common C++ balance in systems, simulation, tooling, and performance-sensitive applications.

```
#include <algorithm>
#include <iostream>
#include <vector>
struct Reading { int id; double value; };
int main() {
    std::vector<Reading> r{{3, 18.2}, {1, 20.1}, {2, 19.4}};
    std::sort(r.begin(), r.end(), [](const Reading& a, const Reading& b){ return
a.id < b.id; });
    for (const auto& x : r) std::cout << x.id << ':' << x.value << '\n';
}
```

How it works.

- `Reading` is a compact user-defined value type.
- `std::vector` supplies contiguous dynamic storage without manual allocation.
- `std::sort` uses a lambda so the policy is explicit at the call site.

Expected behavior/output. Three readings print in ID order: 1, 2, then 3.

Experiment. Add a fourth reading and change the comparator to sort by `value`. Predict the order before running.

1.2 Understanding the C++ Compilation Pipeline

1.2.1 Transforming Source Code Through Preprocessing and Compilation

The preprocessor handles directives such as `#include` and conditional compilation. Conceptually, including a header makes its declarations available in the current source file. The compiler then checks language rules and transforms the resulting translation unit into lower-level instructions. An assembler may produce the final object file as a distinct toolchain step.

Use `g++ -E main.cpp` to inspect preprocessed source, `g++ -S main.cpp` to request assembly, and `g++ -c main.cpp` to create an object file without linking. These are observation tools. Do not edit generated assembly or preprocessed output as though it were the primary source. A diagnostic about an undeclared variable is generally a compilation problem; it is not repaired by adding another object file to the linker command.

A preprocessing directive starts with `#`, but it is not an ordinary runtime statement. In a traditional header-based build, `#include <iostream>` makes declarations from the library header available to the translation process. It does not tell the running program to open a text file named `iostream`. By the time `main` executes, preprocessing is already over.

Use the compiler driver to stop at an intermediate stage. On a GCC-compatible command line, the following commands deliberately produce different kinds of files:

```
g++ -std=c++20 -E source/ch01/main.cpp -o build/ch01.ii
g++ -std=c++20 -S source/ch01/main.cpp -o build/ch01.s
g++ -std=c++20 -c source/ch01/main.cpp -o build/ch01.o
```

Create `build` first. The `.ii` file contains preprocessed C++ text, the `.s` file contains assembly text, and the `.o` file is an object file. The spelling of these options is documented by GCC; the filename extension describes your intent but is not itself a correctness check. [GCC output stages](#)

Open the preprocessed file only to observe its size and the presence of expanded declarations. You are not expected to understand the entire standard library. The important observation is that the translation unit contains more than the few lines you personally wrote. A diagnostic can therefore originate in a header even when your own type or call caused the problem. Begin with the earliest useful diagnostic and follow its reference back to your source.

1.2.2 Combining Object Files Through Linking

An object file contains machine code and metadata, including references to functions or objects that may be defined elsewhere. The linker combines compatible object files and libraries, resolves those references, and constructs an executable or library. A declaration can allow compilation to succeed while a missing definition causes linking to fail.

For example, `int total();` tells the compiler how a call is formed. It does not provide the function body. When separate files define and call `total`, both relevant object files must participate in the link. Link order, library search paths, and binary compatibility become important later. For now, distinguish a source-level name error from a linker message such as an undefined reference.

An object file can contain machine instructions while still depending on definitions supplied elsewhere. Imagine one source file calling a function named `groups_needed` and another source file defining it. The first compiler invocation can check the call if it sees a compatible declaration. The linker later has to find the definition among the supplied objects and libraries.

This gives two different questions. Compilation asks whether the translation unit is acceptable under the chosen rules. Linking asks whether the program's required definitions can be assembled into the requested artifact. A successful compile-only command answers the first question, not the second. Running `g++ -c` and finding no executable is therefore expected, not a mysterious installation failure.

The usual final step for this chapter is:

```
g++ -std=c++20 build/ch01.o -o build/ch01
```

Use the C++ driver for this link step so it supplies the usual C++ runtime and library choices for its environment. Manually invoking a low-level linker is an advanced task, not a repair for an ordinary missing-definition error.

If linking fails, preserve the exact command. A definition may exist in your editor yet be absent from the command's input list. A name may also differ because a declaration and definition disagree about their

namespace or parameter types. Chapter 19 develops those cases. Here, learn to distinguish “the compiler did not accept this source” from “the linker did not receive the complete compatible program.”

Preprocess, Compile, Assemble, and Link One Program

Purpose. Breaking the build into stages makes the compilation pipeline observable instead of treating `g++ main.cpp`` as a black box.

```
g++ -std=c++20 -E main.cpp -o main.ii
g++ -std=c++20 -S main.ii -o main.s
g++ -c main.s -o main.o
g++ main.o -o app
./app
```

How it works.

- `-E`` stops after preprocessing so includes and macros are expanded.
- `-S`` translates the preprocessed source to assembly.
- `-c`` produces an object file without linking.
- The final driver invocation links `main.o`` into an executable.

Expected behavior/output. If `main.cpp`` is valid, the commands create `main.ii``, `main.s``, `main.o``, and finally `app``.

Experiment. Introduce a missing function definition. Compilation still creates an object file, but the final link should fail with an unresolved symbol.

1.3 Understanding the C++ Standard and Compiler Implementations

1.3.1 Understanding Language Standards and Standard-Library Versions

The C++ standard specifies language and library behavior; a compiler toolchain implements it. Select the required dialect explicitly with a setting such as `-std=c++20`. A compiler’s default can differ by version. A language feature and a library feature can also have different implementation status, so successful parsing of modern syntax does not prove that every modern library facility exists.

This series uses a deliberately bounded C++20 baseline for host examples and a separately documented CUDA baseline. It does not assume that the newest published or proposed standard is available on every reader’s machine. Record the compiler version, library implementation, target architecture, and build flags whenever portability matters. The reference directory points to the vendors’ implementation-status pages.

A language standard and an implementation are separate things. The standard describes permitted programs and required behavior. A compiler implements some set of those rules, and a standard-library

implementation supplies facilities such as strings and streams. Their version numbers need not match, and selecting a language mode does not install a missing library facility.

The companion deliberately requests C++20. That is a teaching baseline, not a claim that every chapter uses a C++20-only feature or that C++20 is the newest standard. Most of Book I uses a smaller subset so you can focus on one idea at a time. Later books can introduce more expressive facilities without requiring a different conceptual foundation for every example.

`__cplusplus` is a predefined macro used in the first program to inspect the selected language mode. A macro is a translation-time mechanism; its printed value is not a runtime performance measurement. Some Microsoft compiler configurations require `/Zc:__cplusplus` for the macro to report the selected standard mode as expected. The supplied CMake settings include that option when appropriate.

When reporting a build problem, record the compiler version, selected standard, operating system, architecture, and full command. “I have C++ installed” leaves most of those questions unanswered. The same source can also compile under several standards without demonstrating that every available standard-library feature is present. Test the particular facility your program needs rather than treating one successful greeting program as a complete compatibility certificate.

1.3.2 Comparing GCC, Clang, and Microsoft Visual C++

GCC, Clang, and Microsoft Visual C++ are different compiler implementations with different command-line interfaces, diagnostics, platform integrations, and library arrangements. GCC and Clang commonly accept `-std=c++20`; MSVC uses `/std:c++20`. Clang can use different standard libraries depending on its installation. A familiar compiler name does not uniquely identify the complete environment.

Avoid defining correctness as “my compiler accepted it.” Enable warnings and compare behavior across implementations when practical. Some extensions compile only in one environment; some invalid programs happen to run until optimization exposes their mistake. The companion records the tools actually used for testing rather than claiming universal support. Windows-specific installation instructions are guidance, not evidence of a Windows test run.

GCC, Clang, and MSVC are toolchain families, not three dialects you should mix within a source file. For this book, write to the shared language baseline and label environment-specific commands. A Linux command using `g++` will not become an MSVC command merely by changing the executable’s name; option syntax, environment setup, and output conventions differ.

A compiler may also use another project’s library. For example, a Clang installation can be configured with different standard libraries. Consequently, “works with Clang” is incomplete evidence for a library issue. Record which headers and libraries the build actually used. You do not need to manage every possible combination now; you need to avoid drawing conclusions that the evidence does not support.

Use two compilers as an additional diagnostic perspective, not as a vote on correctness. If one rejects a program and another accepts it, investigate the language rule, selected flags, extensions, and implementation support. Acceptance alone cannot make undefined behavior safe. Likewise, a different warning message can describe the same underlying defect.

For the beginning exercises, choose one working toolchain and keep it stable. Constantly switching editors and compilers introduces extra variables into every experiment. Once you can reproduce a build

from a terminal, a second compiler becomes useful: it tests a recorded project instead of testing your memory of how an editor happened to be configured.

Checking the Active Language Standard and Compiler

Purpose. The C++ standard defines the language and library contract, while GCC, Clang, and MSVC are implementations that expose their own version and extension details.

```
#include <iostream>
int main() {
    std::cout << "__cplusplus=" << __cplusplus << '\n';
    #if defined(__clang__)
        std::cout << "compiler=Clang\n";
    #elif defined(__GNUC__)
        std::cout << "compiler=GCC\n";
    #elif defined(_MSC_VER)
        std::cout << "compiler=MSVC\n";
    #endif
}
```

How it works.

- `__cplusplus` reports the language-mode value selected by the implementation.
- Implementation macros identify the compiler family without changing standard C++ semantics.

Expected behavior/output. With a C++20-capable GCC or Clang build, `__cplusplus` is at least 202002 and the compiler family prints.

Experiment. Compile once with `-std=c++17` and once with `-std=c++20`; compare `__cplusplus`.

Worked example — Identify the language mode before trusting a build

Requirement and simplest approach

The lab needs a program that demonstrates a complete build and reports its selected language mode. The simplest approach is to click Run and accept whatever appears in the output window. That approach hides the source path, compiler selection, and possibility that an older executable is being launched.

The selected design is deliberately smaller: one source file, one named executable, and a visible command. No external dependency download, administrator permission, or network service is required. A source file is the input. The executable is an output. Its text and exit status are observations from a particular run.

Establish the starting state

From the extracted package directory, create a build directory and record the compiler:

```
mkdir -p build
g++ --version
g++ -std=c++20 -Wall -Wextra -Wpedantic \
    source/ch01/main.cpp -o build/ch01
./build/ch01
```

These commands assume a Bash-like shell and a working GCC installation. Chapter 2 covers installation and Windows commands. The backslash continues a shell command onto the next physical line; it is not part of the C++ source. Do not place spaces after it. A beginner using a single-line command can simply omit the backslash and join the lines.

Before running, predict two things: the explanatory sentence will be printed, and the program will either confirm the selected mode or report that the mode is too old. Do not predict a pointer address, timing, or compiler brand: this program does not measure those things.

Trace the implementation

The program inspects a predefined macro rather than guessing the compiler's default. `__cplusplus` is a numeric indicator of the selected language mode, although MSVC requires `/Zc:__cplusplus` for its conforming value. The companion enables that option for MSVC builds. The first conditional is evaluated during preprocessing; the later `if` belongs to program execution.

Compile the same file once with `-std=c++20` and once with `-std=c++17`. The second executable is intentionally rejected by the runtime check. This is a teaching probe, not a comprehensive feature detector. In a real library, test the particular facility you require with its feature-test macro or a configure-time compilation check.

Trace the path as `main.cpp -> translation unit -> object file -> executable -> process`. The executable remains a file after it exits; the process does not. A successful build establishes that the toolchain accepted this program, not that all requirements of a larger application have been met.

```
source + headers -> compiler -> object files
object files + libraries -> linker -> executable
executable + operating system -> running process
```

Complete worked source

source/ch01/main.cpp

```
#include <iostream>

#ifndef __cplusplus
#error This file must be compiled as C++.
#endif
```

```
int main() {
    std::cout << "A source file becomes an executable.\n";
    std::cout << "__cplusplus=" << __cplusplus << '\n';
    if (__cplusplus < 202002L) {
        std::cerr << "Select C++20 or later for this series.\n";
        return 1;
    }
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

```
python3 tools/run_chapter.py 1
```

The executable target is `b01_c01`. Read `source/ch01/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Read the result at the correct level

The final **PASS** means that this program's explicit language-mode check passed. It does not establish that the compiler has no bugs or that the machine supports every example in the six-volume curriculum. The distinction is important because engineers routinely overgeneralize a narrow green check.

Now introduce a controlled defect in a **copy** of the file. Remove a semicolon from an output statement and compile the copy to a new executable name. The translation should fail. Restore the semicolon and compile again. The purpose is not to memorize a diagnostic's exact wording; it is to recognize that a syntax failure occurs before the new program can run.

Next, run a successful compile-only command. You should obtain an object file, not a complete executable. Finally, link that object. These three observations establish the pipeline more effectively than a diagram alone because each observation has an artifact you can inspect.

A misleading success: the stale executable

Suppose `build/ch01` exists from a successful run. You break the source, invoke the compiler, and then run `./build/ch01` anyway. The old executable may still run successfully. The apparent contradiction disappears once you recognize that the failed build did not update it.

A shell can express the intended dependency:

```
g++ -std=c++20 source/ch01/main.cpp -o build/ch01 \
&& ./build/ch01
```

Here `&&` belongs to the shell: run the second command only after the first succeeds. It is not a C++ logical expression. Keep shell syntax and source syntax in separate files and separate mental models. In a larger project, the build system manages these dependencies, but it cannot rescue you from launching an unrelated executable in another directory.

Practice and implementation lab

Create an object file with `g++ -std=c++20 -c main.cpp -o mode.o`, then link it with `g++ mode.o -o mode`. Record which step produces the executable. Separately introduce a misspelled identifier and compare that diagnostic with the result of declaring, calling, but never defining a function.

Explain why installing a text editor does not necessarily install a compiler. List the four environment details you would include in a build defect report.

Starting exercise: solution notes

Compilation produces `mode.o`; linking produces `mode`. A misspelled identifier is diagnosed while compiling the translation unit. A called function with no definition can reach the linker and fail there. An editor manipulates text, while a compiler translates it; an integrated development environment may arrange both but does not make them the same component.

A useful defect report includes compiler and version, standard-library implementation, target platform, and the exact build command. Preserve the complete first failing diagnostic and the smallest source that reproduces it.

Guided lab — Build an evidence trail

Use a clean subdirectory named `build/pipeline-lab`. Produce the preprocessed source, assembly, object, and executable with distinct filenames. After each command, record the requested stopping point, the resulting filename, and whether the next stage has occurred. Do not paste the binary object file into a text editor and expect meaningful prose.

Then change only the message string. Rebuild and run. Your evidence should connect the edited source, successful command, and changed output. A screenshot of the final message is weaker than a record that also identifies the exact executable path. You are learning to make a reproducible claim, not merely to obtain the desired display once.

For a second experiment, intentionally omit the final link step. Explain why there is still no newly linked executable even though the compiler reported no error. For a third, introduce the missing-semicolon defect and confirm that the conditional shell command does not launch the program after a failed compilation. Restore the source before running the normal chapter tests.

Acceptance criteria. Your notebook distinguishes all four build stages; your commands work from the documented directory; your final output reflects a deliberate source change; and your explanation of a failed build does not confuse it with a runtime failure. No claim depends on a specific compiler diagnostic string or an assumed object-file size.

Review and solution reasoning

Can a source file exist without any executable? Yes. Editing and saving text does not invoke translation. The first evidence of a build is a tool invocation and its outputs.

Can an object file be correct while linking fails? Yes. A translation unit may be acceptable but require a definition not supplied to the linker. Correctness at one stage does not imply completeness at the next.

Does a successful run prove the source currently open in the editor was built? No. You must connect the source path and timestamp or content to a successful build of the executable you launched. The stale-executable experiment demonstrates why.

Must every C++20 program use new C++20 features? No. A baseline permits a language feature set; it does not require every available feature in every program.

Why keep generated files in a build directory? It separates replaceable artifacts from authored source. You can remove and regenerate build outputs without deleting the work you are trying to preserve. That directory boundary becomes a practical recovery mechanism throughout the book.

Chapter takeaway. A build is a chain of transformations. Name the stage, preserve the command, inspect the artifact, and connect the executable you run to the source you intended to test.

Technical references: [GCC](#) · [CLANG](#) · [MSVCSTD](#).

Chapter 2 — Creating a Complete C++ Development Environment

A development environment should answer three questions without ambiguity: which compiler is being used, which command builds the program, and which executable is being run. HarborWorks initially has two copies of a program with the same name in different directories. Editing one and running the other creates the illusion that the compiler ignores changes. A small, explicit workspace prevents that class of error.

Begin with one source directory and a separate build directory. Add editor conveniences only after a command-line build works. The command becomes a reproducible explanation of what the editor should do.

A working environment is not defined by the number of installed tools. It is defined by your ability to create a plain-text source file, compile it with known settings, run the resulting program, and locate the first useful diagnostic when something goes wrong. You will establish that path before customizing themes, shortcuts, or extensions.

Learning outcome. Reproduce a build outside the editor, explain the distinction between compiler and editor configuration, and record enough environment information for another learner to repeat the experiment. Installation commands in this chapter are setup instructions, not commands executed by the book’s automated tests.

2.1 Installing and Configuring a C++ Compiler

2.1.1 Installing GCC or Clang on Linux

On a Debian- or Ubuntu-family lab system, a typical starting installation is `sudo apt update` followed by `sudo apt install build-essential cmake ninja-build`. Clang can be installed separately through the distribution’s supported packages. On a RHEL-family system, package names and repository arrangements differ; inspect the distribution documentation rather than copying an unrelated repository configuration.

Installation requires administrative approval, but compiling your own files should not require root. Verify `g++ --version`, `cmake --version`, and `command -v g++`. A successful package installation is not a substitute for compiling a small program. If the compiler is too old for a required feature, use a supported toolchain upgrade or isolated development environment instead of replacing system libraries at random.

On a Debian- or Ubuntu-family lab machine, the conventional starting packages include a compiler toolchain and basic build tools. A representative setup is:

```
sudo apt update
sudo apt install build-essential cmake python3
```

For the optional second compiler, install the distribution's Clang package. On an RPM-family machine, use that distribution's supported repositories and package names rather than pasting the APT command. The important deliverables are a C++ compiler, compatible headers and libraries, a build tool, and Python for the supplied fixture runner. Do not add unrelated third-party repositories merely because a tutorial mentions a different version number.

After installation, inspect what the shell resolves:

```
command -v g++
g++ --version
cmake --version
python3 --version
```

A version command is necessary evidence, but it is not sufficient. Compile and run the chapter's small probe. A binary may be on **PATH** while its headers, runtime, or selected target configuration are unusable. Conversely, an editor may display an error simply because it has not been pointed at the installed compiler.

Keep the compiler installed at the operating-system level rather than copying executable files from another machine. A toolchain consists of cooperating components. Moving only the **g++** driver does not move its complete environment. On managed workstations, obtain approved installation support; this book does not require bypassing workplace controls or replacing a supported system compiler.

2.1.2 Installing Microsoft Visual C++ on Windows

Use the Visual Studio Installer to select the C++ desktop-development workload and the appropriate compiler tools and Windows SDK. A normal command prompt may not contain the environment variables needed by **cl**; a Developer Command Prompt establishes the intended toolchain environment. Confirm the compiler identity before interpreting missing-header errors as defects in your program.

A representative command is **cl /std:c++20 /Zc:__cplusplus /W4 /EHsc /Zi main.cpp**. Options are not interchangeable with GCC's spelling. **/EHsc** selects the usual standard C++ exception model for these examples. Keep a record of the installed workload and toolset. The Windows instructions describe setup; the package's test report states whether that platform was actually executed.

For an MSVC-based Windows environment, install the C++ development workload in Visual Studio or the corresponding Build Tools. The language extension in an editor is not a substitute for those compiler components. Microsoft's command-line walkthrough explains the requirement for the Desktop development with C++ workload and an appropriately initialized developer command prompt. [MSVC setup](#)

Open an x64 developer command prompt for your installed toolset. It prepares paths and other variables for the compiler, headers, libraries, and tools. A plain command prompt that reports **cl** is unknown may simply lack this initialization. Reinstalling everything before testing the developer prompt creates unnecessary work.

From a disposable directory containing a source file, a representative command is:

```
cl /nologo /std:c++20 /Zc:__cplusplus /W4 /EHsc /Zi main.cpp  
main.exe
```

The options request the language mode, a useful language-version macro, warnings, standard C++ exception handling behavior, and debug information. The exact installed toolset remains part of the build record. Do not copy the Linux `-O` or `-std=` spelling into this command and expect the driver to interpret it identically.

Keep source, object files, and executables from different toolchains in separate build directories. Windows can also host other C++ environments, including Linux through WSL, but those environments have distinct paths and binaries. Choose one path for the first exercises; mixing them before you understand which shell is running makes simple failures much harder to classify.

Verifying a GCC or Clang Installation with a C++20 Build

Purpose. A compiler installation is not complete for the learner until the shell can locate the driver and successfully compile and run a known source file.

```
g++ --version  
g++ -std=c++20 -Wall -Wextra -pedantic hello.cpp -o hello  
./hello
```

How it works.

- ``g++ --version`` confirms which driver the shell resolves.
- ``-std=c++20`` selects the language mode.
- The warning flags verify that the compiler is usable with a useful diagnostic policy.

Expected behavior/output. The version command reports the compiler, then ``hello`` builds and runs.

Experiment. Run ``command -v g++`` (or ``where cl`` on Windows) and record the exact compiler path used by your shell.

2.2 Selecting and Configuring Development Tools

2.2.1 Developing with Visual Studio and Visual Studio Code

Visual Studio is an integrated development environment with its own project and debugging facilities. Visual Studio Code is an editor that can invoke compilers, debuggers, and build systems through extensions and tasks. Their similar names do not make their installation or project formats identical.

Configure the editor to use the same compiler and standard as the command-line build. Language-server diagnostics are useful, but they are not the authoritative result of compiling your target. A header search path available to the editor may still be absent from the build. Prefer a CMake project or a compilation

database as a shared source of configuration once you reach larger programs. Avoid maintaining separate, contradictory lists of compiler flags.

An integrated development environment can combine editing, project configuration, building, debugging, and navigation. Visual Studio Code is primarily an extensible editor; its C++ support relies on separately configured tools. The two products share part of a name, not an identical architecture. A green editor status indicator is not proof that your build command uses the intended compiler.

There are usually at least two configurations to consider: the actual build settings and the editor's interpretation of the source. If the build uses C++20 while the editor assumes an older standard, the editor may underline valid code. If the editor knows an include directory but the compiler command does not, the code may look fine and still fail to build. Repair the configuration that is actually wrong rather than suppressing diagnostics indiscriminately.

Begin with one source file and an explicit build command. Only then connect the editor's build task to that command or to CMake. Save the file before building and confirm the output directory. Some editors offer "run current file," which is convenient for one-file lessons but inappropriate for a multi-file application unless its dependencies are included.

When debugging, confirm that the launched executable is the one just built with debug information. A breakpoint that never triggers might indicate a different executable, optimized-away code, or a path mismatch. Those are toolchain questions. They should not immediately send you into rewriting correct program logic.

2.2.2 Developing Directly from the Command Line

The shell makes input and output paths explicit. Create a directory for the chapter, enter it, inspect its files, and compile using a relative or absolute source path you understand. Use a distinct output name and run `./program` on Unix-like systems so that the shell does not select another program from `PATH`.

Treat the build command as data worth saving. A short script or CMake target is more dependable than a remembered sequence of clicks. Keep generated objects and executables outside the manuscript or source tree when possible. A clean rebuild should recreate them. Source control should preserve the material needed to rebuild, not depend on a particular developer's stale executable.

The command line makes inputs explicit. The current directory determines how relative paths are resolved; quotation marks preserve a path containing spaces; environment variables influence which programs and libraries are found. These are not C++ language rules, but they are part of the practical execution environment.

Use a project directory you own. For the examples, the root is the directory containing `source`, `tools`, and `manuscript`. The notation `source/ch02/main.cpp` is relative to that root. Running the same command from inside `source/ch02` would search for a different, probably nonexistent nested path. Before diagnosing a compiler installation, check where you are.

A simple Linux inspection sequence is:

```
pwd
ls source/ch02
mkdir -p build
```

Do not run the chapter programs with **sudo**. They do not need administrative privilege, and using it can create root-owned artifacts or give a mistaken file operation unnecessary authority. Compiling an ordinary program and installing a system package are different activities with different permission requirements.

Maintain a short command history in your notebook, not a huge dump of the environment. Record the few variables relevant to the build, such as the selected compiler and flags. Avoid copying tokens or private paths into a public support question. A reproducible problem report should be specific without leaking unrelated information.

VS Code or IDE Front End Driving the Same Command-Line Toolchain

Purpose. The example separates editor/IDE responsibilities from compiler and debugger responsibilities, which is the key distinction when selecting and configuring development tools.

```
mkdir -p cpp-project/src cpp-project/build
cd cpp-project
code .
g++ -std=c++20 -Wall -Wextra -Wpedantic -g src/main.cpp -o build/app
./build/app
```

How it works.

- The project separates maintained source from generated build output.
- `code .` opens an editor, but it does not replace the compiler.`
- `-g` emits debugger information while warning flags configure compiler diagnostics.`
- `./build/app` proves the executable can run independently of the editor.`

Expected behavior/output. The editor opens the project, the compiler creates `build/app``, and the executable runs from the shell.

Experiment. Remove `-g``, rebuild, and compare debugger symbol information with `file build/app`` or your debugger. Then restore `-g``.

2.3 Compiling Programs Manually

2.3.1 Selecting the C++ Language Standard

The source file's extension identifies its language to many compiler drivers, but it does not select every language-version rule. Use `g++ -std=c++20 main.cpp -o program` or the corresponding

Clang/MSVC setting. In a CMake project, require the feature level on the target and disable nonstandard extensions when portability is intended.

Do not silence an unsupported-feature error by removing the standard setting and hoping that a permissive default will work. First distinguish an old compiler, an incomplete library, a missing header, and an actual source mistake. Each requires a different repair. The companion’s host targets request C++20 consistently; CUDA targets use their own explicit standard setting.

The standard flag belongs to the compilation command, not to a comment inside the source. A comment saying “C++20” documents intent but does not configure a compiler. Similarly, the `.cpp` extension identifies a conventional source type; it does not select a particular language revision.

Use the same standard mode for all translation units in the beginner project. A large organization may intentionally support more complicated combinations, but that requires compatibility reasoning beyond this volume. For now, a consistent setting eliminates a source of uncertainty. Build-system settings should express this once so individual files do not silently diverge.

With CMake, the companion requests `CXX_std_20` on its common settings target. That tells CMake to arrange an appropriate compiler mode for the target’s language requirement. It does not download a newer compiler when the installed one lacks support. Configuration or compilation can still fail, and the correct response is to read that failure rather than changing the program until it accidentally fits an unknown default.

A helpful experiment is to compile the probe once with the documented mode and once with a deliberately older mode. Record the difference. Then restore the documented setting. You are testing the relationship between configuration and observable behavior; you are not trying to maintain multiple standards in every exercise.

2.3.2 Enabling Compiler Warnings and Debug Information

For GCC and Clang, begin with `-Wall -Wextra -Wpedantic -g; -O0` makes an introductory debugging session easier to relate to source lines. Additional checks such as `-Wconversion` and `-Wshadow` can reveal unintended conversions and hidden names, but understand a warning before suppressing it. MSVC’s `/W4` and `/Zi` serve related purposes with different details.

Warnings are engineering signals rather than proofs of correctness. Debug information helps a debugger map machine state to source. Optimization can change that mapping without making the program incorrect. Keep separate debug and release configurations, and eventually test both. A program that only works without optimization often contains a defect that the unoptimized execution happened to hide.

Warnings describe suspicious constructs that the compiler can identify. Some warnings point to definite mistakes; others require context. Treat them as questions that deserve an answer. A clean warning log is useful, but it does not establish that the program’s business rules are correct. A program can add the wrong two perfectly valid integers without producing a warning.

For GCC and Clang, the book’s ordinary baseline is `-Wall -Wextra -Wpedantic`. The name `-Wall` does not literally mean every warning the compiler can produce. Additional warnings and their implications depend on the toolchain. Consult the installed compiler’s documentation rather than treating a copied list of flags as universally safe. [GCC warnings](#)

Debug information, requested with `-g` on the GCC-compatible command line, helps a debugger associate execution with source names and locations. Optimization and debug information are separate choices. The beginner build uses a debugging-oriented configuration because predictable inspection is more valuable than measuring speed at this stage.

Do not silence a signed/unsigned warning by applying an unexplained cast. First identify the permitted input range and the type of each operand. A cast can change a diagnostic while preserving the underlying defect. Later chapters turn this principle into concrete rules for counts and indices. Your immediate habit is simpler: keep warnings visible, change one cause at a time, and rerun the relevant example after the change.

Compiling Two Translation Units and Linking Them Manually

Purpose. Manual compilation exposes the distinction between compilation and linking and makes missing-object-file errors easier to classify.

```
# math.cpp -> math.o
g++ -std=c++20 -Wall -Wextra -c math.cpp -o math.o
# main.cpp -> main.o
g++ -std=c++20 -Wall -Wextra -c main.cpp -o main.o
# object files -> executable
g++ main.o math.o -o calculator
./calculator
```

How it works.

- Each `-c`` command compiles one translation unit without linking.
- The final command gives the linker both object files so calls defined in ``math.cpp`` can be resolved.

Expected behavior/output. Both object files are created, then ``calculator`` links and runs.

Experiment. Omit ``math.o`` from the link step and observe the unresolved-reference error; then restore it.

Worked example — Build an environment fingerprint

Requirement and simplest approach

HarborWorks needs a short environment record before asking a colleague to reproduce a problem. The simplest approach is a screenshot of the editor. That shows the interface, but it may omit the compiler, selected language mode, and target properties that determine the build.

The chapter probe reports a few implementation observations: the selected C++ mode, the sizes of two types, and the binary precision associated with `double`. These values are diagnostic context. They are not requirements that every platform must share. In particular, do not edit a correct program to force `sizeof(int)` or `sizeof(void*)` to match a screenshot from another machine.

Build the probe independently of the editor

On the documented Linux command line:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -g \  
    source/ch02/main.cpp -o build/ch02 \  
    ./build/ch02
```

The program uses `std::numeric_limits`, a standard-library facility for asking about type properties. You need not memorize every property now. Its role here is to replace an assumption with a query. Chapter 4 explains why numeric range and representation affect program design.

Record the compiler's version output beside the probe's output. Then run the same executable from the editor's terminal. The two displays should describe the same binary. If they differ, determine whether you compiled or launched a different path. This is a controlled comparison, not an invitation to reinstall the operating system.

Trace the implementation

This fingerprint distinguishes language settings from representation details. `sizeof(int)` and `sizeof(void*)` are observations of this implementation, not universal constants. `numeric_limits<double>::digits` describes the number of radix digits used for the significand; on a typical binary implementation, those are binary digits, not decimal display places.

Build from a clean directory and save the command beside the output. Change the optimization level and rebuild under a different output name. The representation values normally remain the same for the same target, while debugging behavior and generated instructions may change. None of these numbers measures program speed.

The source deliberately avoids asserting that an integer is four bytes or a pointer is eight bytes. Such assumptions belong in a platform contract only when a requirement actually depends on them. A compiler that reports different values is not automatically wrong.

Complete worked source

source/ch02/main.cpp

```
#include <iostream>  
#include <limits>  
  
int main() {  
    std::cout << "C++ mode: " << __cplusplus << '\n';  
    std::cout << "int bytes: " << sizeof(int) << '\n';  
    std::cout << "pointer bytes: " << sizeof(void*) << '\n';  
    std::cout << "double digits: "  
                << std::numeric_limits<double>::digits << '\n';  
    if (__cplusplus < 202002L) return 1;  
    std::cout << "PASS\n";  
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

```
python3 tools/run_chapter.py 2
```

The executable target is `b01_c02`. Read `source/ch02/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Environment facts and environment claims

A line reporting a pointer size tells you about that build target. It does not prove how much physical memory the machine has. A line reporting C++ mode tells you about the translation setting. It does not prove full implementation of every feature in that standard. The probe is useful precisely because its questions are narrow and its limits are stated.

Your environment record should contain the operating system, compiler identity, language setting, command, and observed result. Also record how to return to the project root. A colleague cannot reproduce a relative path without knowing its starting directory. For the same reason, a build script should determine its own project location instead of assuming the caller's current directory whenever practical.

Failure clinic — Four similar-looking setup failures

Command not found. The shell cannot resolve the named tool. Check installation and `PATH`; the compiler has not yet read your source.

Source file not found. The tool is running but the input path is wrong. Check the current directory, filename, and quoting. Reinstalling the compiler will not create your missing project file.

Standard header not found. The compiler or target configuration cannot locate the required headers. Preserve the full command and verify the toolchain installation. Do not replace a standard header with a random downloaded copy.

Executable cannot be launched. The build may have failed, produced a different filename, targeted a different environment, or written to another directory. Confirm the build's exit status and actual output before changing permissions. On Linux, adding executable permission cannot turn an object file or a Windows executable into a native Linux program.

These distinctions mirror the Linux book's diagnostic method: establish which layer produced the observation, then choose the smallest test that can distinguish the remaining possibilities.

Practice and implementation lab

Create `src` and `build` directories. Place the program in `src/main.cpp`, compile to `build/fingerprint`, and run that exact path. Add a deliberately unused local variable and inspect warnings. Remove the defect rather than disabling warnings globally.

Write down how you would prove that the executable being run came from the file you just edited. Repeat the build from a fresh terminal so that hidden editor configuration cannot help.

Starting exercise: solution notes

An explicit source path, explicit output path, checked compiler exit status, and clean output directory establish a simple provenance chain. A timestamp alone is weaker evidence because an old artifact may survive a failed build. Run the executable only after the compiler succeeds; a shell script can use `&&` to express that dependency.

The unused-variable warning should disappear when the variable is removed or used meaningfully. Reproducing the build from a fresh terminal demonstrates that the command captures the necessary environment, though platform dependencies still need documentation.

Guided lab — Reproduce the environment from a clean directory

Create a second build directory without copying compiled artifacts into it. Compile the probe from the same source and confirm the language-mode check. Record the complete command. Then remove only that second build directory and recreate it. Your source should remain untouched because it lives outside the replaceable build tree.

Next, configure the companion with CMake and build only Chapter 2. Compare the compiler chosen by CMake with the compiler used manually. CMake stores configuration in its build tree; changing an environment variable after configuration does not necessarily replace that cached choice. Use a new build directory when deliberately comparing different compilers.

For the editor exercise, launch the same chapter target through the editor and through a terminal. Put a distinctive, temporary message in a copy of the probe so you can identify the executable being launched. Restore the copy after confirming the path. This is a better test than relying on an ambiguous tab title.

Acceptance criteria. The project builds without administrator privileges; the documented command works from the documented directory; generated artifacts are separated from source; the selected standard is observable; and a second build directory can be recreated without reconstructing undocumented editor state.

Review and solution reasoning

Why is an editor extension insufficient? It can provide language assistance without providing the compiler, headers, and linker needed to build. Verify the actual toolchain rather than inferring it from syntax highlighting.

Why might the editor and compiler disagree? They may use different standards, include paths, macros, or compiler assumptions. Identify which configuration each tool is using before changing source.

Does debug information make code correct? No. It improves inspection. Correctness still requires reasoning about requirements and executing meaningful tests.

Should every warning be removed with a cast? No. A cast changes conversion behavior or tells the compiler to accept a conversion. It does not automatically establish a valid range or a meaningful operation.

What should be included in a setup problem report? A minimal source file, exact command, first useful diagnostic, compiler version, platform, and working directory. Exclude unrelated screenshots, secret-bearing environment dumps, and guesses presented as facts.

Chapter takeaway. A development environment is ready when its build and diagnostic paths are reproducible. The editor is a convenience layer over that path, not a replacement for understanding it.

Technical references: [GCC](#) · [CLANG](#) · [MSVCINSTALL](#) · [MSVCSTD](#) · [CMAKE](#).

Chapter 3 — Writing, Compiling, and Running Your First C++ Programs

The first HarborWorks utility asks for a nonnegative number of devices and prints a small report. That sounds trivial, but it already has a contract: valid input produces a report; invalid input does not silently become a plausible count. A useful beginner program makes both paths visible.

You will read this example from the entry point downward. Follow values, not just lines. Ask what the stream contains before extraction, whether extraction succeeded, and which value is allowed to reach the calculation.

The first useful C++ program should do more than greet the world. It should establish a contract: which input is accepted, which output is produced, and how failure is reported. HarborWorks will begin with a tiny power estimate. The arithmetic is deliberately simple so that the relationship among input, validation, computation, and output remains visible.

Learning outcome. Read a complete console program from top to bottom, explain its entry point and stream operations, distinguish data output from diagnostics, and predict what happens when extraction fails. A few guards appear before the full treatment of conditionals in Chapter 6; here they are small, named safety boundaries rather than unexplained boilerplate.

3.1 Understanding the Structure of a C++ Program

3.1.1 Understanding `main()` as the Program Entry Point

In an ordinary hosted C++ program, `main` is the function through which your application begins its defined work after runtime initialization. The operating system and runtime perform setup before that call. Use a standard form such as `int main();` do not write `void main()` merely because a particular environment accepts it.

Returning zero conventionally reports success to the calling environment. A nonzero return reports a failure category chosen by the program. Reaching the end of `main` has the effect of returning zero. This special rule does not apply to every value-returning function. Keep application success separate from “the process did not crash”: a program can exit normally while correctly reporting invalid input.

For an ordinary hosted console application, `main` is the function through which your program’s main flow begins. The implementation performs startup work before calling it. That distinction will matter later when you study static initialization, but it need not complicate your first program: put the application’s visible sequence in `main` and keep that sequence easy to read.

The return type `int` describes a result communicated back to the execution environment. A zero status conventionally indicates success; this book uses small nonzero values to distinguish documented failure categories. These numbers are not printed automatically. A program can return 2 while producing no text unless it explicitly writes a message.

The braces delimit the function body. Indentation helps a person see that structure but does not replace the braces. C++ is case-sensitive: `main`, `Main`, and `MAIN` are different names. An ordinary hosted application needs the required `main` form, not a function whose name merely looks similar.

Reaching the end of `main` is a special case that has the effect of returning zero. Other value-returning functions do not generally receive that convenience. Writing an explicit `return 0;` is also acceptable. Do not generalize one entry-point rule to every function you will write in Chapter 8.

3.1.2 Understanding Statements, Expressions, and Semicolons

An expression computes a value or performs an operation, such as `count + 1` or `std::cout << count`. A statement describes an executable action or control structure. Many expression statements end with a semicolon. Braces group statements into a block and establish a scope for names declared inside.

A semicolon in the wrong place can change behavior while leaving valid syntax. `if (valid);` has an empty controlled statement; the following block no longer depends on the condition. Format braces consistently and read the structure that the compiler sees. Comments explain intent but do not change execution. The most helpful comments describe why a condition or boundary exists, not merely repeat the operator's name.

An expression computes a value, identifies an object, performs an effect, or combines those activities. A statement participates in the program's control flow. The line `count = 4;` is an expression statement: the assignment expression changes an object, and the semicolon completes the statement. A declaration such as `int count{};` also ends with a semicolon, but its purpose is to introduce an object.

This vocabulary helps you read diagnostics. A missing semicolon can cause the compiler to interpret the next line as an unexpected continuation. The line mentioned in the diagnostic may therefore be where the confusion became visible, not where it began. Inspect the immediately preceding declaration or statement before changing several lines at once.

Whitespace usually separates tokens, but it does not make two statements independent. Placing them on different lines without the required punctuation is not enough. Conversely, several statements can fit on one physical line, but that is usually harder to inspect. This book favors one meaningful action per line because it exposes state changes.

Comments explain intent to a reader; they do not execute. A comment saying “reject invalid input” cannot make an unchecked extraction safe. When reviewing a program, compare each comment with the following operation. The executable behavior comes from the language constructs, and a misleading comment can be worse than no comment because it encourages a false mental model.

Minimal Translation Unit with `main` and a Helper Function

Purpose. The example shows the major structural elements the compiler sees in a small C++ translation unit.

```
#include <iostream>
int square(int x) { return x * x; }
int main() {
    std::cout << square(5) << '\n';
    return 0;
}
```

How it works.

- The include declaration makes the stream interface available.
- `square` is a function definition at namespace scope.
- `main` is the designated program entry point and returns a process status.

Expected behavior/output. The program prints `25` and exits successfully.

Experiment. Move `square` below `main` without adding a prior declaration; explain why compilation then fails and fix it with a declaration.

3.2 Producing Console Output

3.2.1 Writing Text with `std::cout`

`std::cout` is the standard output stream. The insertion operator `<<` sends values to it using formatting appropriate to their types. A string literal such as `"Devices: "` contributes text; an integer contributes its formatted numeric value. Several insertions can be chained in one statement.

Use standard output for the program's normal result and `std::cerr` for diagnostics that should remain separate when output is redirected. A newline character, written `'\n'`, ends a line without necessarily forcing an immediate flush. `std::endl` also requests a flush; use it when that behavior is needed rather than as a ritual replacement for every newline. Output design is part of an application's interface.

`std::cout` names the standard output stream. The `std::` prefix qualifies a name in the standard namespace. It is not a filesystem path or a command to invoke. The insertion operator `<<` sends a formatted representation to the stream. In this context it is a stream operation; with other operand types the same operator spelling can mean something else.

A string literal is text enclosed in double quotes. Escape sequences represent characters that are inconvenient to type directly. The sequence `\n` represents a newline character. The two source characters backslash and `n` are therefore not normally printed as two characters. To print a literal backslash, escape it as `\\`.

```
std::cout << "HarborWorks\n";  
std::cout << "devices=" << 4 << '\n';
```

These are fragments that belong inside a complete program with the appropriate header. The first writes fixed text. The second combines text, a numeric value, and a newline. The operator chain is a useful reading pattern: each insertion contributes another part of the output sequence.

Do not confuse output with verification. Printing `4` after assigning `4` demonstrates one path through the program. A test must compare the observed behavior with an independently stated expectation. If the requirement actually called for a total of five devices, an attractive output line would still be wrong. Formatting makes information legible; it does not establish its truth.

3.2.2 Formatting Output with Stream Operations

Streams maintain formatting state. `std::fixed` selects fixed-point floating formatting, and `std::setprecision(2)` then requests two digits after the decimal point. In other formatting modes, precision has a different interpretation. The state persists on that stream until changed, so a helper that changes formatting can affect later output.

Distinguish displayed precision from stored precision. Printing a value with two decimal places does not round the underlying variable for all subsequent calculations. For reports, choose units and formatting deliberately: `12.50 V` communicates more than an unlabeled number. Later chapters show alternatives for parsing and formatting when localization, throughput, or stable machine-readable output is required.

Formatting is part of an interface. A person reading a report benefits from labels and units; a program consuming that report may require an exact delimiter and field order. Decide which audience the output serves before adding decorative text. The chapter program omits an interactive prompt so its result can be tested easily with redirected input.

`std::fixed` and `std::setprecision(2)` are used together to show two digits after the decimal point for the power estimate. The value being stored remains a floating-point number; display formatting does not change the underlying calculation into exact decimal arithmetic. Chapter 4 explains that difference in detail.

Stream formatting can persist. After setting a stream to fixed notation, subsequent floating-point insertions use that state until it is changed. A helper function that changes shared stream formatting may therefore affect output after the helper returns. In a small program, setting formatting once near the report is easy to reason about. In a reusable library, formatting ownership deserves more care.

Prefer `\n` when you only need a newline. `std::endl` also requests a flush, which is a separate operation. A prompt that must be visible before waiting for input has a different requirement from writing thousands of report lines. Do not sprinkle explicit flushing everywhere in the belief that it improves correctness. State the visibility requirement, then choose the operation that serves it.

Formatting Multiple Values with `std::cout`

Purpose. Console output is more than printing literals: stream manipulators and chained insertions control how typed values are rendered.

```
#include <iomanip>
#include <iostream>
int main() {
    double price = 12.5;
    int quantity = 3;
    std::cout << "quantity=" << quantity
               << " total=$" << std::fixed << std::setprecision(2)
               << price * quantity << '\n';
}
```

How it works.

- Insertion operators send each value to the output stream in order.
- `std::fixed` and `std::setprecision(2)` configure floating-point presentation.
- ```` terminates the line without forcing an unnecessary flush.

Expected behavior/output. The line is `quantity=3 total=$37.50`.

Experiment. Change precision to 3 and replace ```` with `std::endl`; compare formatting and understand the flush behavior.

3.3 Receiving User Input

3.3.1 Reading Values with `std::cin`

`std::cin >> count` attempts formatted extraction from standard input. For an integer, it skips leading whitespace and interprets suitable characters as a number. The expression's stream state can be tested directly: `if (!(std::cin >> count))` detects extraction failure. Initialize the destination anyway so that its initial state is unambiguous.

Formatted extraction consumes the part it recognizes. Input such as `12abc` can successfully produce the integer 12 while leaving `abc` for later. Therefore successful extraction is not necessarily complete validation of a whole line. This chapter accepts a numeric token; later text-parsing examples enforce complete consumption where that is the requirement.

Formatted extraction with `>>` attempts to interpret the next part of a stream according to the destination type. Extracting into an integer is not the same as reading an arbitrary line of text. Leading whitespace is ordinarily skipped, and a numeric prefix may be accepted even when more characters follow.

That last point is easy to miss. Given `4junk`, extracting one integer can obtain 4 and leave `junk` in the stream. The Chapter 3 program's narrow contract is to read one integer token and validate its range; it is not yet a strict whole-line parser. Chapter 11 develops complete-consumption checks, and the capstone

applies them to commands. This boundary is explicit so the small example does not quietly promise more validation than it implements.

Input is not limited to the keyboard. A shell can redirect a file or a pipeline into standard input. The program still uses `std::cin`; the execution environment supplies the bytes. That separation makes console programs convenient to test. A fixture can feed a known input without requiring a person to type it at the right moment.

Always connect the extraction result to the next action. Initializing `count` to zero avoids an uninitialized value, but it does not make a failed extraction semantically equivalent to the user entering zero. The program must recognize the difference instead of silently continuing with a default.

3.3.2 Recognizing and Handling Invalid Input

Treat syntactic validity and domain validity as separate checks. Extraction can fail because the text is not a representable integer. Extraction can also succeed with a value such as `-4` that is not an acceptable device count. Check both before using the value.

For this first program, report an error and exit rather than building a complicated retry loop. An interactive loop must clear the stream's failure state and discard unwanted input before retrying; otherwise it can repeat forever without consuming anything. A robust interface defines whether it accepts one token, one line, or a complete file. That definition determines what counts as extra or malformed input.

Validation has at least two layers here. Syntactic validation asks whether the stream can extract an integer. Domain validation asks whether that integer lies between zero and one thousand. The text `hello` fails the first layer. The integer `1001` passes extraction but fails the domain rule. Those are different errors and deserve different explanations.

The guard `if (!(std::cin >> count))` tests whether extraction succeeded. The parentheses make the operation being tested clear. The logical negation `!` turns a failed stream state into the condition for the error branch. Chapter 6 develops the general conditional structure; here, read it as “stop with a diagnostic when the input cannot be read.”

When a stream enters a failure state, later extractions generally will not solve the problem by themselves. Recovery may require clearing the state and discarding or replacing the offending input. Clearing alone does not consume the characters that caused failure. Repeatedly clearing and retrying the same bytes can create a loop that never makes progress.

This first program exits on invalid input instead of attempting interactive recovery. That is a legitimate policy, especially for a small command-line utility. A later interactive application can choose to report the bad line and continue. The important design choice is to make failure behavior explicit rather than assuming every user will type the example's happy-path input.

Reading and Validating a Numeric Input

Purpose. Input handling must distinguish successful parsing from valid domain values; this example demonstrates both checks.

```
#include <iostream>
int main() {
    int age{};
    std::cout << "Age: ";
    if (!(std::cin >> age) || age < 0) {
        std::cerr << "invalid age\n";
        return 1;
    }
    std::cout << "next year=" << age + 1 << '\n';
}
```

How it works.

- `age{}` value-initializes the variable before extraction.
- Stream extraction returns a state that can be tested immediately.
- The range check rejects negative values even when lexical conversion succeeds.

Expected behavior/output. Input `20` prints `next year=21`; nonnumeric or negative input prints `invalid age` and exits with status 1.

Experiment. Try `abc`, `-1`, and `0`. Explain which check rejects each case.

Worked example — Validate a count before printing a report

Requirement and simplest approach

The lab models each device as consuming 2.5 watts for a teaching calculation. This is an invented coefficient, not a measurement of real equipment. The user supplies a count from zero through one thousand. The program prints the count and estimated power, or it reports why it cannot perform the calculation.

The simplest implementation would extract a count, multiply it, and print the result. That shortcut fails to distinguish unreadable input from zero and can accept quantities outside the stated domain. The selected design puts validation before computation. It does not allocate files, open network ports, or modify inventory.

Trace the data, not just the lines

The input bytes first pass through integer extraction. A successful integer then passes through a range check. Only accepted values reach multiplication. Output formatting is applied after the numerical result

exists. This order is important: checking the printed result is not a substitute for checking the input that produced it.

For input `4`, predict `devices=4` and `watts=10.00`. For input `0`, predict a valid zero estimate. For input `hello`, predict an extraction diagnostic and no normal report. For input `1001`, predict a range diagnostic. These expectations define distinct paths before the program runs.

The program writes diagnostics to `std::cerr`, the standard error stream. Keeping diagnostics separate from normal data makes redirection and automation easier. A test can inspect both streams and the exit status. Their separation is a design decision, not merely a stylistic preference about where text appears on your terminal.

Trace the implementation

The program initializes `count`, checks extraction, validates the accepted range, and only then calculates power using a fictional 2.5-watt-per-device estimate. The estimate is teaching data, not a real hardware rating. Multiplication occurs after validation, so an invalid count cannot become an apparently valid report.

There are three outcomes: zero for a completed report, two for an unreadable integer, and three for a value outside the domain. The fixture tests exercise each category. Run the program with `4` to obtain `devices=4` and `watts=10.00`. Run it with `-1` or `hello` to observe distinct diagnostics and return codes.

Notice what the program does not promise. It does not validate that the entire input line contains only the token. That limitation is explicit rather than hidden. A later parser will reject trailing characters when the interface requires whole-field validation.

Complete worked source

source/ch03/main.cpp

```
#include <iomanip>
#include <iostream>

int main() {
    int count{};
    if (!(std::cin >> count)) {
        std::cerr << "error: expected an integer\n";
        return 2;
    }
    if (count < 0 || count > 1000) {
        std::cerr << "error: count must be in [0,1000]\n";
        return 3;
    }
    const double watts = count * 2.5;
    std::cout << "devices=" << count << '\n';
    std::cout << std::fixed << std::setprecision(2)
              << "watts=" << watts << '\n';
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

```
python3 tools/run_chapter.py 3
```

The executable target is `b01_c03`. Read `source/ch03/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Run the same program without typing interactively

On a Bash-like command line after building the chapter target:

```
printf '4\n' | ./build/bin/b01_c03  
printf 'hello\n' | ./build/bin/b01_c03
```

The shell supplies the input; the executable contains the same C++ logic in both runs. Check the exit status immediately after the command whose status you want to inspect. Running another command replaces the shell's most recent status. The automated fixtures avoid that ambiguity by capturing the executable's return code directly.

Separate the output streams to make their roles visible:

```
printf '1001\n' | ./build/bin/b01_c03 \  
> build/report.txt 2> build/diagnostic.txt
```

The normal report should be absent for the rejected count, while the diagnostic file explains the range error. This test is about interface behavior, not merely about whether the program stays alive.

Failure clinic — The default value that hid a bad read

A learner initializes `count` to zero, removes the extraction guard, and finds that typing text produces a zero-watt report. The initializer did its job: the object had a known initial value. The program still failed its requirement because no valid count was supplied. Initialization and validation solve different problems.

Another learner writes a friendly prompt to standard output before every result. Interactive use improves, but a fixture that expects a machine-readable first line now fails. Neither interface is inherently wrong. The repair is to choose and document one contract, or deliberately separate interactive and batch modes. Randomly changing tests to match each new output hides the absence of a stable interface.

Practice and implementation lab

Predict the results for `0`, `1000`, `1001`, `hello`, and `12abc`. Run each case and explain the difference between token parsing and whole-line validation. Change the permitted maximum in one place and update the boundary test.

Add a label that states the power unit. Do not change the numeric calculation simply to make the formatting look correct.

Starting exercise: solution notes

Zero and 1000 are valid boundaries; 1001 is outside the domain; `hello` fails extraction. `12abc` produces a valid first integer token and therefore passes this program's stated input contract. Rejecting that text requires checking the remaining input, not merely adding another range comparison.

The unit is watts, and output formatting belongs at the reporting boundary. A boundary test should cover the largest accepted value and the smallest rejected value so that a changed condition cannot silently move the interface's limit.

Guided lab — Exercise every input category

Build the original example and run the supplied fixtures. Then make a test table before editing code. Include zero, one, the maximum, a negative integer, one above the maximum, text, empty input, and an integer followed by extra text. For each case, record whether extraction should succeed, whether the domain rule should accept the value, and whether the whole-line interpretation is intentionally outside this chapter's contract.

Change the teaching coefficient in a copy of the program from 2.5 to 3.0. Update the expected results for accepted inputs, not the validation rules. This isolates computation from parsing. Next, change the maximum count and update only the relevant boundary tests. A change should have a reasoned impact rather than forcing every expectation to be rewritten.

For the output exercise, add a label indicating the unit while preserving two decimal places. Decide whether this is a new human-readable interface or a change to an existing machine-readable one. Explain which fixture assertions must change and why. The exercise is complete only when you can distinguish a purposeful interface revision from a regression.

Acceptance criteria. Invalid extraction never produces a normal estimate; out-of-range integers are rejected; zero is accepted; diagnostics and results use their intended streams; and your notes explicitly acknowledge that reading one integer is not the same as validating an entire line.

Review and solution reasoning

Why initialize and still check extraction? Initialization establishes an object state. Extraction checking establishes whether a new input value was obtained. Neither proves the other.

Why can `4junk` be different from `hello`? The first begins with a parseable integer prefix; the second does not. Strict parsing must additionally check what remains.

Does `setprecision(2)` prove exact decimal arithmetic? No. It controls output according to the stream's format state. It does not change the representation used by the calculation.

Is returning a nonzero code enough for a person? Usually not. A concise diagnostic explains the failure, while the code gives automation a structured outcome. They serve complementary audiences.

What is the first debugging step when displayed output is old? Confirm that the new build succeeded and that you launched its exact executable. Do not diagnose stale output as an arithmetic defect.

Chapter takeaway. A console program is an interface between bytes and decisions. Validate before computing, separate data from diagnostics, and test the failure paths as deliberately as the successful one.

Technical references: [LANG](#) · [IO](#).

Chapter 4 — Representing Information with Variables and Fundamental Types

A number becomes useful only when its representation matches its meaning. A device count, a voltage estimate, a character code, and an enabled flag are different kinds of information. Treating them all as interchangeable integers makes accidental operations easy and errors difficult to recognize.

This chapter separates initialization, representation, and constancy. The compiler can help with each, but it cannot infer the physical units or acceptable range of an arbitrary variable name. Those remain part of the program's contract.

A variable is more than a box with a name. It has a type, a value, a lifetime, and rules governing the operations you can perform on it. Many beginner defects arise when those properties are collapsed into the vague idea of “a number.” HarborWorks needs counts, measured quantities, flags, and labels; those are different kinds of information even before classes are introduced.

Learning outcome. Choose fundamental types from a stated domain, initialize every ordinary local object before reading it, recognize narrowing and overflow risks, and distinguish a stored value from its printed representation. You will use small experiments to expose assumptions rather than depending on the type sizes of one computer.

4.1 Declaring and Initializing Variables

4.1.1 Comparing Copy, Direct, and List Initialization

Common initialization forms include `int count = 3;`, `int count(3);`, and `int count{3};`. For simple examples they can produce the same value, but the forms have different rules for class types and conversions. List initialization is particularly useful because it rejects many narrowing conversions, such as initializing an `int` from a nonconstant floating value without an explicit conversion.

Do not confuse initialization with later assignment. `int count{3};` starts an object's lifetime with a value; `count = 4;` changes an existing object. For library containers, braces and parentheses can mean different things: a vector initialized with `{3, 7}` contains two elements, whereas `(3, 7)` requests three copies of 7. Read the selected constructor's contract.

Initialization creates an object's initial state. Assignment changes an object that already exists. The difference is easy to overlook because copy initialization contains an equals sign:

```
int devices = 4;    // Initialization of a new object.
devices = 5;       // Assignment to an existing object.
int racks(2);      // Direct initialization.
int spares{3};     // List initialization.
```

For these simple integers, the resulting values are unsurprising. The choice becomes more significant when conversions or class constructors are involved. List initialization rejects many narrowing conversions that another form might permit. For example, trying to initialize an integer with a nonintegral floating-point value using braces is a useful compile-failure experiment, not a valid calculation to put in the normal companion build.

Braces are a good beginner default, but they are not a claim that every initialization syntax is interchangeable. Later, a vector initialized with parentheses can mean “this many elements,” while braces can mean “these element values.” Learn the object type and selected operation, not merely a punctuation habit.

Declare one object per line when the names or initial values deserve explanation. `int count{}`, `capacity{10}`; is legal, but separate declarations often make later edits and diagnostics clearer. Choose names that identify meaning and, when useful, units. An initialization such as `double duration_seconds{0.25}`; tells a reviewer more than `double d{0.25}`; without requiring a comment for every use.

4.1.2 Preventing Uninitialized Data

A local fundamental object declared as `int count`; is not automatically given a useful value. Reading an uninitialized value is not a technique for obtaining randomness, and its behavior must not be relied upon. Prefer immediate initialization such as `int count{}`;, or declare the object at the point where a meaningful value is available.

Initialization to zero is not a substitute for validating input. A failed read should not silently leave a zero that looks like a valid measurement. Keep the success indicator alongside the value and ensure that invalid paths cannot use it. Shorter scopes reduce the distance between establishing a value and consuming it, making these relationships easier to inspect.

An ordinary local declaration such as `int count`; does not provide the beginner’s intended initial value. Do not read that object before a value has been established. A successful run that happens to print zero is not evidence that the language supplied a safe default. Optimizations and small source changes can expose the defect differently.

For a numeric object that genuinely begins at zero, `int count{}`; expresses that state directly. For an object that must begin from a known input, initialize it at the point where that input is available. Avoid declaring a group of uninitialized variables at the top of a function and hoping every later branch assigns them.

A useful review question is: **What establishes the value on every path reaching this read?** Trace each branch, not only the branch used by your sample input. If a failure path skips an assignment and then continues into the calculation, a default value may prevent an uninitialized read while still producing a false result. The Chapter 3 distinction between initialization and validation still applies.

Prefer constructing the valid state over creating an invalid state and repairing it later. This principle will become a class invariant in Chapters 17 and 18, but it starts with a single integer. A small scope and an explicit initial value reduce the number of possible histories you must consider when debugging that integer.

Initialization Forms and Type Inference

Purpose. The code demonstrates declaration, initialization, and compile-time type deduction in one small expression chain.

```
#include <iostream>
int main() {
    int count{3};
    double rate = 1.5;
    auto total = count * rate;
    std::cout << total << '\n';
}
```

How it works.

- Brace initialization constructs `count` with an explicit integer value.
- Copy initialization assigns the `double` literal to `rate`.
- `auto` deduces `total` from the expression type; it does not make the variable dynamically typed.

Expected behavior/output. The program prints `4.5`.

Experiment. Change `rate` to `int rate{1};` and predict the deduced type and output of `total`.

4.2 Working with Numeric Types

4.2.1 Understanding Signed and Unsigned Integers

Signed integers represent positive and negative values within a finite range. Unsigned integers represent nonnegative values and use modulo arithmetic for overflow. Signed overflow is not a portable wraparound operation. Inspect ranges with `std::numeric_limits<T>` rather than assuming that every implementation gives a type the same width.

Mixed signed and unsigned expressions can be surprising because conversions may occur before comparison. A negative integer can become a large unsigned value. Container sizes commonly use an unsigned size type, but that does not mean every domain quantity should be unsigned. Check ranges before conversion and use an explicit, documented representation at boundaries. Exact-width types such as `std::uint32_t` are useful when the implementation provides them and the interface requires that width.

Separate the mathematical domain from the representation. A device count may be nonnegative, but that fact does not automatically make every unsigned operation convenient. Inputs can contain negative text, differences between counts can be negative, and comparisons between signed and unsigned values can perform conversions that surprise a beginner.

For the bounded inventory exercises, a signed `int` is straightforward because the chosen maximum is small and negative input can be rejected before it becomes a valid quantity. For container sizes, library interfaces commonly use an unsigned size type. At that boundary, validate a signed candidate before

converting it to an index. Do not use a cast as a substitute for checking that the value is nonnegative and below the container's size.

Unsigned arithmetic has modular behavior when a result exceeds its range. That can be useful in a deliberate bit-level algorithm, but an inventory count wrapping to zero is still a domain failure. Signed overflow is not a portable wraparound mechanism. Never test for signed overflow by first performing the overflowing addition and then inspecting the result.

Ask the implementation about limits with `std::numeric_limits<T>`. Do not assume `long` has the same width on every supported platform. A portable requirement should describe the necessary range; the implementation choice must then satisfy it. Chapter 5 shows how to arrange a calculation so that an avoidable overflowing intermediate is never formed.

4.2.2 Understanding Floating-Point Representation

Floating-point types represent a finite set of values with a sign, significand, and exponent in the implementation's format. Many decimal fractions, including 0.1 on typical binary systems, do not have exact representations. Arithmetic therefore includes rounding, and changing operation order can change the final rounded result.

Use floating types for approximate real-valued quantities with an error budget. Do not compare independently computed measurements for exact equality merely because their printed decimal strings look the same. A tolerance must reflect scale and purpose, not an arbitrary universal epsilon. Conversely, exact equality is appropriate for some deliberately exact values and special sentinels. The important question is how the values were produced and what the comparison means.

A floating-point type represents a finite set of values. Many decimal fractions have no exact representation in the common binary formats used by ordinary computers. A calculation can therefore contain rounding even when the source looks like simple school arithmetic. Printing fewer digits can hide that rounding without eliminating it.

The expression `0.1 + 0.2` is a familiar experiment because, on common binary floating-point implementations, printing sufficient precision reveals a value slightly different from the decimal spelling `0.3`. The right lesson is not “never use floating point.” The lesson is to match the representation and comparison policy to the problem.

For measured power, a tolerance can express an acceptable numerical difference. A small teaching check uses an absolute tolerance because its values are near one another and near a known scale. A general scientific application may need both absolute and relative tolerances, a policy for NaNs and infinities, and an error analysis. This chapter does not silently turn a single constant such as `1e-12` into a universal scientific standard.

For exact counts, use an integer. For exact money calculations, a binary floating-point number is not automatically the right representation; the domain may require scaled integers or a decimal design. For sensor values, finite precision may be entirely appropriate. The important decision is which errors your application can tolerate and how those errors are tested.

Inspecting Numeric Limits and Conversion Behavior

Purpose. The section concerns both representation limits and arithmetic behavior, not just choosing names like ``int`` and ``double``.

```
#include <iostream>
#include <limits>
int main() {
    std::cout << "int max=" << std::numeric_limits<int>::max() << '\n';
    int whole = 7;
    int divisor = 2;
    double exact = static_cast<double>(whole) / divisor;
    std::cout << "int division=" << whole / divisor << " floating=" << exact <<
    '\n';
}
```

How it works.

- ``numeric_limits`` reports properties of the implementation's numeric type.
- Integer division discards the fractional part.
- The explicit cast changes one operand so ordinary arithmetic conversion produces a floating-point result.

Expected behavior/output. The second line contains ``int division=3 floating=3.5``.

Experiment. Replace ``int`` with ``std::int32_t`` where available and compare the maximum value.

4.3 Working with Characters, Booleans, and Constants

4.3.1 Representing Character and Logical Data

`char` represents a character-sized unit, not necessarily an entire human-readable Unicode character. Text encodings can use multiple bytes for one displayed character. A string's byte count and a user's notion of character count may therefore differ. Avoid generalizing simple ASCII teaching examples into a complete international-text solution.

`bool` expresses a logical value, `true` or `false`. Use it for a binary condition such as `enabled`, rather than inventing numeric conventions that every caller must remember. When more than two states exist—unknown, ready, failed—an enumeration is usually clearer. Type choice should expose the domain's distinctions instead of making them implicit in comments.

A `char` stores a character-sized code unit, not an abstract human-language character with every possible writing-system property. A narrow string is a sequence of such code units. For simple ASCII-like teaching labels, indexing by code unit is often enough. For general Unicode text, one visible character may involve multiple encoded units, so character counting and byte counting can diverge.

`sizeof(char)` is one by definition, but a C++ byte is defined through the implementation's character storage unit. Do not infer every external file format from that fact alone. This book's source and fixtures

use ordinary UTF-8 text where stated, and its beginner parsing examples deliberately keep their accepted command vocabulary simple.

A Boolean expresses a logical state. Prefer `bool accepted{false};` to using undocumented numeric codes for true and false. The expression `count <= maximum` already produces a Boolean result; comparing that result with `true` usually adds noise rather than meaning.

When printed without special formatting, a Boolean is commonly shown numerically.

`std::boolalpha` requests textual names such as `true` and `false` for stream formatting. That affects display, not the underlying logical operations. A flag named `valid` is also not proof that validation occurred: inspect which condition established it. Good names expose intent, while tests and reasoning establish whether the intent is implemented.

4.3.2 Using `const` and `constexpr`

`const` prevents modification through the declared object or access path after initialization. A `const` value can still be initialized from runtime input. `constexpr` requires a suitable constant-expression initializer for a variable and makes that variable `const`; it is useful for sizes, limits, and computations that must be available during translation.

Neither keyword means “stored in a special fast memory area.” Their primary role is expressing constraints on the program. Use `constexpr int max_devices{1000};` for a fixed policy limit, and `const int count{read_count()};` when a runtime value should not change. Later volumes explain constant evaluation and optimization separately, preventing a semantic guarantee from being confused with a performance guess.

Use `const` when an object should not be modified through its name after initialization. A runtime input can initialize a `const` object; constness does not require the value to be known during compilation. For example, a validated input count can be used to initialize a fixed per-report limit that remains unchanged while the report is generated.

Use `constexpr` for an object whose initializer must satisfy constant-expression requirements. In the chapter example, a small maximum count is a natural named constant. It documents the domain and gives one place to change the value. The compiler can also use such a constant where the language requires a compile-time value.

Do not treat `constexpr` as a general command to “make this faster.” Its semantic requirements matter more than a hoped-for optimization. Likewise, `const` is not a complete security boundary. Other parts of a program may still access the same underlying state through different permitted paths, and pointer constness has several forms discussed in Chapter 14.

A useful beginner sequence is to make a value `const` unless the local algorithm actually changes it. Then keep each necessary mutation close to the code that justifies it. In a ten-line calculation, this makes the changing state easy to identify. In a larger program, the same habit reduces accidental coupling and helps readers see which values are inputs, intermediate results, and mutable accumulators.

Character Classification, Boolean State, and `constexpr`

Purpose. Characters, booleans, and constants each encode different intent: textual code unit, logical state, and nonmodifiable compile-time value.

```
#include <cctype>
#include <iostream>
int main() {
    constexpr char delimiter = ':';
    char ch = 'A';
    bool uppercase = std::isupper(static_cast<unsigned char>(ch)) != 0;
    std::cout << ch << delimiter << std::boolalpha << uppercase << '\n';
}
```

How it works.

- `constexpr` makes the delimiter a compile-time constant.
- The character is passed safely to `std::isupper` through `unsigned char`.
- `boolalpha` prints `true`/`false` instead of `1`/`0`.

Expected behavior/output. The program prints `A:true`.

Experiment. Change `ch` to `a` and then to `7`; predict the boolean result before running.

Worked example — Observe representation without assuming it

Requirement and simplest approach

HarborWorks wants to distinguish three claims: the count lies within a teaching limit, unsigned arithmetic follows its defined wrap behavior, and a floating-point calculation is close to the expected mathematical value. The simplest approach is to print all the values and decide visually that they look reasonable. That can conceal both a domain error and a representation assumption.

The selected design asks each question explicitly. A Boolean expresses the bounded-count decision. A limits query supplies the maximum unsigned value without hard-coding a bit width. A floating-point difference is checked against a stated small tolerance for this particular example. None of these tests grants permission to use wraparound for inventory or to compare arbitrary measurements with the same tolerance.

Trace value, type, and interpretation

For each expression in the worked source, write three columns in your notebook: the type, the mathematical intention, and the expected language behavior. The unsigned maximum plus one has a defined modular result, while the intention in a stock-counting program would normally be “reject a value outside the allowed range.” Those are not the same statement.

For the floating-point sum, first print the default stream representation and then print more significant digits. You have changed what is visible, not recalculated the value more accurately. That distinction will become important when you compare log output with internal calculations in later volumes.

The program's final checks are intentionally narrow. It asks whether the bounded example and representation experiment behave as designed, not whether every number on the machine is exact. A meaningful test should state its scale and interpretation as clearly as its expected result.

Trace the implementation

The unsigned expression demonstrates defined modulo arithmetic without performing signed overflow. The floating expression is printed with enough precision to expose rounding on common implementations. The check uses a tolerance appropriate to this tiny teaching calculation; it is not presented as a universal numerical policy.

`within_limit` is a Boolean result derived from two integers. `count` is `const` because this example never needs to change it; `max_devices` is `constexpr` because the limit is a compile-time policy value. These choices communicate different facts even though both variables are read-only here.

Do not convert this demonstration into a test that demands one exact printed floating-point string or one specific integer width. A portable test checks the promised behavior. Platform-specific representation requirements can be tested separately when a file format, protocol, or hardware interface actually imposes them.

Complete worked source

source/ch04/main.cpp

```
#include <cmath>
#include <iomanip>
#include <iostream>
#include <limits>

int main() {
    constexpr int max_devices{1000};
    const int count{12};
    const bool within_limit = count <= max_devices;
    const unsigned int wrapped =
        std::numeric_limits<unsigned int>::max() + 1U;
    const double sum = 0.1 + 0.2;
    std::cout << std::boolalpha << within_limit << '\n';
    std::cout << "unsigned wrap=" << wrapped << '\n';
    std::cout << std::setprecision(17) << "sum=" << sum << '\n';
    if (!within_limit || wrapped != 0U) return 1;
    if (std::abs(sum - 0.3) > 1e-12) return 2;
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

The executable target is `b01_c04`. Read `source/ch04/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — A cast that changed the symptom

Suppose a program reads a signed candidate index of `-1` and immediately casts it to an unsigned size type. The conversion does not turn it into a valid index. It produces a representable unsigned value according to the conversion rules, and that value is likely far outside the small container's range. A warning may disappear while the original validation defect remains.

The correct reasoning order is to reject negative candidates, compare the remaining value with the permitted bound using an appropriate representation, and only then perform any required conversion. Do not infer that a type named “size” guarantees a valid size. Types constrain representation; domain checks constrain meaning.

Failure clinic — The invisible distinction between constants

A learner writes `const int limit = read_limit();` and expects to use it wherever a compile-time bound is required. The object is not modifiable after initialization, but its value was obtained at runtime. Another learner declares a `constexpr` value from an expression that cannot be evaluated as required at translation time. The compiler rejects the declaration because the contract of `constexpr` is stronger.

These are not contradictory compiler behaviors. The two keywords express different promises. Explain the promise in ordinary language before choosing the spelling: “do not change this initialized object” versus “this initializer must be suitable for constant evaluation.”

A useful counterexample

Print a small exact integer-valued floating-point result and observe that equality can succeed. This prevents the opposite overgeneralization that floating-point equality is always forbidden. The engineering question is whether exact equality matches the operation and data model. Approximate measurements and rounded calculations need a different policy from a deliberately exact sentinel or a value whose representation has been established.

Practice and implementation lab

Attempt `int truncated{2.75};` and record the compilation failure. Then write an explicit conversion and explain what information is lost. Find the minimum and maximum values for `int` on your toolchain using `numeric_limits`.

Explain why an unsigned variable is not a complete validation strategy for a user's device count. Design a test that distinguishes a failed input read from a legitimate count of zero.

Starting exercise: solution notes

List initialization rejects the narrowing conversion. An explicit cast states the conversion intentionally but does not make it lossless; fractional information is discarded for a representable finite value. Validate the range before converting a general floating value to an integer.

An unsigned destination does not establish the user's intended domain or reject all malformed text. The parser's success state and a subsequent range check must remain explicit. A test should supply invalid text and require a nonzero result, while supplying 0 should take the valid-data path.

Guided lab — Design a type sheet for the inventory project

List the information the final application will hold: record identifier, equipment name, quantity, a validity result, and a total quantity. For each item, state the permitted values before choosing a type. A record identifier is positive; a quantity is bounded; a name can contain spaces; a result is logical; a total depends on both maximum quantity and maximum record count.

Compute the largest possible total from the stated limits. Select an accumulator type that can represent it without assuming an unbounded mathematical integer. Keep the calculation in your design notes so a future change to either limit forces the total's range to be reconsidered.

Create three small experiments outside the normal build: a valid brace initialization, an intentionally rejected narrowing initialization, and an attempted assignment to a const object. Record whether each is expected to compile. Restore the valid source before running the companion suite. Compile-failure exercises should remain separate so their intentional errors are not confused with a broken distribution.

For floating point, compare output at several precisions and explain what changed. Do not claim that displaying seventeen digits created seventeen correct decimal digits in the original measurement. The information content of the input, the representation, the calculation, and the display are separate limits.

Acceptance criteria. Every field has a documented domain; every local object is initialized before use; the total's range is justified; invalid conversions are rejected before they become indices or counts; and numerical comparisons have a stated interpretation.

Review and solution reasoning

Why can zero initialization still produce a wrong report? A known default does not prove that a required input or computation succeeded. It prevents one class of defect, not every semantic failure.

Why is unsigned wrap not automatically an error? It is defined arithmetic behavior. Whether it violates a requirement depends on the domain. Stock quantities normally need rejection rather than wraparound.

Why inspect limits rather than assume widths? The program is built for an implementation. Its type ranges are properties to verify against requirements, not facts to infer from another machine's output.

Why name a maximum? A named constant states a rule once, exposes its purpose, and reduces inconsistent copies. Changing it still requires reviewing dependent calculations and tests.

Chapter takeaway. Choose representations from the information they must preserve. Initialization establishes state; validation establishes meaning; formatting merely controls how that state is shown.

Technical references: [LANG](#) · [NUMERIC](#).

Chapter 5 — Building Expressions with Operators

A compact expression can hide several decisions: which type is used, whether division truncates, which operation happens first, and whether an operand changes state. HarborWorks needs to divide a count into complete groups and a remainder. That requirement is easier to implement correctly when the arithmetic model is stated before the expression is compressed.

Prefer expressions that reveal their inputs and leave their surroundings unchanged. A few extra named intermediate values can be cheaper to maintain than one clever line whose evaluation rules nobody remembers.

A calculation can look mathematically familiar and still behave differently in a programming language. Its operands have types, its intermediate results have finite ranges, and its operators follow language rules rather than the layout you intended on paper. HarborWorks’ packing calculation is a small enough problem to inspect every one of those details.

Learning outcome. Translate a word problem into typed expressions, explain integer quotient and remainder, build safe Boolean guards, and distinguish operator grouping from execution order. The goal is not to memorize a precedence chart; it is to write expressions whose meaning survives review and boundary tests.

5.1 Performing Arithmetic Operations

5.1.1 Using Addition, Subtraction, Multiplication, and Division

The arithmetic operators act on values after the language’s conversion rules have selected suitable operand types. `2 + 3` performs integer arithmetic; `2.0 + 3` involves floating-point arithmetic. Assigning an integer-division result to a double does not retroactively make the division floating point. Write `static_cast<double>(total) / count` when that is the intended operation and validate the divisor first.

Finite representations impose limits. Check multiplication and addition bounds when inputs may be large. Do not rely on signed overflow wrapping, and do not assume that choosing a wider type eliminates every possible overflow. A program’s accepted input range can often make the proof simple: two validated small quantities may safely fit a chosen result type.

Begin arithmetic with units and domains. If a quantity means “devices” and another means “devices per crate,” dividing the first by the second produces a crate count. Adding those two original quantities would not express a meaningful requirement even though C++ would happily add two integers. Fundamental types do not encode every domain distinction for you.

Write a small worked example before writing the expression. Twenty-three devices with five spaces per crate require four full crates and one partial crate. That sentence distinguishes the quotient, remainder,

and final count. A variable named `result` hides the distinction; `full_groups`, `remaining`, and `groups` make it inspectable.

Intermediate types matter. Assigning an expression to `long long` does not necessarily make an earlier multiplication use that wider type. The operands determine how the multiplication is performed. If the inputs can exceed the smaller type's safe product range, convert an operand appropriately before multiplying or arrange a checked operation. Do not perform an overflowing signed calculation and hope a larger destination repairs it afterward.

In this chapter's deliberately bounded examples, small positive values keep the arithmetic simple. Still, state those bounds. A formula that is safe for at most one thousand devices may need revision when reused for billions of events. The explanation of the range is part of the algorithm, not a decorative comment that can be discarded when the code is copied.

5.1.2 Understanding Integer Division and Modulus

Integer division truncates toward zero. The remainder operator `%` gives the corresponding remainder for integer operands. For nonnegative quantities, `items / capacity` counts complete groups and `items % capacity` counts leftover items. Require a positive capacity before either operation.

Negative operands need extra care because remainder is not a universally nonnegative mathematical modulo. Also avoid the signed minimum divided by -1 when the positive result is not representable. These are representation constraints, not formatting problems. In the book's grouping example, restricting the domain to nonnegative items and positive capacity keeps the arithmetic aligned with its physical meaning and avoids irrelevant negative-group interpretations.

Integer division produces an integer quotient. With nonnegative counts, `23 / 5` gives four and `23 % 5` gives three. The quotient counts complete groups, while the remainder counts elements left over. Neither operation on its own answers "How many crates are needed to hold everything?" because the remainder may require another crate.

A direct translation is:

```
const int full = items / capacity;
const int remainder = items % capacity;
const int needed = full + (remainder != 0 ? 1 : 0);
```

The conditional operator selects one of two values. Here it contributes one extra crate exactly when a remainder exists. Chapter 6 studies decision-making more broadly; this expression is a compact form of a single binary choice.

A commonly seen alternative is `(items + capacity - 1) / capacity`. It can work for positive values, but the addition can overflow before the division when the input range grows. The quotient-and-remainder formulation avoids that particular intermediate. It still requires a positive capacity and a domain that matches the intended interpretation.

Division by zero is not an error you should discover by running it. Validate the divisor first. Negative integer division also deserves deliberate reasoning: its truncation and remainder behavior do not automatically implement every mathematical rounding rule. This packing problem rejects negative counts, so it does not need to invent a meaning for a negative number of crates.

Integer Division, Remainder, and Floating-Point Arithmetic

Purpose. The same mathematical operands can produce different useful results depending on operator and operand types.

```
#include <iostream>
int main() {
    int items = 17;
    int boxes = 5;
    std::cout << "full=" << items / boxes << " leftover=" << items % boxes <<
    '\n';
    std::cout << "average=" << static_cast<double>(items) / boxes << '\n';
}
```

How it works.

- ``/`` on integers computes the quotient using integer arithmetic.
- ``%`` returns the remainder for integral operands.
- Casting one operand selects floating-point division for the average.

Expected behavior/output. The output reports ``full=3 leftover=2`` and ``average=3.4``.

Experiment. Change ``items`` to 20 and predict both quotient and remainder.

5.2 Comparing and Combining Values

5.2.1 Using Relational Operators

Relational operators such as `<`, `<=`, `>`, and `>=` compare values; equality uses `==`, not assignment's `=`. A comparison produces a Boolean result. Write an interval test as `low <= value && value <= high`. The mathematical-looking `low <= value <= high` instead compares the first Boolean result with `high` and usually does not express the intended interval.

Boundary decisions deserve explicit tests. Does a limit include its endpoint? Is an empty interval allowed? Are units compatible? Two numbers can be legally compared by the language while describing unrelated quantities. Domain types introduced later can prevent some of those comparisons at compile time; until then, names and validation must carry that meaning.

A relational expression produces a Boolean. `items < capacity` asks one question; `items == capacity` asks another. The assignment operator `=` changes an object, whereas `==` compares values. Confusing the two can produce a legal expression with unintended effects, which is one reason warnings and review are useful even when compilation succeeds.

Mathematical interval notation does not transfer directly to C++. The expression `0 <= count <= maximum` does not test a two-sided range in the intended way. It groups into comparisons involving the Boolean result of the first comparison. Write the two conditions explicitly:

```
const bool valid = count >= 0 && count <= maximum;
```

Read this aloud as “count is nonnegative and count is at most the maximum.” The wording exposes both boundaries. A test table should then include just below the lower bound, the lower bound, an interior value, the upper bound, and just above it.

Comparison also interacts with conversions. Comparing a negative signed value with an unsigned container size can yield a result that is surprising when interpreted as ordinary signed arithmetic. Validate the candidate’s domain and use compatible representations. Do not assume that an expression’s appearance as a simple `<` makes its operand types irrelevant.

For a beginner, a useful review method is to annotate each comparison with the exact proposition it is supposed to establish. If the proposition is difficult to state, split the expression into named conditions before making it more compact.

5.2.2 Combining Conditions with Logical Operators

`&&` means logical and, `||` means logical or, and `!` negates a logical value. The built-in `&&` and `||` operators short-circuit: the second operand is evaluated only when needed. This allows a guard such as `capacity != 0 && items / capacity > 2`, though separating validation from calculation can be clearer.

Do not substitute bitwise `&` or `|` for logical operators. They have different purposes and do not provide the same short-circuit protection. Keep conditions free of unnecessary assignments or increments. A condition that only observes state is easier to reuse, negate, and test than one that also consumes input or changes a counter.

For built-in Boolean operations, `&&` evaluates its right operand only when the left operand is true, and `||` evaluates its right operand only when the left operand is false. This short-circuit behavior can protect an operation whose precondition has not yet been established.

For example, a check such as `capacity > 0 && items / capacity > 2` avoids the division when capacity is not positive. Reversing the order defeats that protection. Parentheses can clarify grouping, but they do not repair a dangerous order if the guarded operation still comes first.

The Boolean operators also express different requirements. “The count is negative **or** the capacity is nonpositive” identifies invalid input. “The count is nonnegative **and** the capacity is positive” identifies the corresponding valid domain. Translating between these forms is easier when each individual condition has a clear meaning.

Do not replace `&&` with `&` to save a character. Bitwise and logical operations serve different purposes and have different evaluation behavior. Later low-level chapters explain bit masks. Here, Boolean conditions should use Boolean operators.

A guard can also become too clever. An expression that validates input, changes counters, prints a message, and computes a result through chained operators may be legal but difficult to audit. Keep validation and side effects on separate lines unless the compact form makes the requirement genuinely clearer. Predictability is more valuable than demonstrating how much logic fits into one expression.

Combining Relational Tests into a Range Check

Purpose. Comparisons become useful program logic when boolean results are combined to express domain conditions.

```
#include <iostream>
int main() {
    int temperature = 72;
    bool comfortable = temperature >= 68 && temperature <= 76;
    bool alert = temperature < 50 || temperature > 90;
    std::cout << std::boolalpha << comfortable << ' ' << alert << '\n';
}
```

How it works.

- Relational operators produce boolean values.
- `&&` requires both range boundaries to be true.
- `||` allows either extreme condition to trigger `alert`.

Expected behavior/output. For 72, the program prints `true false`.

Experiment. Try boundary values 68, 76, 50, and 90; identify which comparisons are inclusive and which are strict.

5.3 Understanding Expression Evaluation

5.3.1 Understanding Precedence and Associativity

Precedence determines how operators group syntactically; associativity resolves grouping among operators at the same precedence level. Multiplication binds more tightly than addition, so $a + b * c$ groups as $a + (b * c)$. Parentheses make a different grouping explicit and can also document intent when a reader might hesitate.

Grouping is not a general guarantee about the order in which every operand is evaluated. That distinction matters when operands have side effects. Instead of memorizing a large precedence table as a substitute for clarity, use a small set of familiar expressions and parenthesize less-obvious combinations. The compiler does not reward dense punctuation with better semantics.

Precedence determines how operators group when parentheses do not override that grouping. Multiplication groups more tightly than addition, so $2 + 3 * 4$ means $2 + (3 * 4)$. Associativity resolves grouping among operators at the same precedence level. Neither concept is a universal left-to-right execution schedule.

Use parentheses to express the problem's structure, not to decorate every literal. `(items / capacity) + extra_group` makes the grouping visible, while `((items) / (capacity))` usually adds clutter. If an expression needs several levels of parentheses to remain understandable, intermediate names may communicate better.

For example, the crate calculation has three concepts that deserve names: full crates, leftover devices, and total crates. Naming them also gives you convenient places to inspect state in a debugger. This is not an argument against expressions; it is an argument for keeping their conceptual size appropriate to the reader and the requirement.

When reading unfamiliar code, reconstruct the grouping before calculating a sample result. Then inspect the operand types and range. Correct grouping with the wrong type can still truncate a result; a correct type with an invalid divisor can still violate a precondition. The language has several independent dimensions, and debugging becomes easier when you check them systematically instead of blaming “operator precedence” for every arithmetic surprise.

5.3.2 Avoiding Unexpected Side Effects

A side effect changes program state, such as incrementing a counter or writing to a stream. Combining several changes to the same object within one expression can create unsequenced operations or otherwise make the result difficult to reason about. Avoid examples such as assigning a variable while also incrementing it elsewhere in the same expression.

Use separate statements when order matters: read the current value, perform a calculation, then update the counter. The extra statements give debugging and testing natural observation points. Do not use `volatile` as a repair for sequencing mistakes; it does not supply the missing logical design. Later concurrency chapters distinguish sequencing within one thread from synchronization between threads.

An expression can change state as well as compute a value. Increment operators are a common example. `++count` changes the object and produces a result related to its new value; `count++` also changes the object but has a different value result. In a simple standalone increment statement, the distinction often does not matter. In a larger expression, it can become a source of confusion.

Avoid expressions that modify the same object several times while also depending on the order in which those modifications occur. C++ has detailed sequencing rules, and some constructs have unspecified ordering rather than the left-to-right behavior a reader might imagine. Do not assume that parentheses establish a complete sequencing guarantee. Write separate statements when the sequence itself is important.

A safe beginner transformation is to name each intermediate state:

```
const int before = count;
++count;
const int after = count;
```

Now the relationship is visible and the diagnostic question is simple. Later, when performance work justifies a more compact form, inspect the generated code and language rules. Do not sacrifice clarity in advance for an optimization you have not measured.

Side effects also include stream output and assignments. A function call hidden inside a condition may change external state. During review, underline every operation that changes something. If the expression’s result depends on a subtle interaction among those changes, refactor it into a sequence you can explain without guessing the compiler’s evaluation order.

Making Precedence and Evaluation Order Explicit

Purpose. The example separates operator precedence from programmer-specified grouping and shows why parentheses can document intent.

```
#include <iostream>
int main() {
    int a = 2, b = 3, c = 4;
    int implicit = a + b * c;
    int grouped = (a + b) * c;
    std::cout << implicit << ' ' << grouped << '\n';
}
```

How it works.

- Multiplication has higher precedence than addition, so `b * c` groups first in `implicit`.
- Parentheses explicitly change grouping for `grouped`.
- The code avoids side effects inside a complex expression, keeping evaluation reasoning straightforward.

Expected behavior/output. The output is ``14 20``.

Experiment. Add `int x = 10 - 3 - 2;` and explain associativity before checking the result.

Worked example — Calculate complete groups and a remainder safely

Requirement and simplest approach

Twenty-three items must be packed into groups of capacity five. The input domain is a nonnegative item count and a positive capacity. The output must distinguish complete groups, remaining items, and the number of groups needed to hold all items.

The simplest approach is to divide and stop. It produces four, which is the count of **full** groups rather than the required count of groups. A second shortcut uses floating-point division and rounds visually. That introduces a representation and rounding policy the integer problem does not need.

The selected design uses quotient and remainder. It is precise for the stated domain and gives each intermediate a name. The result is not “correct because the program prints five.” It is correct because four full groups hold twenty items, three items remain, and one additional group can hold those three.

A small proof before execution

For nonnegative items and positive capacity, the quotient accounts for as many complete groups as possible. The remainder is smaller than the capacity. If it is zero, the quotient already covers every item. Otherwise, exactly one more group is sufficient because fewer than one group’s worth remains.

That reasoning explains both branches of the final expression. It also identifies useful tests: an empty input, fewer items than capacity, an exact multiple, and one more than an exact multiple. A formula justified only with the original twenty-three-item example has not yet exposed its boundary behavior.

Trace the implementation

The program validates the arithmetic domain before dividing. It computes complete groups and the remainder separately, then adds one group when leftovers exist. The conditional operator `condition ? first : second` selects one of two values; here it converts the Boolean decision into the integer adjustment 1 or 0.

A common alternative for positive integers is $(\text{items} + \text{capacity} - 1) / \text{capacity}$, but that addition can overflow for large inputs. The separate quotient-and-remainder form makes the boundary behavior easier to audit. The example's tiny constants fit `int`, and a general function would document the supported range explicitly.

The three reported quantities also form a useful invariant: `items == full_groups * capacity + remaining`, with `0 <= remaining < capacity`. Tests can verify that relationship over many inputs instead of checking only one expected answer. This is an early example of deriving a test from the mathematical meaning of the operation.

Complete worked source

source/ch05/main.cpp

```
#include <iostream>

int main() {
    const int items{23};
    const int capacity{5};
    if (items < 0 || capacity <= 0) return 2;
    const int full_groups = items / capacity;
    const int remaining = items % capacity;
    const bool needs_partial_group = remaining != 0;
    const int groups = full_groups + (needs_partial_group ? 1 : 0);
    std::cout << "full=" << full_groups
                << " remaining=" << remaining
                << " total=" << groups << '\n';
    if (full_groups != 4 || remaining != 3 || groups != 5) return 1;
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

```
python3 tools/run_chapter.py 5
```

The executable target is `b01_c05`. Read `source/ch05/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Evidence table — What each intermediate proves

For `items=23` and `capacity=5`, `full_groups=4` shows how many complete groups fit. `remaining=3` shows how much is uncovered. `needs_partial_group=true` translates the remainder into a decision. `groups=5` combines the two responsibilities.

For zero items and positive capacity, both quotient and remainder are zero and the total is zero. This catches an implementation that always adds an extra group. For ten items with capacity five, the total is two, not three. This catches an implementation that mistakes an exact boundary for a partial group.

The test should also reject invalid input before any division. A capacity of zero is a precondition failure, not an alternative arithmetic case. Keep it separate in the test design so a future refactor does not move validation after the dangerous operation.

Failure clinic — Widening too late

Imagine computing a large product in `int` and assigning it to `long long`. A reviewer sees the wide destination and assumes the calculation is safe. The multiplication may already have occurred in the narrower type. Inspect the operands, not only the destination.

The same lesson applies to an overflow-prone numerator in a ceiling-division formula. A mathematically equivalent expression can have different intermediate range requirements. Good C++ arithmetic reasoning includes those intermediate states. This is why the book’s selected design favors a slightly longer quotient-and-remainder calculation over a compact formula with a hidden addition risk.

Failure clinic — A range check that always seems true

A chained comparison modeled after mathematical notation can appear to pass many ordinary tests because the first comparison produces a Boolean value. Add boundary cases that force the intended interval to be false, then inspect grouping. Tests should challenge the logical proposition, not merely repeat values already known to be inside the valid region.

Practice and implementation lab

Evaluate the grouping algorithm for 0, 1, 5, 6, and 25 items with capacity 5. State the expected total number of groups before running a modified program. Explain why dividing first and then converting to double cannot recover the discarded fractional part.

Rewrite a complicated Boolean expression using named intermediate conditions. Make sure the rewritten version does not add or remove side effects.

Starting exercise: solution notes

The expected group totals are 0, 1, 1, 2, and 5. Exact multiples need no partial group, including zero. Integer division has already discarded the remainder from its quotient; a later floating conversion changes representation, not history.

A safe Boolean refactoring preserves which operations occur and when they occur. Pure comparisons can usually be named freely, but moving a potentially failing division out of a short-circuited operand could make it execute unconditionally. Validate the divisor first, then calculate.

Guided lab — Generalize the packing calculation

Create a copy of the example and test item counts zero through twenty for capacities one through seven. For each pair, verify that the reported number of groups has enough capacity and that one fewer group would be insufficient whenever at least one group is needed. Keep the range small so the verification calculations themselves are safe.

Write the two properties in words before translating them into code. “Enough capacity” is different from “minimum number of groups.” An implementation that always returns one thousand may satisfy the first property for these inputs while failing the second. This is an early example of property-based reasoning: several related observations establish a stronger contract than one expected answer.

Next, add an input guard and deliberately test negative counts, zero capacity, and negative capacity. The program must reject them before division. Do not attempt to learn the result of invalid division by executing it. The test is that the invalid operation is never reached.

For the expression exercise, take a dense formula and rewrite it using named intermediates. Predict the output of both versions for valid inputs and explain why they should agree. Then inspect whether either version forms a larger intermediate that could exceed its type’s range. Algebraic equivalence does not remove representation limits.

Acceptance criteria. The result satisfies both sufficiency and minimality in the tested domain; exact multiples do not receive an extra group; zero remains a valid count; invalid divisors are rejected before use; and every side effect has an unambiguous place in the sequence.

Review and solution reasoning

Why not use floating point for this count? The domain and result are integral, and quotient/remainder directly describe the requirement. An additional rounding policy would add complexity without solving a missing problem.

Why can equivalent formulas differ in safety? Their intermediate values may have different ranges. A formula that forms a large sum can overflow even when the final mathematical result would fit.

Does precedence tell you the order of every evaluation? No. Grouping and sequencing are distinct. Keep order-dependent mutations in separate statements unless the relevant guarantee is explicit.

Why use two properties instead of one sample answer? Sufficiency and minimality exclude different wrong implementations. Together they describe the packing requirement more directly than a single expected five.

Chapter takeaway. An expression is an algorithm in miniature. Check meaning, types, ranges, grouping, and side effects before trusting its printed answer.

Technical references: [LANG](#) · [EXPR](#).

Chapter 6 — Controlling Program Decisions with Conditional Logic

HarborWorks classifies a temperature reading as invalid, normal, warning, or critical. The interesting part is not the number of `if` statements. It is whether every accepted input reaches exactly one meaningful result and whether the threshold boundaries match the requirement.

Conditional code is a small decision model. Write the cases in words before encoding them. Then test the points at which the decision changes, because those are where an apparently reasonable comparison most often hides an off-by-one mistake.

A conditional encodes a policy. The syntax is small, but the policy may contain boundary choices, priorities, and assumptions about invalid data. HarborWorks will classify a teaching temperature reading into normal, warning, critical, or invalid. The interesting part is not choosing a color; it is proving that the chosen branches cover the intended domain without hiding contradictory rules.

Learning outcome. Translate a decision table into conditionals, make boundary ownership explicit, distinguish independent decisions from mutually exclusive choices, and simplify nested code without changing its meaning. Every threshold in this chapter is fictional teaching data, not a safety recommendation for physical equipment.

6.1 Making Decisions with `if` Statements

6.1.1 Evaluating Boolean Conditions

An `if` statement executes its controlled statement when the condition converts to true. Conditions often come from explicit comparisons, such as `temperature >= 80`. Prefer a comparison that communicates the domain over a numeric value used merely for its zero/nonzero conversion. `if (device_count > 0)` is usually clearer than `if (device_count)` in beginner code.

Keep validation separate from classification. If negative values are invalid in a teaching model, reject them before asking which valid category they belong to. That structure prevents a catch-all `else` from accidentally treating an invalid value as normal. In other domains negative temperatures are perfectly valid; the permitted range comes from the requirement, not from the syntax.

An `if` statement evaluates a condition and selects whether to execute a controlled statement. Braces turn several statements into a block. Use braces consistently even when a branch currently contains one operation; a later edit should not accidentally leave a newly added statement outside the branch.

A Boolean variable can make a condition easier to review when its name captures a meaningful proposition. `const bool within_range = value >= 0 && value <= 150;` gives a domain rule a name. Do not create names that merely restate syntax without adding meaning, such as `condition1` and `condition2`. The goal is to expose the requirement.

The condition must be based on established data. Testing an uninitialized value is not a valid decision. Testing a default value after failed input may be equally wrong at the domain level. The input chapter's sequence remains important: establish the value, validate its representation and domain, then apply the classification rule.

When a condition is surprising, print or inspect its operands before changing the operator. A negative value converted to unsigned may be the real cause of an unexpected comparison. A stale executable may be the cause of apparently unchanged behavior. Conditional syntax is only one layer of the investigation; the data entering it and the program actually running must also be verified.

6.1.2 Building Multi-Branch `if/else` Logic

An `if/else if/else` chain selects the first matching branch. Ordering therefore matters when conditions overlap. Testing a broad condition such as `value >= 50` before `value >= 80` prevents the later branch from handling high values. Either order thresholds deliberately or write nonoverlapping intervals.

A final `else` can state the remaining valid case after earlier checks establish the domain. This is often simpler than repeating every lower bound. Explain why the cases are exhaustive and mutually exclusive. A future maintainer should be able to insert a new threshold without guessing whether two branches are intended to overlap.

An `if/else if/else` chain selects the first branch whose condition succeeds. This makes branch order part of the policy. If the warning condition is `temperature >= 60` and it appears before the critical condition `temperature >= 80`, a reading of eighty-five will be classified as warning unless the logic accounts for that overlap.

One clear ordering is to reject the invalid domain first, test the most severe valid threshold next, then test the lower threshold, and finally return the normal classification. Another valid design uses nonoverlapping intervals. The best choice is the one whose relationship to the decision table is easiest to inspect.

Independent `if` statements mean something different. A report could legitimately show several independent facts about the same device: it is overdue for calibration and its battery is low. In that case, executing more than one branch is appropriate. A single exclusive temperature label, however, calls for mutually exclusive selection. Do not choose between these structures solely because one uses fewer lines.

Draw the intervals on paper. For this chapter, valid readings zero through fifty-nine are normal, sixty through seventy-nine are warning, and eighty through one hundred fifty are critical. Values outside the valid domain are invalid. Testing fifty-nine, sixty, seventy-nine, and eighty is more informative than testing four random interior values because those cases challenge exactly where responsibility changes from one branch to another.

Guarding a Discount with an `if`/`else` Decision

Purpose. An `if` statement selects behavior based on runtime state, which is the core decision-making mechanism introduced by the section.

```
#include <iostream>
int main() {
    double subtotal = 120.0;
    double total{};
    if (subtotal >= 100.0) total = subtotal * 0.90;
    else total = subtotal;
    std::cout << total << '\n';
}
```

How it works.

- The condition is evaluated once as a boolean expression.
- Exactly one branch assigns `total`.
- The branch condition encodes the business threshold directly.

Expected behavior/output. With 120 the program prints `108`.

Experiment. Test 99.99 and 100.00 to verify the boundary.

6.2 Selecting Among Multiple Cases

6.2.1 Building switch Statements

A **switch** selects among case labels based on an integral or enumeration value. It is well suited to discrete commands and states, not arbitrary ranges or string comparison. Each **case** label identifies a constant value. A **default** branch can handle values not otherwise listed, but a default that silently accepts an invalid command defeats input validation.

A switch does not automatically create a separate scope for every case. Use braces around a case body when it declares local objects whose scope should be limited. Named enumerations, introduced in Chapter 16, make case labels more meaningful than unexplained numeric codes. Until then, document the command mapping at the input boundary.

A **switch** selects behavior based on an integral or enumeration-like value supported by the language rules. It is useful for a small set of discrete cases, such as a menu choice or an enumeration introduced in Chapter 16. It is not a general replacement for every Boolean condition or range test.

Each case label identifies a value. A **default** label can provide fallback behavior when no named case matches. Whether fallback means rejection, logging, or a valid ordinary action is a domain decision. For an external command code, quietly treating an unknown value as a harmless default may conceal bad input.

A range classification such as the temperature policy is naturally expressed with comparisons. Trying to enumerate every possible temperature in a switch makes the structure larger without making the rule clearer. Conversely, using a long string of equality comparisons for a small fixed command enumeration may obscure a clean switch structure.

Place local variables in a braced case block when their scope should belong only to that case. Case labels do not create independent scopes by themselves, and declarations can interact with control flow in ways a beginner does not expect. A case block also communicates that its temporary values are not intended for another case. Keep the lifetime and visibility of decision-local data as narrow as the decision itself.

6.2.2 Understanding Fallthrough and break

Execution can continue from one case into the following case unless a control transfer stops it. **break** exits the nearest enclosing switch or loop; **return** exits the function. Unintended fallthrough is a common defect because the source visually resembles a list of independent branches.

Intentional fallthrough can share behavior, but it should be obvious. Use the standard `[[fallthrough]]` annotation where appropriate to communicate that decision and assist diagnostics. Do not use it to hide a missing break you have not understood. When each case computes a result, returning that result directly can make the control flow easier to verify than mutating a variable and remembering a break in every branch.

After execution reaches a case label, statements continue until control leaves the switch or encounters another control-flow instruction. A **break** ends the switch. A **return** ends the current function. Without an appropriate exit, execution can fall through into the next case's statements.

Sometimes fallthrough is intentional. Several labels may share one implementation, or a carefully designed sequence may reuse work. In modern C++, `[[fallthrough]]` can document an intentional fallthrough where permitted. The annotation is not a command to jump; it expresses intent to readers and diagnostics.

For beginner menu handling, accidental fallthrough is usually the more likely problem. A choice intended to print a report might also execute the reset action placed in the next case. Testing only the final output line could miss the extra side effect. Tests should check that forbidden actions did **not** occur, especially when a branch modifies state.

Do not add **break** blindly after every line. First identify which structure it exits and which behavior the case is supposed to have. In a switch nested inside a loop, **break** exits the switch, not the surrounding loop. If a menu's quit choice must end the loop, use an explicit loop condition or another clearly stated control path. A correct keyword attached to the wrong mental model still produces incorrect behavior.

Dispatching Menu Choices with `switch`

Purpose. A `switch` is clearer than a long equality chain when one value selects among a fixed set of discrete cases.

```
#include <iostream>
int main() {
    int choice = 2;
    switch (choice) {
        case 1: std::cout << "open\n"; break;
        case 2: std::cout << "save\n"; break;
        case 3: std::cout << "quit\n"; break;
        default: std::cout << "unknown\n"; break;
    }
}
```

How it works.

- `switch` compares one integral selector against discrete case labels.
- Each `break` prevents execution from falling through to the next case.
- `default` provides defined behavior for unrecognized input.

Expected behavior/output. With `choice = 2`, the program prints `save`.

Experiment. Remove the `break` after case 2 and observe intentional fallthrough behavior; then restore it.

6.3 Writing Clear Conditional Logic

6.3.1 Simplifying Complex Boolean Expressions

Name meaningful subconditions, remove duplicated comparisons, and use the domain's structure. A condition such as `has_record && is_active && within_limit` can express a policy more clearly than a long expression of unrelated field accesses. Negation rules can help simplify logic, but a mathematically shorter expression is not always more readable.

Test both truth values of each important condition and interactions that affect short-circuiting. Do not assume every possible combination is valid in the domain; document impossible states. When conditions represent a policy likely to change, isolate that policy in a function rather than scattering slightly different versions throughout the program.

Simplification should reduce the number of ideas a reader must track without changing the policy. De Morgan's relationships can help convert a negated compound condition into clearer parts. For example, "not within the interval" can be written as "below the minimum or above the maximum." Test the transformed expression at the same boundaries as the original.

Prefer positive names when possible. `is_valid` is often easier to use than `not_invalid`, especially when another negation appears in a condition. However, do not force every rule into a positive form if the

rejection case is naturally clearer. An early invalid-input guard is often a useful negative statement because it establishes the precondition for the rest of the function.

Repeated comparisons can sometimes be replaced by a named helper function once Chapter 8 introduces functions. The helper should represent a domain concept, not conceal an arbitrary expression. A function named `is_supported_temperature` communicates more than a helper named `check1`.

Avoid simplification that depends on an undocumented assumption. Removing an upper-bound check because “the sensor never sends that value” changes the program’s accepted domain. Either preserve the check or document and enforce the upstream guarantee. In robust software, validation boundaries are deliberately assigned; they do not disappear just because the current sample data happens to be well behaved.

6.3.2 Avoiding Deeply Nested Decision Trees

Early returns can reject invalid input and leave the main path less indented. For example, validate the range, classify the critical threshold, classify the warning threshold, then return normal. This makes each branch’s purpose visible and reduces the amount of state that must be remembered while reading deeper blocks.

Early returns are not an excuse to skip cleanup. At this stage, examples use ordinary values whose lifetimes are automatic. Later, RAII makes resource cleanup follow scope exit even with multiple returns. If a function needs a maze of flags to remember which path was taken, consider dividing it into validation, calculation, and reporting functions with explicit results.

Deep nesting often indicates that the ordinary path is buried beneath repeated validation. Guard clauses can reject invalid cases early, leaving the successful computation at one indentation level. The classification example uses this approach: reject out-of-domain input, then classify known-valid input.

Early return is not automatically superior in every function. Its value depends on whether it makes the function’s responsibilities and cleanup easier to understand. Before resource-owning classes are introduced, several early exits around manually acquired resources can create cleanup mistakes. This is one reason the volume teaches value-based code first and later connects resource lifetime to objects.

A useful refactoring method is to describe the old function as a decision table, rewrite the structure, and run the same table against the new version. Do not compare only line counts. A shorter function that hides branch priorities can be harder to maintain than a slightly longer explicit one.

During review, ask how many facts must remain in your head at the deepest point. If reaching the report requires remembering three nested validation conditions, a single named validation boundary may be clearer. If branches represent genuinely different workflows, forcing them into one expression may make matters worse. The aim is a visible path from requirement to decision, not a blanket rule that all nesting is bad.

Replacing a Nested Condition with Named Predicates and Early Return

Purpose. Clear conditional logic is about preserving the same truth conditions while reducing nesting and making each rule visible.

```
#include <iostream>
bool can_ship(bool paid, bool address_valid, int stock) {
    if (!paid) return false;
    if (!address_valid) return false;
    return stock > 0;
}
int main(){ std::cout << std::boolalpha << can_ship(true,true,3) << '\n'; }
```

How it works.

- Each guard handles one failure reason and exits immediately.
- The final expression states the remaining condition without nesting.
- The function name expresses the higher-level decision the predicates implement.

Expected behavior/output. The supplied call prints `true`.

Experiment. Call the function for each single-failure case and verify which guard rejects it.

Worked example — Classify threshold boundaries without overlapping branches

Requirement and simplest approach

The teaching classifier accepts an integer reading. It must return exactly one of four labels. The valid domain is zero through one hundred fifty. The warning threshold is sixty and the critical threshold is eighty. The upper domain limit distinguishes a very high but accepted reading from an invalid input.

The simplest plausible code tests “at least sixty” and returns warning, then tests “at least eighty” and returns critical. That code is wrong because the first condition already accepts every reading that the second condition would need. This is a policy-ordering defect, not a syntax error.

The selected implementation establishes the domain first and then checks thresholds in descending order. Its returned labels are `std::string_view` objects referring to string literals. That library type is supporting notation here: it observes existing text rather than owning a new string. The literals remain available for the program’s lifetime; Chapter 11 and Book II discuss text ownership more fully. You may focus on the selected label without inventing dynamic memory management for this example.

Trace one boundary from each side

For fifty-nine, the valid-domain check passes, both severity thresholds fail, and the normal label is selected. For sixty, the critical threshold fails but the warning threshold succeeds. For eighty, the critical threshold succeeds before warning is considered. For one hundred fifty-one, the domain guard ends the decision before severity is classified.

That trace explains the order of the source. It also gives each fixture a reason to exist rather than treating testing as an arbitrary collection of numbers.

Trace the implementation

The function is a compact preview of the function syntax developed in Chapter 8.

`std::string_view` here observes fixed string literals; it does not own dynamic text. Every returned view refers to a literal that remains available for the program's lifetime. The example is not returning a view into a temporary local string.

Range validation occurs first. The critical check then removes all values at or above 80 from the remaining path. Consequently the warning check does not need to repeat `temperature < 80`. The final return handles the valid values below 60. This is a decision chain whose coverage can be explained without executing it.

The tests probe immediately below and at each threshold, plus invalid values outside the accepted range. Testing only 20, 70, and 100 would cover the category names but not the boundary rules. The permitted range and thresholds are fictional teaching policy, not a safety specification for real equipment.

Complete worked source

source/ch06/main.cpp

```
#include <iostream>
#include <string_view>

std::string_view classify(int temperature) {
    if (temperature < 0 || temperature > 150) return "invalid";
    if (temperature >= 80) return "critical";
    if (temperature >= 60) return "warning";
    return "normal";
}

int main() {
    if (classify(-1) != "invalid") return 1;
    if (classify(59) != "normal") return 2;
    if (classify(60) != "warning") return 3;
    if (classify(79) != "warning") return 4;
    if (classify(80) != "critical") return 5;
    if (classify(151) != "invalid") return 6;
    std::cout << "59=" << classify(59) << " 80=" << classify(80) << '\n';
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter’s supplied fixtures:

```
python3 tools/run_chapter.py 6
```

The executable target is `b01_c06`. Read `source/ch06/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — Valid syntax, wrong partition

Change the branch order in a copy and run the boundary tests. The program still compiles and returns a legitimate label type. The failure is that the label no longer matches the intended interval. This demonstrates why compilation is not a test of policy correctness.

Next, replace an inclusive comparison with a strict comparison. The exact threshold will move into the neighboring class. An interior value may still pass, so tests centered on the transitions are essential. Read the requirement’s words “at least” and “above” carefully; they describe different boundary ownership.

Finally, remove the upper-domain guard. A reading of one million now looks critical instead of invalid. Whether that is acceptable depends on the contract. In this book’s contract, it is a defect. Do not silently rewrite the requirement to accommodate the current code.

Evidence beyond a single branch

A good classifier test establishes both coverage and exclusivity. Coverage means every possible input category receives a defined result. Exclusivity means the application does not produce contradictory outcomes for one input when the requirement calls for one label. An `if/else` chain helps express exclusivity, but it can still encode the wrong priorities.

For a switch exercise, inspect side effects as well as output. If choosing “show” also increments a reset counter through `fallthrough`, the printed report may look correct while the program’s state is wrong. The test should record the count before and after the choice. Negative evidence—showing that an unintended operation did not occur—is part of a complete behavioral check.

Practice and implementation lab

Add a new `shutdown` category beginning at 120 while preserving invalid values above 150. Write the new boundary tests first. Explain where the additional branch belongs and why.

Write a switch that maps command 1 to `inspect`, 2 to `report`, and all other values to `invalid`. Demonstrate that each accepted command produces one result rather than falling through into the next command.

Starting exercise: solution notes

The shutdown check belongs after range validation and before the critical threshold. Tests at 119 and 120 distinguish critical from shutdown; 150 remains valid and 151 remains invalid. Placing the new branch after `temperature >= 80` would make it unreachable for its intended inputs.

Each switch case can return its label immediately, or assign a result and use `break`. The default branch must reject unknown commands. Both implementations are acceptable when the control flow is explicit and the tests cover an unknown value.

Guided lab — Turn a decision table into tests

Write the four output categories and their accepted intervals before editing code. Add test inputs at every boundary and one step outside each boundary. Include the endpoints of the overall valid domain. For a small domain, you can also loop over all accepted integers later in Chapter 7 and check that each receives the expected category.

Refactor the classifier into nonoverlapping intervals while preserving the same table. Then compare that version with the descending-threshold version. Both can be correct. Explain which version you would prefer for this rule and how your preference might change if the number of categories grew substantially.

Build a separate menu exercise with three discrete choices: show, count, and quit. Use a switch, and ensure each case performs only its intended action. Add an unknown-choice case. Deliberately remove one `break` in a copy, observe the extra action, and restore it. Keep the broken copy out of the normal source build.

For the Boolean exercise, write a valid-domain condition and its invalid-domain counterpart. Verify that they are logical opposites for the tested values. Do not rely on visual similarity alone; a missing equality sign at a boundary can make the pair inconsistent.

Acceptance criteria. The decision table and code agree; all thresholds have tests on both sides; invalid input is distinguished from severe valid input; the menu's cases do not accidentally trigger one another; and each refactoring preserves the documented behavior rather than just the sample output.

Review and solution reasoning

Why does branch order matter? In an exclusive chain, the first satisfied condition wins. Overlapping conditions therefore encode priority through their position.

When are independent if statements appropriate? When several facts or actions may legitimately apply to one input. They are not interchangeable with an exclusive classification chain.

Why does a default case need a policy? It receives values outside the named cases. Treating those values as ordinary without justification can conceal malformed or unsupported input.

What does `break` exit? The innermost loop or switch to which it applies. In a switch inside a loop, it does not automatically end the surrounding loop.

Chapter takeaway. Conditional logic is a partition of possible states. Make the partition explicit, assign its boundaries deliberately, and test both the selected action and the actions that must not happen.

Technical references: [LANG](#) · [STMT](#).

Chapter 7 — Repeating Operations with Loops

A report rarely contains one reading. HarborWorks needs to inspect a sequence, ignore invalid entries, and stop when a termination marker appears. A loop must make progress toward completion while preserving an invariant about the work already done.

The right loop form follows the data source. Use a condition-driven loop when the number of iterations is not known, a counted loop when the bounds are explicit, and a range-based loop when each element of a collection should be visited without index arithmetic.

A loop turns one operation into a repeated process, but repetition creates two new obligations: the state must make progress, and the program must know when to stop. HarborWorks will scan a short sequence of readings, skip rejected values, stop at a teaching marker, and summarize only the accepted prefix. The example is small enough to trace by hand before execution.

Learning outcome. Choose an appropriate loop form, state a termination condition, trace updates through `break` and `continue`, and explain what an accumulator represents after each iteration. You will distinguish an empty result, an invalid input, and an unfinished process rather than treating them as equivalent zeros.

7.1 Repeating While Conditions Remain True

7.1.1 Building while Loops

A `while` loop checks its condition before each iteration. Its body may therefore execute zero times. Identify the state that changes and the condition that eventually becomes false. If neither the body nor an external event can change the condition, the loop may never terminate.

For input processing, `while (stream >> value)` naturally couples progress to successful extraction. Do not loop on `!stream.eof()` and then assume the next extraction succeeds; end-of-file is a state discovered by an attempted operation. For a numeric countdown, maintain a simple invariant such as “remaining is nonnegative and decreases each iteration.” This makes both termination and boundary behavior visible.

A `while` loop tests its condition before each iteration. If the condition is false initially, the body does not run. That makes it suitable when zero repetitions are meaningful, such as processing an input stream that may already be empty.

Every loop needs a progress argument. Identify what changes and why that change can eventually make the condition false. A countdown changes its counter; a stream loop consumes input; a search advances through a finite collection. A condition that stays true while the body changes unrelated state is a likely infinite loop.

For input, `while (std::cin >> value)` combines extraction with the decision to continue. It avoids processing a stale value after a failed read. However, the loop's termination alone does not tell you whether input ended normally or extraction failed on malformed text. Inspect the stream state when that distinction matters to the application's contract.

Do not use `while (!std::cin.eof())` as a general substitute. End-of-file becomes observable through attempted input; checking it before a read does not establish that the next extraction will succeed. The reliable beginner pattern is to test the operation that actually obtains the data, then process only a successfully obtained value.

A loop body should also have a visible effect on the progress state. If a branch can bypass that effect, examine it carefully. The `continue` section later in this chapter shows how a seemingly harmless shortcut can prevent a while-loop counter from ever advancing.

7.1.2 Building do-while Loops

A `do-while` loop executes its body before testing the condition, so it runs at least once. It fits a menu that must be displayed before asking whether to repeat, but it is wrong when an empty collection should cause no work. The syntax ends with a semicolon after the condition.

Choose this form because one initial execution is required, not because it saves a line. In interactive code, a failed input operation must still be handled; repeatedly displaying a prompt does not repair a failed stream. Keep the termination decision and the input-consumption rule together so that the loop cannot become an endless series of prompts over unchanged invalid input.

A `do-while` loop executes its body before testing the continuation condition. It therefore performs at least one iteration. This is useful when the requirement genuinely says “perform an action, then ask whether to repeat,” such as a small menu that must be displayed once.

The guaranteed first iteration can also be a defect. If an empty collection should produce no processing, a `do-while` loop needs an additional guard or a different structure. Do not select it because it seems to save an initialization line. Choose the form that matches the behavior at zero iterations.

Interactive input adds another concern. A menu that repeats until a user selects quit must still have a policy for end-of-input and malformed input. When input has closed, asking the same question forever will not create new bytes. Detect the failure, report or terminate according to the interface, and keep the process from spinning indefinitely.

Read the condition as a continuation rule, not as a success statement. “Repeat while the user wants another operation” is different from “repeat until an operation succeeds.” The latter can become an unbounded retry loop unless progress, limits, and failure handling are defined. Even in a beginner program, a retry policy is an algorithm with observable consequences.

The terminating semicolon in `do { ... } while (condition);` belongs to the syntax. It is different from the accidental semicolon after a plain `while (condition);`, which can create an empty loop body. Inspect punctuation when the visible indented body seems unrelated to actual repetition.

Sentinel-Controlled `while` Loop

Purpose. A `while` loop is appropriate when repetition is controlled by a condition rather than a fixed iteration count known in advance.

```
#include <iostream>
int main() {
    int value = 5;
    while (value > 0) {
        std::cout << value << ' ';
        --value;
    }
    std::cout << "done\n";
}
```

How it works.

- The condition is checked before every iteration.
- The body changes `value`, ensuring progress toward termination.
- When the condition becomes false, execution continues after the loop.

Expected behavior/output. The program prints `5 4 3 2 1 done`.

Experiment. Remove `--value` conceptually and explain why the loop would no longer terminate.

7.2 Performing Counted Iteration

7.2.1 Building Traditional for Loops

A traditional `for` loop groups initialization, continuation condition, and iteration update in one place. The idiom `for (std::size_t i = 0; i < size; ++i)` visits the half-open interval `[0, size)`. The strict less-than comparison is important: index `size` is already beyond the last element.

Use a type and direction that make the bounds safe. An unsigned index cannot become negative in the way a signed countdown might expect, so a condition such as `i >= 0` never becomes false for an unsigned value. For reverse traversal, use a suitable iterator or a carefully stated half-open pattern rather than relying on underflow as an exit mechanism.

A traditional `for` loop collects initialization, condition, and update into one header. The initialization happens once, the condition is tested before each iteration, and the update runs after the body completes normally or reaches a `continue` for that loop. The structure is convenient when a counter clearly describes progress through a finite range.

The half-open interval is a useful pattern: begin at zero and continue while the index is less than the count. It naturally handles an empty collection and avoids treating the count itself as a valid index. Chapter 10 makes the array boundary rule explicit, but the loop's range already expresses the idea.

Avoid changing the counter in several unrelated places. If the header increments it and a branch also increments it, an element can be skipped unexpectedly. A reader should be able to locate the progress rule

without tracing every output statement. More complicated algorithms sometimes need nonuniform progress, but then a while loop or a clearly named update may communicate better.

Choose a counter representation that can express the needed range. Unsigned counters deserve special attention in reverse loops because subtracting from zero wraps rather than becoming negative. A safe reverse-index pattern counts remaining elements down from the size and accesses the element at one less than that count. Do not write `i >= 0` as an unsigned loop's stopping condition and expect it eventually to become false.

7.2.2 Using Range-Based for Loops

A range-based loop visits each element of a range without exposing the traversal machinery. `for (int value : values)` copies each integer into the loop variable. `for (auto& value : values)` can modify existing elements, while `const auto&` observes them without copying larger objects or permitting modification through that reference.

Do not structurally modify a vector during a range-based traversal unless you have carefully established the iterator-validity rules. Appending may reallocate storage and invalidate the loop's hidden iterators. A loop that only transforms existing elements is simpler. Choose an index loop when positions are genuinely part of the algorithm, not merely because indexing is familiar.

A range-based loop expresses “for each element in this sequence” without manually writing an index. It is often the clearest choice when position is irrelevant. In the worked example, each reading is classified independently until the marker is reached, so a range loop exposes the policy rather than the mechanics of indexing.

The loop variable's declaration still matters. `for (int value : readings)` gives the body a value for each iteration. Changing that local value does not change the original integer element. A reference form, introduced more fully in Chapter 13, can permit mutation of the underlying elements. Do not infer mutation behavior solely from the fact that the loop appears to visit each element.

For larger objects, `const auto&` is a common observing form, but you do not need to adopt it blindly before understanding references and type deduction. This volume introduces the ideas in stages. The early integer loops use inexpensive copies deliberately, making ownership and modification easy to reason about.

A range loop does not make structural mutation of the underlying container automatically safe. Adding elements to a vector during iteration can invalidate the traversal state. Chapter 12 discusses container storage and Chapter 13 discusses borrowing. For now, process a stable collection or construct a separate result. The simplest loop is reliable only when the sequence it traverses remains valid for the duration of that traversal.

Index-Controlled `for` Loop

Purpose. A `for` loop groups initialization, continuation test, and progression for a naturally counted task.

```
#include <iostream>
int main() {
    int sum = 0;
    for (int i = 1; i <= 5; ++i) sum += i;
    std::cout << sum << '\n';
}
```

How it works.

- Initialization creates the loop counter once.
- The condition is checked before each iteration.
- The increment expression advances the counter after the body.

Expected behavior/output. The sum of 1 through 5 prints as `15`.

Experiment. Change the increment to `i += 2` and predict the values included in the sum.

7.3 Controlling Loop Execution

7.3.1 Exiting Loops with break

break leaves the nearest enclosing loop or switch. It does not exit every nested loop and does not return from the function. Use it for a clearly defined early-completion condition, such as finding the requested record or encountering a termination marker.

Distinguish “found the answer” from “could not continue.” A Boolean found flag or an optional result can preserve that distinction after the loop. A break that bypasses an important update may invalidate later assumptions, so state the invariant at the exit as well as at the top. Several different break conditions often indicate that the loop is doing too many jobs.

break exits the innermost applicable loop or switch. In the reading example, encountering the marker stops the scan immediately. Values after the marker are not merely rejected; they are outside the processed prefix. That distinction affects both the sum and the count.

A marker value is a protocol choice. The example uses 999 only because the invented accepted readings do not require that value as ordinary data. In a real format, choosing a marker that can also be a valid reading creates ambiguity. A separate length, an explicit record type, or the end of a stream may be a better boundary. Do not copy a teaching sentinel into a real sensor protocol without defining its domain.

Nested loops require careful reading. A break in an inner loop does not automatically stop an outer scan. If the requirement is “stop the entire search when a match is found,” structure the function or state so that the global exit is explicit. Later, a helper function that returns a found result can be clearer than several flags passed through nested loops.

Check the state left behind by an early exit. Does the accumulator describe all data, an accepted prefix, or a partial result that must be labeled incomplete? A program can stop correctly according to control flow and still mislead a caller by presenting the partial result as complete. Name the result according to what was actually processed.

7.3.2 Skipping Iterations with `continue`

`continue` skips the rest of the current iteration and proceeds according to the loop form. In a `for` loop, the iteration expression still runs; in a `while` loop, control returns directly to the condition. This difference matters if progress is performed near the bottom of a `while` body.

A `continue` placed before a required decrement or input read can create an infinite loop. Prefer a structure in which progress is guaranteed independently of optional work. In the worked example, a range-based loop advances automatically, so ignoring a negative value cannot prevent the next element from being considered. Use `continue` to simplify a rejected case, not to scatter hidden jumps throughout a long body.

`continue` skips the rest of the current iteration and proceeds with the loop's continuation behavior. In the worked example, a negative reading is ignored: it contributes neither to the sum nor to the accepted count. The next reading is still considered.

In a traditional `for` loop, the update expression runs after `continue`. In a `while` loop, any manual progress update placed later in the body is skipped. This difference is a common source of infinite loops. If the negative-value branch continues before incrementing an index in a `while` loop, the same negative value may be examined forever.

Place policy decisions before accumulation when rejected values must leave the aggregate unchanged. A sum updated before the rejection check has already incorporated data the program later claims to ignore. Moving a `continue` below the update can therefore change semantics even though the loop still terminates.

A `continue` can improve clarity when it removes a deeply nested ordinary path. It can also hide work if used repeatedly in a long loop. Keep the body small enough that the skipped operations are apparent. If the iteration performs several independent stages, consider a function for processing one item, with an explicit success or rejection result.

For every `continue`, ask two questions: what work is deliberately omitted, and what operation still ensures progress? If either answer is unclear, the loop's correctness depends on an assumption that should be made visible before the code is trusted.

Using `continue` and `break` for Local Loop Control

Purpose. The example shows the distinct meanings of “skip this iteration” and “stop the loop.”

```
#include <iostream>
int main() {
    for (int n = 1; n <= 10; ++n) {
        if (n % 2 == 0) continue;
        if (n > 7) break;
        std::cout << n << ' ';
    }
    std::cout << '\n';
}
```

How it works.

- `continue` skips the remainder of the current iteration for even values.
- `break` terminates the loop completely once an odd value exceeds 7.
- The ordering of the two tests determines which control statement is reached.

Expected behavior/output. The output is `1 3 5 7`.

Experiment. Swap the two `if` statements and reason about whether the output changes.

Worked example — Process a bounded stream of readings

Requirement and simplest approach

The teaching sequence is 4, -2, 7, 0, 9, 999, 50. Nonnegative readings before the marker are accepted. Negative readings are skipped. The marker ends the scan. The program must report the number and sum of accepted readings in that prefix.

The simplest approach is to add every nonnegative value. That would incorrectly include the marker and the later fifty. Another shortcut stops at the marker but increments the count before rejecting negative values. That produces a correct sum with an incorrect accepted count. These are distinct defects, so the result needs at least two checks.

The selected order is marker test, rejection test, then accumulation. The marker has priority over ordinary nonnegative data. The negative-value check has priority over count and sum updates. The fixture is stored in `std::array`, a standard fixed-size container studied in Chapter 12; here it provides a stable sequence for examining loop behavior.

Trace the invariant

Before each iteration, `sum` equals the total of accepted readings already visited, and `accepted` equals their count. The marker branch exits without altering either. The negative branch skips without altering

either. The accepting branch adds exactly one value and increments the count once. That is the invariant the loop preserves.

At termination, four values have been accepted: four, seven, zero, and nine. The sum is twenty. Zero contributes no amount but does contribute one accepted observation. That detail catches a common mistake in which “nonzero” is confused with “valid.”

Trace the implementation

Before each iteration, `sum` equals the total of accepted elements already visited and `accepted` equals their count. A negative value leaves both unchanged. A nonnegative ordinary value extends both. The marker ends processing before the later 50 can contribute.

The array has a fixed, small size and the values keep the sum within `int`’s range. A general measurement service would need an overflow policy and a better framing mechanism than reserving a potentially valid measurement as a magic marker. Chapter 16 introduces named states, and later volumes separate protocol framing from sample values.

The expected result is four accepted values totaling 20. Zero counts as an accepted reading even though it does not increase the sum. That distinction is useful: a test that checks only the sum would fail to detect an implementation that wrongly skips zero. Independent state variables need independent expectations.

Complete worked source

source/ch07/main.cpp

```
#include <array>
#include <iostream>

int main() {
    const std::array<int, 7> readings{4, -2, 7, 0, 9, 999, 50};
    int sum{};
    int accepted{};
    for (int value : readings) {
        if (value == 999) break; // Teaching end marker.
        if (value < 0) continue;
        sum += value;
        ++accepted;
    }
    std::cout << "accepted=" << accepted << " sum=" << sum << '\n';
    if (accepted != 4 || sum != 20) return 1;
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter’s supplied fixtures:

```
python3 tools/run_chapter.py 7
```

The executable target is `b01_c07`. Read `source/ch07/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

A trace table is a debugging tool

Use one row per visited input. Record the input, branch, sum before, sum after, count before, and count after. The row for `-2` should show no change. The row for `0` should show a count increase without a sum increase. The row for `999` should show termination. There should be no processing row for `50`.

This table distinguishes several implementations that might accidentally produce the same sum on a smaller fixture. If the only rejected value were zero, a wrong count could remain hidden. If there were no values after the marker, failure to stop might remain hidden. Good fixtures contain values chosen to expose the rule being tested.

Failure clinic — The loop that never reaches the next item

Rewrite a small scan as a while loop in a copy. Place the index increment after a branch that can continue. When that branch is taken, the index does not advance. Do not leave this broken program running indefinitely; reason through the repeated state or use a bounded diagnostic version that stops after a small number of iterations.

The repair is not to add an arbitrary sleep or increase a timeout. The progress state is wrong. Move or redesign the update so every continuing path advances appropriately, or use a loop form whose update naturally expresses the invariant.

Failure clinic — Averages and empty acceptance

An extension that computes an average must divide by the number of accepted readings, not the number of elements in the original fixture. It also needs a policy when no readings are accepted. Returning zero without a separate indication can confuse an empty result with a genuine average of zero. Later volumes introduce optional results; at this stage, an explicit Boolean or documented branch can represent the distinction.

Practice and implementation lab

Test an empty range, a range beginning with the marker, and a range containing only negative values. Explain what the invariant says after zero iterations. Replace the range-based loop with a while loop and ensure that continue cannot skip its progress step.

Add an average report that handles zero accepted readings without dividing by zero. Do not invent an average of zero unless that is the interface's explicit missing-data policy.

Starting exercise: solution notes

All three special inputs produce a count and sum of zero. The invariant remains true because no accepted values have been processed. A safe while version either increments its index before a possible continue or places progress in a structure that every path executes.

With no accepted readings, report that an average is unavailable. Otherwise convert the sum to a floating type before division. Separating “no average” from the numeric value zero prevents a downstream caller from confusing missing data with a legitimate zero measurement.

Guided lab — Challenge the termination and acceptance rules

Create fixtures for an empty sequence, a marker as the first element, only negative values before a marker, zero as the only accepted value, no marker, and several values after a marker. State what “no marker” means for this finite in-memory fixture: the end of the collection terminates the scan. An external protocol might require a different rule.

For each fixture, compute expected count and sum manually. Then run the program and compare both. Add the average extension only after the original aggregate is correct. Handle the no-accepted-values case explicitly rather than permitting division by zero or inventing an ordinary result.

Implement the same policy with a traditional for loop and a while loop. Keep the data and expected results identical. The purpose is to compare how each form expresses progress, not to change the algorithm while changing syntax. Annotate where each form advances after continue and where it exits after break.

For the nested-loop exercise, scan two small rows of values and stop only the current row at its marker. Then change the requirement to stop the entire scan at the first marker. Explain what structural change makes the broader exit explicit. Do not assume an inner break automatically expresses both requirements.

Acceptance criteria. Every loop has a stated progress argument; empty and marker-first fixtures terminate correctly; zero is counted as valid where required; rejected values leave aggregates unchanged; and the meaning of a partial result is documented.

Review and solution reasoning

Why use an invariant? It states what the changing variables mean at a stable point in the loop. That gives you a precise property to preserve and inspect after each iteration.

Why test a value after the marker? It proves that termination occurs at the marker rather than merely that the marker itself is skipped.

Why is zero a useful fixture? It separates validity from contribution to a sum. A valid zero should affect the count even though it does not change the total.

Why not repair an infinite loop with a delay? A delay changes how quickly the wrong state repeats. It does not establish progress or correct termination.

Chapter takeaway. A loop is a state transition repeated under a rule. Make the invariant, progress, acceptance policy, and stopping condition visible before relying on its aggregate result.

Technical references: [LANG](#) · [STMT](#).

Chapter 8 — Decomposing Programs with Functions

A report becomes difficult to test when reading input, calculating results, and printing text are all interleaved. HarborWorks separates its grouping calculation into a function that accepts validated values and returns an answer. The shell, a test, and a future graphical interface can then reuse the same calculation.

A function is more than a named block. Its parameter and result types form a contract. Decide which data it observes, which data it may modify, and how it reports failure before choosing a convenient signature.

Functions let you replace a long sequence of statements with named responsibilities. That is useful only when the name and interface tell the truth about the work. A function named `calculate_groups` should not unexpectedly read from the terminal, change a global counter, and print a report. HarborWorks will separate the packing calculation from the bookkeeping that records successful reports.

Learning outcome. Write declarations and definitions that agree, distinguish parameters from arguments, choose value or reference deliberately, and state what a function returns or changes on success and failure. You will learn to treat a function as a contract rather than as a convenient place to hide lines.

8.1 Declaring and Defining Functions

8.1.1 Understanding Function Signatures

A function declaration introduces its name, parameter types, and return type. The definition supplies its body. For ordinary overload selection, the parameter list is central; changing only a return type does not create a distinct overload. Parameter names help readers but need not appear in a declaration used only to describe the interface.

Keep a function focused on a coherent operation. `groups_needed(items, capacity)` communicates a calculation. A function called `process` that reads a file, updates global state, and prints an error has a less useful contract. The declaration should let a caller understand the required inputs without inspecting implementation details. Later chapters place these declarations in headers.

A function declaration introduces the information needed to use a function, including its name, return type, and parameter types. A definition supplies the body. In a one-file program, a definition placed before its first use also serves as the needed declaration. In a multi-file program, a header commonly supplies declarations while an implementation file supplies definitions.

A parameter is the local name in the function's interface and body. An argument is the expression supplied by a caller. In `groups_needed(23, 5)`, the two numbers are arguments. Inside the function, `items` and `capacity` are parameters that receive the corresponding values.

Choose a function name that describes one coherent responsibility. `groups_needed` tells the reader that the function produces a count. `process` or `handle` may be too vague unless the surrounding type

supplies the missing context. A useful naming test is whether you can complete the sentence “This function ...” without listing several unrelated jobs.

The phrase “function signature” has precise language meanings that depend on context, so do not reduce overload reasoning to a casual checklist of every visible token. For the ordinary functions in this chapter, parameter types distinguish overloads; changing only the return type does not create a separate overload selected by the caller’s desired destination. The interface must be unambiguous before the function can be used.

8.1.2 Returning Values from Functions

A value-returning function uses `return expression;` to provide its result. Ensure that every reachable normal path returns an appropriate value. A `void` function reports no result through the return channel, although it may still modify referenced objects or perform I/O.

Prefer returning a result to storing it in unrelated global state. It makes dependencies explicit and lets callers decide what to do next. For multiple related results, a small structure is often clearer than several output parameters. This volume first uses simple numeric and Boolean results; later volumes introduce optional values and tagged alternatives for richer success and failure models.

A return statement ends the current function and supplies its result when the return type requires one. Returning a value is often clearer than modifying a caller-owned output parameter because the result is visible at the call site. `const int groups = groups_needed(items, capacity);` makes the data flow explicit.

Every path through an ordinary value-returning function must meet the function’s return requirements. A missing branch is not repaired by whatever value happens to remain in a processor register. Unlike the special rule for reaching the end of `main`, ordinary functions should not fall off their end expecting an automatic meaningful result.

The chapter function returns `-1` for invalid input. That sentinel is usable because every valid result is nonnegative. This is a narrow design choice, not a universal error convention. A function whose valid domain includes negative results would need another representation. Book II introduces types that can express absence or alternatives more directly.

A value return also protects lifetime boundaries. Returning the computed integer gives the caller its own result. Returning the address or reference of a local integer would leave the caller observing an object whose lifetime has ended. Chapter 9 explains the lifetime issue in detail. For now, return ordinary values unless the interface genuinely needs to expose an existing object with an established longer lifetime.

Separating a Function Declaration from Its Definition

Purpose. This is the simplest demonstration of the declaration-versus-definition distinction used throughout multi-file C++ programs.

```
#include <iostream>
int add(int, int);
int main() { std::cout << add(2, 3) << '\n'; }
int add(int a, int b) { return a + b; }
```

How it works.

- The declaration introduces the function signature before it is called.
- `main` can type-check the call using only the declaration.
- The later definition supplies the executable body that the linker associates with the declaration.

Expected behavior/output. The program prints `5`.

Experiment. Change the declaration to `double add(int,int);` without changing the definition and observe the conflicting-declaration diagnostic.

8.2 Passing Information into Functions

8.2.1 Passing Arguments by Value

A value parameter is a separate object initialized from the argument. Changing that parameter does not change the caller's original value. This is a natural choice for small scalar inputs such as counts and limits. It also establishes independence when a function intentionally needs its own copy of a larger value.

Do not assume every value parameter has the same cost. Copying an integer and copying a large vector are different operations. Start with the semantics you require, then consider cost. Modern C++ can avoid some copies or transfer resources, but those optimizations do not change the basic distinction between a function's parameter object and the caller's original object.

Passing an integer by value gives the function a parameter object initialized from the argument. Modifying that parameter does not modify the caller's integer. This creates a useful boundary: a calculation can reorganize its local working values without surprising the caller.

For example, a helper might repeatedly subtract capacity from a local copy of `items`. The caller's original count remains unchanged. That particular algorithm is not as direct as quotient and remainder, but it illustrates the parameter model. The parameter's name is not another name for the caller's object unless the interface explicitly uses a reference.

Small scalar values are usually natural value parameters. Do not replace every `int` with `const int&` in the belief that references are always faster. A reference introduces aliasing and lifetime considerations;

avoiding a tiny copy is not automatically an improvement. Performance claims require evidence, while clarity and the intended mutation model are immediately relevant.

Passing a large object by value can have a different cost, especially when a copy is required. Later chapters explain references and move semantics. The beginner rule is not “always copy” or “never copy.” It is to identify whether the function needs its own value, needs to observe an existing object, or must modify caller-owned state. The interface should make that choice visible instead of hiding it behind a blanket optimization rule.

8.2.2 Passing Arguments by Reference

A reference parameter names an existing caller-owned object. A nonconst reference allows the function to modify that object, so mutation becomes part of the interface. A const reference permits observation without copying through that access path. Neither reference form extends an arbitrary caller object’s lifetime for later use.

Use references deliberately. `void increment(int& count)` advertises mutation more clearly than hiding it through a global. However, a function that returns the incremented value may be easier to compose when mutation is unnecessary. Avoid returning references to local automatic objects; those objects cease to exist when the function returns, leaving the caller with an invalid access path.

A reference parameter such as `int& successful_reports` refers to an existing integer supplied by the caller. Incrementing that parameter changes the caller’s object. The ampersand in this declaration is part of a reference type, not the address-of expression used later with pointers.

This makes reference parameters appropriate for deliberate mutation, but mutation should be evident in the function’s name and contract. `record_success(reports)` suggests bookkeeping. A function named `groups_needed` that also increments an unrelated counter through a hidden reference would mix two responsibilities.

Specify failure behavior for every output or in/out parameter. A parser that fails should not leave the caller guessing whether the destination contains its old value, a partial result, or an arbitrary default. A common pattern is to compute into a local candidate, validate it completely, and assign to the caller’s object only on success. The string parser in Chapter 11 uses that pattern.

References also permit aliasing: two parameters can refer to the same object. A function that assumes they are independent may behave unexpectedly when the caller passes the same variable twice. For a beginner interface, either design the operation to remain correct under aliasing or state and enforce an appropriate precondition. Do not infer independence from two different parameter names; names do not create separate underlying objects.

Pass-by-Value Versus Pass-by-Reference Mutation

Purpose. Parameter passing determines whether a function receives an independent value or direct access to an existing object.

```
#include <iostream>
void increment_copy(int x) { ++x; }
void increment_original(int& x) { ++x; }
int main() {
    int n = 10;
    increment_copy(n);
    std::cout << n << ' ';
    increment_original(n);
    std::cout << n << '\n';
}
```

How it works.

- The value parameter receives a separate `int`.
- The reference parameter aliases the caller's object.
- Only the reference-taking function changes `n` in `main`.

Expected behavior/output. The output is `10 11`.

Experiment. Change the second parameter to `const int&` and explain why incrementing it no longer compiles.

8.3 Designing Flexible Functions

8.3.1 Using Function Overloading

Overloading gives multiple functions the same name when their parameter lists differ. The compiler chooses an applicable overload using the argument types and conversion rules. It is useful when the functions express the same conceptual operation for different types, such as formatting an integer or a floating measurement.

Do not create overloads whose behavior differs in surprising ways or whose implicit conversions make calls ambiguous. A call with an integer literal may select a different overload from a call with a double. Read the diagnostic rather than adding casts until compilation succeeds. An explicit, distinct name is sometimes a better interface than a complicated overload set.

Overloading lets several functions share a name when their parameter lists distinguish meaningful variations. The compiler chooses among candidates using the argument expressions and the language's conversion rules. This is a compile-time selection mechanism, not a runtime search through function names.

Use overloads when the operations represent the same concept. A function that computes a quantity from integers and another that computes a fundamentally different report from text should not share a name

merely because overloading is available. Consistent semantics matter more than a tidy list of matching names.

Implicit conversions can create ambiguity or select an overload you did not intend. A literal such as `0` has a type; a literal such as `0.0` has a different type. Overloads involving numeric types should therefore be tested with representative argument forms, not only with variables already declared in the expected type.

Do not overload solely on return type. At a call site, the compiler generally needs to identify the function from its name and arguments before using the result. Two ordinary functions with identical parameter lists but different returns do not become a convenient “choose the result type” interface.

For an introductory library, fewer overloads with explicit contracts are often better than many clever combinations. Add an overload when it removes a real burden for callers without obscuring which operation is selected. If a reviewer must simulate a complicated conversion ranking for every call, the interface may be too ambitious for its purpose.

8.3.2 Using Default Arguments

A default argument supplies a value when a caller omits a trailing argument. `int groups_needed(int items, int capacity = 5);` allows `groups_needed(12)` while retaining an explicit-capacity form. Place the default in the interface declaration visible to callers, not redundantly in every definition.

A default is part of the API’s policy. Changing it can change behavior at call sites that did not mention the value. Do not use defaults to hide an essential safety parameter or a choice that varies by deployment. When several adjacent parameters have the same type, a configuration structure or named operation may be clearer than relying on readers to remember their positions.

A default argument supplies an argument expression when the caller omits that trailing argument. In `groups_needed(int items, int capacity = 5)`, a call with one argument uses the documented default capacity. A call with two arguments supplies an explicit capacity instead.

Defaults belong to the declaration information visible at the call site. They are not a second runtime overload that magically receives missing data. This matters in multi-file code: place the default in the public declaration and do not repeat it inconsistently in the implementation. Chapter 19 shows the header arrangement.

A default is also a policy. Choosing five means that callers who omit capacity accept the teaching packing convention. If real hardware or configuration determines capacity, silently embedding a constant default can hide an important dependency. Require an explicit argument when omission would make the caller’s intent ambiguous.

Combining overloads and defaults can make calls ambiguous. Before adding both mechanisms, write the expected call forms and verify that each selects one clear function. A small API should be easy to use correctly without a detailed understanding of overload resolution.

Finally, changing a default can change behavior for recompiled callers even when the function body is unchanged. Treat that as an interface revision, not a harmless cosmetic edit. In this volume, the project is rebuilt together, but the principle prepares you for library evolution in Book V: declarations carry behaviorally important information.

Overloads and Default Arguments for Related Operations

Purpose. Function flexibility in C++ is usually expressed with well-defined overloads and defaults rather than runtime inspection of arbitrary arguments.

```
#include <iostream>
int area(int side) { return side * side; }
int area(int width, int height) { return width * height; }
int scale(int value, int factor = 2) { return value * factor; }
int main(){ std::cout << area(4) << ' ' << area(4,5) << ' ' << scale(3) <<
'\n'; }
```

How it works.

- Overload resolution selects `area` by argument count.
- The default argument lets callers omit a common `factor`.
- The signatures remain type-checked at compile time.

Expected behavior/output. The output is `16 20 6`.

Experiment. Add `double area(double)` and call `area(4.5)`; inspect which overload is selected.

Worked example — Separate validation, calculation, and caller-owned mutation

Requirement and simplest approach

The application must compute the number of groups and record a successful report only when the calculation accepts its inputs. The simplest approach is to put calculation, error handling, bookkeeping, and output into one long `main`. That works for one path but makes the calculation hard to test without reproducing the entire interaction.

The selected design has two functions. `groups_needed` computes a value and returns a documented invalid-input sentinel. `record_success` deliberately modifies the caller's report counter. The first expresses a calculation; the second expresses an effect. Keeping them separate makes the point at which bookkeeping occurs visible.

Trace the call boundary

When `groups_needed(23)` is called, the omitted capacity uses the default five. The function validates the arguments and returns five. Only then does the caller increment the success counter. When capacity is zero, the function returns the failure sentinel before division, and the caller must not record success.

The function's return value is not automatically displayed. Reporting remains the caller's responsibility. This separation allows the same calculation to be reused by a terminal interface, a unit test, or a later application layer without adding unwanted output.

Test the function directly with zero items, exact multiples, partial groups, and invalid capacity. Then test the caller's integration rule: successful results increment the counter exactly once; rejected results do not increment it. A correct function does not guarantee a correct caller, and a correct caller cannot compensate for a function that violates its contract.

Trace the implementation

`groups_needed` takes its inputs by value because they are small and the function does not need to modify the caller. Its `-1` result is a deliberately simple failure convention; valid group counts are nonnegative. A later `optional` or domain result type can make that distinction more explicit.

`record_success` takes a nonconst reference because its purpose is to update the caller's counter. The call site can see the function name and signature, so the mutation is not hidden. Neither function performs I/O, allowing the main function to control reporting independently.

The default capacity is a fixed teaching policy. The explicit `(4, 0)` call verifies that a bad capacity is rejected before division. This demonstrates why default arguments do not remove the need to validate explicitly supplied values. A caller can still provide a value outside the accepted domain.

Complete worked source

source/ch08/main.cpp

```
#include <iostream>

int groups_needed(int items, int capacity = 5) {
    if (items < 0 || capacity <= 0) return -1;
    return items / capacity + (items % capacity != 0 ? 1 : 0);
}

void record_success(int& successful_reports) {
    ++successful_reports;
}

int main() {
    int reports{};
    const int groups = groups_needed(23);
    if (groups >= 0) record_success(reports);
    if (groups != 5 || reports != 1) return 1;
    if (groups_needed(0) != 0) return 2;
    if (groups_needed(4, 0) != -1) return 3;
    std::cout << "groups=" << groups << " reports=" << reports << '\n';
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

The executable target is `b01_c08`. Read `source/ch08/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — The parameter that changed nothing

A learner writes `void record_success(int reports)` and increments `reports` inside the function. The caller's value remains unchanged. The function received a value parameter, so the increment changed its local parameter object. This is not a compiler optimization removing useful work; it is the declared parameter model.

The repair depends on the intended interface. A reference parameter can express in-place bookkeeping. Returning the new count can also work if the caller assigns it. Choose one clear model and test the observable result. Do not add a global variable merely to make the change visible across the function boundary.

Failure clinic — A sentinel treated as an ordinary answer

The calculation returns `-1` for invalid input, but the caller prints `"groups=-1"` and increments the success counter anyway. The function met its contract; the integration violated it. This is why result handling belongs in tests. A result representation is useful only when callers interpret it correctly.

A stronger later design can encode success and failure in a dedicated type. For this volume, the explicit branch is sufficient because the domain is small and the sentinel is outside the valid range. The text states that limitation rather than presenting sentinel values as an ideal design for every API.

Refactoring without changing behavior

When extracting a function from existing code, begin with the current inputs, outputs, and side effects. Move one coherent calculation, preserve its tests, then remove duplicated logic from callers. Do not simultaneously change the rounding rule, error convention, and output format. Smaller refactorings make a failed test informative because fewer dimensions changed at once.

Practice and implementation lab

Add tests for one less than, exactly, and one more than a multiple of the capacity. Write a value-taking version of `record_success`, call it, and explain why the caller's counter does not change.

Refactor the failure convention into a Boolean-returning function with an output reference, then compare the clarity of the two interfaces. Identify which output must remain unchanged when validation fails.

Starting exercise: solution notes

For capacity 5, inputs 9, 10, and 11 require 2, 2, and 3 groups. A value parameter is a separate object, so incrementing it cannot update the caller. This is a semantic difference, not a compiler optimization issue.

A Boolean/output-reference interface should validate first and assign the output only on success.

Otherwise callers may observe partially updated state after failure. Returning a structured result is often clearer, but this exercise makes the mutation contract visible before richer result types are introduced.

Guided lab — Specify and test a function contract

Write a contract for `groups_needed` in ordinary language. Include accepted inputs, the result's meaning, the failure sentinel, and the guarantee that invalid capacity is rejected before division. Then create a table of direct calls and expected results. Use at least one omitted-default call and one explicit-capacity call.

Add a helper that computes remaining spaces in the final group, but define what it should return when there are zero items. Several interpretations are possible; choose one and make it visible. The exercise is not complete when the code compiles. It is complete when another reader can predict the answer without inspecting the implementation.

For the mutation exercise, implement the success counter once using a reference parameter and once using a returned value. Keep the integration tests the same. Explain which interface makes the change clearer at the call site. There is no requirement to force both versions into one production program; the comparison teaches you to recognize the tradeoff.

Finally, move the pure calculation into a helper while leaving input and output in `main`. Run the previous fixtures unchanged. A successful refactoring preserves behavior while changing organization. An intentional behavior change should have a separately stated requirement and updated tests.

Acceptance criteria. Each function has one coherent responsibility; declarations and definitions agree; valid and invalid results are distinguished; reference mutations are deliberate; and every caller handles the result according to the contract.

Review and solution reasoning

Why prefer a return value for a calculation? It exposes the result at the call site and avoids unnecessary mutation of caller-owned state. The choice remains subject to the actual data model.

Why can two reference parameters alias? They are names for existing objects supplied by the caller. Different parameter names do not guarantee distinct arguments.

Why not repeat default arguments everywhere? Defaults are part of the visible declaration contract. Repetition creates opportunities for inconsistency and invalid redeclarations.

Why test the caller as well as the function? The caller can ignore failure, update state twice, or mislabel the result even when the function itself is correct.

Chapter takeaway. A function earns its name by making a responsibility and its data flow clear. State the contract, keep effects visible, and test the boundary between caller and callee.

Technical references: [LANG](#) · [FUNC](#).

Chapter 9 — Understanding Scope, Lifetime, and Storage Duration

A name can disappear from view while its object still exists, and an access path can remain visible after its object has died. Confusing those facts causes some of C++’s most serious defects. HarborWorks encounters a helper that returns an address of a local result: the address has a plausible numeric value, but the object it once identified no longer exists.

This chapter separates visibility from existence. Draw both timelines whenever a pointer or reference crosses a function boundary.

“Can I use this name here?” and “Does the object still exist?” are different questions. A name may disappear while an object continues to exist, or a pointer may remain visible after its target has died. This chapter separates scope, storage duration, and lifetime before pointers make the consequences more visible.

Learning outcome. Trace when names are visible, when objects are created and destroyed, and which state persists between calls. Explain why returning an ordinary value is different from returning access to a local object, and why hidden persistent state can make otherwise simple tests depend on execution order.

9.1 Understanding Where Names Are Visible

9.1.1 Working with Block and Function Scope

A local name is generally visible from its declaration to the end of its enclosing scope, subject to more detailed language rules. Nested blocks can introduce their own names, and an inner declaration can hide an outer name. Hiding is legal in many cases but can make a program difficult to read; warning options such as `-Wshadow` help identify it.

Function parameters are available within the function’s definition. A loop variable declared in a for initializer is limited to the loop’s scope. Prefer the narrowest useful scope so that readers do not have to remember a variable long after its last use. Visibility is a compile-time name-lookup property, not a statement about where storage physically resides.

A block introduces a region in which local names can be declared. A name declared inside an inner block is not generally available after that block ends. The braces therefore serve two related but distinct purposes: they organize control flow and they establish boundaries for declarations and ordinary local objects.

A local name can hide an outer name. For example, declaring another `total` inside a nested block creates a separate object rather than resetting the outer `total`. A debugger can show a plausible value while you are stopped in the inner block, then show the unchanged outer value after leaving it. The apparent disappearance is a naming issue, not lost memory.

```
int total{};
{
    int total{7}; // A distinct inner object.
    std::cout << total << '\n';
}
std::cout << total << '\n'; // The outer object remains zero.
```

This is a diagnostic fragment, not a recommended naming style. Prefer names that make distinct roles clear, especially when an outer accumulator is important. Warnings for shadowing can help, but the stronger habit is to keep each scope small and purposeful.

Declare an object close to the first point where its valid initial value is known. This narrows the region in which later code can accidentally use it. A variable declared at the beginning of a long function becomes another piece of state every branch must consider, even when only one small block needs it.

9.1.2 Working with Namespace Scope

A namespace groups names and reduces accidental collisions. A name declared at namespace scope can be used through qualification, such as `harbor::limit`, when its declaration is visible. Headers commonly use namespaces to organize interfaces shared across translation units.

Namespace scope does not mean every declaration is mutable global state. A function declaration, a type, and a constexpr policy constant can all live there. The concern is uncontrolled dependence on mutable objects whose initialization, access, and synchronization are difficult to reason about. Avoid broad `using namespace` directives in headers, because they change the name-lookup environment of every file that includes them.

A namespace organizes names at a broader level than a function block. In the companion's later multi-file application, `harbor::Inventory` identifies the inventory type in the project's namespace. The namespace communicates ownership of an interface and reduces collisions with unrelated libraries.

Namespace scope does not mean every name is a mutable global variable. A namespace can contain functions, types, constants, and other declarations. Keeping related names together is different from giving all functions unrestricted access to shared mutable data.

Avoid placing a broad `using namespace std;` directive in a public header. It can affect code that includes the header and introduce names or ambiguities beyond the header's intended interface. Writing `std::vector` and `std::string` makes the dependency explicit and is a useful beginner habit.

A qualified name also helps diagnosis. If a declaration is in `harbor` but the definition is accidentally placed in the global namespace, the linker may not find the expected function. The spelling before `::` is part of the identity, not decorative text. Chapter 19 returns to this when declarations and definitions are split across files.

For now, use namespaces to organize interfaces, not to hide a pile of global state. A well-named namespace can make code easier to locate, but it does not establish a safe lifetime or synchronization policy for the objects declared inside it. Those responsibilities remain separate.

Block Scope Hiding and Name Visibility

Purpose. Scope controls where a name can be used; this example makes two different objects share spelling while remaining distinct.

```
#include <iostream>
int main() {
    int value = 10;
    {
        int value = 20;
        std::cout << value << ' ';
    }
    std::cout << value << '\n';
}
```

How it works.

- The inner declaration creates a different object with block scope.
- Inside the block, the inner name hides the outer name.
- After the block ends, the outer `value` becomes visible again.

Expected behavior/output. The output is `20 10`.

Experiment. Rename the inner variable and compare readability without changing behavior.

9.2 Understanding How Long Objects Exist

9.2.1 Understanding Automatic Storage Duration

Ordinary local objects usually have automatic storage duration: their storage is managed as execution enters and leaves the relevant block. Construction establishes the object's lifetime, and destruction ends it for class objects. The standard's storage-duration model is more precise than saying every local variable is physically on a stack; optimization can change placement or eliminate storage entirely.

Returning a local object by value is safe because the result is a distinct returned value, with copy elision or movement available where applicable. Returning a pointer or reference to that same local object is not safe. The caller receives an access path to an object whose lifetime has ended, regardless of whether the old bytes remain unchanged for a while.

An ordinary block-local object without a special storage specifier usually has automatic storage duration. Its lifetime begins when initialization completes under the relevant rules and ends when control leaves its scope, including an early return. The conceptual lifetime matters even when the optimizer keeps the value in a register or eliminates physical storage entirely.

This is why “local variable equals stack slot” is only an introductory implementation model, not the full language contract. The compiler can transform representation while preserving required observable behavior. Reason about the object and its lifetime first; use stack diagrams as explanatory models rather than as guarantees about every optimized binary.

Two calls to a function ordinarily create separate automatic parameter and local objects. A local counter initialized to zero therefore starts from zero on each call. That is often exactly what a calculation needs: its result depends on explicit inputs rather than on previous calls.

When the function returns a value, the caller receives a result according to the return semantics. It does not need the function's local integer to remain alive. When the function returns a pointer or reference to that local integer, however, it exposes an access path to an object whose lifetime is ending. The existence of an address value does not prolong the target's life.

Keep these questions separate in a trace: where was the object declared, when was it initialized, and what access paths remain after its lifetime ends? That trace becomes the foundation for diagnosing dangling references and pointers.

9.2.2 Understanding Static Storage Duration

Objects with static storage duration have storage associated with the program's execution rather than one ordinary block entry. A local static object is initialized when control first reaches its declaration, under the applicable initialization rules, and persists across later calls. Its name can remain block-local even though its lifetime is much longer.

A mutable local static counter therefore remembers previous calls, which can be useful but introduces hidden state. Concurrent initialization of a function-local static is handled by the language, but subsequent unsynchronized mutation is not automatically safe. Static duration should not be confused with automatic thread safety or with a universal guarantee about initialization order across unrelated translation units.

A function-local static object is initialized according to static-local initialization rules and persists across calls. Its name remains local to the function, but its storage duration is not the same as an ordinary automatic local. This is a direct example of scope and duration being independent dimensions.

The chapter's `persistent_next` function begins with one static integer and increments the same object on successive calls. The first two results are one and two. Moving the declaration to an ordinary local would change the behavior because each call would initialize a new counter.

Persistent state can be useful, but it introduces history. A test that assumes the next call returns one may fail if another test already called the function. A program that reloads data cannot necessarily reset the static object through its public interface. These are design considerations, not bugs in the storage-duration feature.

Do not infer thread-safe mutation from the special initialization rules for a local static. Initializing an object and coordinating later concurrent updates are separate operations. Concurrency is developed in Book IV; this introductory counter is single-threaded and bounded by its small test sequence.

A real identifier service would also need a range, reset policy, persistence policy, and collision model. The example provides none of those guarantees. It exists to make state persistence visible, not to supply a production identifier generator. Keeping that boundary explicit prevents a useful teaching experiment from being mistaken for a complete service.

Observing Automatic Object Lifetime with Constructor and Destructor Messages

Purpose. Object lifetime is a runtime property distinct from the lexical visibility of the name.

```
#include <iostream>
struct Trace {
    Trace(){ std::cout << "construct\n"; }
    ~Trace(){ std::cout << "destroy\n"; }
};
int main(){
    std::cout << "before\n";
    { Trace t; std::cout << "inside\n"; }
    std::cout << "after\n";
}
```

How it works.

- `t`` begins its lifetime when execution reaches its declaration.
- Its destructor runs automatically when the block is left.
- The printed order exposes lifetime boundaries directly.

Expected behavior/output. The order is ``before``, ``construct``, ``inside``, ``destroy``, ``after``.

Experiment. Return early from inside the block and verify that the destructor still runs.

9.3 Separating Scope from Lifetime

9.3.1 Recognizing Lifetime-Related Bugs

A dangling pointer or reference does not become safe merely because it is nonnull. It may point to an object that has been destroyed, a vector element invalidated by reallocation, or a temporary whose lifetime ended. Checking the numeric address cannot reconstruct the missing ownership and lifetime relationship.

Trace where the object is created, who owns it, and which event ends its lifetime. Then ask whether every use occurs before that event. A returned reference to a local fails this test immediately. Runtime sanitizers can reveal some executed lifetime defects, but absence of a report is not a proof. The design must make valid lifetimes understandable without relying on lucky memory contents.

A dangling access path is one that no longer refers to a live object in the way the program intends to use it. The pointer or reference variable may still be in scope, which can make the defect visually deceptive. A local name's visibility does not certify its target's lifetime.

Consider a function that returns the address of a local integer. The integer's storage may appear to contain the old value briefly after return, but using that apparent persistence is not valid lifetime reasoning. A later function call, optimization, or unrelated edit can change the symptom. "It printed the expected number once" is therefore not evidence of safety.

Returning an ordinary integer value solves that problem when the caller only needs the result. Returning a container by value can also be a sound design; do not introduce an unsafe reference merely to avoid a copy you have not measured or that the implementation may optimize appropriately.

Temporary lifetime rules have important special cases, including some reference bindings, but those rules should not be generalized into “a reference keeps an object alive.” It does not provide universal ownership. Book II explores the details. The beginner’s reliable question is: which object owns the storage, and what guarantees that owner remains alive for the entire use?

Diagnostic tools can expose some lifetime errors when the failing path executes. They do not replace a correct ownership model. A sanitizer pass on one fixture does not validate every possible escape of a local object’s address.

9.3.2 Avoiding Unnecessary Global State

Global mutable state hides inputs and outputs. A function that appears to calculate a result from one parameter may secretly depend on a configuration object modified elsewhere. Tests then influence one another, and concurrent callers may race over shared state.

Pass required values explicitly or place related state inside a clearly owned object. A small counter object can make persistence intentional while allowing each test to construct a fresh instance. Constants and immutable lookup tables can still be appropriate at namespace scope. The goal is not a ban on every global declaration; it is a clear ownership and dependency model for values that can change.

Global mutable state makes dependencies implicit. A function that reads a global limit appears to accept one input while actually depending on two: its argument and the current global configuration. A function that changes a global counter also affects later calls that do not mention the counter.

For a small application, pass the needed values explicitly or group related state in an object with a clear owner. A calculation that accepts `previous` and returns `next` can be tested independently of the order in which other calculations ran. The chapter’s `local_next` function illustrates that simpler model.

Not every global-scope declaration is bad. A compile-time constant or a type declaration can be entirely appropriate. The concern is unnecessary mutable state whose initialization, access, and reset behavior are hard to see. A blanket ban on namespaces or constants would miss the actual problem.

When refactoring a global dependency, first list every reader and writer. Then decide which component should own the state and which functions need it as input. Replacing a global variable with a globally accessible singleton can preserve the same hidden dependency under a different spelling. The architecture improves only when ownership and access become more explicit.

A useful test is whether two independent instances of the application logic can run with different configuration. If they cannot because every function reaches into one shared global object, the design may be more coupled than the requirement demands. The inventory capstone deliberately keeps its collection inside an explicit object so tests can create fresh independent instances.

A `static` Local Object Outliving Each Function Call

Purpose. The example separates name visibility from storage duration: small scope does not imply short lifetime.

```
#include <iostream>
int next_id() {
    static int id = 0;
    return ++id;
}
int main(){ std::cout << next_id() << ' ' << next_id() << '\n'; }
```

How it works.

- The name `id` is visible only inside `next\_id`.
- The object has static storage duration, so it survives between calls.
- Each call observes and modifies the same object despite the narrow scope.

Expected behavior/output. The output is `1 2`.

Experiment. Replace `static int id` with `int id` and explain why both calls then return 1.

Worked example — Compare persistent state with caller-owned state

Requirement and simplest approach

HarborWorks wants to compare a counter that remembers its previous calls with a calculation whose history is supplied explicitly. The simplest approach is to print a number from each and conclude that both “increment.” That description hides the important difference: one operation depends on persistent internal state, while the other depends on an argument.

The selected experiment calls `persistent_next` twice and `local_next` with a known value. It also creates a value inside an inner block to show a scope boundary. No address of a dead object is dereferenced. The example teaches lifetime through valid code rather than requiring a crash to make the point.

Build a time-ordered object map

At the first call to `persistent_next`, the static object is initialized and then incremented. At the second call, that same object is incremented again. For `local_next(10)`, the parameter and local result belong to that call; the returned value is eleven. After return, the local objects no longer serve as objects the caller may access.

Inside the nested block, `scoped` is available and can be tested. Outside, its name is unavailable. The static counter still exists even though its name is not directly visible in `main`. These observations separate visibility from persistence without relying on implementation-specific memory addresses.

The trace's key columns are object identity, scope of its name, start of lifetime, end of lifetime, and who can use it. A diagram that contains only arrows and addresses would omit several of those necessary facts.

Trace the implementation

The name `value` is visible only inside `persistent_next`, but the object persists between calls. The second call observes the state left by the first. In contrast, `local_next` constructs a local result and returns its value without retaining hidden state. Its result depends only on the supplied input, within the small supported range used here.

The inner `scoped` variable demonstrates a short visibility region. Trying to name it after the block would be a compile-time lookup error. Returning its address from a function would be a different problem: a lifetime error that can compile while still being invalid to use.

The example intentionally does not call the persistent counter from multiple threads. Incrementing the static integer would require synchronization in that setting. The lesson is persistence, not an assertion that static objects are a convenient replacement for an ownership model.

Complete worked source

source/ch09/main.cpp

```
#include <iostream>

int persistent_next() {
    static int value{};
    return ++value;
}

int local_next(int previous) {
    const int next = previous + 1;
    return next; // Return a value, not its address.
}

int main() {
    const int first = persistent_next();
    const int second = persistent_next();
    const int explicit_result = local_next(10);
    if (first != 1 || second != 2 || explicit_result != 11) return 1;
    {
        const int scoped = local_next(2);
        if (scoped != 3) return 2;
    }
    std::cout << "persistent=" << first << ',' << second << '\n';
    std::cout << "explicit=" << explicit_result << '\n';
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter’s supplied fixtures:

```
python3 tools/run_chapter.py 9
```

The executable target is `b01_c09`. Read `source/ch09/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — Tests that depend on their order

Suppose a test expects the first call to `persistent_next` to return one. It passes when run alone but fails after another test calls the same function. The function is behaving according to its persistent-state design; the test assumed a fresh state that was never promised.

The repair could be to isolate the test in a fresh process, expose a deliberate reset policy, or redesign the counter as an instance owned by the test. The appropriate choice depends on the requirement. Randomly changing the expected value to whatever appeared on the last run merely encodes the accidental execution order.

Failure clinic — The local value that seemed to survive

A learner returns a reference to a local integer and sees the expected result in a debug run. The storage has not been visibly overwritten yet, so the defect appears harmless. The language’s lifetime rule does not depend on whether yesterday’s bytes remain in memory. The program has lost a valid target for the reference.

The right repair is to return a value or arrange a genuinely longer-lived owner. Making the local static can extend its lifetime, but it also introduces sharing and history. That may change the application’s semantics. A lifetime repair is not complete until the new ownership model fits the requirement.

Refactoring checkpoint

Take one calculation that currently reads a global setting and add an explicit parameter. Run two tests with different settings in the same process. If the results no longer depend on which test ran first, you have made an important dependency visible. The source may have one more parameter, but the behavior now has fewer hidden inputs.

Practice and implementation lab

Draw separate scope and lifetime intervals for the local static counter. Rewrite the counter as a function that accepts and returns its state. Explain how independent tests can reset that version without a special reset function.

Write a nonexecuted example of a function returning a pointer to a local integer. Mark the exact event that invalidates the pointer. Do not run undefined behavior to obtain an expected answer.

Starting exercise: solution notes

The local static's name is available only in its function body, while its object survives after a call returns. Passing state explicitly allows each test to choose an initial value, removing dependence on test order.

The local integer's lifetime ends as its block exits during return. The pointer value can be copied to the caller, but it no longer identifies a live object available for dereferencing. A null check is irrelevant to that defect; returning the integer by value or using an appropriately owned longer-lived object is the repair.

Guided lab — Draw lifetimes before writing pointers

Create an object table for the worked source. Include the static counter, each call's parameter, each ordinary local result, and the inner-block value. Mark when each name is usable and when the corresponding object exists. Use separate columns; a single "scope/lifetime" column would hide the distinction the exercise is meant to teach.

Modify the explicit calculation to accept a configurable increment. Test two independent sequences by keeping separate previous values in the caller. Then compare that with one persistent static counter. Explain which design naturally supports two independent users and which shares a history.

For the shadowing experiment, introduce an inner variable with the same name as an outer accumulator in a copy. Print both at appropriate points and explain why changing the inner value does not update the outer one. Restore clear names before adding the example to your own working project.

Do not execute an invalid lifetime example merely to obtain a dramatic crash. Write a short explanation of why returning an address to a local object would be invalid, then implement the value-returning repair. Later sanitizer exercises can use isolated intentional defects with explicit boundaries.

Acceptance criteria. The table distinguishes scope from lifetime; ordinary locals are not assumed to occupy fixed physical stack addresses; persistent state is recognized as history; and no returned access path depends on an expired local object.

Review and solution reasoning

Can a name be local while its object persists? Yes. A function-local static has a local name and persistent storage duration.

Does a reference universally extend lifetime? No. Some language constructs have specific extension rules, but a reference is not a general ownership mechanism.

Why can returning a value be safer than returning a reference? The caller receives a result without needing the callee's local object to remain alive. A reference requires a valid longer-lived target.

Why is changing a local to static not always a proper fix? It changes sharing, persistence, and possibly concurrency behavior. A lifetime extension must still match the application's intended state model.

Chapter takeaway. Names tell you where an object can be mentioned; lifetime tells you when it can be used. Keep those questions separate and make persistent state an explicit design choice.

Technical references: [LANG](#) · [LIFETIME](#) · [GUIDELINES](#).

Chapter 10 — Storing Fixed Collections with Arrays

HarborWorks receives four readings per diagnostic frame. The count is fixed by this teaching interface, so an array is a natural representation. The important contract is not simply that storage is contiguous. Every index must remain inside the collection, and functions must know how many elements they are allowed to inspect.

Raw arrays expose the underlying model, while standard containers make several of its constraints easier to express. Learn the model without assuming that the least-protected representation is the best default.

An array gives a sequence a fixed shape, but it does not make every integer a valid position in that sequence. HarborWorks will inspect a short frame of readings and a small two-dimensional grid. The values are ordinary integers; the new responsibility is preserving the relationship between a collection and its bounds.

Learning outcome. Declare and initialize fixed arrays, traverse half-open index ranges, interpret multidimensional arrays as arrays of rows, and recognize when an array’s size information is lost at an interface. The chapter uses raw arrays to expose the mechanism before Chapter 12 presents safer standard-container interfaces.

10.1 Creating and Accessing Arrays

10.1.1 Declaring Fixed-Size Arrays

A raw array such as `int readings[4]{1, 2, 3, 4};` contains four adjacent integer elements. Its extent is part of its type. With an initializer that supplies fewer values, the remaining elements are value-initialized in this case. An omitted initializer for a local fundamental array does not establish useful element values.

A standard C++ raw-array bound must be a suitable constant expression; variable-length arrays accepted by some compilers are extensions, not a portable substitute for vector. A zero-length raw array is also not the standard mechanism for representing an empty collection. Use `std::array<T, N>` for a fixed-size value-like container and `std::vector<T>` when the size is determined dynamically.

A built-in array stores a fixed number of elements of one element type. In `int readings[4]{2, 4, 6, 8};`, the bound is four and the elements are initialized from the list. In `int readings[]{2, 4, 6, 8};`, the initializer determines the bound. That bound is part of the array type; the object does not grow when you need a fifth reading.

For ordinary built-in arrays in standard C++, the bound must meet constant-expression requirements. A compiler extension that accepts a runtime-sized local array does not make that code portable C++. The book requests a standard mode and warnings partly to make such accidental dependencies visible. Use a vector for an ordinary runtime-sized collection when Chapter 12 introduces it.

An empty brace initializer zero-initializes ordinary numeric elements. A shorter initializer list initializes the remaining numeric elements appropriately, but the array still has its full declared bound. Distinguish “this position contains zero” from “this position has not been logically filled with an input record.” A partially used fixed buffer may need a separate logical count.

Built-in arrays do not provide the ordinary zero-element collection model that standard containers can provide. Do not invent a zero-length raw array and rely on a compiler extension. A `std::array<T, 0>` or an empty vector has a defined container interface for that case. Choosing the right collection type can remove an awkward special case from the program.

10.1.2 Accessing Elements by Index

Array indexing begins at zero. For an extent of four, valid indices are 0, 1, 2, and 3. The expression `readings[4]` is not a request for a fifth slot that the implementation will create. It accesses beyond the array and must not be evaluated as an ordinary element access.

A raw subscript does not perform an automatic bounds check. Validate external indices before using them. For standard containers, `at` offers checked access with an exception on an invalid position, while `operator[]` retains a precondition that the index is valid. Neither form removes the need to define how an application should report malformed input.

For a four-element array, valid indices are zero through three. The bound four is a count, not the last valid index. This is the reason the loop condition normally uses `index < count`, not `index <= count`.

Indexing a built-in array does not automatically perform a runtime bounds check. An out-of-range access is not guaranteed to throw an exception or crash at the offending line. It can corrupt another object, appear to work, or behave differently after optimization. A familiar printed result is not proof that the index was valid.

Treat an external index as untrusted until its domain has been checked. If it is read into a signed integer, reject negative values before converting to an unsigned size type. Then establish that the resulting position is below the bound. The order matters because converting a negative input first can produce a large unsigned value rather than a useful negative sentinel.

A useful mental model is a half-open interval `[0, count)`: zero is included, count is excluded. This notation also describes iterator ranges later in the series. The model scales from arrays to slices and subranges because the end position can represent “one past the last element” without representing an element you may access.

Write tests for the first and last valid indices and for both invalid neighbors. Interior-only tests do not challenge the boundary contract that makes indexing safe.

Fixed-Size Raw Array Initialization and Indexed Access

Purpose. Raw arrays introduce fixed-size indexed storage and make zero-based element addressing concrete.

```
#include <iostream>
int main(){
    int scores[4]{90, 85, 88, 92};
    scores[1] = 87;
    std::cout << scores[0] << ' ' << scores[1] << ' ' << scores[3] << '\n';
}
```

How it works.

- The array contains four adjacent `int` elements.
- Valid indices range from 0 through 3.
- Indexed assignment changes one existing element in place.

Expected behavior/output. The output is `90 87 92`.

Experiment. Attempt to reason about `scores[4]` without running it: the expression is outside the array and has undefined behavior.

10.2 Processing Collections of Array Elements

10.2.1 Traversing Arrays Safely

Use a range-based loop when every element should be processed and the array object is still known. `std::size(array)` can obtain the extent without duplicating a numeric constant. Hard-coding a second size in a loop makes later edits fragile: the declaration can change while traversal still assumes the old count.

When a raw array is passed to many ordinary function parameters, it decays to a pointer and its extent is no longer carried by that parameter. A declaration that visually resembles `int values[4]` in such a parameter does not enforce a four-element argument. Prefer an array reference, a standard container, or a span with a clear lifetime and extent contract as your interfaces become more advanced.

When a calculation needs every element, derive the traversal bound from the actual collection instead of duplicating a numeric constant. The chapter source uses `std::size(frame)` with the appropriate header. If the initializer later changes length, the loop's bound remains connected to the array.

The older expression `sizeof(array) / sizeof(array[0])` can compute an element count when the expression really denotes the array object. It does not work as a universal function-interface technique because an array parameter often adjusts to a pointer. At that point, `sizeof` measures the pointer representation rather than the original array's total storage.

A range-based loop can avoid explicit indexing when position is irrelevant. It makes the traversal intention clear and removes one place for an off-by-one defect. It still requires a valid range and does not authorize changing the collection's structural lifetime during iteration.

For a sum, state the accumulator's range. Four small integers are safe in the teaching example, but a long collection of large integers may require a wider or checked accumulator. Bounds safety and arithmetic safety are separate obligations. Successfully staying inside the array does not guarantee that the total is representable.

Trace the loop invariant: before each iteration, the accumulator equals the sum of exactly the earlier elements. After the final iteration, every element has contributed once. This catches both skipped positions and double-counted positions, not merely out-of-bounds access.

10.2.2 Processing Multidimensional Arrays

A declaration such as `int grid[2][3]` is an array of two arrays, each containing three integers. Traverse rows and columns within their respective bounds. The nested structure matters to the type system; it is not interchangeable with every pointer-to-pointer representation.

For large numerical data, a flat vector with an explicit indexing function can make dimensions and storage ownership easier to manage. The row-major mapping `row * columns + column` must still be protected against invalid dimensions and arithmetic overflow. At this stage, use tiny fixed arrays to understand the relationship between shape and indexing before generalizing the representation.

A declaration such as `int grid[2][3]` describes an array of two rows, each row being an array of three integers. The first index selects a row; the second selects an element within that row. Thinking in nested arrays is more precise than imagining a generic two-dimensional object with arbitrary indexing rules.

A nested range loop reflects that structure directly. The outer loop observes each row; the inner loop processes the row's elements. The source uses a reference for the row so it remains an array expression rather than being treated as an unrelated copied value. Chapter 13 explains references in detail; here the notation preserves the row object being traversed.

Do not assume that pointer arithmetic from the first row may freely cross every nested-array boundary simply because the complete object occupies contiguous storage. C++ pointer arithmetic is defined in relation to array objects and permitted bounds. The straightforward nested traversal avoids relying on an invalid flattening interpretation.

When an algorithm requires a flat buffer, choose and document a flat representation deliberately, such as a one-dimensional container plus row and column dimensions. The mapping `row * columns + column` then needs its own bounds and overflow checks. That design is developed in later numerical work; it is not necessary for the two-by-three teaching grid.

Name dimensions according to meaning. `rows` and `columns` communicate a shape; two variables both called “size” force readers to infer which index each constrains.

Summing and Averaging an Array with a Range-Based Loop

Purpose. Processing arrays usually means applying the same operation to every element while keeping the extent correct.

```
#include <iostream>
#include <iterator>
int main(){
    int scores[]{80,90,100};
    int total = 0;
    for (int s : scores) total += s;
    std::cout << total << ' ' << static_cast<double>(total)/std::size(scores) <<
    '\n';
}
```

How it works.

- The range-based loop visits each element exactly once.
- `std::size` obtains the compile-time array extent without repeating a magic number.
- The cast prevents integer division in the average.

Expected behavior/output. The program prints ``270 90``.

Experiment. Add a fourth score and verify that `std::size` and the average adjust automatically.

10.3 Understanding the Limitations of Raw Arrays

10.3.1 Recognizing Out-of-Bounds Access

Out-of-bounds access is not defined to return zero, throw an exception, or crash immediately. It can read unrelated data, overwrite another object, or appear harmless until an optimized build changes the layout. A program that prints the expected result after an invalid access is still defective.

Use bounds-aware interfaces, tests at zero and at the final valid index, compiler diagnostics, and sanitizers as complementary defenses. Do not create a test whose expected behavior depends on reading just past the end. Instead, test the validation path that rejects the invalid index before access. The contract is what must be verified, not an accidental symptom of violating it.

Out-of-bounds behavior is dangerous because the program may not fail where the defect occurs. A write one element beyond an array can change nearby state, and the visible symptom may appear much later. A read can produce a plausible number that came from an unrelated location. Neither result belongs to the array's contract.

Do not test the invalid access by intentionally reading arbitrary memory in the normal suite. Instead, test the validation function or loop structure that prevents it. An isolated sanitizer demonstration can be useful later, but it must be labeled as an intentional defect and excluded from the valid examples.

Off-by-one mistakes often occur at the boundary between human counting and zero-based indexing. “The fourth item” is at index three. “There are four items” gives an excluded end of four. Write both interpretations in a trace table when the code is unfamiliar.

A separate logical count must never exceed physical capacity. If a fixed buffer has room for ten elements and only three have been filled, iterating to ten may process zero-initialized but logically absent records. This may remain memory-safe while still being semantically wrong. The program needs to distinguish capacity from populated size, a theme that returns with vectors.

Review bounds where they are established and where they can change. A check performed before an unrelated mutation may no longer justify an access afterward. Safety depends on the relevant state remaining valid between the check and the use.

10.3.2 Comparing Raw Arrays with Standard Containers

`std::array` retains fixed size while supporting familiar container operations and value semantics. `std::vector` manages a dynamically sized contiguous sequence and owns its storage. Both integrate with standard algorithms and expose size explicitly. Raw arrays remain useful for understanding layout, interfacing with certain APIs, and some tightly constrained representations.

Choose based on ownership, size policy, and interface requirements rather than assuming that raw syntax is inherently faster. Compilers can optimize standard-container access effectively when the operations have equivalent semantics. A clearer size and ownership contract often prevents mistakes without adding a runtime cost to the hot loop.

Raw arrays are useful for understanding layout and for some low-level interfaces, but their interface is limited. They do not have a member `size`, do not resize, and do not support ordinary whole-array assignment in the way a standard value container does. When passed through certain function parameter forms, their bound is not carried automatically.

`std::array` wraps a fixed-size collection in a standard-container interface while keeping a compile-time extent. `std::vector` owns a runtime-sized sequence and manages its storage. These types do not eliminate all mistakes, but they make important operations and metadata available through consistent interfaces.

The correct comparison is not “raw is fast, standard is slow.” For a fixed collection, the standard wrapper may express the same essential representation with a clearer interface. Performance questions require a workload and measurement. In beginner code, losing bounds information to save a few characters is rarely a sound tradeoff.

Choose a raw array when an interface or learning objective specifically calls for it. Choose a standard container for ordinary application ownership unless there is a reason not to. Chapter 12 shows the practical operations; this chapter gives you the mechanism needed to understand what those operations are protecting.

The final inventory application does not replace a vector with a homemade raw buffer. Its goal is to model records and enforce rules, not to reimplement a storage manager before the application’s requirements are stable.

Array Decay Versus `std::span` Size Preservation

Purpose. Raw arrays decay to pointers in many interfaces, which is why safer range abstractions become useful.

```
#include <iostream>
#include <span>
void inspect_raw(const int* p) { std::cout << "raw first=" << p[0] << '\n'; }
void inspect_span(std::span<const int> s) { std::cout << "span size=" <<
s.size() << '\n'; }
int main(){ int a[]{1,2,3,4}; inspect_raw(a); inspect_span(a); }
```

How it works.

- Passing the raw array to `const int*` loses its element count.
- `std::span` is a nonowning view that carries the range length.
- Both functions avoid copying the array, but only one preserves size information.

Expected behavior/output. The output contains `raw first=1` and `span size=4`.

Experiment. Change the span call to `std::span{a}.first(2)` and inspect the reported size.

Worked example — Sum a fixed frame and a small matrix

Requirement and simplest approach

The lab must total a four-value frame and a two-by-three grid. The frame's expected total is twenty, and the grid's expected total is twenty-one. The simplest approach is to write each addition manually. That works for the current literals but duplicates the shape in the calculation and becomes fragile when the fixture changes.

The selected design traverses the actual collections. The frame loop uses a bound derived from the frame. The grid loop follows rows and then elements. The program has no external index input, so its immediate safety argument rests on the loop structure and the validity of the fixed objects.

Trace the frame and the grid separately

For the frame, record the accumulator after each of two, four, six, and eight. The sequence should be two, six, twelve, twenty. For the grid, first inspect the row totals of six and fifteen, then the combined total of twenty-one. These intermediate observations provide a stronger diagnosis than checking only the final sum.

A program that skips one element and accidentally counts another twice can occasionally reach the same total. Use nonrepeated values and, in practice exercises, a second fixture whose values make each position's contribution distinguishable. Tests should expose traversal shape as well as arithmetic.

The complete source uses `std::size` rather than a separate four. If you add a fifth initializer and preserve an inferred bound, the loop should adapt. That experiment demonstrates the value of keeping metadata connected to the object it describes.

Trace the implementation

The frame traversal derives its bound from the array itself. The matrix traversal preserves each row as an array reference, then visits its elements. Both forms avoid duplicating the extent in an unrelated loop condition. The small input values make integer overflow irrelevant to this specific demonstration.

The outer matrix loop uses `const auto& row` so that the row's array structure is retained without mutation. A pointer-to-pointer is not involved. This distinction becomes important when calling C libraries, mapping files, or designing numerical interfaces: similar-looking indexing notation can hide different memory layouts.

The tests verify independent totals for the one- and two-dimensional data. To test bounds handling, add a separate checked-access function and require it to reject invalid positions. Do not add a deliberate out-of-bounds read to the normal test suite and call a crash the expected educational result.

Complete worked source

source/ch10/main.cpp

```
#include <iostream>
#include <iterator>

int main() {
    const int frame[][2]{2, 4, 6, 8};
    int total{};
    for (std::size_t i = 0; i < std::size(frame); ++i) {
        total += frame[i];
    }
    const int grid[2][3]{{1, 2, 3}, {4, 5, 6}};
    int matrix_total{};
    for (const auto& row : grid) {
        for (int value : row) matrix_total += value;
    }
    if (total != 20 || matrix_total != 21) return 1;
    std::cout << "frame=" << total << " matrix=" << matrix_total << '\n';
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

```
python3 tools/run_chapter.py 10
```

The executable target is `b01_c10`. Read `source/ch10/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — The count mistaken for an index

A loop written with `i <= count` visits the excluded end. The error is in the range definition, not in the array's allocation. Increasing the array's capacity may hide the symptom while preserving the wrong logical traversal. Repair the loop contract instead of creating unexplained spare storage around it.

A related error begins at one because the author is thinking of “item one.” That skips the first element and may still overrun at the other end. Convert between human numbering and program indices at a clearly defined boundary rather than switching conventions inside the loop.

Failure clinic — The array that became a pointer

A function parameter written as `int values[]` does not carry the caller's bound in the way a beginner may expect. Attempting to recover the count with `sizeof(values)` inside that function measures the adjusted parameter representation. The result can look like a small plausible number and still be unrelated to the number of elements.

A repair can pass an explicit count, use an interface that retains the array extent, or use a suitable standard container or view. Each design has a lifetime and validation contract. Chapter 13 discusses references; Book II introduces nonowning range views. For this volume, do not hide the count in an assumption known only to the caller.

A practical review technique

Annotate every subscript with the bound that justifies it. For a grid, annotate the row bound and the column bound separately. If the justification cannot be found in the surrounding code or interface, the access is depending on undocumented state. This small review habit scales to buffers and numerical kernels much larger than the teaching arrays.

Practice and implementation lab

Change the frame's extent by adding one initialized element and confirm that traversal adapts without editing a bound. Write a function that accepts an index and rejects any value outside `[0, size)` before reading.

Draw the six elements of the matrix in row-major order. Explain why a two-dimensional row array is not automatically compatible with `int**`.

Starting exercise: solution notes

Deriving the bound from `std::size` keeps the loop aligned with the declaration. The checked accessor must reject the index equal to size as well as larger indices; negative external indices must be rejected before conversion to an unsigned type.

The matrix stores two adjacent row arrays. `int**` instead describes a pointer to pointers, whose pointees may be unrelated allocations. Converting between the two representations by force does not create the missing pointer objects or establish a valid layout.

Guided lab — Make shape changes safe

Change the frame values and length in a copy, then compute a new expected total manually. Keep the traversal bound derived from the object. The build and tests should continue to work without editing a duplicated element count elsewhere.

Next, add a function that accepts a candidate index and reports whether it is valid for a known bound. Test a negative candidate, zero, the last valid index, the excluded end, and a large positive value. Do not dereference rejected indices. The purpose is to prove that invalid positions are prevented, not to discover what unrelated memory contains.

For the grid, compute each row total and the overall total. Verify that the sum of the row totals equals the overall total. Then deliberately swap the dimension assumptions in a copy and explain the defect from the shape before running anything unsafe. A correct diagnosis can prevent the invalid access entirely.

Finally, compare a raw array declaration with a `std::array` declaration from the next chapter. List which metadata and operations become available through the container interface. Do not claim a performance difference without measuring equivalent work under a documented build.

Acceptance criteria. The first and last valid positions are handled; the excluded end is never accessed; dimension changes do not leave stale traversal constants; logical populated size is distinguished from capacity; and function interfaces do not pretend a pointer carries an array bound.

Review and solution reasoning

Why is the count excluded from the index range? Zero-based indexing assigns positions zero through count minus one. The count is useful as an end boundary, not as an element position.

Can processing an unused zero-initialized slot be wrong without being out of bounds? Yes. Memory safety and logical validity are different properties. An absent record is not necessarily a valid zero record.

Why use nested loops for a nested array? They follow the actual row structure and make each bound visible without relying on questionable flattening assumptions.

Why can a standard container improve clarity without changing the task? It carries size and ownership through a defined interface, reducing assumptions that would otherwise be passed informally.

Chapter takeaway. A collection and its bounds form one contract. Preserve that relationship at every traversal and function boundary, and distinguish valid storage from valid application data.

Technical references: [LANG](#) · [ARRAY](#) · [LIFETIME](#).

Chapter 11 — Manipulating Text with `std::string`

Text is an interface, not just a bag of characters. HarborWorks receives a field such as `sensor=42` and must distinguish a complete identifier from a prefix followed by garbage. The program should own text that must remain available and avoid treating a successful partial conversion as complete validation.

This chapter introduces string ownership, searching, and strict numeric parsing. The same distinctions later protect configuration files, command-line interfaces, and network protocols.

Text is an interface between people, files, and typed data. A string can own those bytes safely while the application still misinterprets their meaning. HarborWorks will parse a small `name=value` field and accept an identifier only when the complete value obeys a stated grammar and range.

Learning outcome. Construct and modify strings, check search results before using positions, distinguish substring length from endpoint, and parse numeric text without accepting an unintended suffix or destroying an earlier valid result. The chapter’s command vocabulary is deliberately narrow; it is not presented as a complete internationalized text-processing system.

11.1 Creating and Modifying Strings

11.1.1 Constructing and Assigning Strings

`std::string` owns a sequence of character elements and manages the storage required for them. Construct it from a literal, another string, or a suitable character range. Assignment replaces the current value while preserving the string object’s identity. An empty string is a valid value with size zero; it is not a null pointer.

A string can contain embedded null characters, so its size need not equal the length observed by a C API that stops at the first null. The `c_str()` view provides a null-terminated sequence for appropriate interoperability, but its pointer is borrowed and may be invalidated by later modifications. Keep ownership and external access lifetimes explicit.

`std::string` owns a sequence of character code units and manages the storage needed for that sequence. A string object can be empty, can grow, and can be copied as a value. You do not need to allocate a raw character buffer merely to store an equipment name.

Construction establishes a new string; assignment replaces the value of an existing string. After assigning one ordinary string to another, changing one does not mean that the other automatically changes as an alias. The ownership and value model is different from simply copying a raw pointer to the same character storage.

A string literal and a string object are related but not identical. The literal supplies characters with its own language-defined storage properties. A `std::string` initialized from that literal owns its string value

through the library's interface. Later, `std::string_view` observes existing character storage without owning it, so its lifetime contract requires separate attention.

The size of a string counts its code units, not necessarily the number of visible human-language characters. This book's identifiers and command keywords use a simple byte-oriented grammar. Equipment names can be stored as text, but operations such as Unicode normalization, grapheme counting, and locale-sensitive comparison are outside Book I. State that boundary rather than claiming that indexing a byte string solves every text problem.

For beginner code, prefer the owning string when the text must remain available after the input buffer changes. Introduce a nonowning view only when its owner and lifetime are clear.

11.1.2 Concatenating and Appending Text

Use `+`, `+=`, or `append` to combine text according to the desired ownership behavior. Repeatedly constructing large temporary strings can be more expensive than appending into one result, but clarity comes first and performance should be measured for the actual workload. `reserve` can request capacity when a credible size estimate is available; it does not change the string's logical length.

Appending can invalidate pointers and references into the string if storage changes. Do not hold a `c_str()` pointer across arbitrary mutation. Also avoid using concatenation to build shell commands from untrusted input. A string that is safe as data can become dangerous when interpreted as a command language; systems chapters keep argument vectors separate from shell parsing.

Concatenation combines text values; appending changes an existing string. `name += " sensor";` extends `name`. A separate result such as `const std::string label = name + " sensor";` leaves the original `name` unchanged. Choose the form that matches the intended state change.

Do not assume that `+` converts every operand into text. Two raw string literals do not form a general runtime string-concatenation expression through the same rules as `std::string` objects. A literal plus an integer can mean pointer arithmetic rather than numeric formatting. Use a string object and an explicit conversion such as `std::to_string` when combining a number with text.

```
const int id{42};
const std::string label = std::string{"sensor="}
    + std::to_string(id);
```

The extra explicit string construction clarifies which abstraction owns the concatenation. A string stream is another formatting option, useful when several values require stream formatting rules. Later language/library facilities offer more choices, but this volume does not require them for a small identifier label.

Repeated concatenation in a large loop can involve allocation and copying. Do not optimize that possibility prematurely in a program with a few short names. First make the grammar and output correct. When the workload becomes significant, measure and consider appropriate capacity planning or a different assembly method. A faster malformed record is still malformed.

Any pointer, iterator, or view into a string's storage may be affected by mutation. Keep borrowed access short-lived and reestablish it after operations that can invalidate it.

Building and Editing a `std::string`

Purpose. The example demonstrates creation, growth, and in-place modification through the string abstraction.

```
#include <iostream>
#include <string>
int main(){
    std::string name = "Ada";
    name += " Lovelace";
    name.replace(0, 3, "Augusta Ada");
    std::cout << name << '\n';
}
```

How it works.

- `std::string` owns a variable-length character sequence.
- `+=` appends text and grows storage when necessary.
- `replace` edits a selected range without manual character shifting.

Expected behavior/output. The program prints `Augusta Ada Lovelace`.

Experiment. Use `erase` to remove `Augusta` and verify the resulting string.

11.2 Searching and Extracting Text

11.2.1 Finding Characters and Substrings

`find` returns a position when it locates the requested character or substring, and `std::string::npos` when it does not. Check that sentinel before using the result in arithmetic or as a substring boundary. Since the position type is unsigned, treating not-found as an ordinary small negative integer can create surprising results.

Search semantics matter. Finding `sensor=` somewhere in a field is different from requiring it at the beginning. Use the check that matches the interface: exact equality, prefix matching, or a search with a documented range. A parser should not accidentally accept `notsensor=42` because it found a familiar sequence in the middle.

`find` returns a position when it locates the requested character or substring and a special `npos` value when it does not. Position zero is a valid successful result. Therefore, `if (text.find('='))` does not correctly distinguish success from failure: it treats a match at the beginning differently from other matches and does not compare with the documented absence value.

Always test against `std::string::npos` before using the returned position. This is especially important before adding one to the position or subtracting positions to compute lengths. The position type is unsigned, and arithmetic on the absence sentinel is not a meaningful substitute for an explicit missing-separator branch.

A parser also needs a rule for multiple separators. The first `=` may divide a key from a value that itself permits equals signs, or multiple separators may be invalid. `find` supplies a location; it does not choose the grammar. For the small numeric field, the value parser rejects extra nonnumeric characters, so an additional equals sign will not be accepted as part of an identifier.

Test missing, leading, trailing, and repeated separators. Each reveals a different boundary: no split point, empty key, empty value, or unexpected additional syntax. A parser tested only with `sensor=42` has not yet shown that it can reject structurally different input.

Search is a mechanism. Validation decides whether the structure found by that mechanism is meaningful for the application.

11.2.2 Extracting Substrings

`substr(position, count)` creates a new owning string containing the selected portion. The position must be valid; a count extending beyond the end is limited to the available remainder. This is useful when the extracted value must outlive or change independently of its source.

A nonowning `std::string_view` can refer to a portion without allocating, but it does not extend the source's lifetime. Returning a view into a local string creates a dangling view. Prefer an owning result until the ownership relationship is clear. A zero-copy design is only correct when the backing storage remains valid for every consumer.

`substr(position, count)` takes a starting position and a number of code units to copy. The second argument is not an ending index. To extract the interval from `begin` up to but excluding `end`, the length is `end - begin` after establishing that the positions are ordered and valid.

For `sensor=42`, the key begins at zero and ends before the separator. The value begins one position after the separator and continues to the end. These operations are easy to express once the separator has been checked. Performing them first and hoping that an exception catches every grammar error makes the parser harder to reason about.

An empty substring is not automatically an error. It may be a valid value in some formats. The identifier grammar in this chapter rejects empty text because an identifier requires digits. Keep the domain decision separate from the fact that the string library can represent an empty sequence perfectly well.

A copied substring owns its resulting text. A string view of the same interval would observe the original storage instead. Both can be useful, but they have different lifetime requirements. The worked example stores `name` and `value` as owning strings before passing the value to a nonowning parser view, keeping the owner alive throughout the call.

When diagnosing a bad split, print quoted fields and their lengths rather than relying on an unmarked output line. An empty field or a trailing space is easier to see when its boundary is explicit. The capstone later uses quoted names for the same practical reason.

Locating a Delimiter and Extracting a Substring

Purpose. Search and extraction are linked operations: first prove the boundary exists, then slice relative to that boundary.

```
#include <iostream>
#include <string>
int main(){
    std::string record = "user:charles";
    auto pos = record.find(':');
    if (pos != std::string::npos) std::cout << record.substr(pos + 1) << '\n';
}
```

How it works.

- `find` returns an index or the sentinel `npos`.
- The check prevents treating a failed search as a valid position.
- `substr(pos + 1)` extracts the text after the delimiter.

Expected behavior/output. The program prints `charles`.

Experiment. Remove the colon from the input and verify that nothing is extracted.

11.3 Converting Between Strings and Other Types

11.3.1 Parsing Numeric Values from Text

Numeric parsing has at least three questions: were any characters recognized, was the value representable, and was the entire required field consumed? `std::from_chars` reports an error code and the position where parsing stopped without throwing for ordinary parse failure. For integer parsing, it does not skip leading whitespace; that behavior can be useful for strict field formats.

Check both the error code and that the returned pointer reaches the field's end. Then apply domain constraints such as a positive identifier range. A successful conversion of `42junk` into `42` is not success when the format requires a complete integer. Avoid relying on conversion functions' partial-consumption behavior without examining the remainder.

Numeric parsing needs both conversion success and complete consumption when the contract requires the entire field to be numeric. A conversion routine may successfully parse a prefix and stop before a suffix. Accepting `42junk` as identifier forty-two would violate the chapter's strict field grammar even though the initial digits are valid.

The worked parser uses `std::from_chars` for a bounded integer conversion. It receives a beginning and end pointer, a destination, and returns information containing an error code and the position where parsing stopped. The `auto` declaration lets the compiler deduce that result type; it does not make the object dynamically typed.

For this integer overload and decimal interpretation, leading whitespace is not skipped automatically, and a leading plus sign is not accepted as an ordinary signed decimal prefix. A minus sign can be parsed for a signed destination, but the positive-identifier domain then rejects negative results. Overflow during conversion is reported as an error rather than being treated as a valid wrapped identifier.

The parser first writes into a local candidate. Only after the error code, stopping position, and range checks succeed does it assign to the caller's result. This gives a strong, simple failure contract: a failed parse leaves the earlier result unchanged. Test that property explicitly; returning false alone does not prove that the output parameter was preserved.

Leading zeros are a separate grammar choice. The supplied decimal parser accepts them when the resulting identifier is within range. Document such choices instead of allowing callers to infer them from one example.

11.3.2 Formatting Values as Strings

`std::to_string` provides a simple conversion for common arithmetic values, while streams offer more control over formatting and locale behavior. Stable machine-readable formats need explicit rules for precision, separators, and accepted special values. A human-friendly report and a protocol field need not use the same representation.

Keep formatting at the boundary. Store an identifier as a meaningful numeric or domain type while calculating, then convert it to text for output. Repeatedly parsing and formatting inside the core algorithm adds work and obscures type information. Round-trip tests can be useful, but they must reflect the format's intended precision and normalization rules rather than demanding identical spelling in every case.

Formatting is the reverse boundary: a typed value becomes text according to an output contract. `std::to_string` is convenient for a simple integer identifier. Stream formatting is useful when labels, widths, bases, or floating-point presentation need coordination. Choose one representation deliberately so later parsing or comparison is predictable.

Do not assume every formatting/parsing pair is a perfect round trip for every type and format. A floating-point value printed with a small number of digits may lose information. A formatted identifier with a prefix or thousands separator may not be accepted by a strict decimal parser. The grammar on both sides must agree.

For the identifier exercise, define the output as a name, a colon, and a decimal integer. That is a small, stable format. It does not preserve the input's original spelling if the input contained leading zeros, because the program is reporting the parsed value rather than reproducing the raw text. Decide whether that normalization is desirable for the actual application.

When output is intended for another program, keep diagnostics out of the data stream and avoid decorative changes without revising the interface. A new sentence added before a record can break a consumer that expects the record first. Human-readable output and machine-readable output can coexist, but they need distinct modes or clearly separated channels.

A formatter should not silently repair invalid domain objects. Validate the object before it becomes a record, then make the formatter responsible for a faithful representation of that valid state.

Parsing an Integer with `std::from_chars` and Formatting It Again

Purpose. Conversion must validate lexical success and consumption of the intended input before a numeric value is trusted.

```
#include <charconv>
#include <iostream>
#include <string>
int main(){
    std::string text = "42";
    int value{};
    auto [ptr, ec] = std::from_chars(text.data(), text.data()+text.size(),
value);
    if (ec == std::errc{} && ptr == text.data()+text.size())
        std::cout << std::to_string(value + 8) << '\n';
}
```

How it works.

- `from_chars` parses without locale-dependent stream formatting.
- Both the error code and final pointer are checked so partial input is not silently accepted.
- `to_string` converts the computed integer back to owned text.

Expected behavior/output. The program prints `50`.

Experiment. Change the text to `42x` and observe why the full-consumption check rejects it.

Worked example — Parse a complete numeric field

Requirement and simplest approach

HarborWorks receives the field `sensor=42`. It must split the key and value, verify the expected key, parse a positive identifier no greater than one million, and produce `sensor:42`. Invalid numeric text must not overwrite an earlier accepted result.

The simplest approach finds the separator and converts the suffix without checking how much text was consumed. That can accept a numeric prefix while ignoring malformed data. Another shortcut assigns directly into the caller's result before checking the domain, leaving the caller with a rejected candidate after failure.

The selected design separates structure, conversion, domain validation, and commit. The owning strings hold the split fields. The parser observes the value through `std::string_view`, whose backing string remains alive during the call. A local integer is the candidate state. The final assignment is the commit point.

Read the supporting notation

`text.data()` identifies the beginning of the observed characters, and adding the view's size forms the excluded end. The parser does not read beyond that range. The returned `ptr` identifies where conversion stopped; `ec` reports conversion status. Comparing the stopping pointer with the supplied end checks complete consumption.

These pointer operations are small, controlled uses of an interface studied more fully in Chapter 14. They do not require manual allocation. The parser borrows existing text for the duration of one call and returns an integer value through an explicitly documented output parameter.

Trace the implementation

The parser writes into a local candidate first and updates the caller's result only after every check succeeds. That gives failure a useful guarantee: the previous result remains unchanged. It is an early form of commit-after-validation design, which later appears in resource ownership and transactional updates.

The `string_view` parameter borrows the supplied text only during the call. The function neither stores the view nor returns a pointer into it. Its lifetime requirement is therefore simple: the argument's characters must remain valid for the duration of the call. The empty case is rejected before pointer-range construction.

The field splitter checks for the separator before taking substrings. It also verifies the expected field name instead of assuming that any prefix is acceptable. Strict parsing rejects trailing junk and leading whitespace under this intentionally narrow format. A different interface may permit whitespace, but that should be a deliberate normalization step rather than an accidental side effect.

Complete worked source

source/ch11/main.cpp

```
#include <charconv>
#include <iostream>
#include <string>
#include <string_view>
#include <system_error>

bool parse_id(std::string_view text, int& result) {
    if (text.empty()) return false;
    int candidate{};
    const char* begin = text.data();
    const char* end = begin + text.size();
    const auto parsed = std::from_chars(begin, end, candidate);
    if (parsed.ec != std::errc{} || parsed.ptr != end) return false;
    if (candidate < 1 || candidate > 1000000) return false;
    result = candidate;
    return true;
}

int main() {
    const std::string field{"sensor=42"};
    const auto separator = field.find('=');
    if (separator == std::string::npos) return 1;
```

```

const std::string name = field.substr(0, separator);
const std::string value = field.substr(separator + 1);
int id{-1};
if (name != "sensor" || !parse_id(value, id) || id != 42) return 2;
if (parse_id("42junk", id) || parse_id(" 42", id)) return 3;
if (parse_id("", id) || parse_id("0", id)) return 4;
if (id != 42) return 5; // Failed parses preserve the previous result.
std::cout << name + ":" + std::to_string(id) << '\n';
std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root, build and run this chapter’s supplied fixtures:

```
python3 tools/run_chapter.py 11
```

The executable target is `b01_c11`. Read `source/ch11/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Trace rejection without corrupting prior state

Start with an output integer containing a known valid value. Parse `42`, then attempt `42junk`, a leading-space form, an empty string, and zero. After each rejected attempt, check that the result still contains forty-two. This distinguishes “reported failure” from “reported failure while unexpectedly changing state.”

Add an integer too large for the destination type. The conversion error must be handled before the domain comparison relies on the candidate as a valid identifier. Add the largest permitted identifier and one above it to distinguish representation limits from application limits.

Failure clinic — Absence treated as position zero

A learner uses the search result as a Boolean. A separator at the first position appears false, while the absence sentinel may appear true. The parser then extracts the wrong fields. The library did not reverse its meaning; the caller used the wrong success test. Compare explicitly with `npos`.

Failure clinic — Parsing succeeded, the record did not

Another learner checks only the conversion error code. The prefix of `42junk` is a valid integer, so conversion can report success up to the stopping point. The complete-field contract still fails. The stopping position is part of the result for a reason.

Diagnostic evidence for text

When investigating a malformed field, record its length and an escaped or quoted representation. Invisible whitespace, empty strings, and embedded control characters can disappear in ordinary output.

Do not copy sensitive input records into a public report; reduce the problem to a small synthetic string that preserves the parsing behavior. The book’s fixtures are invented for exactly that purpose.

Practice and implementation lab

Add tests for the maximum valid identifier, one beyond it, a minus sign, a plus sign, and an integer too large for `int`. State which spellings the format permits. Add a test proving that failed parsing does not modify the output parameter.

Replace the owning substring for the value with a view into the original field. Identify which object must remain alive and unmodified while the parser runs.

Starting exercise: solution notes

The domain accepts 1 through 1000000. `from_chars` for a signed integer accepts a minus sign but the domain rejects negative values; a leading plus is not accepted by this integer parser. Overflow produces a conversion error. The output remains unchanged because assignment occurs only after all checks.

A view can refer directly to the field’s character range, provided the owning field is not destroyed or mutated in a way that invalidates its storage during the call. Keeping a view after the field is gone would trade an allocation for a lifetime defect.

Guided lab — Build a strict field parser

Write a grammar sheet for the field before adding features. State the expected key, separator rule, numeric syntax, allowed range, leading-zero policy, whitespace policy, and behavior on failure. A parser with a written grammar is easier to test than one whose accepted language is whatever the conversion function happens to tolerate.

Construct a test table with a correct field, missing separator, empty key, empty value, wrong key, suffix text, leading whitespace, a leading plus sign, a negative value, zero, the maximum, one above the maximum, and a very long integer. For each rejected numeric case, verify preservation of the prior output value.

Extend the program to parse a second named field with a different range. Keep structural splitting separate from domain validation so the two fields can share a mechanism without silently sharing the same policy. A device identifier and a quantity may both be integers but have different accepted domains.

For the formatting exercise, accept a leading-zero identifier and print the normalized decimal value. Explain whether the program is preserving textual identity or numerical identity. Then decide which behavior is appropriate for an equipment identifier in your proposed application.

Acceptance criteria. Missing separators are detected before position arithmetic; conversion success and complete consumption are both checked; range validation is separate; rejected input leaves the previous result unchanged; and every accepted formatting choice is documented.

Review and solution reasoning

Why is `npos` not an ordinary index? It is the library's absence result. Treating it as a location or performing arithmetic on it does not establish a valid substring boundary.

Why use a local candidate? It delays mutation of caller-owned state until all validation succeeds. That makes failure behavior predictable and testable.

Why can a string view be safe here? The owning string remains alive and unchanged for the parser call. The view does not escape beyond that lifetime.

Why distinguish grammar from range? Text can be syntactically numeric while representing an invalid identifier. Each boundary rejects a different class of input.

Chapter takeaway. Text handling is not finished when a conversion returns a number. Validate structure, consume exactly the intended field, enforce its domain, and commit the result only after the complete contract succeeds.

Technical references: [LANG](#) · [STRING](#) · [CHARCONV](#).

Chapter 12 — Managing Dynamic Collections with Standard Containers

A fixed diagnostic frame and a growing inventory list have different size policies. HarborWorks should not allocate a large raw array and hope that it will always be large enough for the inventory. A container can encode the intended policy while managing its own storage.

This chapter establishes a practical default: use `std::array` when extent is a fixed part of the type, and use `std::vector` for a dynamically sized contiguous sequence unless another requirement gives a reason to choose differently.

A collection has at least three properties worth separating: how many elements are logically present, how much storage is available, and who owns that storage. Standard containers make those properties easier to manage, but they do not remove the need to understand them. HarborWorks will use a fixed calibration array and a growing sequence of readings to compare the two models.

Learning outcome. Choose between fixed and dynamic containers, use size and capacity correctly, modify collections without assuming stale access paths remain valid, and validate external growth requests before allocation. The aim is ordinary application ownership, not premature construction of a custom container.

12.1 Using `std::array` for Fixed-Size Collections

12.1.1 Creating Type-Safe Fixed Arrays

`std::array<T, N>` is a standard container containing exactly N elements. Its extent is part of the type, so `std::array<int, 4>` and `std::array<int, 5>` are distinct types. It supports value copying, size queries, iterators, and checked access through `at`.

The elements reside within the array object rather than in a separately managed dynamic buffer. Where that object itself resides depends on its storage duration and ownership. A `std::array` can be a local variable, a class member, or part of a dynamically allocated object. Do not equate the container name with one physical memory region. A zero-element array is representable, but it has no element available to access.

`std::array<T, N>` represents a fixed number of elements through a standard-container interface. The element type and extent are part of its type. Unlike a raw array in many function parameter forms, the object carries its extent through its type and offers operations such as `size`, `begin`, `end`, and `at`.

For a small calibration table whose number of entries is a design constant, this is a natural choice. A `std::array<int, 3>` cannot become a four-element array by appending. If the requirement says the number of calibration values varies at runtime, that is a reason to choose another representation rather than to fight the fixed type.

Initialization still matters. A value-initialized array of integers has a known initial state. An uninitialized automatic array of fundamental values should not be read merely because it is wrapped in a standard type. The container interface does not erase the element type’s initialization rules.

A zero-extent `std::array` can represent a valid empty fixed collection. Iterating from begin to end performs no element work. Operations that require an existing element, such as accessing the first element, still need the relevant precondition. “The container object exists” is not the same as “an element exists.”

Ordinary value copying and assignment are available when the element type supports them. This makes the container easier to use as part of a structured value than a raw array with a separate informal size convention.

12.1.2 Iterating Through `std::array`

Use range-based iteration or standard begin/end iterators to visit array elements. The size does not change, so operations that resize a vector have no equivalent on array. Assigning one array to another of the same type copies or assigns corresponding values according to the element type’s behavior.

Choose a const reference in a loop when observing potentially expensive elements and a nonconst reference when modifying them. A loop variable copied by value does not update the original element. Test that distinction explicitly when learning: incrementing `value` in `for (auto value : values)` changes only the local copy, while `auto&` reaches the stored object.

A range loop expresses traversal without duplicating the extent. For observation, copy small scalar elements or use a const reference when appropriate. For deliberate element modification, a reference loop variable makes the mutation affect the array rather than a temporary copy.

```
std::array<int, 3> calibration{1, 2, 3};
for (int& value : calibration) {
    value *= 2;
}
```

After this loop, the stored elements are two, four, and six. Removing the ampersand would leave the original array unchanged because each iteration would modify its local value. Chapter 13 develops this distinction fully; the example makes the effect visible now.

`operator[]` provides unchecked element access according to its preconditions. `at` provides checked access and reports an out-of-range index through the library’s exception mechanism. A checked operation can be useful at an uncertain boundary, but it does not replace a clear logical bound in a simple internal loop. Choose the operation according to where invalid positions might arise and how the program handles failure.

A fixed extent makes structural invalidation simpler than with a growing vector, but an array can still outlive or be outlived by references to its elements. If the owning array is destroyed, its element references are no longer valid. Fixed shape is not permanent lifetime. Keep representation, mutation, and lifetime as separate properties in the design.

Fixed-Extent Collection with `std::array` Algorithms

Purpose. `std::array` keeps fixed-size semantics while providing the container interface expected by modern library algorithms.

```
#include <algorithm>
#include <array>
#include <iostream>
int main(){
    std::array<int,4> a{4,1,3,2};
    std::sort(a.begin(), a.end());
    for(int v:a) std::cout << v << ' ';
}
```

How it works.

- The size is part of the type `std::array<int,4>`.
- The container exposes iterators compatible with standard algorithms.
- Storage remains embedded in the array object rather than dynamically sized.

Expected behavior/output. The values print as ``1 2 3 4``.

Experiment. Use `a.at(4)` inside a try/catch and compare its checked behavior with unchecked `operator[]`.

12.2 Using `std::vector` for Dynamic Collections

12.2.1 Adding and Removing Elements

`std::vector` owns a contiguous sequence whose logical size can change. `push_back` appends an element, `emplace_back` constructs one from arguments, and `pop_back` removes the final element when the vector is nonempty. Erasing in the middle shifts later elements and has iterator-invalidation consequences.

Do not retain references to elements across arbitrary structural changes. Reallocation can move every element to new storage, and erasure can invalidate positions at or after the removed element. Use indices or stable identifiers when the collection's structure may change, and still validate those indices before use. A vector's ownership simplifies allocation, but it does not make every borrowed access permanent.

A vector owns a sequence whose size can change at runtime. `push_back` appends an element, `pop_back` removes the last element when one exists, and `clear` removes all elements. These operations affect the logical contents; their effects on storage and borrowed access require separate attention.

Check the nonempty precondition before removing the last element or accessing `back`. A vector object can be perfectly valid while containing no elements. Empty input is an ordinary case, not an exceptional accident to ignore until a test crashes.

Initialization syntax can change meaning. `std::vector<int> values(3, 7);` creates three sevens. `std::vector<int> values{3, 7};` creates two elements with values three and seven. Both are useful, but substituting braces mechanically can change the program's data. Read the selected constructor's meaning, not only the punctuation style.

Insertion and erasure can move elements and invalidate positions or access paths. An index may remain within the new size while now referring to a different logical record after erasure. Memory validity and identity stability are different properties. For inventory records, a stable identifier is often a better external reference than "the third element."

Validate application limits before appending. A vector manages storage; it does not know that HarborWorks permits at most one hundred records. The application must enforce that bound and decide how to report a rejected addition without corrupting the existing collection.

12.2.2 Understanding Size and Capacity

Size is the number of constructed elements available to the program. Capacity is the amount of element storage available before another reallocation is required. `reserve(100)` may provide capacity for at least 100 elements without creating them; accessing element 99 is still invalid until the size includes it. `resize(100)` changes the number of elements and initializes new ones as specified.

Capacity growth is an implementation strategy, not an exact doubling promise. Repeated appends have amortized constant complexity under the container's contract, while an individual growth can move many elements. Reserve when a credible upper estimate is known, but do not call reserve before every append in a way that defeats geometric growth.

Size is the number of elements currently present. Capacity is the amount of element storage available before certain growth operations require reallocation. Calling `reserve(8)` on an empty vector does not create eight elements. Its size remains zero, and indexing position zero is still invalid until an element exists.

`resize(8)` changes the logical size to eight, constructing additional elements as required. For integers grown through ordinary value initialization, those new elements have zero values. That does not mean eight external readings have been received. As with the fixed buffer in Chapter 10, storage state and application meaning must not be confused.

The implementation may provide capacity larger than the requested reservation. Test `capacity() >= requested`, not equality with a particular observed growth factor. The standard's vector capacity interface describes guarantees without requiring every implementation to use the same allocation strategy. [Vector capacity](#)

A reserve operation that reallocates invalidates existing pointers, references, and iterators to elements. Reserving at a deliberate setup point can avoid some later reallocations, but it does not grant unlimited validity for every subsequent mutation. Erasing or inserting elements has its own rules.

Do not call `reserve(size() + 1)` before every append as a reflex. Repeatedly defeating the container's growth strategy can harm performance. Reserve when you have a meaningful estimate or known bound, then measure a real workload before introducing more elaborate allocation policy.

Observing `std::vector` Size, Capacity, and Growth

Purpose. A dynamic collection separates logical element count from allocated capacity, which explains both growth behavior and iterator invalidation.

```
#include <iostream>
#include <vector>
int main(){
    std::vector<int> v;
    for(int i=0;i<8;++i){
        v.push_back(i);
        std::cout << "size=" << v.size() << " capacity=" << v.capacity() <<
        '\n';
    }
}
```

How it works.

- `size()` counts constructed elements.
- `capacity()` reports currently reserved element slots.
- `push_back` may trigger reallocation when size would exceed capacity.

Expected behavior/output. Size increases by one each iteration; capacity grows in implementation-defined steps.

Experiment. Call `v.reserve(8)` before the loop and compare the capacity changes.

12.3 Choosing Between Basic Containers

12.3.1 Comparing Fixed and Dynamic Storage

A fixed-size container makes the extent a compile-time property and avoids a separate resizing policy. A vector accepts a runtime size and owns dynamic storage. Both provide contiguous elements suitable for cache-friendly traversal and many library interfaces.

Consider total object size, maximum input size, copying behavior, and ownership. A huge local fixed array may exceed a platform's practical stack budget, while an unnecessarily dynamic tiny object can add allocation overhead. These are workload and platform questions. Avoid memorizing “array is fast, vector is slow” as a design rule; equivalent element loops can have very similar generated code when the surrounding storage policy is appropriate.

A `std::array` owns its fixed element storage as part of the array object. A vector object manages separately allocated element storage as needed. The vector object itself can still be an ordinary automatic local. Describing every vector as “on the heap” collapses the owner object and the storage it manages into one misleading picture.

The distinction matters for lifetime. When an automatic vector is destroyed, it destroys its elements and releases its owned allocation through its resource-management behavior. You do not call `delete` on the

vector's `data()` pointer. That pointer is an observation of storage owned by the container, not a transfer of ownership to the caller.

Fixed storage can make bounds and resource needs predictable when the extent is genuinely fixed. Dynamic storage accommodates input-dependent sizes and ordinary application growth. Neither choice is automatically faster for every workload. Element type, access pattern, allocation frequency, and the surrounding algorithm all matter.

For this volume, prefer a fixed array for a small table whose extent is part of the design and a vector for the inventory's variable number of records. These choices are driven by the requirements, not by a hierarchy in which one container is universally more professional. A well-chosen simple container leaves attention available for validation, interfaces, and correctness.

When memory must be strictly bounded, enforce an application limit and account for all relevant storage. A maximum logical count is useful, but it is not by itself a complete real-time allocation or memory-certification argument.

12.3.2 Selecting Containers Based on Requirements

Write the operations the program actually needs: append, index, traverse, insert in the middle, preserve references, or look up by key. Then choose a representation whose guarantees fit those operations. For a growing list primarily appended and traversed, vector is a strong starting point. For exactly four calibration coefficients, array communicates the invariant directly.

Do not optimize hypothetical operations that the application never performs. A linked list's inexpensive node insertion does not help if every task scans all elements and allocation dominates. Book III compares data structures systematically, and Book V measures locality. Here the goal is selecting a clear ownership and size model before adding complexity.

Write the collection's requirements before choosing its type. Does its extent vary? Must elements be contiguous? Do callers need stable identities? Are insertions mainly at the end? What is the maximum accepted input? The answers may be simple for a beginner application, but asking them prevents cargo-cult container selection.

For HarborInventory, a bounded vector is sufficient. It stores a modest number of records, supports straightforward iteration, and owns their lifetime. A linear search for duplicate identifiers is acceptable at the teaching limit. Replacing it with a linked structure merely because insertion is possible would add allocation and pointer-management complexity without a demonstrated need.

Stable record identity should be modeled explicitly. An identifier belongs to the record; a vector index is a current position. If a later operation sorts or erases records, positions can change. This is a semantic issue even when all memory accesses remain within bounds.

A container decision should also include failure behavior. An append may fail because the application limit is reached or because storage allocation cannot complete. Those are different categories. The capstone reports ordinary validation rejection and documents its broader runtime boundary; it does not promise recovery from every resource-exhaustion scenario.

Choose the smallest interface that satisfies the current requirements and leave a clear boundary around it. Encapsulation in Chapter 17 will make it possible to change representation later without forcing callers to rewrite every operation.

Choosing Fixed or Dynamic Storage from the Requirement

Purpose. Container choice should follow the data-shape requirement: fixed extent versus runtime-variable extent.

```
#include <array>
#include <iostream>
#include <vector>
int main(){
    std::array<int,3> rgb{255,128,0};
    std::vector<int> samples{4,8,15};
    samples.push_back(16);
    std::cout << rgb.size() << ' ' << samples.size() << '\n';
}
```

How it works.

- RGB has a fixed cardinality encoded as `std::array<int,3>`.
- Samples can grow, so `std::vector` models the requirement directly.
- Both support range iteration and size queries, but their growth semantics differ.

Expected behavior/output. The output is `3 4`.

Experiment. Try to “push” a fourth RGB component and explain why the type intentionally has no such operation.

Worked example — Distinguish reserved storage from live elements

Requirement and simplest approach

The lab has three calibration values whose extent is fixed and a sequence of readings that grows as values arrive. It must double the calibration values, reserve space for a small reading batch, append three values, remove the last, and verify the resulting contents.

The simplest shortcut is to call `reserve` and then assign by index as if the elements already existed. That confuses capacity with size. The selected design appends actual elements after reservation and checks both the logical size and a surviving value after removal.

Trace two different kinds of change

The calibration loop changes existing element values but does not change the array's extent. The vector operations change the number of elements and may involve storage management. Treat those as distinct transitions in your notebook.

Immediately after `reserve`, the vector is empty. After three `append`s, its size is three. After one `pop`, its size is two and its second element remains twenty. Capacity is reported as an observation, but the test does not demand a particular exact value. This avoids encoding an implementation's growth choice into the application's correctness contract.

The complete program performs no manual allocation or deallocation. Its owners are ordinary standard-library objects. Chapter 15 will contrast this with raw `new` and `delete`, but the ordinary application path is already resource-managing rather than manually owning pointers.

Trace the implementation

Immediately after `reserve`, the vector remains empty even though it can accommodate several future elements without another allocation. The test checks a lower bound on capacity rather than an exact value because the implementation may reserve more than requested. Only the two remaining appended elements are valid after `pop_back`.

The calibration loop uses a reference to modify the array's elements in place. Replacing `int&` with `int` leaves the stored coefficients unchanged and causes the corresponding check to fail. This makes a semantic choice visible through a concrete test.

The example does not hold an element pointer across `push_back`. A separate experiment can record capacity changes and observe when reallocation happens, but it must stop using any borrowed pointer once invalidation occurs. The address change is an observation, not permission to dereference the old address to see what remains there.

Complete worked source

source/ch12/main.cpp

```
#include <array>
#include <iostream>
#include <vector>

int main() {
    std::array<int, 3> calibration{1, 2, 3};
    for (int& value : calibration) value *= 2;
    std::vector<int> readings;
    readings.reserve(8);
    if (!readings.empty() || readings.capacity() < 8) return 1;
    readings.push_back(10);
    readings.push_back(20);
    readings.push_back(30);
    readings.pop_back();
    if (readings.size() != 2 || readings.at(1) != 20) return 2;
    if (calibration[0] != 2 || calibration[2] != 6) return 3;
    std::cout << "size=" << readings.size()
```

```
        << " capacity=" << readings.capacity() << '\n';  
std::cout << "PASS\n";  
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

```
python3 tools/run_chapter.py 12
```

The executable target is `b01_c12`. Read `source/ch12/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — Reserved storage mistaken for elements

A learner reserves room for eight integers and immediately writes `readings[0] = 10`. The vector's size is still zero, so the access violates the element-access precondition. The fact that storage may have been allocated does not create a logically present integer element at that position through the vector interface.

The repair is to append an element or resize when the requirement truly calls for a sequence of existing elements. Those are different operations with different meanings. Do not pick `resize` merely to silence a crash if the application should be recording actual arrivals one at a time.

Failure clinic — A reference survives in syntax only

Another learner binds a reference to the first element and appends enough values to force reallocation. The reference variable still exists, but its target is no longer the original live element in the relocated vector storage. Reacquire access from the container after a potentially invalidating operation, or redesign the algorithm so borrowed access does not cross that operation.

Evidence about capacity

Run the example with two compilers or standard-library configurations and compare capacity output. A difference can be entirely valid. The test's contract is sufficient capacity after reservation and correct logical contents after operations, not a fixed internal growth ratio.

This distinction is the same one used throughout the Linux template: an observed implementation detail is evidence about one environment, while a documented guarantee is the basis for portable reasoning. Good tests do not accidentally convert incidental observations into universal requirements.

Practice and implementation lab

Call `resize(5)` on an empty vector of integers and compare its size and contents with an empty vector that only received `reserve(5)`. Explain which indices are valid in each case.

Design a collection for exactly eight channel coefficients and another for an unknown number of inventory records. State the size invariant and the operations that justify each choice.

Starting exercise: solution notes

Resizing creates five integer elements, value-initialized to zero in this case, so indices 0 through 4 are valid. Reserving alone leaves size zero and no valid element index. Capacity describes storage, not object existence.

An eight-element array expresses the fixed coefficient count as part of its type. A vector accommodates an inventory discovered at runtime and supports efficient append and traversal. Neither choice guarantees correctness without validation, but each makes the intended size policy explicit.

Guided lab — Model a bounded reading batch

Create a vector that accepts at most ten valid readings. Reject an eleventh without changing the first ten. Test an empty batch, one value, exactly the limit, and one attempted addition beyond the limit. Keep the limit check in application logic rather than assuming the vector's capacity enforces it.

Compare two construction paths: reserve then append, and resize then assign. Explain what each path claims about when elements exist. For a known-length generated sequence, resize may be appropriate. For a stream of accepted observations, append often expresses the model more clearly.

For the fixed-table exercise, copy a `std::array`, modify the copy, and confirm that the original value remains unchanged. Then observe an array through a reference and demonstrate that deliberate mutation affects the original. This prepares the value-versus-alias distinction for Chapter 13.

Finally, record an index for a particular record, erase an earlier record in a copy, and inspect which record now occupies that index. Stay within valid bounds. The experiment demonstrates that a valid position can identify a different object after a structural change. Propose an identifier-based lookup as the semantic repair.

Acceptance criteria. Size and capacity are reported separately; no element is accessed merely because space was reserved; application limits are enforced before mutation; borrowed access does not cross invalidating operations; and record identity is not confused with current position.

Review and solution reasoning

Why does reserve not make indexing valid? It addresses allocation capacity, not the number of constructed elements in the vector's logical sequence.

Why is exact capacity a poor portable assertion? The interface permits implementation choices beyond the requested minimum. Correctness should depend on the guarantee the program actually needs.

Why not delete data() manually? The container owns that allocation. Manually releasing borrowed storage would violate the ownership model and corrupt later container behavior.

Why can an index remain valid but become misleading? Structural operations can move a different record into that position. Bounds validity does not establish stable identity.

Chapter takeaway. Let containers own storage, but keep size, capacity, lifetime, and domain limits distinct. Choose the container from the workload and preserve valid access across each operation.

Technical references: [LANG](#) · [ARRAY](#) · [VECTOR](#).

Chapter 13 — Sharing Existing Objects with References

Copying a long list just to inspect it is unnecessary, while modifying a caller’s list accidentally is a defect. References let the interface distinguish those intentions. HarborWorks needs one function that computes a total without changing readings and another that deliberately applies an offset.

The central question is whether the function owns an independent value or borrows an existing object. Reference syntax is compact; the lifetime and mutation contract behind it must remain explicit.

Copying a value and observing an existing object are different operations. References let an interface express the second operation directly, but the convenience comes with a lifetime obligation. HarborWorks will pass a reading collection to an observing function and to a modifying function, then inspect exactly which object changes.

Learning outcome. Distinguish value parameters from aliases, read const-reference interfaces correctly, preserve the owner’s lifetime, and choose whether a failed modifying operation leaves the original object unchanged. References are introduced as a semantic tool, not as an automatic performance upgrade.

13.1 Understanding Reference Semantics

13.1.1 Binding References to Existing Objects

A reference is initialized to refer to an existing object or another suitable entity. For an ordinary lvalue reference, write `int& alias = value;`. Subsequent assignment through `alias` assigns to the referred-to object; it does not reseal the reference to a different object.

A valid reference must have a valid referent for each use. The language does not provide a normal null-reference state corresponding to a nullable pointer. This makes a reference useful for a required argument, but it is not a magical lifetime guarantee. A reference can still dangle after its object is destroyed or invalidated. Trace the owner even when the parameter syntax looks safe.

A reference declaration binds a name to an existing object. In `int& alias = count;`, `alias` and `count` denote the same integer object for the relevant uses. The reference does not create a second independent integer value that must later be synchronized.

A reference must be bound appropriately when created. It is not a nullable handle that you initialize to “nothing” and fill later. A function taking `int&` ordinarily requires the caller to supply a valid integer object. If the relationship is optional, a pointer or a different explicit representation may be more suitable; Chapter 14 develops that choice.

Once bound, a reference is not reseated by assignment. If `alias` refers to `count`, the statement `alias = other;` assigns the value of `other` to `count`. It does not make `alias` start referring to `other`. This distinction is one of the most useful small experiments in the chapter.

The target must remain alive while the reference is used. A reference to an element of a vector depends not only on the vector object's lifetime but also on operations that can invalidate that element access. The reference's own scope does not preserve the element through reallocation.

Think of binding as establishing a relationship with an owner, not acquiring ownership. Write down which object supplies the lifetime guarantee and what operations could end or invalidate that guarantee before using a reference beyond a short local expression.

13.1.2 Understanding References as Aliases

An alias is another way to access the same object. Observing a mutation through the original name and through the reference should therefore give the same stored value. This differs from a copy, which is an independent object with its own state.

Avoid turning that mental model into unsupported layout claims. A reference may be implemented with a pointer-like mechanism in some contexts, or optimized away in others; its specified semantics are what matter. You cannot treat a reference as a separately reseatable pointer object. Use a pointer or reference wrapper when a design genuinely needs an indirect access path that can change targets.

An alias provides another way to access the same state. That can make code clearer when a long expression identifies the object of interest, but it can also make mutation harder to track if several names appear unrelated.

Consider a reference to the first reading. Assigning through that reference changes the vector element, so a later total observes the new value. There is no separate “copy back” step. This is exactly why reference parameters can express caller-visible modification.

Aliasing can also create surprising interactions between parameters. A function receiving two references may be called with the same object for both. If the algorithm reads one after modifying the other, the order can affect the result. A robust interface should either support this relationship or state a justified precondition.

The word “reference” is sometimes used informally for any indirect access. In C++, a language reference and a pointer have distinct syntax and capabilities. Both can observe existing objects, but a pointer is itself a value that can be null or redirected under its type rules. Do not transfer every pointer habit to references or every reference assumption to pointers.

For ordinary application code, keep aliases close to their use and give them role-based names. A temporary `current_record` reference inside a loop is easier to reason about than a long-lived alias stored elsewhere while the collection is repeatedly modified. The smaller the borrowing interval, the fewer lifetime and invalidation events must be considered.

Two Names Referring to the Same Object

Purpose. A reference is an alias, not a second independent integer or a nullable handle.

```
#include <iostream>
int main(){ int value=10; int& alias=value; alias=25; std::cout << value << ' '
<< alias << '\n'; }
```

How it works.

- `alias` must bind to an existing `int` when declared.
- Assignment through the reference modifies the original object.
- Reading through either name observes the same stored value.

Expected behavior/output. The output is `25 25`.

Experiment. Create a second reference to `value` and modify through it; all aliases should observe the change.

13.2 Using References in Functions

13.2.1 Modifying Caller-Owned Data

A nonconst reference parameter makes mutation possible and should be used when changing the caller's object is part of the operation's purpose. Name that purpose clearly, such as `apply_offset(readings, amount)`. Validate prerequisites before making partial changes whenever a failure would leave the caller in an inconsistent state.

Two reference parameters can alias the same object unless the interface forbids it. A function that assumes distinct objects may fail when called with the same argument twice. State and test aliasing assumptions. For simple arithmetic updates, a local temporary can sometimes make the behavior correct even when input and output refer to the same object.

A nonconst reference parameter communicates that the function may modify the caller's object. The parameter alone does not describe **which** modification is allowed, so the contract must supply that meaning. `apply_offset` changes every reading by a stated amount; it does not resize the collection or reorder elements.

This narrower contract helps callers preserve other assumptions. If the function only changes element values, an existing position still describes the same element. If it can also append or erase, callers must reconsider both validity and identity. Do not use one vague "may modify" statement to conceal several unrelated operations.

Failure behavior matters. An offset operation that checks and updates one element at a time might change the first elements before discovering that a later addition would exceed the permitted range. Returning false then leaves a partially modified collection. That may be acceptable in some streaming designs, but it is not the same as all-or-nothing validation.

For a small bounded collection, a two-pass design can validate every proposed change first and perform the updates only after all checks succeed. Another design creates a separate candidate collection and replaces the original after validation. These choices have different storage and cost implications. The contract, not a preference for references, determines the appropriate approach.

The worked source uses deliberately small values whose additions and totals are representable. Its simple `apply_offset` is not a general checked numeric library. The lab asks you to make the stronger rejection-and-preservation contract explicit.

13.2.2 Avoiding Expensive Copies

Passing a large container as a const reference avoids copying its elements merely to inspect them. The function borrows the container for the call and must not retain that reference beyond the owner's lifetime. A const reference also avoids mutation through that particular parameter.

The benefit depends on type and use. Passing a small integer by const reference often adds indirection without useful savings. If a function needs to take ownership of a value or store an independent copy, a value parameter may express that requirement more naturally. Modern move semantics later make ownership transfer efficient; references should not be used to disguise an operation that actually needs ownership.

A const reference can let a function inspect an existing larger object without making an ordinary value copy of that object. For a vector, this is often a clear observing interface: the caller owns the collection, and the callee reads it during the call.

Avoid turning this into the rule that every parameter should be a reference. Small scalar values are inexpensive and easy to reason about by value. A value parameter also isolates the callee's local changes. The appropriate choice depends on size, ownership, lifetime, and whether the function needs an independent value.

A copy is not always wasted work. A function that sorts a collection without changing the caller's original may deliberately take or create a value copy. That copy supports a semantic requirement. Removing it for a presumed optimization can change the interface from "produce a sorted value" to "reorder my existing data."

Performance comparisons must preserve the same behavior. Comparing a copying function that leaves input unchanged with a reference function that mutates input is not a fair measurement of one implementation choice; they provide different services. Establish equivalent contracts before interpreting timings.

Returning values is also not automatically inefficient. Modern C++ has rules and implementation techniques that can avoid unnecessary copying in many return paths. Book II develops value and move semantics. At this stage, do not create a dangling reference merely to avoid a hypothetical copy. A clear valid ownership model comes before speculative savings.

Mutating Caller State Through an Output Reference

Purpose. Reference parameters allow functions to operate directly on caller-owned objects when mutation is part of the contract.

```
#include <iostream>
void clamp_to_zero(int& value){ if(value < 0) value = 0; }
int main(){ int balance=-5; clamp_to_zero(balance); std::cout << balance <<
'\n'; }
```

How it works.

- The parameter aliases the caller's `balance`.
- The function can update the existing object without returning a replacement.
- The conditional mutation is visible to the caller immediately after the call.

Expected behavior/output. The program prints `0`.

Experiment. Change the parameter to pass-by-value and verify that the caller's `balance` remains -5.

13.3 Protecting Referenced Data

13.3.1 Using const References

A const reference prevents modification through that access path. It does not make the object globally immutable if another nonconst alias exists. It also does not automatically make concurrent access safe: another thread changing the object without synchronization can still cause a data race.

Const references can bind to some temporaries, with lifetime-extension rules that depend on the exact context. Do not generalize those rules into “a reference keeps anything alive.” A function storing a reference to a temporary argument can leave a dangling member after the full expression ends. Prefer owning storage when an object must retain data after a call.

A const reference restricts modification through that access path. It does not necessarily make the underlying object globally immutable. Another nonconst alias may still modify the same object when the language and program's synchronization rules permit it. This is why constness and ownership are related but different concerns.

For a function such as `total(const std::vector<int>& values)`, the interface says that this function observes the vector rather than changing it through `values`. It does not promise that the total is cached, that other threads cannot modify the vector, or that the reference remains valid after the call returns. Those would require additional design guarantees.

Const-reference parameters can bind to some temporary values, which makes many calls convenient. The temporary's permitted lifetime for the call does not mean the function may store the reference for later

use. An observing function should normally finish its observation before returning unless a longer-lived ownership relationship is explicitly established.

Do not cast away constness as a routine way to reuse a modifying function. That defeats the interface's stated restriction and can lead to invalid behavior when the underlying object is genuinely const. Refactor the shared calculation or provide appropriate overloads instead of treating const as an obstacle to be bypassed.

A useful review question is: "What can this function change through each parameter?" The answer should be visible from the type and refined by the documented contract, not discovered by surprise after the call.

13.3.2 Choosing Value or Reference Parameters

Use values for small independent inputs and for intentional ownership or copying. Use const references to inspect larger existing objects without copying. Use nonconst references for required, explicit mutation. Use pointers when nullability or reseating is meaningful, and express ownership separately rather than inferring it from punctuation alone.

These are design defaults, not inflexible laws. A lightweight view such as span can pass by value while borrowing many elements. A reference to a large object can be cheap to pass but dangerous to retain. Evaluate size, lifetime, mutation, ownership, and aliasing together. A signature should help a caller answer those questions without reading the implementation.

Choose value when the function needs a small input or an independent working value. Choose const reference when it observes an existing object whose copy is unnecessary and whose lifetime covers the call. Choose nonconst reference when it deliberately modifies a required caller-owned object. Use a different explicit representation when absence or ownership transfer is part of the requirement.

These are starting points, not absolute laws. An API can accept a value intentionally so that callers may provide either a copy or a movable temporary, as Book II explains. The beginner's task is to recognize the semantic choice before considering its advanced optimization possibilities.

Document aliasing when it can affect behavior. A function that computes from one vector and writes into another may receive the same vector twice unless the interface prevents or rejects it. A two-phase algorithm or an independent candidate can make that case safe. An unexplained assumption that "input and output are obviously different" is not a contract.

Keep returned references especially deliberate. Returning access to a member of a longer-lived object can be appropriate, but callers must understand the owner's lifetime and invalidation rules. Returning a reference to a local temporary is not a valid shortcut. Chapter 17 will use observing member functions with explicit ownership boundaries.

The practical decision sequence is meaning first, validity second, cost third. A fast parameter choice that changes the promised behavior or exposes an expired object is not an improvement.

Read-Only Access with `const` References

Purpose. A `const&` expresses efficient read-only borrowing, which is a central tool for protecting referenced data.

```
#include <iostream>
#include <string>
void print_length(const std::string& s){ std::cout << s.size() << '\n'; }
int main(){ std::string name="C++"; print_length(name); }
```

How it works.

- The reference avoids copying the string.
- `const` prevents the function from modifying the referenced string through this parameter.
- The caller retains ownership and lifetime responsibility.

Expected behavior/output. The program prints `3`.

Experiment. Attempt `s += "20"` inside the function and inspect the compiler diagnostic.

Worked example — Observe and update the same collection through different contracts

Requirement and simplest approach

The lab has readings one, two, and three. It must compute a total without modifying the collection, add an offset of two to each element, compute a second total, and then deliberately replace the first element through an alias.

The simplest approach passes the vector by value to every function. That preserves the original during observation but also prevents an intended modifying function from changing the caller's readings unless it returns and replaces the collection. Another shortcut uses nonconst references everywhere, making an observing function's mutation permissions unnecessarily broad.

The selected design uses const reference for the total and nonconst reference for the offset. The local element reference is created only after the vector's structural operations are complete. No append or erase occurs while that reference is used.

Trace object identity through the calls

Before the offset, the sum is six. The modifying call changes the original elements to three, four, and five, so the new sum is twelve. Binding `first` to the first element and assigning ten changes that original element. A separately copied integer would not have that effect.

The key observation is not the existence of an ampersand. It is the identity of the object being accessed at each step. Draw one vector with three elements and show the observing and modifying access paths pointing to that same owner. Do not draw a second vector unless a value copy actually occurs.

Trace the implementation

`total` borrows the vector as a const reference and returns an independent integer. `apply_offset` borrows it mutably because changing elements is the advertised operation. The loop's reference variable reaches the actual element rather than a copy.

The final alias `first` demonstrates that assigning through a reference changes the element observed through the vector. The vector is not resized or erased while that reference is in use, so no structural invalidation occurs. A later `push_back` that reallocates would require abandoning the old reference before further use.

The arithmetic is intentionally bounded by small test data. A general offset operation needs an overflow policy and may need to validate every proposed result before committing any updates. That issue is separate from whether the vector was passed efficiently. Avoid treating a low-copy interface as proof of a robust numeric contract.

Complete worked source

source/ch13/main.cpp

```
#include <iostream>
#include <vector>

int total(const std::vector<int>& values) {
    int result{};
    for (int value : values) result += value;
    return result;
}

void apply_offset(std::vector<int>& values, int offset) {
    for (int& value : values) value += offset;
}

int main() {
    std::vector<int> readings{1, 2, 3};
    const int before = total(readings);
    apply_offset(readings, 2);
    const int after = total(readings);
    int& first = readings.at(0);
    first = 10;
    if (before != 6 || after != 12 || readings[0] != 10) return 1;
    std::cout << "before=" << before << " after=" << after << '\n';
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

The executable target is `b01_c13`. Read `source/ch13/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — The loop that modified copies

A learner writes a modifying function taking a vector by reference but uses `for (int value : values)` inside it. The function can access the original collection, yet each loop variable is still a local integer copy. Incrementing the loop variable does not update the element.

The repair is to use an element reference when mutation is intended. This example shows that reference semantics must be traced through each boundary. A reference parameter does not automatically make every local variable inside the function an alias.

Failure clinic — A failed operation left half the data changed

A stronger offset function rejects values that would exceed a permitted range. If it updates early elements before checking later ones, failure can leave a mixed state. A caller that retries with another offset may then apply the operation to an already partly changed collection.

The all-or-nothing repair validates every candidate first. Only after that phase succeeds does mutation begin. For fixed integer bounds, safe precondition comparisons can avoid forming an overflowing addition. The lab's solution pattern checks positive offsets against the upper bound and negative offsets against the lower bound before performing any addition.

Borrowing boundary

Do not keep `first` across an operation that might reallocate the vector. A reference cannot be checked against null to determine whether its target is still valid. Its validity must follow from the owner and operation sequence. Shortening the reference's scope is often the simplest way to make that guarantee apparent.

This is the same discipline as keeping a Linux file descriptor or mount assumption within a verified operating context: an access path is useful only while the conditions that justify it still hold.

Practice and implementation lab

Change `apply_offset` to take its vector by value and predict which tests fail. Then add an observer that returns the largest reading without modifying the collection and specify what happens for an empty collection.

Explain why a `const` reference to a vector does not guarantee that another thread cannot modify the same vector. Identify which additional concept is needed for concurrent access.

Starting exercise: solution notes

A value-taking update modifies only its local copy, so the caller's post-update total remains 6 rather than 12. An empty maximum has no ordinary numeric answer; use an explicit success indicator now or an optional result in Book II.

Const constrains one access path, not every alias. Concurrent mutation requires a synchronization and ownership design, such as a mutex protecting the shared object or immutable snapshots published safely. Reference syntax alone does not establish a happens-before relationship.

Guided lab — Make mutation transactional

Extend the offset operation to return success or failure and preserve the entire vector on rejection. State a permitted element range. Validate all proposed results before applying the offset. Include a fixture where only the last element would fail; that case exposes a one-pass implementation that has already changed earlier elements.

Test zero offset, positive offset, negative offset, an empty vector, exact upper and lower boundaries, and one-step overflow of the domain. Keep the arithmetic checks themselves within representable ranges. For a positive offset, compare against an appropriately computed upper threshold before addition; for a negative offset, compare against the lower threshold.

For the alias experiment, bind a reference to one integer, assign another integer through it, and confirm that the reference did not reseal. Print or test both underlying integers to identify which object changed. Then repeat with a pointer in Chapter 14 to compare the two models.

For the interface review, classify several functions as value-producing, observing, or modifying. Rewrite one misleading interface so its parameter types and name agree with its actual responsibility. Preserve the previous behavioral tests unless the requirement deliberately changes.

Acceptance criteria. Observing functions do not mutate through their inputs; modifying functions change the intended owner; a failed batch update leaves all elements unchanged; aliases remain within valid lifetimes; and parameter choices are justified by meaning rather than a blanket speed claim.

Review and solution reasoning

Why does assignment through a reference not reseal it? The reference remains bound to its original target; assignment changes that target's value according to the target type's operation.

Why can const reference still observe a changing object? Constness restricts that access path. Other permitted aliases may affect the underlying state, so broader immutability requires a broader design.

Why test a failure at the last element? It reveals whether earlier elements were modified before the operation discovered the rejection condition.

Why keep element references short-lived? Fewer intervening operations can invalidate the target, making the lifetime argument easier to inspect.

Chapter takeaway. A reference is a relationship to an existing object. Make the owner, permitted mutation, failure behavior, and borrowing interval explicit before treating that relationship as safe.

Technical references: [LANG](#) · [LIFETIME](#) · [GUIDELINES](#).

Chapter 14 — Working Directly with Memory Through Pointers

A pointer is an access path whose validity depends on more than its numeric value. HarborWorks uses one to represent an optional selected reading. Null means no selection, but a nonnull pointer is safe only while the selected object remains alive and accessible.

This chapter teaches pointer operations without treating raw pointers as automatic owners. Most raw pointers in well-structured interfaces are borrowers. The program must know who owns the object and what event invalidates the borrow.

Pointers make indirect access explicit. They are essential to understanding many C++ interfaces, but an address is not a complete description of a valid object. HarborWorks will select an existing reading through a pointer, modify that reading, and traverse a bounded array using beginning and end pointers.

Learning outcome. Distinguish a pointer object from its target, use null as an explicit absence value, preserve lifetime and bounds, and explain why a nonnull check is necessary in some interfaces but never a universal proof of validity. No pointer in the worked example owns or allocates storage.

14.1 Understanding Memory Addresses

14.1.1 Obtaining an Address with &

The address-of operator `&` obtains a pointer to an object, as in `int* selected = &reading;`. The pointed-to type determines what object can be accessed through the pointer. The pointer itself is a separate object that can be copied or changed to refer elsewhere.

An address is meaningful within an execution environment, not a durable identifier for storage or a network protocol. Processes have their own address spaces, and allocation can reuse addresses after objects die. Do not write pointer values to a file and expect them to identify the same objects after restart. Use stable logical identifiers when identity must survive beyond one object's lifetime.

The address-of operator `&` obtains a pointer to an existing object in the ordinary cases used here. In `int* selected = &reading;`, `selected` is a pointer object and `reading` is an integer object. The pointer stores an access value that identifies the target; it is not the target integer itself.

This explains why changing the pointer and changing the target are different operations. Assigning another valid address to `selected` changes which object it observes. Assigning through `*selected` changes the observed integer. A diagram should show both objects rather than drawing one box labeled “pointer/value.”

An address printed during a run is implementation and execution context, not a stable identifier for a record. Address-space layout, allocation, and object lifetimes can change between runs. Do not store a printed address in a file and expect it to identify the same object in a later process.

Declare one pointer per line when the distinction matters. In a declaration containing several names, the `*` syntax can make it easy to misread which names are pointers. Clear declarations are cheaper than debugging an incorrect type assumption.

The ampersand also appears in reference declarations, but its role differs by context. `int& alias` declares a reference type; `&reading` is an expression obtaining an address. Read the grammatical context before deciding what the symbol means. C++ reuses several symbols across declarations and expressions, so punctuation alone is not a sufficient mental model.

14.1.2 Dereferencing an Address with `*`

The indirection operator `*` accesses the object a valid pointer points to. `*selected = 7;` modifies that object; `selected = other;` changes the pointer's target. These operations have different effects even though both contain assignment.

Dereferencing requires a suitable live object and valid access permissions under the language's rules. A nonnull pointer can still dangle, point one past an array, be misaligned, or refer to an incompatible representation. Do not repair an unexplained crash by casting until the types match visually. Establish provenance, lifetime, bounds, and type before accessing the object.

Dereferencing a valid pointer with `*` gives access to the object it points to. The operation is valid only when the pointer identifies an appropriate live target for that use. A null pointer, an expired target, or an out-of-bounds position does not become safe merely because the syntax compiles.

Parentheses can clarify which object an operation affects. `++(*selected)` increments the pointed-to integer. `++selected` advances the pointer according to pointer-arithmetic rules. A compact expression such as `*selected++` can obscure that distinction for a beginner. Prefer the explicit form until you can explain both the value result and side effect.

Pointer constness has two common dimensions. `const int* p` permits changing the pointer while prohibiting modification of the integer through that access path. `int* const p` fixes the pointer's stored address while permitting modification of the target integer when otherwise valid. `const int* const p` restricts both through that name.

These declarations do not establish ownership or synchronization. A pointer-to-const can still dangle, and another permitted alias may still modify the underlying nonconst object. Constness describes allowed access through a type, not the entire history of the storage.

Before dereferencing, state the target's owner and valid range. In this chapter, the owner is the local array, which remains alive for the complete pointer use. That simple ownership relationship is the safety argument; the pointer's printed address is not.

Inspecting an Object Address Through a Pointer

Purpose. Pointers make addresses explicit while the pointed-to type tells the compiler how to interpret the referenced storage.

```
#include <iostream>
int main(){ int value=42; int* p=&value; std::cout << "value=" << *p << " same="
<< std::boolalpha << (p==&value) << '\n'; }
```

How it works.

- `&value` obtains the object's address.
- `p` stores that address with type `int*`.
- `*p` dereferences the address to access the original integer.

Expected behavior/output. The program prints `value=42 same=true`.

Experiment. Assign `*p = 99` and verify that reading `value` produces 99.

14.2 Representing Optional or Indirect Objects

14.2.1 Initializing Pointers with `nullptr`

`nullptr` represents a null pointer value and communicates absence more clearly than an integer zero used for that purpose. Initialize optional pointers explicitly so that the absence state is defined. A pointer's nullability belongs in the interface contract: can the caller omit the object, and what does the function do when it is absent?

A null pointer does not own an object and must not be dereferenced. A nonnull raw pointer does not automatically own its target either. Keep those two concepts separate. Later smart pointers encode ownership, while optional values encode absence without necessarily using indirection. Choose the representation that matches the domain rather than using pointers for every missing value.

`nullptr` is the language's explicit null pointer literal. It represents the absence of a target in an appropriate pointer context. Prefer it to using integer zero as a pointer spelling because its type and intent are clearer, especially when overloads are involved.

A default-initialized raw pointer without an initializer is not automatically a safe absence value. Use `int* selected = nullptr`; or an equivalent clear initialization when no target has yet been selected. As with ordinary integers, initialization establishes state but does not validate an eventual assignment.

Optionality belongs in the interface. A function accepting a pointer may permit null to mean “no result,” or it may require a nonnull pointer. The type alone does not specify every precondition, so document the convention. A reference is often appropriate when a valid existing object is required and absence would make no sense.

Null is not an error code with unlimited meanings. If null might mean missing record, invalid index, allocation failure, or expired data, callers cannot distinguish those outcomes without another channel. For the small selection example, null means that no in-range element was selected. More expressive error types are introduced in Book II.

Assigning null to one pointer after releasing a resource affects only that pointer object. It does not clear other aliases that copied the old address. This is why nulling a pointer is a useful local cleanup convention but not a complete solution to dangling-pointer problems.

14.2.2 Checking Pointer Validity Before Dereferencing

An `if (pointer != nullptr)` check establishes only that the pointer is not null. It cannot prove that the object is still alive. Correct pointer use depends on a design that bounds the pointer's lifetime relative to the owner's lifetime and prevents invalidation while access is in progress.

For a pointer into a local array used within the same block, the relationship is straightforward. For a pointer stored in a long-lived object or passed between threads, it is not. Document whether a callee retains the pointer, whether a container may reallocate, and whether another thread can destroy the object. Prefer interfaces that make those relationships difficult to violate.

A null check answers one question: does this pointer contain the null value? It does not prove that a nonnull pointer refers to live, correctly typed, accessible storage. A pointer can remain nonnull after its object is destroyed, after a vector reallocates, or after it moves outside the permitted array range.

The validity argument therefore has several parts. The pointer must originate from an appropriate object or interface. The object must still be alive. The operation must stay within the permitted bounds. The access type and constness must be suitable. In this introductory example, those conditions follow from selecting an element of a known local array and avoiding structural mutation.

Do not invent a generic function that tries to test arbitrary addresses by reading them. Attempting the read is already the operation whose validity is in question. Operating-system memory mapping information would not establish C++ object lifetime or type correctness either.

For a selection function, validate the index before forming the element pointer. Returning null for a rejected index is clear. Forming an out-of-range pointer first and checking afterward can already violate the pointer-arithmetic rules. The guard must protect the operation, not merely the later dereference.

A practical design technique is to keep optional pointer selection close to its use. The longer the gap, the more intervening operations can invalidate the target. If the access must persist, the interface needs a stronger ownership or stable-identity design than a casual local nonnull check.

Using a Nullable Pointer to Represent “No Selected Object”

Purpose. Pointers are appropriate when an interface needs identity or an explicit “no object” state and ownership remains elsewhere.

```
#include <iostream>
struct Item{ int id; };
void print(const Item* item){
    if(item) std::cout << item->id << '\n';
    else std::cout << "none\n";
}
int main(){ Item x{7}; print(&x); print(nullptr); }
```

How it works.

- A pointer can hold an object address or the null pointer value.
- The null check occurs before member access.
- ``->`` combines dereference and member selection for the non-null case.

Expected behavior/output. The output is ``7`` followed by ``none``.

Experiment. Remove the null check only as a thought experiment and explain why dereferencing ``nullptr`` is invalid.

14.3 Understanding Pointer Arithmetic

14.3.1 Moving Between Contiguous Elements

Pointer arithmetic is defined within an array object and its one-past position. Adding one to a pointer to an element advances by one element, not one byte, according to the pointed-to type. The one-past pointer is useful as an end sentinel but must not be dereferenced.

Subtracting suitable pointers into the same array gives an element distance. Do not subtract pointers from unrelated allocations and interpret the result as a portable distance. A scalar object has limited array-like treatment for these rules, but that does not make neighboring unrelated objects part of one collection. Standard iterators and spans express many of these boundaries more clearly.

Pointer arithmetic advances in units of the pointed-to element type. Adding one to an `int*` that points into an array advances to the next integer element, not the next raw byte. This is why a beginning pointer and excluded-end pointer can describe a range of elements.

A pointer one past the final element may be formed for permitted range operations, but it must not be dereferenced as an element. The loop in the worked source compares the current pointer with that end and stops before accessing it. The half-open interval from Chapter 10 appears again in a different notation.

Subtraction between appropriate pointers into the same array can express an element distance. Do not subtract unrelated pointers or compare their numeric-looking printouts to infer a useful distance. Object adjacency in one run is not a license to traverse between unrelated objects.

The array itself supplies the relationship that makes the arithmetic meaningful. A bare pointer does not carry its count. If a function receives a pointer range, the caller must provide a valid related beginning and end or another explicit size convention. Later range abstractions package these ideas more conveniently.

In beginner code, prefer an index or range-based loop when it states the task more clearly. This chapter uses pointer traversal to teach the mechanism, not to claim it is superior for every sum. A correct higher-level loop often communicates bounds and ownership more directly.

14.3.2 Recognizing Pointer Arithmetic Hazards

Forming a pointer outside the permitted array range can already be invalid, even before dereferencing it. Therefore “I will check it after adding the index” is not always a safe bounds strategy. Validate the index against the known size before forming the resulting element pointer.

Arithmetic also depends on a correct element type and a valid backing object. Casting a byte buffer to a larger type does not automatically satisfy alignment, object-lifetime, or aliasing requirements. Those advanced representation issues appear in Book IV. For ordinary collection processing, retain a pointer together with its extent or use a standard view that carries both.

Pointer arithmetic has a validity boundary before dereferencing. Forming a pointer far beyond an array and promising never to read through it is not a general safe technique. Keep every intermediate pointer within the permitted array positions, including the one-past end where allowed.

An off-by-one loop can therefore fail in two ways: it may dereference the excluded end, or it may advance beyond the permitted range. Trace both the access and the update. A loop that checks too late can perform an invalid operation before its condition notices the mistake.

Pointers into nested arrays also follow the relevant array-object boundaries. Do not flatten a multidimensional raw array by assuming that any arithmetic from the first scalar element may cross every row. The nested traversal from Chapter 10 expresses the actual structure without that assumption.

A pointer to a vector element is additionally subject to the vector’s invalidation rules. Even if the old allocation’s address still looks readable, the pointer is not a valid access path to the new relocated element after reallocation. Reacquire access or avoid holding the pointer across the change.

Do not release storage through a pointer unless the ownership contract says you own the exact allocation in the required form. An element pointer, a one-past pointer, or a pointer to a local object is not a **delete** target merely because its type is `int*`. Chapter 15 distinguishes observing access from dynamic ownership explicitly.

Walking Within One Array Using Pointer Arithmetic

Purpose. Pointer arithmetic is defined relative to array elements, which is why element type and bounds matter.

```
#include <iostream>
int main(){
    int a[]{10,20,30};
    int* p=a;
    for(std::size_t i=0;i<3;++i) std::cout << *(p+i) << ' ';
}
```

How it works.

- The array expression supplies a pointer to its first element.
- Adding ``i`` advances by ``i`` elements, not by raw bytes.
- Dereferencing is valid only while the resulting pointer refers within the same array object (or one-past for comparison, not dereference).

Expected behavior/output. The program prints ``10 20 30``.

Experiment. Print ``p+1 - p`` and confirm the difference is one element.

Worked example — Use a nullable borrower and a half-open pointer range

Requirement and simplest approach

The lab contains readings three, six, and nine. It must select index one when valid, increase that reading by one, and total the resulting range. The expected modified reading is seven and the total is nineteen.

The simplest approach immediately forms `&readings[index]` and dereferences it. That assumes the index is valid. The selected design starts with null, checks the index against the array size, and only then forms the element pointer. It checks the optional selection before use and keeps the array alive throughout.

Follow the two objects

Before selection, the pointer has no target. After validation, it observes the second array element. Incrementing the pointed-to value changes the array from three, six, nine to three, seven, nine. The pointer object still identifies that same element.

The traversal then uses a beginning pointer and a one-past end pointer. The current pointer visits exactly three live elements and is compared with the excluded end before each dereference. The program neither allocates nor releases memory. Every pointer is a borrowed access path into the local array.

This is a deliberately stronger explanation than “the null check makes it safe.” The null check handles optional selection, while the array’s lifetime and the range construction establish the other required conditions. Each part of the safety argument answers a different question.

Trace the implementation

The index is checked before the element address is formed. The selected pointer borrows an element of the local array; it does not allocate or free anything. All uses occur while the array is alive, and the array cannot reallocate.

The traversal uses a half-open range from the first element to one past the last. The loop compares with the end before dereferencing, so the sentinel is never read as an element. Incrementing the last valid element pointer produces the permitted one-past pointer and terminates the loop.

The selected element changes from 6 to 7, making the total 19. This confirms that indirection changes the underlying object. A copy of the integer would not have the same effect. The example does not print addresses as expected test output because address values can vary across runs and are not part of the behavior being taught.

Complete worked source

source/ch14/main.cpp

```
#include <iostream>
#include <iterator>

int main() {
    int readings[]{3, 6, 9};
    int* selected = nullptr;
    const std::size_t index{1};
    if (index < std::size(readings)) selected = &readings[index];
    if (selected == nullptr) return 1;
    *selected += 1;
    int sum{};
    const int* begin = readings;
    const int* end = readings + std::size(readings);
    for (const int* current = begin; current != end; ++current) {
        sum += *current;
    }
    if (readings[1] != 7 || sum != 19) return 2;
    std::cout << "sum=" << sum << '\n';
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter’s supplied fixtures:

```
python3 tools/run_chapter.py 14
```

The executable target is `b01_c14`. Read `source/ch14/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — Nonnull but expired

Suppose the array were replaced with a vector, a pointer were selected, and then enough elements were appended to force reallocation. Testing that pointer against null afterward would not detect the invalidation. The pointer's value did not automatically update to the vector's new storage.

The repair is to avoid the intervening invalidation or reacquire the element by a valid index or identifier after the change. Do not attempt to repair the pointer by guessing the difference between old and new printed addresses. Ownership and valid operations, not numeric address arithmetic, determine the correct access path.

Failure clinic — The pointer moved instead of the value

Changing `++(*selected)` into `++selected` advances the pointer rather than modifying the selected reading. The program may still remain within the array, so it can produce a plausible but wrong total. A test must inspect the intended element change, not merely whether execution completed without a crash.

Failure clinic — A missing optional-result branch

A selection function returns null when the index is invalid, but the caller dereferences unconditionally. The callee honored its contract; the caller ignored it. As with the sentinel result in Chapter 8, a representation only helps when callers handle every permitted outcome.

The review question is therefore two-sided: does selection return a valid pointer or the documented absence value, and does every caller distinguish those cases before accessing a target?

Practice and implementation lab

Set the selected index equal to the array size and require a clean rejection before address formation. Write a function that accepts a pointer and count, and state whether a null pointer with count zero is allowed by your function's contract.

Explain why replacing the array with a vector and appending after taking `selected` may invalidate the pointer. Identify a safe alternative for locating the same logical record later.

Starting exercise: solution notes

An index equal to size is outside the element range. A pointer/count function can explicitly permit an empty range, but its implementation must avoid invalid pointer operations and dereferences on that path. A contract should define this rather than relying on caller intuition.

A vector append can reallocate its backing storage. A stable record identifier followed by a fresh lookup avoids retaining the invalidated address. An index can also work when ordering and erasure rules preserve its meaning, but it still needs validation before reuse.

Guided lab — Select without escaping the range

Extract the index-selection logic into a function that receives a valid array range and a candidate position according to a documented interface. Return an element pointer for an accepted position and null otherwise. Test the first position, the last position, the excluded end, and a large invalid position. Do not form a rejected element pointer before checking its bound.

For signed external input, add a validation boundary before conversion to a size type. Test negative one explicitly. Keep the parsing layer separate from the selection mechanism so each has a clear contract.

Compare three ways to calculate the same sum: index traversal, range-based traversal, and pointer traversal. Use identical values and expected results. Explain the bounds and lifetime argument for each rather than declaring one inherently faster. This is a correctness comparison, not a benchmark.

For the pointer-versus-reference exercise, bind each to an integer and then assign another integer or address as appropriate. Observe which operation changes the original value and which changes the pointer's target. Keep all target objects alive, so the experiment teaches semantics without relying on invalid memory.

Acceptance criteria. Optional selection is handled; indices are validated before pointer formation; the excluded end is never dereferenced; no borrowed pointer is deleted; and the explanation identifies both the owner lifetime and the range boundary.

Review and solution reasoning

Why is nonnull insufficient? A nonnull value can still refer to expired, invalidated, out-of-range, or unsuitable storage. Nullness is only one part of validity.

Why may an end pointer exist without being dereferenced? It represents the boundary of a permitted half-open range, not an additional element.

Why does assigning null not repair every alias? It changes one pointer object. Other copies of the old pointer value remain unchanged.

Why distinguish the pointer from its target? Operations can modify either one. Confusing them creates defects that may remain memory-safe while changing the wrong state.

Chapter takeaway. An address is an access path, not an ownership certificate. Validate its origin, lifetime, bounds, and intended operation before using it.

Technical references: [LANG](#) · [EXPR](#) · [LIFETIME](#).

Chapter 15 — Allocating and Releasing Dynamic Memory

Some objects must outlive the block that decides to create them, and some collections have a size known only at runtime. Dynamic storage can meet those requirements, but someone must own the responsibility for ending the lifetime and releasing the storage.

This chapter examines manual allocation so that its failure modes are recognizable. It then connects the same requirement to a standard container. Understanding `new` and `delete` is essential; using them as the default ownership interface is not.

Dynamic allocation separates an object’s lifetime from the block in which an access variable was created. That flexibility is useful, but it introduces ownership obligations that ordinary local values handle automatically. HarborWorks will calculate the same small sum with manual allocation and with a vector so you can see both the mechanism and the safer application-level alternative.

Learning outcome. Explain dynamic object lifetime, match allocation and release forms, identify leaks and dangling aliases, and prefer an existing resource-owning abstraction for ordinary code. The manual examples are controlled lessons for understanding legacy and low-level code, not a recommendation to replace standard containers with raw owning pointers.

15.1 Understanding Stack and Heap Storage

15.1.1 Comparing Automatic and Dynamic Storage

Automatic storage duration ties ordinary local object management to execution of a scope. Dynamic storage is obtained and released through allocation mechanisms and can support lifetimes that cross ordinary block boundaries. “Stack” and “heap” are useful implementation terms, but the language’s storage-duration and lifetime rules are the more portable model.

A pointer variable can be automatic while the object it points to has dynamic storage. Destroying that raw pointer variable does not destroy the pointed-to object. Conversely, a vector declared locally owns a dynamic buffer whose cleanup is performed by the vector’s destructor. Separate the location of the owner from the location of the owned storage.

“Stack” and “heap” are useful implementation terms, but automatic and dynamic storage duration describe the language-level distinction more precisely. An ordinary local vector object may have automatic lifetime while managing element storage obtained dynamically. The owner and the owned allocation need not occupy the same region.

A local integer normally ends its lifetime when its scope is left. An integer created with a successful `new` expression remains a dynamic object until it is released appropriately or otherwise managed through a valid lifetime mechanism. The raw pointer variable that stores its address does not automatically release the object when that pointer variable goes out of scope.

This is the source of a common leak: the last owning access path disappears while the allocation remains unreleased. The operating system may reclaim process memory when the entire process ends, but that is not a sufficient resource policy for a long-running application. Repeated leaks can accumulate throughout its operation.

Do not allocate dynamically merely because an object is “important” or might be large. Choose storage according to lifetime, size requirements, and ownership. A standard container can manage a runtime-sized collection without requiring the caller to write raw allocation code. A local class object can own files or buffers through its members while still having a simple automatic lifetime.

The best beginner default is the simplest owner that matches the requirement. Dynamic allocation becomes a deliberate mechanism when the lifetime or representation actually needs it.

15.1.2 Understanding Object Lifetime on the Heap

A new-expression ordinarily obtains storage and constructs an object in it. The object’s lifetime continues until it is destroyed, commonly by a matching delete-expression or by an owning abstraction. Losing the final owning access path does not cause raw dynamic storage to be reclaimed automatically.

Allocation can fail, and construction can fail after storage is obtained. Standard new-expression rules handle certain construction-failure cleanup, but a sequence of several manually acquired resources can still leak if a later operation fails. Associate each successfully acquired resource with an owner immediately. Book II’s RAII design makes that rule systematic rather than dependent on every future return path.

A successful **new** expression obtains suitable storage and initializes an object according to the expression’s form. The returned pointer provides access to that object. It does not carry a built-in label identifying which function must eventually release it, so the program’s ownership design must answer that question.

Keep an ownership ledger for manual examples: allocation site, owning variable, nonowning aliases, release site, and every path between allocation and release. If more than one path can release the same allocation, explain how they are made exclusive. If a path can exit without release, identify whether ownership was transferred or lost.

The dynamic object’s lifetime is not prolonged by copying its address into more raw pointers. Those pointers merely create additional access paths. Conversely, destroying one pointer variable does not destroy the target. This independence makes pointers flexible and also makes implicit ownership assumptions dangerous.

A container or smart pointer connects release to an owner’s destructor. That is the central resource-management idea developed through RAII in Book II. You can use the idea now by choosing **std::vector** for an owned sequence instead of writing a manual array manager.

Avoid returning a raw pointer without stating whether ownership is transferred or merely observed. A caller that guesses incorrectly can either leak the object or release storage it never owned. Good interfaces reduce the need for that guess; the type and documentation should agree about the resource relationship.

Contrasting Automatic Storage with Dynamic Storage

Purpose. The distinction is about storage duration and ownership, not simply “small versus large” variables.

```
#include <iostream>
#include <memory>
int main(){
    int automatic = 1;
    auto dynamic = std::make_unique<int>(2);
    std::cout << automatic << ' ' << *dynamic << '\n';
}
```

How it works.

- ``automatic`` has automatic storage duration tied to the block.
- ``make_unique`` obtains dynamically allocated storage and returns an owning object.
- The smart pointer itself is automatic, while the owned integer lives in dynamic storage until released by the owner.

Expected behavior/output. The program prints ``1 2``.

Experiment. Move ``dynamic`` into another ``unique_ptr`` and inspect which object retains ownership.

15.2 Performing Manual Dynamic Allocation

15.2.1 Allocating Objects with `new`

`new int{7}` creates one dynamically allocated integer initialized to 7 and returns a pointer to it. `new int[count]{}` creates an array of value-initialized integers when the count is valid and the allocation succeeds. Validate sizes before converting external values to allocation counts; a negative signed input converted to a large unsigned value is not a sensible request.

Ordinary throwing `new` reports allocation failure with an exception. Do not assume it returns null in that form. Nothrow allocation has a different interface and still requires a failure policy. For most application code, a vector or smart pointer expresses the ownership requirement more clearly and composes better with exceptions.

The expression `new int{7}` creates an initialized integer, while an array form such as `new int[3]{2, 4, 6}` creates a sequence of initialized integers. An uninitialized allocation of fundamental elements does not supply meaningful input data merely because storage exists. Choose initialization deliberately.

Ordinary throwing allocation reports failure through an exception rather than returning null as its normal failure mechanism. A nonthrowing allocation form exists, but choosing it requires a different explicit failure contract. Do not write a null check after ordinary `new` and assume that this alone handles every allocation failure.

Validate requested sizes before allocating. An external count of several billion elements may be syntactically valid and still violate the application's resource budget. Also avoid an overflowing byte-count calculation in low-level code. The inventory exercises impose small application bounds so that resource use remains deliberate and testable.

Allocation should occur at a point where an owner can be established immediately. A raw pointer left between acquisition and a later handoff is vulnerable to early returns or exceptions in that interval. Existing owning types reduce that gap by combining acquisition with ownership construction.

Do not use a dynamic single integer where an ordinary value already has the required lifetime. The manual scalar form is useful to understand syntax, but writing more `new` expressions does not make an application more capable. It usually creates more cleanup paths to prove.

15.2.2 Releasing Objects with `delete`

Use `delete pointer` for a matching single-object new-expression and `delete[] pointer` for a matching array new-expression. Mixing those forms is invalid. Deleting a null pointer is permitted, but deleting the same allocation twice or deleting storage not obtained through the matching mechanism is not.

After deletion, pointers that referred to the destroyed object become dangling. Assigning null to one pointer can make that particular variable's state clearer, but it does not update other aliases. Therefore `pointer = nullptr` is not a complete defense against use-after-free. A single, explicit owner and bounded borrowing relationships are the more reliable design.

Release must match the allocation form. A single-object `new T` is paired with the corresponding single-object delete expression. An array `new T[n]` is paired with `delete[]`. Mixing those forms violates the required lifetime and deallocation relationship even when an integer array appears to behave harmlessly in one run.

The release operation must use an appropriate owning pointer to the allocation. A pointer advanced into an array is not a substitute for the original allocation pointer. A pointer to a local object or to vector-owned storage is not manually deletable by the observer. Pointer type alone does not establish release authority.

Deleting a null pointer is permitted, but that convenience does not solve ownership ambiguity. Deleting the same live allocation twice is invalid, and deleting through one pointer does not update aliases that still contain the old address.

After release, a local owner variable can be set to null to make its own empty state visible. This prevents an accidental second release through that same variable only if later code respects the state. It does not repair other aliases, and it does not make use-after-free through those aliases valid.

For ordinary application code, arrange release through an owning object's destructor instead of manually placing delete expressions along every path. The source's vector version demonstrates the same sum without an explicit delete because the container owns and releases its storage through its defined lifetime.

Allocating and Releasing a Dynamic Array Explicitly

Purpose. Manual allocation exposes the responsibility that containers and smart pointers normally encapsulate.

```
#include <iostream>
int main(){
    std::size_t n=3;
    int* p = new int[n]{10,20,30};
    for(std::size_t i=0;i<n;++i) std::cout << p[i] << ' ';
    delete[] p;
}
```

How it works.

- `new[]` allocates storage and constructs three integers.
- The returned pointer is the only handle used here to reach the allocation.
- `delete[]` must match `new[]` so the allocation is released correctly.

Expected behavior/output. The program prints `10 20 30` and then releases the allocation.

Experiment. Rewrite the program with `std::vector<int>` and list which manual responsibilities disappear.

15.3 Recognizing Memory-Management Failures

15.3.1 Preventing Memory Leaks

A leak occurs when allocated resources remain unreleased and the program no longer has an appropriate way to reclaim them. Overwriting an owning raw pointer before deleting its allocation is one simple cause. Returning early or throwing between allocation and cleanup is another.

Review every path, including failure paths, but recognize the limits of manual review. A resource-owning object whose destructor performs cleanup makes new return paths safer by construction. Leak detectors can identify some allocations left behind in executed tests, yet untested paths can still leak. Prefer designs where ownership is represented in a type rather than in a comment that each maintainer must remember.

A leak occurs when a resource that should be released remains acquired without a usable ownership path or policy to release it. The simplest memory leak loses the only pointer to an allocation. More subtle leaks retain objects forever through a container or ownership structure even though the application no longer needs them.

Manual code must account for every exit after acquisition. A normal return, an error return, and an exception can take different paths. Adding a new validation branch between `new` and `delete` can introduce a leak into a function that was previously correct on its only path.

A useful review exercise is to draw the control-flow paths from allocation to release. If the drawing becomes complicated, the design is asking the reader to manually prove what an owning type could

enforce structurally. RAII does not eliminate every resource bug, but it substantially narrows the gap between scope exit and cleanup.

Leak detection tools observe executed paths. A passing run cannot establish that an untested early return releases correctly. Conversely, a report should be interpreted with the tool's platform and configuration in mind. The companion's verification report records the actual sanitizer runs rather than claiming that every possible memory history has been proved.

Do not rely on process termination as routine cleanup for a reusable function. The function may later run repeatedly inside a server or test process. A resource policy that only works when the program immediately exits is not a sound foundation for reusable application logic.

15.3.2 Preventing Dangling Pointers and Double Deletion

A dangling pointer retains an address after the object's valid lifetime has ended. Double deletion attempts to release one allocation more than once. Both often arise when several raw pointers ambiguously behave as owners. A shallow copy of an owning pointer can create two objects that each believe they must delete the same resource.

Do not resolve this by forbidding all copying without understanding the desired semantics. Some resources should have unique ownership, some values should be deeply copied, and some lifetimes genuinely require shared ownership. Book II introduces types for these choices. For now, keep manual examples short, identify exactly one owner, and never execute deliberate invalid accesses as ordinary sample behavior.

A dangling pointer retains an access value after its intended target is no longer valid. A double deletion attempts to release the same allocation more than once. Both often arise when several raw pointers are treated as owners without an explicit transfer or sharing model.

Copying a raw pointer copies the address, not the pointed-to allocation. If two objects each store that copied pointer and both destructors delete it, their cleanup logic conflicts. Book II explains copy and move semantics and ownership types; this volume avoids presenting a naive raw-owning class as a finished design.

Use-after-free is not reliably visible as a crash. The released storage may be reused, remain readable briefly, or trigger a diagnostic under instrumentation. The validity rule does not depend on the symptom. A nonnull check after deletion does not prove the target is still alive.

A modern exclusive owner such as `std::unique_ptr` makes one ownership relationship explicit and transfers that ownership through defined operations. A vector similarly owns its elements and storage. These are not magical protections against every invalid borrowed pointer, but they make release responsibility far clearer than several unannotated raw pointers.

For this book's manual experiment, keep exactly one owner, do not let aliases outlive release, and return the computed value rather than the pointer. Then prefer the container version in the ordinary application. Learning how a failure happens is most useful when paired with a design that avoids needing to manage that failure manually.

Replacing a Leak-Prone Raw Owner with `std::unique_ptr`

Purpose. Leaks and double deletes arise when ownership is unclear; RAII converts cleanup from a remembered action into deterministic object lifetime.

```
#include <iostream>
#include <memory>
struct Resource{ ~Resource(){ std::cout << "released\n"; } };
void work(){ auto r = std::make_unique<Resource>(); std::cout << "working\n"; }
int main(){ work(); }
```

How it works.

- `unique_ptr` records exclusive ownership in the type.
- No explicit `delete` is needed on the normal path or on exceptions.
- The destructor message proves release occurs when the owner leaves scope.

Expected behavior/output. The output is `working` then `released`.

Experiment. Replace the smart pointer with `new Resource` and omit `delete`; run under AddressSanitizer to observe the leak report, then restore RAII.

Worked example — Compare manual ownership with container ownership

Requirement and simplest approach

The lab must sum two, four, and six and return twelve. The manual version allocates a three-element array, computes the result, releases the array, and returns the ordinary integer. The container version stores the same values in a vector and lets its owner manage cleanup.

The simplest but incorrect comparison would claim that both are equivalent because they print twelve once. A complete comparison includes lifetime and failure responsibilities. The manual version has an explicit release obligation. The vector version associates that obligation with the vector object's lifetime.

Trace ownership separately from arithmetic

In the manual function, `values` is the local owning pointer for this teaching allocation. The sum is copied into a local integer before release. After `delete[]`, the function no longer accesses the array. Setting the pointer to null documents its local empty state, and returning the integer does not expose the expired storage.

In the vector function, the local vector owns its sequence. The loop computes the same total. When the function exits, the vector's lifetime ends and its resources are released through the library's ownership behavior. No caller receives or deletes its internal data pointer.

The arithmetic fixtures are small and valid. This example does not test allocation exhaustion or every exception path on every platform. Its purpose is to make the release responsibility visible and show why the higher-level owner is the natural choice for this application.

Trace the implementation

The manual example has one allocation, a calculation that does not throw, and one matching array deletion. It is intentionally narrow enough to inspect. Nulling the local pointer after deletion clarifies that variable's state but contributes little here because the function immediately returns and has no aliases.

The vector example delegates cleanup to the owning container. If a later change introduces a throwing operation after construction, the vector still releases its buffer during stack unwinding. The manual version would need additional protection. This difference is why code that is correct today can become fragile when ownership is encoded only by a distant delete statement.

Both functions return an integer value, so their local storage does not escape. The tests compare behavior, not allocation addresses. The purpose is not to prove that one implementation is faster; it is to show that equivalent results can be obtained with a stronger cleanup structure.

Complete worked source

source/ch15/main.cpp

```
#include <iostream>
#include <vector>

int manual_sum() {
    int* values = new int[3]{2, 4, 6};
    const int result = values[0] + values[1] + values[2];
    delete[] values;
    values = nullptr;
    return result;
}

int owned_sum() {
    const std::vector<int> values{2, 4, 6};
    int result{};
    for (int value : values) result += value;
    return result;
}

int main() {
    const int a = manual_sum();
    const int b = owned_sum();
    if (a != 12 || b != 12) return 1;
    std::cout << "manual=" << a << " owned=" << b << '\n';
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

The executable target is `b01_c15`. Read `source/ch15/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — Returning before cleanup

Imagine adding an early validation return after the manual allocation but before `delete[]`. The new path leaks the allocation unless ownership is transferred or cleanup occurs on that path. A reviewer who only scans the final lines may see a delete and incorrectly conclude that every execution releases the resource.

The structural repair is to establish an owner immediately, usually an existing container or smart pointer. Adding repeated delete statements can work in a tiny example, but it multiplies the places that must remain synchronized as the function evolves. A resource-owning object reduces that maintenance burden.

Failure clinic — Two aliases, one release

Copy the owning address into a second raw pointer in a **diagram**, then release through the first. Mark the second pointer as dangling. Do not dereference it to see whether the old data remains. The ownership fact is already sufficient to diagnose the invalid use.

If both pointers are later treated as owners, a second delete becomes possible. Nulling only the first pointer does not stop that path. The repair must resolve ownership, not merely change one local variable's spelling or value.

Verification with instrumentation

The companion can be rebuilt with AddressSanitizer and UndefinedBehaviorSanitizer using the documented CMake option. The resulting executable still needs meaningful fixtures. Instrumentation can detect many invalid accesses and some other undefined operations on executed paths, but it does not prove a complete ownership design. See the tool references and the package's actual test report for the supported checks and recorded environment.

Do not ship an intentionally broken memory example as part of the normal application or treat sanitizer runtime settings as production security policy. These tools support development diagnosis; the default companion remains valid, bounded code.

Practice and implementation lab

Draw the owner and any borrowers in each function. Add an early-return requirement after allocation and explain how cleanup would remain correct. Do not intentionally leak in the delivered test suite; sketch the faulty path and then repair it.

Explain why `delete` and `delete[]` are not interchangeable even when the element type is `int`. Identify which representation you would choose for a runtime-sized collection of readings.

Starting exercise: solution notes

The raw pointer is the sole manually designated owner in `manual_sum`; the vector object is the owner in `owned_sum`. An early return in the vector version is naturally safe. A manual version needs a resource guard or a different owning type, not another informal promise that callers will remember cleanup.

Array and single-object allocation have distinct matching deallocation requirements. The element's simplicity does not erase those rules. A vector is the natural default for a runtime-sized contiguous reading collection because it expresses both ownership and logical size.

Guided lab — Write an ownership ledger

For both worked functions, record which object owns the storage, which objects merely observe it, when allocation occurs, and what event releases it. Explain why returning the integer result remains valid after the storage is gone. The returned value and the old element objects are not the same lifetime dependency.

Refactor the manual function to use a vector while preserving its observable result. Run the same fixtures before and after. Then add a deliberate validation failure before the sum and explain how each design handles cleanup. Keep intentional leak examples out of the normal build; the exercise can be completed by reasoning about paths and testing the repaired design.

For a bounded dynamic-size extension, accept a small validated count, create an owning container of that size, fill it, and calculate a checked or range-justified total. Reject oversized requests before allocation. Do not interpret successful allocation as approval of an otherwise invalid application input.

Finally, review a hypothetical function returning `int*`. Write the questions a caller must ask: Is null possible? Who owns the target? How many elements exist? Who releases it? Which release form applies? How long may an observer retain it? Propose a clearer interface when the caller only needs an ordinary result value or an owned sequence.

Acceptance criteria. Allocation and release forms match; no borrowed container storage is manually deleted; every path has one clear owner; rejected input does not create an unowned allocation; and the preferred application solution uses an appropriate resource-owning type.

Review and solution reasoning

Why does a raw pointer's destruction not release its target? The pointer is an address-holding object, not an automatic ownership policy. Release responsibility must be supplied by the program's design.

Why is a vector local not simply "all on the stack"? The owner object can have automatic lifetime while managing dynamic element storage. Owner location and owned storage are distinct.

Why can two raw pointers cause double deletion? Copying an address does not create independent ownership. If both are treated as owners of one allocation, their release paths conflict.

Why prefer a container in the capstone? It expresses the needed sequence and its resource ownership without requiring manual cleanup along every control-flow path.

Chapter takeaway. Dynamic memory is a lifetime and ownership problem before it is an allocation syntax problem. Know who releases every resource, and use an existing owner whenever it fits the requirement.

Technical references: [LANG](#) · [LIFETIME](#) · [GUIDELINES](#).

Chapter 16 — Modeling Structured Data with Structures and Enumerations

A device record has an identifier, a name, and a state. Keeping those in unrelated parallel variables makes it easy to update one without the others. HarborWorks introduces a record type so that the compiler and the reader can see which values belong together.

A domain model should eliminate accidental combinations. An enumeration is not merely a replacement for typing a number; it gives a set of choices a name and makes inappropriate mixing more visible.

Several variables can describe one thing without the language knowing that they belong together. A device identifier, name, and state stored in parallel collections can drift apart after one insertion or deletion. HarborWorks will replace that fragile arrangement with a structured record and a named state type.

Learning outcome. Group related data into a value, initialize aggregates deliberately, use scoped enumerations for discrete choices, and distinguish a valid representation from a valid domain object. The chapter prepares the data model that the inventory class will protect in Chapters 17 through 20.

16.1 Grouping Related Data with struct

16.1.1 Declaring Data Members

A struct groups named members into one type. `struct Device { int id; std::string name; };` says that an identifier and a name form one device value. Use member access such as `device.id` to refer to a component. Different instances have independent member objects unless a member itself refers to shared state.

A struct is a C++ class type with public access by default. It can have functions and constructors; it is not limited to passive C-style records. Use the simplest interface that matches the invariant. Public members are appropriate when the values can be freely combined, while protected invariants may call for private representation and checked operations.

A structure groups named members into one type. A `Device` containing an identifier, name, and state gives the program one object to copy, pass, or store. The member names describe the relationship more clearly than unrelated arrays such as `ids`, `names`, and `states` whose positions must remain synchronized manually.

The dot operator accesses a member of an object: `device.name` names the name member of that device. This is not a global variable shared by every device. Two ordinary `Device` values have their own member values, subject to the semantics of the member types.

A C++ struct can contain member functions and constructors; it is not restricted to passive data. The main language distinction from a class is default access, not an absolute separation between “data” and “behavior.” This chapter uses simple public aggregates because the learning objective is data modeling. Chapter 17 introduces a restricted interface when invariants need protection.

Choose members that form a coherent concept. A structure containing an equipment record should not also contain unrelated user-interface settings merely because they are available in the same function. Cohesion makes copying and passing the value meaningful.

The structure’s memory representation is not a portable file format. Padding, member representation, and objects such as strings make raw byte dumping inappropriate for a general persistent record. Serialize fields according to an explicit format instead of writing `sizeof(Device)` bytes and expecting another process or compiler to interpret them safely.

16.1.2 Initializing Aggregate Objects

An aggregate can be initialized by listing member values in declaration order, such as `Device{7, "scope"}`. Default member initializers provide meaningful defaults for omitted fields where the aggregate rules permit. C++20 also supports designated initialization for suitable aggregates, subject to ordering and other restrictions.

Do not mistake brace syntax for automatic semantic validation. A negative identifier can still initialize an integer member unless the type prevents it. Define which combinations are valid and decide whether validation belongs in a parser, constructor, factory, or mutation method. Public aggregate records work especially well at simple data boundaries when validation is performed explicitly before the record is trusted.

Aggregate initialization supplies member values in the type’s declared order for the positional form used in this book. A default member initializer can provide a sensible starting state when a member is omitted. The visible initializer should still make important domain values explicit.

For example, an identifier of zero may be a useful staging default that means “not yet assigned,” but it is not automatically a valid stored device identifier. A default-initialized record can therefore be a temporary candidate rather than an accepted inventory object. Validation must occur before the candidate enters a collection whose contract requires valid records.

Positional initialization can become fragile when a structure contains several adjacent members of the same type. Swapping quantity and identifier may compile while changing meaning. Clear member names, small aggregates, named construction functions, or later constructors can reduce that risk. C++20 also has designated initialization under specific rules, but this volume does not require it to understand the basic value model.

When adding a member, review all initialization sites and serialization code. A default may preserve compilation while silently changing behavior. A successful build does not prove that every old initializer expresses the intended new state.

Test the default object, a fully specified valid object, and a rejected candidate. Those cases reveal whether defaults are domain-valid, whether field order is correct, and whether the validation boundary actually protects the collection rather than relying on callers to remember every rule.

Modeling One Record with a `struct`

Purpose. A `struct` replaces parallel unrelated variables with one domain record that can be passed and stored as a unit.

```
#include <iostream>
#include <string>
struct Employee{ int id; std::string name; double hourly_rate; };
int main(){ Employee e{17,"Ada",42.5}; std::cout << e.id << ' ' << e.name << ' '
<< e.hourly_rate << '\n'; }
```

How it works.

- The fields are grouped because they describe one conceptual record.
- Aggregate initialization supplies values in member order.
- Member access uses the object and field name directly.

Expected behavior/output. The record prints as `17 Ada 42.5`.

Experiment. Add a `hours` field and write a function that computes weekly pay from one `Employee`.

16.2 Representing Named Choices with Enumerations

16.2.1 Using Traditional Enumerations

An unscoped enumeration gives names to integral choices and introduces its enumerator names into the surrounding scope. It can participate in implicit conversions to integer in ways that are convenient for older interfaces but can also allow accidental mixing with unrelated numeric values.

When reading legacy code, identify the intended domain before treating an enum as an arbitrary integer. A stored underlying value may not correspond to a listed enumerator, especially after unchecked deserialization. Validation at external boundaries remains necessary. The named constants improve readability, but they do not automatically make every cast from an integer a valid domain conversion.

A traditional unscoped enumeration introduces named values and permits certain implicit conversions that can be convenient in older interfaces. It improves readability over raw numeric literals, but its names and conversion behavior can also create collisions or accidental mixing with unrelated integers.

For a small teaching example, `offline`, `ready`, and `failed` are clearer than `zero`, `one`, and `two` because they state the domain choices. The important improvement is the meaning carried by the names. A comment mapping numbers to states is weaker because the comment and code can diverge independently.

Do not assume that an enumeration validates external input by itself. Reading an integer from a file and casting it to an enumeration can produce a value that does not correspond to an accepted named state. Validate the external code before treating it as a domain choice.

An enumeration's underlying representation is also not automatically a stable protocol specification. If a file or network format uses numeric state codes, define the mapping explicitly and version it as needed. Reordering declarations or adding a value should not accidentally redefine persisted data.

Traditional enums remain important when reading existing C and C++ interfaces. You should understand them, but new application code can often use `enum class` to make the intended type boundary clearer. The next subsection explains why that stronger boundary is useful without pretending it removes the need for validation.

16.2.2 Using Type-Safe `enum class`

A scoped enumeration such as `enum class State { offline, ready, failed };` keeps its names qualified and does not implicitly convert to an integer. `State::ready` therefore communicates both the choice and its domain. Two unrelated scoped enum types cannot be casually compared as if their numeric encodings were interchangeable.

Choose an explicit underlying type when a binary interface requires one, and validate any incoming numeric encoding before conversion. Do not serialize an enum by dumping its object bytes and assuming a stable cross-platform protocol. Named choices describe program meaning; wire representation and compatibility need a separate, explicit contract.

A scoped enumeration keeps its enumerators associated with the enumeration type, so the source writes `State::ready`. It also prevents ordinary implicit conversion to an integer in contexts where an unscoped enum might convert. That makes accidental arithmetic or mixing of unrelated state types less likely.

The type boundary is useful in a function interface. A function taking `State` communicates a discrete domain choice more clearly than a function taking an undocumented `int`. A caller must deliberately produce a state value rather than pass any convenient number and hope it has the right meaning.

Scoped enums can still hold values requiring defensive treatment after explicit casts or external decoding. The label function includes a fallback for an unrecognized state value. That fallback provides a diagnostic representation; it does not mean such a state should be accepted into the valid inventory domain.

A switch over all named states can also help compiler diagnostics identify missing handling when a new enumerator is added. Do not assume every compiler configuration guarantees the same warning behavior. Preserve tests for the known states and review the conversion boundary when the type evolves.

A named state should represent one dimension. Combining readiness, calibration status, and ownership into one enormous enumeration can create an unwieldy cross-product of choices. Model independent facts independently unless the domain truly requires an exclusive combined state. The type should clarify the real model rather than force unrelated properties into one label.

Using `enum class` for a Closed Set of States

Purpose. Named choices communicate valid states more clearly and safely than magic integer constants.

```
#include <iostream>
enum class Status{ queued, running, done };
const char* name(Status s){
    switch(s){case Status::queued: return "queued";case Status::running: return
"running";case Status::done: return "done";}
    return "unknown";
}
int main(){ std::cout << name(Status::running) << '\n'; }
```

How it works.

- `enum class` creates named values in a scoped type.
- The switch handles each known enumerator explicitly.
- Callers cannot accidentally substitute an unrelated integer without a conversion.

Expected behavior/output. The program prints `running`.

Experiment. Add a `failed` state and let compiler warnings help identify switches that need updating.

16.3 Designing Meaningful Domain Types

16.3.1 Replacing Unrelated Variables with Structured Models

A collection of `Device` objects keeps related fields together. This is easier to copy, return, and validate than three vectors whose matching indices must remain synchronized. A function accepting one record can state a coherent requirement instead of relying on callers to pass fields in the correct order.

Model only the distinctions that matter to the application. A small teaching record does not need a deep inheritance hierarchy. As requirements grow, separate stable identity from mutable status and avoid using a display name as the only key. A good type reduces the number of invalid states the rest of the program must consider.

A domain type groups information according to the problem being solved. A device record is not just “some integers and a string”; it represents one identified piece of equipment with a state. That meaning determines which fields belong together and which combinations are valid.

Parallel arrays require an invariant that all arrays have matching lengths and that position `i` refers to the same device in each. A vector of `Device` values makes the relationship structural. Adding one record adds its fields together, and copying a record preserves that association through the member types’ value semantics.

The structure alone does not enforce uniqueness of identifiers across a collection. That is a collection-level invariant, so it belongs in a component that owns or validates the collection. Similarly, a valid state value does not prove that a state transition is allowed. The model has several levels of rules, and each needs an appropriate owner.

Avoid building a large inheritance hierarchy merely because the data has acquired a name. A simple value type is often enough for a record. Inheritance and polymorphism arrive in Book II when behavior actually requires them. The beginner objective is to make data relationships explicit without adding architecture that the problem has not requested.

A useful design test is to explain what one complete object means in a sentence. If that sentence lists several unrelated concepts, divide the type or reconsider its responsibility before adding more members.

16.3.2 Replacing Magic Numbers with Enumerations

A condition such as `state == 2` forces readers to remember an external mapping. `state == State::failed` states the meaning locally. It also makes switch branches and diagnostic functions easier to review when new states are added.

Not every numeric constant belongs in an enum. A voltage threshold is a quantity, not necessarily one of a closed set of named choices. Use `constexpr` values with units or domain types for quantities, and enumerations for alternatives. At protocol boundaries, document both the symbolic meaning and the encoded value so that a future change does not silently reinterpret stored data.

Magic numbers hide policy inside representation. A condition such as `state == 2` forces the reader to remember a mapping. `state == State::failed` carries the interpretation directly. The same principle applies to named quantity limits, menu choices, and validation results.

Not every number needs an enumeration. A measured temperature varies across a numeric range and is not naturally a list of named alternatives. A threshold is often a named constant; a small exclusive set of modes is often an enumeration. Choose the abstraction from the domain rather than converting every literal into a different kind of ceremony.

Be careful with casts at external boundaries. A cast expresses a conversion, not an approval. A parser should map accepted text or numeric codes to named states and reject unsupported input. That mapping can return a success flag and commit to an output state only after validation, following the candidate pattern from Chapter 11.

When logging a state, use a stable label function so the representation is consistent across call sites. Repeating slightly different switch statements throughout an application creates drift when the enumeration changes. Centralizing the mapping also gives tests one place to check every accepted alternative.

The final goal is fewer hidden conventions. A type-safe label, a named limit, and an explicit decoding function make different parts of the contract visible. None alone replaces the full validation and ownership design, but together they reduce the amount of undocumented knowledge a caller must carry.

Strong Domain Wrapper Preventing Unit Mixups

Purpose. Meaningful domain types use the type system to encode distinctions that comments alone cannot enforce.

```
#include <iostream>
struct Meters{ double value; };
struct Seconds{ double value; };
double speed(Meters d, Seconds t){ return d.value / t.value; }
int main(){ std::cout << speed(Meters{100.0}, Seconds{9.58}) << '\n'; }
```

How it works.

- `Meters` and `Seconds` are distinct types even though both contain `double`.
- The function signature documents units and prevents swapping raw doubles accidentally.
- The calculation unwraps values only at the domain operation boundary.

Expected behavior/output. The program prints a speed a little above 10.4 meters per second.

Experiment. Try calling `speed(Seconds{9.58}, Meters{100})`; the type mismatch should be rejected at compile time.

Worked example — Report device states without magic numeric codes

Requirement and simplest approach

The lab must report two devices, one ready and one failed, and count the ready devices. It must also be able to label the offline state even though no offline device appears in the initial collection. The data is fictional teaching data.

The simplest design stores identifiers, names, and numeric state codes in separate arrays. That creates a positional synchronization requirement unrelated to the reporting task. Another shortcut scatters raw state numbers and string labels throughout the loop, making the mapping easy to change inconsistently.

The selected design uses a **Device** aggregate, a scoped **State**, and one label function. The vector owns complete records. The loop observes each record, tests a named state, and prints the fields in a predictable order.

Trace data and behavior separately

The aggregate defines what information travels together. The enum defines the named choices for one field. The label function defines the external text representation. The reporting loop decides which records contribute to the count. Each responsibility can be tested without pretending it belongs to the others.

The ready count should be one. The output should preserve each identifier/name/state relationship. The offline label check exercises a named alternative absent from the sample records. That is a small but important testing habit: the sample data does not necessarily cover every supported branch.

The use of `std::string_view` for returned labels observes static string literals and does not transfer ownership. The device's name itself remains an owning string, because the record must retain that text as part of its value.

Trace the implementation

The record groups each device's identity and state, while the enum prevents casual substitution of an unrelated integer in ordinary assignments. The label function centralizes the human-readable spelling rather than scattering it throughout reporting code.

The switch lists all declared states and still returns `invalid` if an out-of-domain value reaches it. That defensive branch is not a substitute for validating deserialized input, but it prevents the function from falling off a value-returning path. The demonstration constructs only valid enumerators.

The vector's records are observed through `const` references. Counting ready devices does not need to copy strings or mutate records. The test checks the count and an additional state label, showing that the model can support more than one independent reporting operation without parallel arrays or numeric conventions.

Complete worked source

source/ch16/main.cpp

```
#include <iostream>
#include <string>
#include <string_view>
#include <vector>

enum class State { offline, ready, failed };

struct Device {
    int id{};
    std::string name;
    State state{State::offline};
};

std::string_view label(State state) {
    switch (state) {
        case State::offline: return "offline";
        case State::ready: return "ready";
        case State::failed: return "failed";
    }
    return "invalid";
}

int main() {
    const std::vector<Device> devices{
        {1, "scope", State::ready}, {2, "meter", State::failed}};
    int ready{};
    for (const Device& device : devices) {
        if (device.state == State::ready) ++ready;
    }
}
```

```

        std::cout << device.id << ':' << device.name
                  << ':' << label(device.state) << '\n';
    }
    if (ready != 1 || label(State::offline) != "offline") return 1;
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root, build and run this chapter’s supplied fixtures:

```
python3 tools/run_chapter.py 16
```

The executable target is `b01_c16`. Read `source/ch16/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — The fields drifted apart

Suppose a name is erased from a names array while the corresponding identifier remains in a separate identifiers array. Every access may stay within some valid array bound, yet the report associates the wrong name with the wrong identifier. Memory safety alone does not protect relational meaning.

Grouping the fields in one record removes that particular synchronization problem. Erasing one record removes its related fields together. The collection still needs its own uniqueness and validity rules, but the basic association no longer depends on parallel editing discipline.

Failure clinic — A cast accepted an unsupported state

An external integer is cast directly to `State` and stored. The label function later returns `invalid`. The fallback prevented misleading label output, but the accepted-domain boundary already failed. The repair belongs at decoding or insertion, where unsupported values should be rejected before they become stored state.

Failure clinic — A default that was mistaken for approval

A default record contains identifier zero and an empty name. Those defaults can be useful while parsing a candidate, but they do not satisfy the final inventory’s acceptance rules. A constructor or insertion function must distinguish “a C++ object exists” from “the application accepts this record.”

This distinction becomes the central topic of the next chapter. Public aggregate fields are convenient for transport and parsing; a class with a restricted interface can protect a stronger invariant after the candidate has been validated.

Practice and implementation lab

Add a `maintenance` state and update every state-to-text conversion. Compile with switch-related warnings enabled and inspect whether the toolchain helps identify missing cases. Add a record with the new state and test that it is not counted as ready.

Design an input mapping from the strings `offline`, `ready`, and `failed` to the enum. Reject unknown text rather than choosing a silent default.

Starting exercise: solution notes

The additional state belongs in the enum and in the centralized label function. A switch that enumerates known values can let diagnostics expose missing branches, depending on enabled warnings. A maintenance record must not increment the ready count.

The parser should compare complete accepted spellings and return an explicit failure for anything else. A default of ready would convert malformed input into an operationally meaningful state and violate the validation boundary.

Guided lab — Model and validate a device record

Write the record's field rules: identifier domain, name policy, and accepted states. Then write collection rules separately, including identifier uniqueness. This separation prevents a record-level helper from pretending it can validate relationships that require access to the whole collection.

Add a third device and verify that its fields remain associated in the output. Copy one record, modify the copy's name, and confirm that the original value remains unchanged. Do not confuse value copying of a string member with copying a raw pointer to shared character storage.

Implement a state decoder that accepts the exact strings `offline`, `ready`, and `failed`. Reject empty text, unknown words, and unintended case variants unless the grammar explicitly permits them. Preserve the previous output state on rejection. This reuses the validation-and-commit pattern in a nonnumeric domain.

For the evolution exercise, add a new named state in a copy and identify every place that needs review: label conversion, input decoding, tests, reporting rules, and any persisted format. The compiler can help find some omissions, but it cannot decide the new state's business meaning for you.

Acceptance criteria. Related fields travel as one value; named states replace unexplained numeric codes; decoding rejects unsupported values; defaults are not mistaken for validated records; and collection-level rules have a clear owner.

Review and solution reasoning

Why is a struct not automatically a valid domain object? The language can create its members even when their combination violates application rules. Validation or construction policy must establish those rules.

Why prefer enum class for new discrete choices? It keeps names scoped and strengthens the type boundary against accidental mixing. Explicit conversions still require validation.

Why not dump the struct's raw bytes to a file? Its in-memory representation can contain padding and nonportable member state, including string internals. A file format needs an explicit field representation.

Why test a state absent from the sample vector? Supported behavior extends beyond one fixture. Every named alternative needs coverage even when the initial demonstration does not use it.

Chapter takeaway. Good data types preserve relationships and name the domain. They reduce hidden conventions, while validation and ownership establish which combinations may become accepted application state.

Technical references: [LANG](#) · [CLASS](#).

Chapter 17 — Designing Classes and Objects

HarborWorks now needs a quantity that never becomes negative. A public integer member cannot enforce that invariant because any caller can assign an invalid value. A class can keep the representation private and expose only operations that preserve the rule.

Encapsulation is not secrecy for its own sake. It is a way to reduce how many places must be trusted to maintain an object's valid state.

A record groups data, but a public field can still be assigned an invalid value from anywhere that can reach it. A class becomes useful when its interface protects a rule that must remain true. HarborWorks will model a quantity constrained to zero through one thousand and prevent rejected updates from changing the previously valid value.

Learning outcome. Define a class around an invariant, distinguish observers from modifiers, explain access control without treating it as a security boundary, and keep the public interface smaller than the private representation. The chapter's goal is not “make everything a class”; it is to give a real rule a clear owner.

17.1 Defining Classes

17.1.1 Declaring Data Members

A class's nonstatic data members describe the state held by each object. Initialize them to a valid starting state. A member such as `int quantity_{0};` is private by default in a class definition and can be accessed directly by that class's member functions.

Choose members that represent the invariant with minimal redundancy. Storing both a list and a separate count can create two values that must always agree; deriving the count from the list may be safer. When cached or redundant data is necessary for performance, make the synchronization rule explicit and test it. Representation design determines how difficult every later operation will be to keep correct.

A class declaration introduces a type whose objects contain member state and support operations. Each ordinary `Quantity` object in the example has its own stored integer value. The maximum is a named class-level constant because it describes the type's accepted domain rather than an independent setting for every object.

Start with the invariant in words: the stored quantity is always between zero and one thousand whenever callers can observe the object through its normal interface. Then choose the representation that makes that rule straightforward. One private integer is sufficient for the teaching type; additional caches and flags would create more relationships to maintain.

A class is not inherently a dynamically allocated object. `Quantity quantity;` creates an ordinary object with the lifetime determined by its declaration context. You do not need `new Quantity` merely

because the type uses the `class` keyword. The resource principles from Chapter 15 apply to classes just as they do to integers and vectors.

Avoid duplicating derivable state without a reason. Storing both `value_` and an `is_empty_` flag means every update must keep them consistent. If emptiness is simply `value_ == 0`, an observing function can compute it. A smaller representation often supports a simpler invariant and fewer failure paths.

Member names are implementation details when access is private. A consistent suffix such as `value_` can help distinguish stored state from parameters without exposing that naming convention to callers.

17.1.2 Implementing Member Functions

A member function operates in the context of an object and can access its private representation. A getter can return the current quantity, while a checked update can reject a value outside the allowed range. The function name should describe the domain operation rather than merely expose arbitrary storage mutation.

Keep I/O out of simple domain methods when possible. A quantity object should report whether an update succeeded; the caller can decide whether to print, log, retry, or reject a request. This separation lets tests exercise the invariant without capturing terminal output and lets the same class serve several interfaces.

A member function operates in the context of an object. Calling `quantity.set(12)` requests a state transition on that particular quantity. Calling `quantity.value()` observes its current state. The syntax makes the relationship between operation and owner visible.

The observer is declared `const` because it does not modify the object's ordinary state through that member-function interface. The `const` qualifier appears after the parameter list. It is different from returning a `const` integer, which would describe the result type rather than the member function's access to its object.

A modifying function should validate before committing the change. In the example, `set` rejects a candidate outside the permitted interval and returns `false` without replacing `value_`. An accepted candidate is assigned once and the function returns `true`. The control flow mirrors the contract directly.

Not every related operation must be a member. A formatter that only uses public observers can be a separate function. A calculation combining several independent quantities may also belong outside the class. Give privileged access to operations that need it rather than turning the class into a container for every nearby utility.

Keep member functions focused enough to test through their public behavior. If testing a quantity update requires starting a user interface or reading a file, the class may be mixing domain state with infrastructure. The small quantity type has no stream dependency in its own definition, making its invariant easy to exercise directly.

Encapsulating State and Behavior in a Class

Purpose. A class packages state with the operations that maintain and expose that state.

```
#include <iostream>
class Counter{
    int value_{};
public:
    void increment(){ ++value_; }
    int value() const { return value_; }
};
int main(){ Counter c; c.increment(); std::cout << c.value() << '\n'; }
```

How it works.

- Private data stores representation that callers cannot modify directly.
- Public member functions define the supported operations.
- The `const` observer guarantees it does not modify the object.

Expected behavior/output. The program prints `1`.

Experiment. Add a `reset()` member and decide whether it belongs in the public interface.

17.2 Protecting Object State

17.2.1 Understanding public, private, and protected

Public members form the callable interface. Private members are implementation details available to the class and its friends. Protected members are additionally accessible in appropriate derived-class contexts. These are compile-time access-control rules, not operating-system security boundaries.

Do not make data protected merely because inheritance might be useful someday. It exposes representation to subclasses and can make invariants harder to maintain. Prefer private data and deliberate operations. A struct's default public access and a class's default private access are conventions that help signal intended use; both constructs otherwise support the same broad language mechanisms.

public members form the interface ordinary callers may use. **private** members restrict direct access to the class's own implementation and other specifically permitted access. **protected** introduces access for derived-class relationships, which Book II studies. For this small value type, private state and a public interface are sufficient.

Access control is a compile-time language mechanism for organizing code. It is not encryption, process isolation, or a defense against an attacker who can arbitrarily modify the program's memory. Its value here is preventing ordinary callers from accidentally bypassing the type's validation rule.

A public mutable reference to a private member can effectively reopen the bypass. If `value()` returned `int&`, a caller could assign a negative number without calling `set`. The member remains spelled private,

but the interface would expose mutation authority. Encapsulation is about the operations you permit, not merely the label above a field.

Returning an inexpensive integer by value is a natural observer for this type. For larger containers, an observing reference can be appropriate with explicit lifetime and invalidation rules. Do not expose internal representation simply because copying might cost something; choose the smallest observation the caller actually needs.

Protected data can also spread invariant maintenance across derived classes. Prefer behavior-based interfaces unless direct derived access is genuinely necessary. A class whose rule depends on many outside components remembering to update several fields together has not fully assigned ownership of that rule.

17.2.2 Maintaining Class Invariants

An invariant is a condition that must hold whenever an object is available for ordinary use. For a bounded quantity, it might be `0 <= value <= maximum`. Constructors establish it, and public operations preserve it. A failed update should leave the previous valid state intact rather than partially changing the object.

Validate candidates before assigning members. If an operation changes several related fields, compute or validate the complete new state before committing it. Tests should cover successful changes, rejected changes, and the state remaining after rejection. Checking only return codes can miss an implementation that reports failure after already corrupting the object.

An invariant is a property the class promises to maintain at its defined observable boundaries. The quantity invariant is an interval constraint. A more complex inventory invariant includes unique identifiers, valid names, bounded quantities, and a maximum record count. Each operation must either establish or preserve the relevant rule.

For a modifier, identify the pre-state, candidate, validation, commit point, and post-state. On rejection, the example preserves the pre-state. On success, the new stored value satisfies the invariant. Writing those steps before the code makes it easier to detect a function that assigns first and checks later.

Construction is another invariant boundary. The chapter uses a default member initializer that creates a valid zero quantity. Chapter 18 considers types that require constructor arguments and can reject invalid construction. An invariant cannot depend on callers remembering to invoke a separate initialization method before every observer unless that two-phase state is explicitly part of the type's model.

An invariant does not mean every transient internal instruction must independently describe a complete externally visible state. More advanced algorithms can make controlled internal transitions. However, such transitions must not leak invalid observations or leave the object inconsistent when an operation fails. For beginner code, a small validation-before-assignment sequence keeps the proof simple.

Tests should include successful updates, rejected updates, and observations after both. Testing only the return flag misses an implementation that reports false but has already stored the invalid candidate.

Maintaining a Nonnegative Balance Invariant

Purpose. Encapsulation is valuable when it protects invariants rather than merely hiding syntax.

```
#include <iostream>
class Account{
    double balance_{};
public:
    bool withdraw(double amount){ if(amount < 0 || amount > balance_) return
false; balance_ -= amount; return true; }
    void deposit(double amount){ if(amount > 0) balance_ += amount; }
    double balance() const { return balance_; }
};
int main(){ Account a; a.deposit(50); std::cout << a.withdraw(70) << ' ' <<
a.balance() << '\n'; }
```

How it works.

- `balance_` is private so callers cannot force an invalid negative value directly.
- Mutation functions validate their own preconditions.
- The observer exposes state without exposing writable representation.

Expected behavior/output. The withdrawal fails (0) and the balance remains 50.

Experiment. Add a valid withdrawal of 20 and verify the invariant still holds.

17.3 Understanding Object Behavior

17.3.1 Using the `this` Pointer

Inside a nonstatic member function, `this` points to the object on which the function was called. Member names commonly use that object implicitly. Explicit `this->member` can resolve naming situations or improve clarity, but it is not required before every access.

A member function marked `const` receives a `const`-qualified view of its object for ordinary member access and promises not to modify its nonmutable representation through that path. This supports calling observers on `const` objects. Do not use casts or mutable fields simply to bypass a design you have not resolved; logical constness and caches require deliberate reasoning about observable behavior and concurrency.

Inside a nonstatic member function, `this` provides access to the current object. Most ordinary member access can be written without spelling it explicitly because the member name is resolved in that context. `value_ = candidate;` and the corresponding explicit `this->value_` form refer to the same member in this simple example.

The arrow notation accesses a member through a pointer. You have already seen dereferencing in Chapter 14; `this->member` is the conventional member-access form for the current-object pointer. The pointer itself is supplied by the language's member-call mechanism, not allocated by your function.

A common beginner mistake occurs when a parameter has the same name as a member and the function assigns `value = value;`. Both names may resolve to the parameter, leaving the member unchanged. A naming convention or explicit `this->value` can make the intended target clear. The repair should be based on name lookup and object identity, not on random use of extra punctuation.

In a const member function, access through `this` respects the const-qualified object interface. That is why an ordinary observer cannot assign to a nonmutable data member through that path. The compiler helps enforce the promise expressed by the member-function declaration.

Do not return `this` or a reference to the object merely because fluent interfaces are possible. Such APIs can be useful, but the beginner quantity type gains nothing from a chained mutation style. Keep the interface aligned with the operation's simple success/failure contract.

17.3.2 Separating Interface from Implementation

The public interface states what callers can ask the object to do. The private representation states how those operations are implemented. Callers that use only the interface should not need rewriting when the internal representation changes while behavior stays the same.

This boundary also improves tests. Verify that accepted updates produce the promised result and invalid updates preserve state, rather than inspecting private fields or depending on a particular layout. Later chapters place declarations and definitions in separate files; separation in files is useful, but the conceptual boundary exists even when a small class is defined in one source file.

The interface describes what callers may request and what they may rely on. The implementation chooses how the type fulfills those promises. A caller should know the quantity's accepted range and update behavior without depending on the private member's name or exact layout.

This separation makes future changes possible. The class might later store a different representation or add instrumentation while preserving `set` and `value` behavior. If callers directly manipulate the representation, every such change becomes a coordinated edit across the application.

An interface can still be too broad even when all fields are private. Providing a getter and setter for every field may reproduce the same unrestricted data model through functions. Prefer operations that represent meaningful allowed transitions. For a more realistic stock type, `receive` and `remove` with validation may communicate better than an arbitrary setter, depending on the requirements.

Do not promise behavior the implementation cannot maintain. Returning a reference to an internal vector while claiming callers can retain it across all future operations is incompatible with ordinary mutation and lifetime constraints. Document observation boundaries precisely and avoid exposing unnecessary internal structure.

The quantity example intentionally remains small enough that the interface and implementation fit on one page. Chapter 19 moves definitions into separate files. Physical separation is useful, but conceptual separation comes first: a header is not a good interface merely because the implementation happens to be stored elsewhere.

Behavior Derived from Object State

Purpose. Objects are useful when behavior depends on persistent state governed by the object's invariant.

```
#include <iostream>
class Thermostat{
    double target_{21.0};
public:
    explicit Thermostat(double t):target_(t){}
    bool should_heat(double current) const { return current < target_; }
};
int main(){ Thermostat t{22}; std::cout << std::boolalpha << t.should_heat(20.5)
<< '\n'; }
```

How it works.

- The constructor establishes the object's target state.
- `should\_heat` combines current input with stored state to produce behavior.
- The member is `const` because deciding does not change the thermostat.

Expected behavior/output. The program prints `true`.

Experiment. Create a second thermostat with a lower target and compare results for the same current temperature.

Worked example — Enforce a bounded quantity invariant

Requirement and simplest approach

The quantity begins at zero. A caller may request a value from zero through one thousand. An accepted request replaces the stored value; a rejected request returns false and preserves the old value. Observers can read the quantity without acquiring direct mutation access.

The simplest approach stores a public integer and trusts every caller to validate it. That places the invariant in many locations and makes one forgotten check sufficient to corrupt the model. The selected class owns the rule and offers one validated modification path.

Trace the state transitions

Create a default quantity and confirm zero. Request twelve and confirm both success and the new value. Request negative one and confirm rejection **and** preservation of twelve. Request one thousand one and confirm the same preservation. Finally, observe through a const reference to show that read-only access uses the same invariant-preserving object.

The test's structure is as important as its numbers. Every modifier result is paired with a state observation. This catches implementations that return the right Boolean but perform the wrong assignment. It also makes the failure contract explicit rather than treating rejection as an unspecified interruption.

The maximum is a named constant in the class's interface. Its role is to define the accepted domain consistently. Changing it requires reviewing the tests and any dependent arithmetic, not merely editing a number and assuming the rest of the application remains correct.

Trace the implementation

The class begins in a valid zero state. Its only mutator checks a candidate before changing the stored value. A caller cannot directly assign `value_`, so the number of places responsible for the invariant is small and visible.

The failed-update tests check both the false result and the unchanged value. This is stronger than merely confirming that a negative input is recognized. It establishes a useful failure guarantee that future implementations must preserve.

The const reference at the end can call the const observer but not the mutator. That distinction lets functions advertise observation without mutation. The example uses a public static `constexpr` `maximum` because callers may legitimately need to know the accepted bound; exposing the bound does not expose mutable representation.

Complete worked source

source/ch17/main.cpp

```
#include <iostream>

class Quantity {
public:
    static constexpr int maximum{1000};
    bool set(int candidate) {
        if (candidate < 0 || candidate > maximum) return false;
        value_ = candidate;
        return true;
    }
    int value() const { return value_; }
private:
    int value_{};
};

int main() {
    Quantity quantity;
    if (quantity.value() != 0) return 1;
    if (!quantity.set(12) || quantity.value() != 12) return 2;
    if (quantity.set(-1) || quantity.value() != 12) return 3;
    if (quantity.set(1001) || quantity.value() != 12) return 4;
    const Quantity& observed = quantity;
    std::cout << "quantity=" << observed.value() << '\n';
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter’s supplied fixtures:

```
python3 tools/run_chapter.py 17
```

The executable target is `b01_c17`. Read `source/ch17/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Failure clinic — Validation after assignment

A learner writes the candidate into `value_`, then checks whether it lies in range and returns false when invalid. The reported result looks correct, but the object is already invalid. Any later observer can see the rejected value.

The repair is to move the commit after validation or use a candidate representation that is only installed when valid. This is the same pattern used by the string parser and transactional batch update. The chapter brings the pattern under one object’s authority so callers cannot easily bypass it.

Failure clinic — Private storage with a public escape hatch

Another learner returns a nonconst reference to `value_` to “avoid copying.” A caller assigns negative five through that reference. The class’s private declaration did not prevent the interface from granting direct mutation access. Returning the small integer by value preserves observation without exposing the bypass.

Failure clinic — A class that owns too many jobs

Suppose `set` reads a line from the terminal and prints a report as well as validating a quantity. Testing a state transition now requires an input/output environment, and another application cannot reuse the quantity without unwanted interaction. Separate parsing, domain mutation, and reporting. The quantity should receive a candidate value and enforce its rule; the outer application decides how the candidate arrived.

This separation is not abstract architecture for its own sake. It directly improves testability and makes it possible to diagnose whether a defect lies in input syntax, domain validation, or presentation.

Practice and implementation lab

Add an `increase(int amount)` operation that rejects negative amounts and any result above the maximum. Avoid computing an overflowing sum before checking it. Test the maximum boundary and verify unchanged state after rejection.

Explain why providing a public `int&` getter for the stored value would undermine the invariant even if the data member itself remained private.

Starting exercise: solution notes

For a nonnegative amount, first check `amount <= maximum - value_`, then perform the addition. Because the existing value is already within the invariant, the subtraction is safe and the accepted addition fits. Reject before modifying the member.

A mutable reference getter would let callers assign arbitrary integers directly, bypassing validation. Private storage alone is not encapsulation when the interface hands out unrestricted mutation of that storage.

Guided lab — Extend an invariant without exposing the representation

Add an `is_empty` observer that derives its answer from the stored quantity. Do not add a redundant Boolean member. Test zero and nonzero quantities, including observations after rejected updates. The observer should not change the object.

Then design a `remove(amount)` operation. State the domain for amount, the behavior when amount exceeds the available quantity, and the preservation guarantee on rejection. Validate before subtraction and test exact depletion, partial removal, zero removal if permitted, negative input, and excessive removal.

Compare that operation with a public setter. Which application rule does each express? There is no requirement to eliminate all setters, but you should recognize when an operation-specific interface prevents misuse more clearly than arbitrary replacement.

For the access-control exercise, write a small separate compile-failure example that attempts to assign directly to the private member. Keep it outside the default build. Then inspect whether any public function still exposes a mutable reference or pointer that bypasses the same rule. The interface review must consider behavior, not only declarations.

Acceptance criteria. The default state is valid; every accepted transition preserves the invariant; rejection leaves the object unchanged; observers do not grant unintended mutation authority; and parsing or printing responsibilities remain outside the quantity model.

Review and solution reasoning

Why is a Boolean return insufficient evidence? The function can return false after already changing state. Tests must inspect both the reported result and the object's post-state.

Why not store an empty flag? It is derivable from the quantity. Duplicating it creates another invariant that every update must preserve.

Why is private not a security boundary? It controls ordinary source-level access within the language. It does not provide memory isolation or protect against arbitrary modification of a running process.

Why prefer meaningful operations? They expose allowed transitions and can enforce domain rules more directly than unrestricted field replacement.

Chapter takeaway. A class is valuable when it owns a rule. Keep the representation private, make transitions explicit, and verify that both success and failure leave a state callers can trust.

Technical references: [LANG](#) · [CLASS](#) · [GUIDELINES](#).

Chapter 18 — Controlling Object Construction and Destruction

An object’s first observable state should already be valid. HarborWorks does not want a quantity that must be remembered as “unconfigured” until a separate setup call succeeds. Construction can establish the invariant, while destruction connects scope exit to cleanup.

The ordering rules matter: members are constructed in declaration order and destroyed in reverse order. A carefully designed owner can rely on those rules rather than on scattered cleanup calls.

In Chapter 17, a default member initializer gave the quantity a valid starting value. That solution was sufficient because zero was a meaningful state. A more general question remains: what should happen when the caller supplies an invalid starting value, or when creating an object requires a resource that cannot be acquired?

Construction answers whether the object can begin its usable life. Destruction answers how an already-constructed object’s responsibilities end. They are related, but they are not symmetrical calls that the application should scatter manually throughout its code. The language connects them to object lifetime, making predictable cleanup possible when the ownership design is sound.

Learning outcome. Explain when members are initialized, establish a class invariant during construction, centralize validation with delegation, and trace cleanup on ordinary scope exit and exception unwinding. Distinguish a completed object from a construction attempt that failed. Recognize when no default object should exist rather than inventing a plausible but invalid placeholder.

18.1 Constructing Valid Objects

18.1.1 Creating Default and Parameterized Constructors

A constructor initializes an object of its class. A default constructor can establish a sensible baseline, while a parameterized constructor accepts the information needed for a different valid state. If a requested state is invalid, the constructor can reject it rather than creating an object that violates its invariant.

This chapter previews exceptions with `std::invalid_argument` and a matching catch in the test. An exception prevents normal completion of construction. The partially constructed complete object is not available for ordinary use, while already constructed subobjects are cleaned up according to language rules. Book II develops resource safety across these failure paths in detail.

A constructor has the class’s name and no return type. It is selected when an object is initialized; it is not an ordinary member function you call later to reset an existing object. In `Quantity stock{7};`, the argument participates in creating `stock`. By the time the declaration completes successfully, the public quantity invariant should already hold.

A default constructor is callable without arguments. That does not require the class to have no other constructors. In this example, `Quantity{}` represents a valid zero quantity, while `Quantity{7}`

represents a valid supplied quantity. Both construction paths must establish the same interval rule. Their different argument lists do not justify different definitions of validity.

The parameterized constructor is marked `explicit`. This prevents certain implicit conversions from an integer to the domain type. A function expecting a `Quantity` should not silently accept an unrelated integer expression merely because a converting constructor exists. Writing `Quantity{7}` at the call site makes the domain conversion deliberate. Explicit construction remains possible; the keyword does not prohibit direct initialization.

A constructor cannot report failure by returning false because it has no ordinary return value. The teaching implementation throws `std::invalid_argument` when the supplied quantity is outside the permitted interval. Later designs can use a separate factory that returns an optional or another result type. For now, understand the chosen contract: either construction completes with a valid quantity, or no usable quantity is delivered by that declaration.

Catch an exception at a boundary capable of deciding what failure means. The unit test catches the specific invalid-argument exception because rejection is the expected outcome. A user interface might translate it into an input message. Do not catch everything and continue as though the object were created successfully. There is no completed local variable to inspect after its initialization throws.

A useful review question is whether a constructor's arguments describe everything required for a valid object. If a type cannot operate without a real identifier, require that identifier. A separate `initialize()` call creates an additional state and additional obligations unless the application genuinely needs a two-phase lifecycle.

18.1.2 Using Member Initializer Lists

A member initializer list follows the constructor's parameter list and initializes base classes and members before the constructor body runs. Use it to initialize members directly rather than default-constructing them and assigning later. Const members and references require appropriate initialization rather than later reseating.

Actual member construction follows declaration order in the class, not the order written in the initializer list. Keep the written order consistent with declaration order to avoid misleading readers and compiler warnings. Do not initialize an earlier member from a later member that has not yet been constructed. If dependencies are complicated, reconsider the representation or compute validated inputs before construction.

Members are initialized before the constructor body executes. The member initializer list appears after a colon, before the body. In `Quantity(int value) : value_(value)`, the stored member is initialized from the parameter rather than first being created in some other state and then assigned inside the body.

Initialization and assignment are not interchangeable operations. A reference member must be bound as part of initialization; assigning through it later would affect the referenced object rather than reseat the reference. A const data member also needs its initial value established appropriately. Library members such as strings and vectors can often avoid unnecessary intermediate work when constructed directly with their desired values.

For the ordinary nonstatic members used here, initialization follows their declaration order in the class, not the visual order of entries in the initializer list. Write the initializer list in declaration order to keep the source consistent with execution. A warning about reordered initialization is valuable because a dependency may otherwise appear to work in an order the language does not use. [Constructor initialization reference](#).

Consider a class with a stored capacity and a vector whose initial size depends on that capacity. If the vector is declared first but its initializer reads the later capacity member, rearranging the initializer-list text does not repair the dependency. Establish a sensible declaration order or compute both members directly from a validated constructor argument. Prefer avoiding unnecessary member-to-member initialization dependencies when a simpler representation exists.

The constructor body can validate a fundamental member after that member's initialization without making an invalid object available to ordinary callers. If validation throws, the enclosing construction fails. However, do not publish `this`, call arbitrary external callbacks, or otherwise expose the partially constructed object. The example's constructor only checks its own argument and establishes its own rule.

A default member initializer supplies a value when the selected constructor does not initialize that member explicitly. An explicit member initializer takes precedence for that construction path. Use this feature for genuine common defaults, not as a second competing place to encode the same validation algorithm.

Constructor Rejecting an Invalid Range

Purpose. A constructor should establish the conditions required for every valid object of the class.

```
#include <iostream>
#include <stdexcept>
class Range{
    int low_, high_;
public:
    Range(int low,int high):low_(low),high_(high){ if(low>high) throw
std::invalid_argument("low > high"); }
    int width() const { return high_-low_; }
};
int main(){ Range r{3,8}; std::cout << r.width() << '\n'; }
```

How it works.

- The member-initializer list establishes both members before the constructor body.
- The validation prevents a constructed object from surviving with `low > high`.
- After construction succeeds, member functions can rely on the invariant.

Expected behavior/output. The valid range prints width `5`.

Experiment. Try `Range{8,3}` and catch `std::invalid_argument` to observe failed construction.

18.2 Simplifying Constructor Logic

18.2.1 Delegating Between Constructors

A delegating constructor calls another constructor of the same class to perform common initialization. A default constructor can delegate to a parameterized constructor with a standard starting value, ensuring that validation and invariant establishment are not duplicated.

Delegation is useful when the constructors represent different convenient entry points into the same valid state model. It should not create a maze of constructors that repeatedly transform partially understood inputs. Keep the main validation path clear. When construction requires several fallible external operations, a factory or builder may make the staged process more explicit, but the final object should still emerge in a valid state.

Delegation lets one constructor use another constructor of the same class to perform the shared construction work. The example's default constructor delegates to the parameterized constructor with zero. As a result, there is one interval check to maintain rather than a duplicated check in every constructor body.

The target constructor completes before the delegating constructor's body runs. Any additional work in the delegating body therefore occurs after the delegated construction has succeeded. Keep that additional work small and justified. The benefit disappears if every constructor then makes unrelated changes that can violate the invariant established by the shared path.

Do not create a delegation cycle. A default constructor that delegates to an integer constructor, which in turn delegates back to the default constructor, has no valid construction endpoint. Draw the constructor paths as a small directed graph: every path should reach a nondelegating constructor that actually initializes the members.

A delegating initializer names the class itself rather than one of its members. It is not a request to create a temporary and assign it into the object. The distinction matters because assigning a temporary in the body would operate only after the current object's members had already been initialized by some other path.

Delegation is appropriate when constructors differ mainly in how they supply arguments to the same valid representation. It is not a substitute for deciding whether an overload should exist. A constructor with ten loosely related optional parameters can still be difficult to use, even if all its overloads delegate correctly. Prefer an interface that makes required information and defaults clear at the call site.

In the quantity example, zero is the chosen default because an empty count is meaningful. A network endpoint, device serial number, or authenticated session may have no equivalent universally meaningful default. Such a type should not gain a default constructor merely for stylistic symmetry with another class.

18.2.2 Providing Sensible Default Values

Default member initializers and default constructors reduce uninitialized state, but a default should have a real domain meaning. Zero quantity is sensible for an empty stock count. A fake network port or an empty required credential may create a misleading object that cannot perform its advertised role.

When no useful default exists, require the necessary parameter or represent the absence explicitly with a separate type. Do not invent a magic invalid value merely to make default construction possible. A type that cannot be default-constructed can be easier to use correctly than a type that silently begins in an undocumented unusable state.

A sensible default is a valid state with a clear meaning. Zero quantity means nothing is currently counted. An empty vector means the collection currently contains no records. These defaults support useful operations without requiring an invisible setup protocol.

A fabricated identifier such as one, a fake filename, or an arbitrary network address is different. It may satisfy a superficial type check while pointing to the wrong resource. The program can then run long enough to cause a misleading result. Requiring the caller to supply the value often produces a much earlier and more understandable failure.

Separate a default value from an absence marker. An optional reading may legitimately have no value; a numeric zero reading is still a value. Treating zero as missing when zero is valid merges two states the application needs to distinguish. Book II introduces dedicated optional types, but the design decision is already visible here.

The same question applies to public APIs. A default capacity of five can be reasonable for a teaching grouping function whose domain defines groups of five. It would be surprising in a general storage allocator with no such convention. Defaults communicate assumptions and should be documented as part of the contract, not concealed as implementation conveniences.

A default should also preserve the type's invariant after future maintenance. If the accepted quantity range later changes to begin at one, the zero default becomes invalid. A single validation path catches the inconsistency immediately. Separate duplicated constructors may drift apart and allow one creation path to bypass the revised rule.

Test the default state explicitly, just as you test supplied values. Do not assume that an empty pair of braces proves domain validity. Braces control initialization syntax; the constructor and member rules determine what the resulting object means.

Default Member Initializers and Delegating Constructors

Purpose. Constructor delegation and member defaults reduce duplicated initialization paths that can otherwise drift apart.

```
#include <iostream>
#include <string>
class User{
    std::string name_;
    int level_{1};
public:
    explicit User(std::string n):name_(std::move(n)){}
    User(std::string n,int level):User(std::move(n)){ level_=level; }
    void print() const { std::cout << name_ << ' ' << level_ << '\n'; }
};
int main(){ User a{"Ada"}; User b{"Linus",3}; a.print(); b.print(); }
```

How it works.

- The default member initializer supplies the common level.
- The two-argument constructor delegates common name initialization to the simpler constructor.
- Shared initialization logic is written once instead of duplicated.

Expected behavior/output. The output contains `Ada 1` and `Linus 3`.

Experiment. Change the default level to 2 and identify which constructor results change.

18.3 Cleaning Up Objects

18.3.1 Understanding Destructor Execution

A destructor runs as a class object's lifetime ends through ordinary scope exit, deletion by an owner, or other specified lifetime-ending operations. Local objects are destroyed in reverse order of completed construction. This ordering lets an inner helper finish before the outer context it depends on is destroyed.

Do not promise destructor execution after every conceivable process failure. Abrupt termination, certain fatal signals, or power loss do not provide normal stack unwinding. Destructors are a language-level cleanup mechanism, not a replacement for durable storage protocols or external recovery. Also avoid allowing cleanup destructors to throw during stack unwinding, because that can terminate the process.

A destructor ends an object's class-specific responsibilities. For a local class object whose construction completed, leaving its scope normally invokes its destructor. This includes reaching the closing brace and returning from the function. During ordinary exception propagation, stack unwinding destroys completed automatic objects in the scopes being exited before control reaches a matching outer handler.

The example makes lifetime visible with a borrowed counter rather than printing from the destructor. Constructing a probe increments the counter; destroying it decrements the counter. Nested scopes

produce the sequence zero, one, two, one, zero. Predict this sequence before running the source, then connect each transition to one declaration or scope boundary.

The outer counter must outlive every probe that refers to it. The probe's destructor accesses that borrowed object. A cleanup function can itself contain a lifetime defect when its dependencies have already ended. Resource management does not eliminate lifetime reasoning; it makes the ownership and dependency relationships explicit enough to reason about.

An object's destructor body executes before its nonstatic data members are destroyed. Those members are subsequently destroyed in reverse construction order. This is why a class can normally rely on its members while its own destructor body is running. Inheritance adds further ordering rules, which belong to Book II rather than this first lifetime experiment. [Destructor reference](#).

Do not explicitly call the destructor of an ordinary local object to make cleanup happen early. The language still expects to destroy that local object at its normal lifetime boundary, and manually disrupting the lifetime requires specialized reasoning not needed here. Introduce a smaller scope when you need a local resource to finish sooner.

Cleanup on normal scope exit is not a promise about every way a process can stop. Forced termination, operating-system failure, and abrupt termination paths do not necessarily run ordinary C++ destructors. Persistent data and external systems therefore require their own durability and recovery design. A destructor is a lifetime mechanism, not a complete disaster-recovery strategy.

18.3.2 Connecting Destruction to Resource Management

Resource Acquisition Is Initialization, or RAII, associates ownership with an object's lifetime. A file wrapper closes its descriptor when destroyed; a lock guard unlocks its mutex; a vector releases its buffer. The owner should acquire or adopt the resource in a valid state and perform cleanup exactly once.

A custom destructor is not automatically necessary for every class. When members already own their resources correctly, their destructors compose and the compiler-generated destructor may be enough. Book II calls this the Rule of Zero. The useful question is which resource needs an owner, not how many destructor bodies a programmer can write.

The probe resembles a tiny resource owner because acquiring its responsibility changes a count and releasing it reverses that change. Its copy operations are deleted. An implicitly copied reference member would refer to the same counter without running the probe's ordinary incrementing constructor, while both objects' destructors would decrement. The result would violate the counting rule.

This is an early demonstration of why resource-like types need deliberate copy and move policies. It is not an invitation to implement every special member immediately. Book II develops the rule of zero, unique ownership, and move semantics. In Book I, recognize that a class's generated operations must agree with what ownership means for that class.

A class containing a `std::vector` or `std::string` usually needs no handwritten destructor just to release those members' storage. Their own destructors already manage it. Adding a destructor that manually deletes memory obtained from a vector would be incorrect because the vector remains its owner. Prefer composing reliable owning members rather than duplicating their cleanup mechanisms.

When a constructor throws, the destructor of the not-fully-constructed enclosing object is not called. Members and bases whose construction completed are cleaned up according to the language rules. Consequently, acquiring a raw resource in the constructor body and relying on the enclosing destructor to clean it if a later step throws is fragile. An owning member can keep the acquired resource covered even when the whole constructor fails.

Destructors should not normally let failures escape. An exception escaping cleanup during another active exception can terminate the program. Operations whose failures require an actionable result may need an explicit operation before destruction, with destruction retaining a safe final cleanup role. The simple counter decrement cannot fail under its stated lifetime and counting preconditions.

The professional habit begins now: for every resource, name the owner, the acquisition point, the release point, and the behavior of each failure path. If two objects both appear responsible for release, or nobody does, improve the ownership design before adding more error handling.

Destructor-Driven Cleanup with a File Stream

Purpose. Destructors are the normal mechanism for deterministic resource cleanup in C++ classes.

```
#include <fstream>
#include <iostream>
void write_report(){
    std::ofstream out("report.txt");
    out << "complete\n";
}
int main(){ write_report(); std::ifstream in("report.txt"); std::string s;
std::getline(in,s); std::cout << s << '\n'; }
```

How it works.

- `std::ofstream` acquires a file resource when opened.
- Leaving `write\_report` destroys the stream and closes the file automatically.
- The subsequent input stream can then reopen and read the completed file.

Expected behavior/output. The program writes and then prints `complete`.

Experiment. Return early from `write\_report` after writing and verify that the stream still closes when the function exits.

Worked example — Establish an invariant once and observe cleanup order

Requirement and simplest approach

HarborWorks needs a quantity that is valid immediately after construction and a visible demonstration that completed local objects are cleaned up when their scopes end. A tempting shortcut is to provide an

empty constructor plus a setter and hope every caller remembers to invoke the setter. That creates a state that the type does not need and distributes responsibility across callers.

The selected design makes the quantity constructor enforce the interval. A separate lifetime probe observes scope behavior without confusing the domain quantity with a fake external resource. Keeping these teaching responsibilities separate prevents the example from suggesting that every value class needs a custom destructor.

Construction and cleanup ledger

Before reading the implementation, write four questions: What is initialized first? What state exists if validation fails? Which objects have completed construction when the exception is thrown? Which object owns the counter used by cleanup? The answers should identify actual declarations and paths, not rely on a vague notion that C++ “cleans everything.”

Trace the implementation

The default quantity delegates to the validated constructor. The `explicit` keyword prevents unintended implicit conversion from an integer in contexts where that would obscure the domain operation. Invalid construction throws, and the test catches that specific exception to confirm rejection.

`LifetimeProbe` borrows a counter that outlives both probes. Construction increments it and destruction decrements it. Copying is disabled because an implicit copy would not perform the increment while its destructor would still decrement, breaking the counting invariant. This is a preview of why resource-like classes need deliberate special-member behavior.

Nested scopes make the cleanup sequence observable without printing from destructors. The inner probe disappears first, then the outer probe. Returning early from inside either scope would still destroy completed automatic objects on that normal return path. The example never relies on cleanup after abrupt process termination.

Complete worked source

source/ch18/main.cpp

```
#include <iostream>
#include <stdexcept>

class Quantity {
public:
    Quantity() : Quantity(0) {}
    explicit Quantity(int value) : value_(value) {
        if (value < 0 || value > 1000) {
            throw std::invalid_argument("quantity out of range");
        }
    }
    int value() const { return value_; }
private:
    int value_;
};

class LifetimeProbe {
```

```

public:
    explicit LifetimeProbe(int& live) : live_(live) { ++live_; }
    ~LifetimeProbe() { --live_; }
    LifetimeProbe(const LifetimeProbe&) = delete;
    LifetimeProbe& operator=(const LifetimeProbe&) = delete;
private:
    int& live_;
};

int main() {
    Quantity empty;
    Quantity stock{7};
    if (empty.value() != 0 || stock.value() != 7) return 1;
    bool rejected = false;
    try { Quantity invalid{-1}; }
    catch (const std::invalid_argument&) { rejected = true; }
    if (!rejected) return 2;
    int live{};
    {
        LifetimeProbe first{live};
        { LifetimeProbe second{live}; if (live != 2) return 3; }
        if (live != 1) return 4;
    }
    if (live != 0) return 5;
    std::cout << "live=" << live << '\n';
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root, build and run this chapter’s supplied fixtures:

```
python3 tools/run_chapter.py 18
```

The executable target is `b01_c18`. Read `source/ch18/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Evidence and interpretation

The printed live count of zero is the final observation. The checks inside the nested scopes are equally important: they establish that both acquisitions occurred before both releases. A counter that always stayed zero could produce the same final printout while failing to demonstrate construction entirely.

The invalid-quantity test must observe the specific rejection path. A program that simply skips construction for a negative value has not tested the constructor. Put the declaration itself inside the try block, catch the intended exception, and check a separate rejection flag after the handler.

Failure clinic — cleanup during an exception

Add a completed probe, then throw `std::runtime_error` before the scope’s closing brace. Catch it outside the probe’s scope and check that the counter is again zero. The probe should be destroyed while the exception propagates; the code after the throw inside that scope should not execute. This is different from throwing inside the probe’s own constructor before that probe has completed construction.

For a second experiment, attempt to copy a probe in a separate scratch source. Compilation should fail because copying is deliberately deleted. This expected compiler failure belongs outside the default runnable examples. Restore the source before rebuilding the passing suite.

Practice and implementation lab

Add a test that throws an ordinary exception while a probe is in scope and catches it outside that scope. Verify that the live count returns to zero. Explain why the referenced counter must outlive the probe.

Design a class whose default state would be meaningless. State which constructor parameter should be mandatory rather than assigning a fake default.

Starting exercise: solution notes

Stack unwinding destroys the completed probe before control reaches the outer catch, restoring the live count. The destructor accesses its referenced counter, so destroying the counter first would make cleanup itself invalid.

A connection configuration requiring a real endpoint should require that endpoint explicitly. A fabricated default that looks valid but targets the wrong service is worse than a compile-time requirement to supply the necessary information.

Guided lab — audit an object's complete lifetime

Begin with the delivered construction test. Add boundary quantities zero and one thousand, then reject negative one and one thousand and one. For each case, classify the outcome as a completed valid object or rejected construction. Do not inspect a variable whose declaration did not complete.

Next, write a timeline for two probes in nested scopes and repeat it with an ordinary exception leaving the inner scope. Record the count before construction, after each completed constructor, inside the outer handler, and after the outer scope. Keep the borrowed counter outside all probe scopes.

Finally, design a configuration type requiring a nonempty equipment name. Decide whether an empty default configuration is meaningful. Describe its constructor contract before writing code. A justified decision to omit default construction is a successful design outcome, not a missing feature.

Acceptance evidence. Boundary construction behaves as documented; rejected construction is observed explicitly; completed probes release on ordinary scope exit and tested exception unwinding; no handwritten cleanup duplicates standard-member ownership; and the dependency used by each destructor outlives the destructor call.

Review and solution notes

Does a constructor-body assignment initialize a reference member? No. Reference binding occurs during initialization before the body. Assignment through a bound reference operates on its referent.

Does reversing the initializer-list text reverse member construction? No. For these ordinary members, declaration order determines construction order. Match the written initializer order to that order and avoid relying on an uninitialized later member.

Why is the probe noncopyable? Its acquisition and release accounting would not match an implicitly generated copy. Deleting copying makes that unsupported operation visible at compile time.

Does a failed enclosing constructor run the enclosing destructor? Not for an object that never completed construction. Already-constructed member owners provide the important cleanup coverage during that failure.

Chapter takeaway. A valid lifetime begins with a successful construction contract and ends through a deliberate ownership model. Defaults, delegation, cleanup order, and copy policy should all support the same invariant.

Technical references: [LANG](#) · [CLASS](#) · [LIFETIME](#) · [GUIDELINES](#).

Chapter 19 — Organizing Programs Across Multiple Files

HarborWorks separates a calculation from the program that uses it. The change should not alter behavior; it should make the interface reusable and the build dependencies explicit. Multiple files introduce a new boundary: each source file is compiled independently, so declarations must be available where calls are checked and definitions must be available when references are linked.

A header is an interface artifact, not a place to paste arbitrary copies of implementation until the linker stops complaining.

A single source file hides an important engineering boundary: all of its declarations are visible to one compiler invocation after preprocessing. Splitting the same behavior into several files requires you to make that information flow explicit. The calling source needs an interface, and the final executable needs the corresponding implementation.

The purpose of this chapter is not to maximize file count. It is to separate a reusable calculation from a particular program entry point without changing the calculation's contract. HarborWorks will move the grouping function into a small library-shaped component, then deliberately distinguish compilation failures from linking failures.

Learning outcome. Create a self-contained header, include that header from both caller and implementation, compile independent translation units, link the required objects, and protect repeated textual inclusion. Explain why a clean rebuild is useful evidence but not a substitute for understanding the dependency that was wrong.

19.1 Separating Declarations from Definitions

19.1.1 Creating Header Files

A header normally contains declarations needed by more than one translation unit: functions, class definitions, constants with appropriate linkage, and templates whose definitions must be visible for instantiation. Include the headers required by the declarations themselves so that the header can be used without relying on an accidental include order.

Keep unnecessary implementation dependencies out of public headers. Every additional include can increase build coupling and expose details callers should not need. Avoid global using-directives in a header. A small header that states a complete interface is easier to test by compiling a file that includes only that header first.

A header describes information that more than one translation unit needs to share. For the grouping component, that information is a function declaration inside the `harbor` namespace. The caller needs to know the function's name, parameter types, and result type before the compiler can check a call. It does not need to see every implementation statement.

Make a header self-contained: a source file should be able to include it without first including some unrelated header by accident. If a declaration uses `std::string`, the header should include the standard header that makes that name available. Relying on another file to include the right dependency first creates order-sensitive builds that fail far from the actual omission.

Do not put `using namespace std;` into a shared header. Every source file that includes it would inherit that broad name exposure in the affected scope. Qualify standard-library names explicitly and put your project's declarations in a deliberate namespace. An interface should provide its intended names, not change unrelated lookup behavior for all of its consumers.

A declaration and a definition answer different questions. `int groups_needed(int items, int capacity);` tells the caller how the function can be used. The body that computes the result supplies the definition. Many declarations of the same function can be visible in compatible contexts, but this ordinary non-inline function should have one intended defining implementation in the program.

Headers can legitimately contain more than declarations of free functions. Class definitions, templates, and appropriate inline definitions commonly belong there. The beginner rule is therefore not “headers never contain code.” It is “put shared interface information where consumers need it, and respect the definition rules for each entity.” Templates and library design are developed more fully in Book II.

A header is a source dependency, not ordinarily the object file that resolves an external call. Adding a header filename to an editor project may help navigation, but it does not replace compiling the implementation source that defines the function.

19.1.2 Creating Implementation Files

An implementation file includes its own header and supplies out-of-line definitions. Including the corresponding header first helps detect mismatches between the declared interface and the implementation. Other required headers should be included directly rather than obtained accidentally through unrelated includes.

Do not include a `.cpp` file as a substitute for adding it to the build. That changes translation-unit boundaries and can produce duplicate definitions when the same file is also compiled separately. There are specialized unity-build techniques, but they are deliberate build strategies, not a beginner repair for an undefined reference.

The implementation file includes its own public header and defines the declared behavior. Including the header first helps reveal missing interface dependencies and keeps the implementation connected to the exact declaration consumers use. It does not magically prove that every intended declaration has a matching definition, so the link step and tests remain necessary.

Keep the function's namespace and parameter types consistent with its declaration. Accidentally defining a global `groups_needed` while the header declares `harbor::groups_needed` creates a different function. Likewise, changing an integer parameter to a floating-point parameter may define an overload instead of satisfying the original declaration. The source can contain valid C++ while the intended executable still lacks the required definition.

Implementation-only helpers should not become public merely because placing them in the header is easy. A small helper used only by `groups.cpp` can remain local to that translation unit, for example in

an unnamed namespace when that concept becomes useful. The public interface should expose the operations consumers need, not every step used to implement them.

Avoid including an implementation `.cpp` file from another `.cpp` file as a routine way to solve linking errors. Textually including the definition may make one build appear to work while another separately compiles the same implementation and produces duplicate definitions. The dependency belongs in the build target, not in an accidental source inclusion arrangement.

The grouping implementation should remain independent of terminal input and output. It returns a result under its existing contract. The demonstration application decides what to print and how to report a failed test. This separation makes the calculation usable in a second program without importing the first program's `main` function or console protocol.

An implementation change that preserves the declared contract should not require edits in every caller. That is the practical benefit of separating interface from implementation: the codebase gains a controlled boundary for change rather than merely another file to open.

Header Declaration with Source-File Definition

Purpose. Separating interface from implementation lets multiple translation units agree on a declaration without duplicating the definition.

```
// math.hpp
#pragma once
int add(int a, int b);

// math.cpp
#include "math.hpp"
int add(int a, int b) { return a + b; }

// main.cpp
#include "math.hpp"
#include <iostream>
int main(){ std::cout << add(2,3) << '\n'; }
```

How it works.

- The header exposes the function declaration to every consumer.
- The source file owns the one non-inline definition.
- `main.cpp` depends on the interface, not the implementation body.`

Expected behavior/output. When compiled and linked together, the program prints ``5``.

Experiment. Change the header parameter type without changing `math.cpp`` and observe the consistency failure.

19.2 Understanding Translation Units

19.2.1 Understanding Separate Compilation

Each source file, after preprocessing its included material, forms a translation unit that the compiler checks independently. A function used in one translation unit must have an appropriate declaration visible there. The compiler does not automatically inspect every other source file in the directory to discover declarations.

Separate compilation supports incremental builds: changing one implementation may require recompiling only affected targets. Changing a widely included header can trigger much more work. This cost encourages stable, minimal interfaces, but do not sacrifice clarity merely to avoid every rebuild. Correct dependency tracking is more important than a fast build that accidentally reuses stale objects.

A translation unit is the source presented to the compiler after preprocessing has incorporated the relevant includes and other preprocessing effects. Two `.cpp` files usually produce two translation units even when they include the same header. Each is checked with the information available in that unit.

Compiling only `main.cpp` with `-c` can succeed because the header supplies a valid declaration for the call. The resulting object file can still contain an unresolved reference to the function. That is not an incomplete compiler error; it is a normal intermediate build artifact whose external references are handled later.

The manual sequence makes the boundary visible:

```
# From source/ch19 in a disposable build exercise:
g++ -std=c++20 -Wall -Wextra -Wpedantic \
    -g -c main.cpp -o main.o
g++ -std=c++20 -Wall -Wextra -Wpedantic \
    -g -c groups.cpp -o groups.o
g++ main.o groups.o -o groups_demo
./groups_demo
```

These commands write build artifacts into the exercise directory. The supplied CMake workflow instead keeps them under a separate build tree. Use the manual sequence to learn the stages, then prefer the organized build for ordinary work. Removing generated `.o` files should never remove the only copy of authored source.

A header change can require recompiling every translation unit that depends on it. A change confined to the implementation body may require rebuilding only the implementation object and relinking its consumers. Build systems record these dependencies so you do not have to remember them manually.

Separate compilation does not guarantee that incompatible compiler options or binaries can be mixed. The language mode, architecture, standard-library implementation, configuration definitions, and binary interface may matter. In this book, keep one coherent toolchain and build configuration per build directory. Book V studies those compatibility questions in detail.

When a program behaves as though a change did not happen, verify the source path and executable path first. The editor may be displaying one checkout while the shell builds another. A timestamp is useful

context, but the actual compile command and test result provide stronger evidence about what was rebuilt.

19.2.2 Understanding the Linker's Role

The linker resolves references among compiled objects and libraries. Every required noninline function definition must be supplied in a compatible form. A missing source file in the link can cause an undefined reference even when all individual compilations succeeded.

The One Definition Rule constrains how entities are defined across the program. Putting an ordinary noninline function definition in a header included by several source files can create multiple definitions. Inline functions and templates have different permitted patterns when their requirements are met. Do not resolve duplicate symbols by arbitrarily deleting object files; understand which entity was defined more than the language permits.

The linker combines the selected object files and libraries into the final program and resolves compatible external references. The grouping example's caller refers to a function whose definition is supplied by the implementation object. Omitting that object leaves a reference the final link cannot satisfy.

Read the stage and the symbol name in a diagnostic. An undeclared-name error during compilation means the current translation unit lacks a usable declaration. An undefined-reference error during linking often means the definition is missing from the selected inputs or differs from the declaration the caller used. The correct repair depends on which question remains unanswered.

A multiple-definition error is another category. Two separate non-inline definitions of the same ordinary function may have entered the link. Common causes include putting a function body in a widely included header without the appropriate design, including a `.cpp` file and also compiling it, or listing duplicate implementation sources through an incorrect build arrangement.

Do not respond to every linker error by adding more include directives. Includes affect the source information seen by a compiler. Link inputs determine which separately compiled definitions participate in the executable. The two mechanisms cooperate but are not substitutes for one another.

The compiler driver is normally the convenient way to invoke the link for a C++ executable. It arranges the standard C++ runtime and library options expected by that toolchain. Calling a low-level linker directly is possible, but it adds platform-specific responsibilities that do not help this beginner exercise.

An executable normally has one program entry point defined by its application. A second file containing another `main` should become a different executable target, not be linked into the first merely because both use the same calculation. Share the calculation's implementation, not competing entry points.

Successful linking proves that the selected definitions satisfy the build's symbol-resolution needs. It does not prove the calculation is correct. Keep the grouping tests after the link so the full learning chain is declaration, implementation, combination, and observed behavior.

Two Translation Units Sharing One Header Contract

Purpose. Translation units explain why declarations must be shared consistently and definitions must be linked exactly where needed.

```
// counter.hpp
#pragma once
int next_value();

// counter.cpp
#include "counter.hpp"
int next_value(){ static int n=0; return ++n; }

// main.cpp
#include "counter.hpp"
#include <iostream>
int main(){ std::cout << next_value() << '\n'; }
```

How it works.

- `counter.cpp` and `main.cpp` are preprocessed and compiled independently.
- Both see the same declaration from `counter.hpp`.
- The linker combines the object files and resolves the call to the one definition.

Expected behavior/output. With both source files linked, the program prints `1`.

Experiment. Compile only `main.cpp` and observe that compilation succeeds but linking fails because the definition is absent.

19.3 Preventing Header Problems

19.3.1 Using Include Guards

An include guard wraps a header in a unique macro condition so that its contents are not repeatedly processed within one translation unit. The pattern uses `#ifndef`, `#define`, and a matching `#endif`. Choose a project-specific guard name that is unlikely to collide with another header and avoid reserved identifier forms.

A guard prevents repeated inclusion in one translation unit; it does not give an ordinary external definition permission to appear in multiple translation units. This distinction explains why a guarded header can still cause linker duplicate-definition errors. Header idempotence and program-wide definition rules solve different problems.

An include guard uses a preprocessor macro to suppress repeated processing of the guarded header within one translation unit. On the first inclusion, the macro is not yet defined, so the guarded contents are processed and the macro becomes defined. A later inclusion sees the macro and skips the guarded contents.

Use a distinctive macro name based on the project and header. A generic macro such as `HEADER_H` can collide with another header's guard, silently suppressing declarations that should be present. Avoid inventing names reserved to the implementation. A readable project prefix is both clearer and safer than unusual underscore patterns.

The guard protects repeated textual inclusion in a translation unit. It does not give the entire program one global copy of every definition in the header. Two different `.cpp` files each start their own preprocessing context and may each process the guarded header once. This distinction explains why include guards alone do not repair inappropriate ordinary function definitions in shared headers.

The deliberate repeated include in the worked example demonstrates the local guard behavior. It should not create an error from processing the same guarded declarations twice. More realistically, repeated inclusion can arrive indirectly when one header includes another that the source also includes directly.

Do not use inclusion order to hide a broken dependency. If header A requires a type from header B, make that requirement explicit in the header design rather than hoping every source includes B first. Some cases permit a forward declaration, but choosing between a forward declaration and a full include requires understanding what the declaration actually uses.

For this chapter, the grouping header has no unnecessary standard-library dependency because its interface uses only fundamental types. That small example makes the boundary easy to see. As interfaces grow, review their includes and declarations as part of the same public design.

19.3.2 Using `#pragma once`

`#pragma once` is a widely supported implementation facility that requests single processing of a header in a translation unit. It is concise, but it is not the standard-language include-guard mechanism. Toolchains and filesystem arrangements can have details worth considering in unusual build environments.

Either approach can work in a documented project. The companion uses ordinary include guards for small public teaching headers so that the mechanism is visible. Consistency is useful, but blindly adding both forms is not a substitute for understanding the header's contents. Neither approach fixes circular conceptual dependencies between classes that need a better interface design.

`#pragma once` is a widely implemented directive that asks the compiler to process a header once per translation unit. It is concise and supported by the compiler families discussed in the book. It is not itself a portable directive specified by the C++ language standard, so projects targeting unusual toolchains may prefer conventional guards.

Both approaches address repeated inclusion, not arbitrary duplicate-definition problems across different translation units. Replacing an include guard with `#pragma once` will not make a non-inline function body safe to define independently in every compiled source file. The definition's language rules still apply.

Choose one consistent project convention unless there is a concrete compatibility reason to differ. Combining both mechanisms is possible but is not necessary for the small examples. Consistency helps reviewers focus on the declarations and dependencies rather than treating every header as a new stylistic puzzle.

Filesystem identity and unusual include arrangements can complicate implementation-specific once behavior. Do not use symlink tricks, copied headers with conflicting definitions, or case-dependent paths to test the limits of a beginner build. Keep a single authoritative public header and a clear include path.

The larger lesson is to know which guarantees come from the language and which come from a toolchain convention. A directive can be useful without being part of the language standard. Record a project's supported compilers so such choices remain deliberate instead of becoming hidden assumptions.

Headers are also read by people. Keep a short contract near the declaration when the behavior is not obvious: accepted ranges, invalid-input result, and side effects. Do not repeat the entire function implementation in comments. A useful contract remains valid even when the internal arithmetic is refactored.

Preventing Multiple Inclusion with `#pragma once`

Purpose. Header guards prevent duplicate declarations caused by transitive and repeated includes.

```
// point.hpp
#pragma once
struct Point { int x; int y; };

// a.hpp
#pragma once
#include "point.hpp"

// main.cpp
#include "point.hpp"
#include "a.hpp"
int main(){ Point p{1,2}; return p.x+p.y==3 ? 0 : 1; }
```

How it works.

- Both include paths eventually request `point.hpp`.
- `#pragma once` ensures the header's declarations are processed once per translation unit.
- Without an inclusion guard, the class definition would appear twice in `main.cpp` after preprocessing.

Expected behavior/output. The program compiles and exits successfully.

Experiment. Temporarily remove `#pragma once` from `point.hpp` and inspect the redefinition diagnostic.

Worked example — Compile and link a small library interface

Requirement and simplest approach

HarborWorks wants two potential applications to share the grouping calculation without copying its source into each application. The simplest shortcut is to paste the same function into both programs. That works until one copy is fixed and the other is forgotten. The selected design gives the calculation one interface and one implementation source that each relevant target can compile or link.

Build dependency view

```
groups.hpp -> declaration visible to main.cpp
           -> declaration visible to groups.cpp
main.cpp   -> main.o -----+
groups.cpp -> groups.o ---+--> groups_demo -> behavior checks
```

The arrows describe build dependencies, not execution order. The header is processed during compilation; it does not run before the function. At runtime, `main` calls the linked function according to the program's control flow.

Small controlled change

First build the delivered version and record its passing result. Then omit the implementation object only from a separate manual link experiment. Preserve the working source. This creates a bounded failure whose cause is known and whose repair is restoring the missing build input, not editing unrelated language syntax.

Trace the implementation

The header declares the function inside a namespace. `groups.cpp` defines it, and `main.cpp` calls it through the declared interface. The repeated include in `main` is intentional: it demonstrates the guard without creating another definition.

A manual build can use `g++ -std=c++20 -c groups.cpp, g++ -std=c++20 -c main.cpp`, and then `g++ groups.o main.o -o groups_demo`. Omitting `groups.o` from the final command leaves the caller's reference unresolved. Compiling both source files in one driver command is also valid; the driver still manages the underlying compilation and link stages.

The companion CMake target lists both implementation files and provides a reproducible alternative to manual object commands. The header itself is not a separately linked implementation. This distinction prepares you to build libraries without relying on source-file inclusion tricks.

Complete worked source

source/ch19/groups.hpp

```
#ifndef HARBOR_GROUPS_HPP
#define HARBOR_GROUPS_HPP
namespace harbor {
int groups_needed(int items, int capacity);
}
#endif
```

source/ch19/groups.cpp

```
#include "groups.hpp"
namespace harbor {
int groups_needed(int items, int capacity) {
    if (items < 0 || capacity <= 0) return -1;
    return items / capacity + (items % capacity != 0 ? 1 : 0);
}
}
```

source/ch19/main.cpp

```
#include "groups.hpp"
#include "groups.hpp" // The guard makes repeated inclusion harmless here.
#include <iostream>

int main() {
    if (harbor::groups_needed(23, 5) != 5) return 1;
    if (harbor::groups_needed(0, 5) != 0) return 2;
    if (harbor::groups_needed(3, 0) != -1) return 3;
    std::cout << "groups=" << harbor::groups_needed(23, 5) << '\n';
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

```
python3 tools/run_chapter.py 19
```

The executable target is `b01_c19`. Read `source/ch19/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Evidence and interpretation

The useful observations are distinct: compiling the caller object succeeds; linking that object alone fails; linking both objects succeeds; running the resulting program satisfies the grouping checks. Together, they demonstrate why a declaration can be enough for compilation but insufficient for a complete executable.

A diagnostic naming a different namespace or parameter type points to a different hypothesis than a diagnostic naming the exact declared function with no defining object in the command. Compare the header, definition, and link command before changing anything. Save the first meaningful error rather than only the last build-summary line.

Failure clinic — the stale object

Change the implementation in a scratch copy, but deliberately run the old executable without rebuilding. Its unchanged output is expected: a source edit does not alter an existing executable. Now rebuild the correct target and run that target's path. This experiment reconnects Chapter 1's compilation pipeline to a real multi-file project.

The supplied CMake target provides repeatable source selection. Use `cmake --build build --target b01_c19 --verbose` to inspect actual compile and link commands when diagnosing a mismatch. Verbose output is evidence about the requested build, not permission to copy every generated flag into an unrelated project.

Practice and implementation lab

Build only `main.cpp` and observe the linker failure. Then build both source files and confirm success. Introduce a mismatch between the header's parameter type and the definition, and explain how including the header in the implementation helps reveal interface defects.

Create a second caller that uses the same function. Decide whether it belongs in the same executable or a separate target before writing its own main function.

Starting exercise: solution notes

The standalone caller compiles because it sees a declaration, but linking fails because the definition is absent. Adding the defining object resolves the reference. A mismatch may create a distinct overload rather than define the declared function, leaving an unresolved reference; using the shared header and targeted tests makes the discrepancy visible.

An executable has one ordinary program entry point. A second independent main belongs in a separate target that links the same library implementation. Reusable code should not require copying the implementation into each application's source.

Guided lab — produce a reusable calculation boundary

Create a clean exercise directory and copy the chapter's three source files. Compile each implementation source to an object separately, link the objects, and run the checks. Record the command that creates each artifact and the input files that artifact depends on.

Next, write a second small application that calls the same declared calculation for another input. Give it its own executable target or manual link command. Do not link both application entry points together. Confirm that a single implementation fix can be rebuilt into both applications without copying the function body.

Perform three bounded failures separately: remove the header include from the caller, omit the implementation object from linking, and accidentally define the function in the wrong namespace. Classify each observed diagnostic by stage. Restore the working copy after each experiment and finish with a clean passing build.

Acceptance evidence. The public header is self-contained; caller and implementation agree on one contract; each application has one entry point; the correct defining object reaches each link; repeated inclusion does not break compilation; and tests still verify behavior after the file split.

Review and solution notes

Does an include guard prevent duplicate definitions throughout an executable? No. It suppresses repeated processing within each translation unit. Cross-translation-unit definition rules remain relevant.

Why include the public header in the implementation? It connects the implementation to the shared interface and reveals many inconsistencies early. A mismatch that creates a different overload can still require linking and tests to expose the missing intended definition.

Why not include groups .cpp? The implementation belongs in the target's compiled source inputs. Textually including it and also compiling it can introduce duplicate definitions and an unclear dependency model.

What should a clean build prove? That the selected authored inputs can reproduce the executable without relying on leftover intermediate files. It does not prove every supported platform or every behavior has been tested.

Chapter takeaway. Make interface visibility and implementation availability explicit. A multi-file program is reliable when its source boundaries, build dependencies, and behavioral contracts describe the same system.

Technical references: [LANG](#) · [ODR](#) · [GCC](#) · [CMAKE](#).

Chapter 20 — Building a Complete Multi-File C++ Application

HarborInventory turns the previous lessons into a small, usable command-line program. It accepts equipment records, rejects duplicate identifiers and invalid quantities, lists the accepted inventory, and reports the total quantity. The goal is a complete, bounded application whose behavior can be explained and tested—not a collection of disconnected syntax demonstrations.

The source is divided into a domain library and an interface. The library does not read the terminal or print diagnostics. The interface translates text commands into library operations and reports results. That division lets tests exercise the rules independently of interactive input.

The capstone is where isolated language features become a design that another person can inspect. HarborInventory has a small command vocabulary, one clear owner of its records, and measurable acceptance conditions. The application does not need inheritance, templates, threads, or a graphical interface to demonstrate sound C++ foundations. It needs the simpler features to cooperate without hidden assumptions.

Treat the application as a chain of boundaries: input bytes become parsed candidates; candidates are checked against domain rules; approved operations change owned state; observers produce results; and tests establish whether the entire chain behaved as specified. A defect at one boundary should not require guessing about every other boundary.

Learning outcome. Build and explain a complete multi-file program, translate requirements into tests, distinguish parsing from domain validation, preserve state after rejection, and present evidence of what the finished application actually does. The final acceptance report should be candid about excluded features rather than describing future aspirations as implemented capabilities.

20.1 Defining Application Requirements

20.1.1 Converting Requirements into Program Features

Write acceptance rules before implementation. An identifier must be positive, a name must be nonempty, and quantity must be between zero and 1000. At most 100 records are accepted. An existing identifier cannot be inserted again. A rejected insertion leaves the collection unchanged. The total is the sum of accepted quantities and therefore remains within a simple, proven integer bound.

The interface supports `add`, `list`, `total`, and `quit`. Names containing spaces are quoted. Unknown commands and malformed arguments produce an error rather than a partially inserted record. Persistence, authentication, concurrent writers, and remote access are outside this capstone's scope. Naming exclusions prevents a working classroom program from being mistaken for a finished enterprise service.

Start with behavior a user can observe. HarborWorks wants to add equipment records, list accepted records, and see the total quantity. Each record has an identifier, a name, and a quantity. A request with

invalid data should be rejected without corrupting the inventory. A duplicate identifier should not silently replace an existing record.

Convert those sentences into precise rules. An identifier must be positive and unique within this inventory. A name must not be empty. A quantity must be between zero and one thousand, inclusive. The collection may contain no more than one hundred records. These limits define the exercise's data model; they are not claims about an industry-wide inventory standard.

Now define the command grammar. Interactive mode reads one line at a time. An **add** command supplies an integer identifier, a name that may be quoted, and an integer quantity. **list**, **total**, and **quit** take no additional arguments. An extra token after an otherwise complete command is rejected. Empty lines are ignored. End-of-file ends the input loop normally unless the input stream reports a different failure condition.

Write examples before implementation. **add 1 "bench scope" 2** should add one record. A second **add** using identifier one should be rejected, even when its name and quantity differ. **add 2 "meter" -1** has syntactically readable integers but fails domain validation. **add x "meter" 1** fails parsing before the domain library receives a valid candidate. These are different failure paths and deserve different tests.

Decide what a rejected command means for the process. This teaching application continues after an ordinary malformed or rejected interactive command, prints a diagnostic, and permits the next line. Therefore, a process exit code of zero does not mean every command in a session was accepted. The session output carries command-level results. Automated consumers must check the contract at the appropriate level.

Requirements also include exclusions. The application does not save records between runs, authenticate users, contact a server, or coordinate concurrent writers. A process restart begins with an empty inventory. State these boundaries prominently so a reader does not mistake a classroom command processor for a durable equipment-management service.

Avoid vague requirements such as “handle all bad input” or “be fast.” Replace them with a defined grammar, limits, and expected outcomes. Then identify residual risks explicitly: an arbitrarily long input line can still consume memory before domain limits are evaluated, for example. A maximum record count is not a universal bound on every allocation in the program.

20.1.2 Modeling Application Data

A **Record** groups identifier, name, and quantity. **Inventory** owns a vector of records and controls insertion. Callers can inspect the records through a const reference but cannot directly mutate the vector through the public interface. The identifier, rather than the display name, determines uniqueness.

This representation favors simple insertion and listing for a small bounded collection. A linear duplicate search is adequate under the 100-record teaching limit. Book III later evaluates maps and indexing when lookup requirements change. Do not add a hash table merely to look advanced; choose complexity in proportion to the workload and keep the invariant easy to inspect.

The **Record** structure gives related values one identity. Its identifier, name, and quantity travel together through validation and storage. This prevents the synchronization problem of parallel containers in which an identifier at one index might accidentally be paired with the wrong name or quantity.

A record is a candidate until the inventory accepts it. The structure itself has public fields and does not individually enforce every inventory rule. That is acceptable here because the invariant belongs to the **Inventory** boundary: only validated records enter the owned collection. Creating an invalid candidate is not the same as allowing invalid persistent domain state inside the inventory.

The vector owns the accepted records, and each string owns its text. The program does not allocate a separate record with **new** for each insertion. The ownership graph is short: a local inventory owns its vector; the vector owns its elements; each record owns its name. When the inventory's lifetime ends, its standard-library members clean up their storage.

Uniqueness is checked by scanning existing identifiers before insertion. With at most one hundred records, a linear scan is easy to understand and sufficient for the exercise. A hash table could improve some asymptotic costs but would add another container choice and possibly another consistency relationship. Book III studies the tradeoff using explicit workload constraints rather than assuming that a more elaborate structure is always better.

The total is computed from accepted quantities using a **long long** accumulator. Its mathematical bound is one hundred multiplied by one thousand, or one hundred thousand. This reasoning is more useful than an unqualified claim that "an integer is big enough." The domain bounds and selected result type jointly justify the computation.

The **records()** observer returns a const reference to the vector. It avoids copying the whole collection and prevents ordinary mutation through that access path. It does not transfer ownership. References, pointers, and iterators to elements remain subject to the vector's lifetime and invalidation rules. The report function consumes the observation immediately rather than retaining element addresses across later insertion.

Name validation is intentionally narrow: nonempty text is required, but the teaching library does not normalize whitespace, enforce Unicode policy, or distinguish visually confusing names. A whitespace-only nonempty string therefore satisfies the delivered domain check. A future requirement can strengthen this policy, but the documentation and tests should not pretend that the current **empty()** check performs those additional validations.

Turning Requirements into Executable Acceptance Checks

Purpose. Requirements become engineering constraints when they are stated precisely enough to test.

```
#include <cassert>
int shipping_cost(int items){
    if(items <= 0) return 0;
    return items <= 3 ? 5 : 8;
}
int main(){
    assert(shipping_cost(0)==0);
    assert(shipping_cost(3)==5);
    assert(shipping_cost(4)==8);
}
```

How it works.

- Each assertion encodes one boundary from the requirement.
- The implementation is intentionally small so the requirement can be evaluated independently.
- The tests cover zero, the upper boundary of the first tier, and the first value of the second tier.

Expected behavior/output. The program produces no output and exits successfully when all acceptance checks hold.

Experiment. Change the boundary rule to “up to 5 items costs 5” and update tests before modifying the function.

20.2 Implementing the Application

20.2.1 Dividing Responsibilities Across Classes and Functions

The inventory library validates records and owns accepted state. The command loop parses one line at a time, ensuring that a failed command does not leave a stream in a confusing partially consumed state for the next command. Reporting is a separate function that formats existing records without changing them.

Parsing produces a local candidate record. Only a fully parsed command is passed to the inventory. The library then validates domain rules before committing the insertion. This creates two boundaries: text syntax and domain validity. A command can be syntactically valid while still attempting a duplicate identifier, and those cases should remain distinguishable in tests.

The program separates the domain library from the command-line adapter. `Inventory::add` decides whether a candidate record may enter the collection. It does not ask the user for a replacement value or print a message. The adapter parses a line, calls the library, and chooses the diagnostic text. This separation keeps domain behavior testable without a terminal session.

Validation should occur before mutation. The library checks the candidate's identifier, name, quantity, collection capacity, and duplicate status before calling `push_back`. Rejected candidates therefore do not change the accepted-record count or total. Tests must inspect state after rejection, because a false return value alone would not expose an implementation that modified the vector first.

The add parameter is passed by value. It represents a local candidate the function may transfer into storage on success. The implementation uses `std::move` at the insertion point, allowing suitable members to transfer their resources. This is a preview of Book II's move semantics, not a statement that the word `move` physically relocates arbitrary objects or guarantees zero work.

When a caller supplies an lvalue record to a by-value parameter, the local candidate is ordinarily a copy of that caller-owned value. Moving from the candidate does not mean the caller's original object was moved from. At other call sites, a temporary candidate may avoid that initial copy. Ownership decisions should follow the actual parameter boundary rather than the presence of a familiar utility name.

The input adapter uses a fresh `std::istream` for each line. Parsing failure on one command therefore does not leave the next line's parser in the failed state. This is a practical reason to isolate line parsing instead of attempting to repair a single long-lived parser after every malformed token.

`std::quoted` allows a name containing spaces to occupy one logical field. It also accepts an unquoted token according to its extraction behavior. The protocol is not a complete CSV format and should not be described as one. Escaping and quoting behavior belongs to the chosen stream operation; arbitrary commas do not create fields automatically.

The adapter checks for an unexpected trailing token after the expected fields. This closes a gap from Chapter 3, where simple formatted integer extraction accepted a numeric prefix. Here, an input such as `add 1 "scope" 2junk` leaves unread text after the quantity and is rejected by the trailing-token check. The comparison between chapters shows how a stronger grammar requires an explicit stronger validation step.

Keep observers simple. Listing formats the accepted records in insertion order; total sums quantities without altering them. A const reference at the library boundary helps the compiler enforce that intent. If an observer later needs a cache, that is a separate design choice with its own consistency obligations, not a reason to casually remove const correctness.

20.2.2 Organizing Source and Header Files

`inventory.hpp` contains the record and class declarations. `inventory.cpp` implements insertion and total calculation. `main.cpp` contains the command interface and a deterministic self-test entry point. Each implementation includes the declarations it depends on, and the build target compiles both implementation files.

The test entry point is intentionally modest so that the complete program remains understandable after this first volume. Later volumes separate test executables, introduce richer test tools, and automate build matrices. The important property already exists: a fresh checkout can compile the same source and run checks without manually entering a particular terminal session.

The public header contains the domain types and declarations. The implementation source contains the validation and aggregation logic. The application source contains the entry point, reporting, self-test

demonstration, and command processing. These file boundaries correspond to responsibilities rather than arbitrary line-count limits.

A useful dependency direction is application to domain, not domain to application. The domain implementation includes its header and the library facilities it needs. It should not include the application's `.cpp` file or depend on a global terminal object. A second adapter, such as a file importer, should be able to call the same domain operations.

The header exposes only what consumers need to compile calls. The private vector is an implementation detail even though the current `const` observer reveals the container type. That observer is a deliberate simplicity tradeoff for Book I. A more stable long-lived API might expose narrower traversal or copying operations so consumers do not depend on the exact storage container.

Build the two implementation sources into the capstone executable through one declared target. The supplied CMake configuration lists both files. A source that merely exists in the project directory is not necessarily compiled into every target. Conversely, indiscriminately collecting every `.cpp` can accidentally include multiple entry points or experimental code.

Maintain separate build directories for different compilers or materially different configurations. For example, `build-gcc`, `build-clang`, and `build-sanitize` make the evidence easier to interpret. Do not replace a compiler underneath an old configured build tree and assume that every generated setting will remain appropriate.

Keep fixtures and generated results separate from source. A command-session fixture is authored test input; a temporary directory containing an execution result is generated evidence. Neither should be confused with a persistent inventory database because the application does not implement one. The test runner intentionally executes cases in fresh temporary working directories.

A handoff package should include source, build instructions, tests, a clear license or usage decision by the owner, and an honest verification record. Do not ship only the executable or only a screenshot of passing output. The recipient needs enough material to reproduce the behavior and inspect what was actually tested.

Small Expense Tracker Using the Earlier Building Blocks

Purpose. The example integrates variables, classes of data, containers, loops, strings, and console output into one coherent application flow.

```
#include <iostream>
#include <string>
#include <vector>
struct Expense{ std::string name; double amount; };
int main(){
    std::vector<Expense> expenses{{"food",12.50},{ "fuel",30.00}};
    double total=0;
    for(const auto& e:expenses){ total += e.amount; std::cout << e.name << ": "
    << e.amount << '\n'; }
    std::cout << "total=" << total << '\n';
}
```

How it works.

- A `struct` models each expense as one record.
- A `vector` stores a runtime-sized collection.
- The range loop performs both reporting and accumulation over the application data.

Expected behavior/output. The final total is `42.5`.

Experiment. Add interactive input for a third expense and preserve the same accumulation logic.

20.3 Testing and Improving the Finished Program

20.3.1 Debugging Functional Defects

Test ordinary behavior, boundaries, and failure preservation. Insert a valid record, reject a duplicate, reject a negative quantity, and verify that size and total remain correct. Exercise the command interface with quoted names, malformed numbers, trailing junk, and unknown commands. A successful library test does not prove that the text interface passes the correct data to it.

When a defect appears, preserve the exact command sequence and expected state. Reduce the sequence until the smallest failing case remains. Check parsing before changing the inventory rule, and check the inventory before changing output formatting. The layer at which the observation first diverges from the requirement is the useful place to investigate.

Debugging begins with an observable mismatch. Suppose adding a duplicate identifier increases the total. Record the exact command sequence, expected output, actual output, and state before and after the duplicate request. This is a narrower and more useful report than “the inventory is broken.”

First decide whether parsing or domain logic is responsible. Call the domain operation directly with a duplicate candidate in a unit-style test. If it rejects and preserves state there but the interactive session behaves differently, inspect how the adapter builds or interprets the candidate. If the direct test also fails, focus on the validation and insertion sequence.

Use the smallest reproducing input that retains the defect. For a duplicate bug, two records with the same identifier are enough. A hundred-line session adds distraction unless the failure depends on capacity or history. Preserve the reduced reproducer as a regression case before making the fix.

Boundary tests are especially valuable because they distinguish inclusive and exclusive comparisons. Test quantities zero, one thousand, negative one, and one thousand and one. Test an empty inventory and a full inventory. Test positive and nonpositive identifiers. Inspect both the return result and retained state after each rejected operation.

Capacity testing should establish the exact transition. Insert one hundred distinct valid records, verify their count and total, then attempt another valid distinct record. The final request should be rejected without changing the first hundred. Testing only a random large input does not prove where the boundary lies.

Malformed input needs protocol-level tests. Try a missing field, invalid numeric token, trailing junk, an unknown command, and extra arguments after `total`. Follow each failure with a valid command to verify that the line-based processor recovers as intended. The recovery behavior is part of the interface, not an optional courtesy.

Sanitizers complement these tests by detecting certain executed memory and undefined-behavior problems. They do not know that duplicate identifiers are forbidden unless the tests encode that rule. A memory-clean program can still compute the wrong total or accept the wrong grammar. Use semantic checks and runtime instrumentation for their different purposes.

When fixing a defect, change the smallest responsible mechanism and rerun the related test first. Then run the entire Book I suite to detect unintended effects. A local passing result is encouraging, but a reusable library change may affect other call paths that the reduced reproducer does not exercise.

20.3.2 Refactoring Repetitive or Fragile Code

Refactoring changes internal organization while preserving intended behavior. Extract repeated reporting, centralize limits, and keep validation in one domain operation. Run the existing tests after each small change. A rewrite that also changes the accepted command syntax is a feature change and needs updated requirements, not merely a refactoring label.

The finished capstone provides a foundation for later volumes: resource-safe persistence in Book II, indexes and algorithms in Book III, concurrent service boundaries in Book IV, and release engineering in Book V. Preserve the simple reference behavior while adding capabilities. Complexity is easier to justify when a smaller correct implementation remains available for comparison.

Refactoring changes internal structure while preserving intended observable behavior. Moving repeated report formatting into a function is a refactor when the protocol remains the same. Changing duplicate identifiers from rejection to replacement changes behavior and requires a requirement decision, not merely a code-cleanup label.

Before refactoring, establish a passing baseline and save representative output. The tests become a guardrail for the existing contract. They are not a proof that every behavior is preserved, so inspect the design change as well. An incomplete test suite can allow an unintended change to pass silently.

Prefer removing duplicated decisions before creating elaborate abstractions. If two input paths independently enforce the quantity range, move the authoritative rule into the domain boundary rather than inventing a class hierarchy for validators. Adapters may perform early checks for better messages, but accepted domain state should not depend on which adapter was used.

Keep ownership visible during cleanup. Replacing a value member with a raw owning pointer might reduce the size of one class declaration while making lifetime management more fragile. A shorter file is not automatically a simpler system. Evaluate the number of states, dependencies, and failure paths that a maintainer must understand.

A useful extension is a lookup operation. Returning a record copy gives the caller independent data but incurs copying. Returning a const pointer avoids that copy but creates a borrow whose validity depends on the inventory and its modifications. Either can be justified if the contract is explicit. Do not return a mutable reference that allows callers to rewrite an identifier and bypass uniqueness checks.

A quantity-update operation should locate the target, validate the new value, and assign only after validation succeeds. Rejected updates must preserve the old quantity and total. This is the same validation-before-commit pattern used for insertion, now applied to an existing record. The operation should not remove the record and attempt to reinsert it unless that more complex transaction is actually necessary.

Persistence would introduce a different class of requirement: file format, versioning, partial-write handling, replacement strategy, and recovery after interruption. Do not add a casual destructor write and declare the application durable. The final exercise asks you to design the boundary before implementing a storage feature that would need substantially more testing.

The capstone ends when the current contract is clear, implemented, and reproducible. It does not need every feature in the six-volume curriculum. A deliberately bounded application is a better foundation for later modern C++, algorithms, and systems engineering than an unfinished framework that nobody can fully explain.

Regression Tests Around a Refactored Calculation Function

Purpose. Improvement should preserve behavior intentionally; small regression tests provide evidence that a cleanup did not change results.

```
#include <cassert>
#include <vector>
double total(const std::vector<double>& values){ double sum=0; for(double
v:values) sum+=v; return sum; }
int main(){
    assert(total({})==0.0);
    assert(total({2.5})==2.5);
    assert(total({1.0,2.0,3.0})==6.0);
}
```

How it works.

- The calculation is separated from console I/O so it can be tested directly.
- The tests include empty, one-element, and multi-element cases.
- Refactoring implementation later is safe only if the observable contract remains covered.

Expected behavior/output. All assertions pass with no output.

Experiment. Replace the loop with `std::accumulate` and rerun the same tests to validate the refactor.

Worked example — Build HarborInventory as a tested multi-file application

Requirement and simplest approach

HarborWorks needs a local interactive inventory demonstration that accepts valid records, rejects invalid changes, and reports the accepted state. The smallest defensible design is a domain class plus a command adapter. One enormous loop containing parsing, validation, storage, and formatting would be quicker to type but harder to test without replaying an entire terminal session.

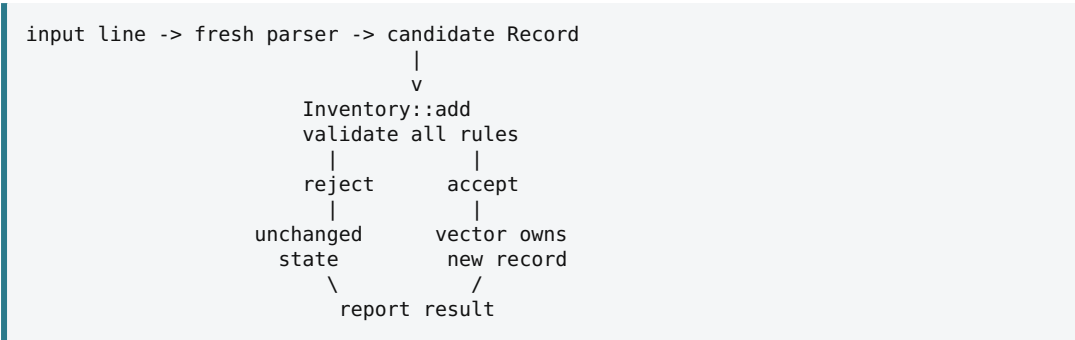
Contract ledger

Boundary	Accepted information	Required result
Command line	Default mode or - - interactive	Selected behavior or usage error
Input line	Recognized command with its exact fields	Candidate operation or diagnostic
Domain insertion		

Boundary	Accepted information	Required result
	Positive unique ID, nonempty name, bounded quantity	One accepted record or unchanged state
Collection	At most 100 accepted records	Stable ownership and bounded domain total
Observation	Existing inventory	Ordered listing or total without mutation

These are original teaching requirements. They are not claims that a real organization would choose the same capacity, name policy, or command vocabulary.

Mechanism view



The input adapter owns the temporary line and parser. The inventory owns accepted records. A rejected candidate never gains membership merely because its fields were successfully read.

Prepare a reproducible session

Use a disposable text fixture rather than relying only on hand typing. A useful first session adds two distinct records, requests a listing and total, attempts a duplicate, and asks for the total again. Predict the exact total before execution. Save the fixture with the test so another reader can reproduce the same sequence.

Trace the implementation

Run without arguments for deterministic checks, or use `--interactive` to enter commands. An example session adds `add 1 "bench scope" 2`, adds another distinct record, then uses `list` and `total`. The interface accepts a quoted name as one field and rejects trailing tokens after a complete command.

The library uses a long long total. At most 100 records each contribute at most 1000, so the maximum is 100000, comfortably within the guaranteed range of that result type. The bound follows from the domain limits rather than from an assumption that every implementation gives int at least 32 bits.

Insertion validates before `push_back`. The vector and string own their storage; no manual delete is needed. `std::move` in the implementation requests transfer from the local candidate where the member types support it, a concept developed fully in Book II. It does not move from the caller's original object when the parameter was copied by value.

The delivered fixture suite checks both the self-test mode and a command sequence containing an accepted record, a duplicate, malformed input, and a final total. These are real checks of the supplied application, not evidence of persistence, concurrent access, or production deployment features that were never implemented.

```
text commands -> parser -> validated Record -> Inventory
Inventory -> const observations -> report
fixture tests -> parser and domain boundary
```

Complete worked source

source/ch20/inventory.hpp

```
#ifndef HARBOR_INVENTORY_HPP
#define HARBOR_INVENTORY_HPP
#include <string>
#include <vector>

namespace harbor {
struct Record {
    int id{};
    std::string name;
    int quantity{};
};

class Inventory {
public:
    static constexpr int maximum_quantity{1000};
    static constexpr std::size_t maximum_records{100};
    bool add(Record record);
    long long total() const;
    const std::vector<Record>& records() const { return records_; }
private:
    std::vector<Record> records_;
};
}
#endif
```

source/ch20/inventory.cpp

```
#include "inventory.hpp"
#include <utility>

namespace harbor {
bool Inventory::add(Record record) {
    if (record.id <= 0 || record.name.empty()) return false;
    if (record.quantity < 0 || record.quantity > maximum_quantity) return false;
    if (records_.size() >= maximum_records) return false;
    for (const Record& existing : records_) {
        if (existing.id == record.id) return false;
    }
}
```

```

        records_.push_back(std::move(record));
        return true;
    }

    long long Inventory::total() const {
        long long result{};
        for (const Record& record : records_) result += record.quantity;
        return result;
    }
}

```

source/ch20/main.cpp

```

#include "inventory.hpp"
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>

void report(const harbor::Inventory& inventory) {
    for (const auto& record : inventory.records()) {
        std::cout << record.id << ' ' << std::quoted(record.name)
                    << ' ' << record.quantity << '\n';
    }
}

int self_test() {
    harbor::Inventory inventory;
    if (!inventory.add({1, "bench scope", 2})) return 1;
    if (!inventory.add({2, "meter", 3})) return 2;
    if (inventory.add({1, "duplicate", 4})) return 3;
    if (inventory.add({3, "invalid", -1})) return 4;
    if (inventory.records().size() != 2 || inventory.total() != 5) return 5;
    report(inventory);
    std::cout << "total=" << inventory.total() << '\n';
    std::cout << "PASS\n";
    return 0;
}

int main(int argc, char** argv) {
    if (argc == 1 || (argc == 2 && std::string{argv[1]} == "--self-test")) {
        return self_test();
    }
    if (argc != 2 || std::string{argv[1]} != "--interactive") {
        std::cerr << "usage: inventory [--self-test|--interactive]\n";
        return 2;
    }
    harbor::Inventory inventory;
    std::string line;
    while (std::getline(std::cin, line)) {
        std::istringstream input{line};
        std::string command;
        if (!(input >> command)) continue;
        if (command == "add") {
            harbor::Record candidate;
            std::string extra;
            if (!(input >> candidate.id >> std::quoted(candidate.name)
                    >> candidate.quantity) || (input >> extra)) {
                std::cout << "error: malformed add\n";
                continue;
            }
            std::cout << (inventory.add(candidate) ? "added\n" : "rejected\n");
        } else {

```

```

        std::string extra;
        if (input >> extra) {
            std::cout << "error: unexpected arguments\n";
            continue;
        }
        if (command == "list") report(inventory);
        else if (command == "total") std::cout << "total=" << inventory.total()
<< '\n';
        else if (command == "quit") break;
        else std::cout << "error: unknown command\n";
    }
}
if (std::cin.bad()) return 3;
return 0;
}

```

Run and verify

From the extracted package root, build and run this chapter's supplied fixtures:

```
python3 tools/run_chapter.py 20
```

The executable target is `b01_c20`. Read `source/ch20/cases.json` for the exact input and expected results. Tests execute in temporary working directories. A passing check demonstrates those cases, not every possible input or platform.

Run a complete command session

After building the capstone target, the following session can be entered interactively or redirected from a fixture file:

```

add 1 "bench scope" 2
add 2 "reference meter" 3
list
total
add 1 "duplicate scope" 9
total
quit

```

The two accepted quantities sum to five. The duplicate does not replace the first record and does not add nine to the total. A second total of five is therefore evidence of preserved state after rejection, not merely a repeated formatting result.

```

# Run from the extracted project root after building:
./build/bin/b01_c20 --interactive < session.txt

```

Create `session.txt` explicitly with the lines above before using the command. The supplied fixture runner performs analogous checks automatically in an isolated working directory. Its output records whether actual stdout, stderr, and process status matched the case definition.

Evidence and interpretation

Distinguish the application's demonstration self-test from the external regression suite. The no-argument mode runs a small set of built-in checks. External cases exercise command sessions and can test error messages and recovery behavior without modifying the application. The expanded package also supplies direct domain regression checks, including the capacity boundary and state preservation after rejected candidates.

Passing these finite tests establishes the observed behaviors for the tested builds and cases. It does not establish durable storage, protection from malicious input of arbitrary size, simultaneous access by multiple users, or compatibility with every compiler. The verification report records the actual environment and exclusions.

Failure clinic — acceptance reported after a partial parse

Suppose a modified adapter accepts a valid-looking prefix and ignores the rest of the command. A fixture ending with `2junk` or an extra word exposes the gap. The fix is not to adjust the inventory's quantity range; the adapter must establish that the complete intended line was consumed according to its grammar before calling the domain operation.

Now suppose the adapter correctly rejects the line but the next valid line is never processed. That points to parser recovery or loop control. The delivered design creates a new line parser each iteration, isolating an ordinary malformed command from the next line. Tests should contain a valid command after the malformed one to demonstrate that recovery.

Recovery and application boundary

A rejected insertion leaves existing records intact. A process restart does not restore them because persistence is outside the delivered contract. Recovery from a bad command and recovery from process loss are different requirements. Explain both before anyone relies on the program for valuable data.

Keep manual experiments on invented equipment records. Do not use the exercise as the only inventory of real assets. A learning application can be complete for its educational purpose while still lacking the controls required for a production service.

Practice and implementation lab

Extend the application with a `find ID` command that does not expose mutable internal state. Add tests for existing and missing identifiers. Next, add a quantity-update operation that preserves uniqueness and rejects invalid quantities without changing the prior record.

Write a one-page acceptance report stating the input grammar, domain limits, supported operations, test evidence, and excluded capabilities. Explain which component would change first if records needed to be loaded from a file.

Starting exercise: solution notes

A const lookup can return a borrowed pointer whose lifetime is limited by the inventory's structural changes, or it can return an owning copy. State that choice explicitly. A quantity update should locate the record, validate the new quantity, and only then assign it. A failed update must leave both size and total unchanged.

File loading belongs in an input adapter that produces candidate records and calls the same domain operation. It should not bypass validation by writing directly into the vector. The acceptance report must distinguish demonstrated classroom behavior from future requirements such as durable storage, authentication, and concurrency.

Guided lab — finish with an acceptance review

Begin by building and running every delivered Book I test from a clean build directory. Then run the capstone interactively with a small invented inventory. Compare the observed listing with the source of truth you wrote before the test: identifier, name, quantity, insertion order, and expected total.

Create a test matrix covering valid insertion, duplicate rejection, lower and upper quantity boundaries, invalid identifiers, empty names, capacity exhaustion, malformed numeric fields, unexpected trailing tokens, unknown commands, blank lines, and ordinary end-of-file. For every rejection that reaches the domain library, verify that size and total remain unchanged. For parsing failures, verify that no candidate is inserted.

Next, implement one extension in a separate branch or copied lab directory: a read-only lookup or a validated quantity update. Write its expected behavior before the code. Add a regression that would fail if the extension exposed mutation authority or changed state after a rejected update. Keep the unmodified reference implementation available for comparison.

Finally, give the source and instructions to a second reader or simulate that handoff by using a clean directory yourself. Build without copying old object files. Run the fixtures using only the documented commands. Record any assumptions you had previously supplied from memory, such as a missing compiler flag or an undocumented working directory, and repair the instructions.

Acceptance evidence. The program builds from its authored files; all supported commands have specified behavior; invalid input is classified at the correct boundary; ownership requires no manual deletion; rejection preserves the required state; the fixture suite is reproducible; and the handoff report clearly excludes unimplemented capabilities.

Solution notes for the extension choices

For a lookup returning an owning copy, the caller can retain the result across later insertions without depending on vector element addresses. For a borrowed lookup, document that the inventory must outlive the borrow and that structural changes may invalidate it. A const borrow restricts mutation through that access path; it does not independently solve lifetime.

For a quantity update, do not erase the record before checking the replacement quantity. Validate the identifier lookup and candidate value first, then perform a single assignment to the existing record. Test

both the selected record and the aggregate total afterward. A failure should preserve the exact prior record, not merely the number of records.

For future file import, parse file content into candidates and submit them through the same domain operations. Decide whether the import is all-or-nothing or may accept a valid prefix. That transaction decision must be explicit before adding the feature, because the two choices require different rollback and reporting behavior.

Review and solution notes

Why separate parsing from domain validation? Parsing establishes whether a line expresses a candidate operation. Domain validation establishes whether that operation is permitted for the current inventory. Different adapters can then share one authoritative rule boundary.

Does a const vector reference make retained element references permanent? No. It restricts ordinary mutation through that reference, but element lifetime and invalidation still depend on the owning vector's operations and destruction.

Does a successful session exit mean every add succeeded? No. Interactive command rejection is reported within the session while the program continues. Check the appropriate output as well as the process status.

What is the final engineering outcome? A reader can explain the input grammar, state invariants, ownership graph, build pipeline, test evidence, and limits. That is a stronger foundation than a larger program whose behavior is only guessed.

Chapter takeaway. A complete application connects requirements to explicit boundaries and observable evidence. Build the smallest system that satisfies its stated contract, verify its failure behavior, and leave the next stage of development a clear foundation rather than hidden debt.

Technical references: [LANG](#) · [VECTOR](#) · [STRING](#) · [GUIDELINES](#) · [CMAKE](#).

Appendix A — A practical diagnostic desk reference

Start with the failed boundary

A failed command is evidence about a stage, not a complete explanation. Begin by identifying whether the failure occurred before compilation, during preprocessing or compilation, during linking, when launching the executable, or while processing input. The same word, such as “not found,” can refer to a compiler executable, an included header, a library, or a runtime file. Each belongs to a different search path.

For a missing compiler command, inspect the active environment and the exact command name. For a missing header, inspect include spelling, installed development dependencies, and include directories. For an undefined reference, inspect declarations, definitions, namespaces, and link inputs. For a program that runs but computes the wrong answer, stop rearranging build flags and examine the algorithm’s contract and data.

Write a one-sentence hypothesis before the next change. “The implementation object was omitted from this link command” predicts that compiling and supplying that object will resolve the corresponding reference. “Something is wrong with C++” predicts nothing useful. A small hypothesis makes the result of each experiment interpretable.

A compact failure map

Observation	First question	Bounded next check
Compiler command unavailable	Which environment and command are active?	Record compiler path and version
Header cannot be opened	Which include is missing and who provides it?	Check spelling and include configuration
Name is undeclared	Is its declaration visible in this translation unit?	Inspect scope and required header
Undefined reference	Is the exact defining object part of this link?	Compare signature, namespace, and inputs
Duplicate definition	Was an ordinary definition supplied more than once?	Inspect shared headers and source lists
Executable appears unchanged	Which source and binary paths were used?	Rebuild the named target and run its path
Input is rejected		

Observation	First question	Bounded next check
	Is the grammar wrong or the value outside its domain?	Save the exact bytes and parse result
Wrong total after rejection	Was state changed before validation completed?	Compare full pre-state and post-state
Intermittent memory fault	Does a borrow outlive or exceed its owner?	Trace lifetime and bounds; run instrumentation

These are starting questions, not universal diagnoses. An unavailable header can also result from selecting the wrong target architecture or toolchain environment. An apparent arithmetic error can originate in parsing. Follow the evidence through the specific program instead of treating the table as a substitute for investigation.

Read a diagnostic without losing the first cause

Compilers often emit many messages after one missing brace or declaration. Start with the earliest relevant diagnostic in the source being compiled and inspect its surrounding context. A later error may be a consequence of how the compiler attempted to continue parsing after the first failure. Correct one cause and rebuild before trying to repair every line named in the output.

Do not suppress a warning simply because the executable runs. A signed/unsigned comparison or narrowing conversion may work for the one supplied value and fail at a boundary. Translate the warning into a question about ranges, representation, or lifetime. Then either fix the code or explain the deliberately safe conversion in a way the code makes visible.

When asking another person for help, provide the smallest complete source, exact build command, compiler version, input, expected result, and actual diagnostic. Screenshots can illustrate an interface, but copyable text makes a compiler error much easier to reproduce and search. Remove confidential paths and data without removing the mechanism that causes the failure.

A test is a claim with a witness

A good test states what behavior matters and supplies an input that would expose a plausible wrong implementation. Testing only an ordinary valid quantity does not distinguish `candidate <= 1000` from `candidate < 1000`. Testing exactly one thousand does. Testing a rejected update without observing the old value does not expose mutation-before-validation. Checking the retained value does.

Consider both examples and properties. For grouping, an answer must provide enough capacity and avoid an unnecessary extra group. For parsing, success must mean the complete intended token was consumed. For a rejected inventory insertion, the collection's prior records and total should remain unchanged. Properties can reveal more than an isolated expected number, but they must themselves match the application's actual contract.

Avoid writing the expected result by calling the same implementation under test. That only checks a function against itself. Derive a small expected value by hand, compare against an independently

designed reference when appropriate, or assert an invariant that the implementation must satisfy. Keep the reasoning with the fixture so future changes do not turn it into an unexplained magic number.

Close an investigation with evidence

A repair is not finished when the most recent command stops failing. Rerun the original reproducer, the boundary cases nearest the change, and the broader relevant suite. Record the actual cause, the changed mechanism, and the evidence that the intended behavior was restored. Retain the regression case so the same defect has a better chance of being caught before it returns.

A concise failure report can use five paragraphs: observed symptom, expected contract, minimal reproducer, cause and repair, verification and remaining limits. The format works for a classroom loop and for a much larger native application. Its value comes from connecting claims to observable evidence, not from formal wording.

Appendix B — A worked solution for transactional batch updates

Requirement and design

Chapter 13's original offset example uses deliberately small values. A reusable integer batch update needs a stronger rule: either every element can accept the offset, or no element is changed. The companion's `checked_offset` program implements that rule with two passes. The first pass validates; only a completely successful validation permits the second pass to mutate.

Checking for overflow by performing the overflowing addition first would defeat the purpose. Instead, the code compares the existing value with a safe bound derived from the offset. For a positive offset, the largest permitted starting value is `maximum - offset`. For a negative offset, the smallest permitted starting value is `minimum - offset`. The sign guards make those bound calculations representable for the relevant cases.

The function assumes no concurrent mutation between or during the passes. Book I programs are single-threaded; this is an explicit precondition rather than an accidental claim of thread safety. Adding concurrency later would require a separate synchronization or ownership design.

`source/labs/checked_offset.hpp`

```
#ifndef HARBOR_CHECKED_OFFSET_HPP
#define HARBOR_CHECKED_OFFSET_HPP
#include <vector>

namespace harbor {
// Returns false without changing any element when an addition would overflow.
// Precondition: no concurrent mutation of values during either pass.
bool apply_offset_checked(std::vector<int>& values, int offset);
}
#endif
```

`source/labs/checked_offset.cpp`

```
#include "checked_offset.hpp"
#include <limits>

namespace harbor {
bool apply_offset_checked(std::vector<int>& values, int offset) {
    const int low = std::numeric_limits<int>::min();
    const int high = std::numeric_limits<int>::max();
    for (const int value : values) {
        if (offset > 0 && value > high - offset) return false;
        if (offset < 0 && value < low - offset) return false;
    }
    // Validation of the entire batch precedes all mutation.
    for (int& value : values) value += offset;
    return true;
}
```

```
}  
}
```

source/labs/checked_offset_demo.cpp

```
#include "checked_offset.hpp"  
#include <iostream>  
#include <limits>  
#include <vector>  
  
int main() {  
    std::vector<int> ordinary{1, 2, 3};  
    if (!harbor::apply_offset_checked(ordinary, 2)) return 1;  
    if (ordinary != std::vector<int>{3, 4, 5}) return 2;  
  
    const int maximum = std::numeric_limits<int>::max();  
    const int minimum = std::numeric_limits<int>::min();  
    std::vector<int> overflow{5, maximum};  
    const auto original = overflow;  
    if (harbor::apply_offset_checked(overflow, 1)) return 3;  
    if (overflow != original) return 4;  
  
    std::vector<int> underflow{minimum, 8};  
    const auto before_underflow = underflow;  
    if (harbor::apply_offset_checked(underflow, -1)) return 5;  
    if (underflow != before_underflow) return 6;  
  
    std::vector<int> edge{0};  
    if (!harbor::apply_offset_checked(edge, minimum)) return 7;  
    if (edge.front() != minimum) return 8;  
    if (!harbor::apply_offset_checked(edge, maximum)) return 9;  
    if (edge.front() != -1) return 10;  
  
    std::vector<int> empty;  
    if (!harbor::apply_offset_checked(empty, maximum)) return 11;  
    std::cout << "checked offset: valid, overflow, underflow, "  
                "extreme offset, and empty cases PASS\n";  
}
```

Trace the evidence

The valid fixture changes one, two, and three into three, four, and five. The overflow fixture places a maximum integer after an ordinary first value, then requests an offset of one. This ordering matters: a one-pass implementation that updates as it checks might change the first value before discovering the impossible last value. Comparing the entire vector with its saved copy exposes that partial update.

The underflow case checks the opposite boundary. Extreme offsets and an empty vector exercise additional paths without assuming a fixed width for `int`. The implementation uses numeric limits from the actual selected type, keeping the argument about representability connected to that type rather than a handwritten machine-specific constant.

Run and extend

Build the `checked_offset` target and run its CTest fixture. Then add a mixed positive/negative vector and offsets of zero, one, and negative one. For every rejected case, compare the complete prior collection rather than only its size. Equal size does not prove equal contents.

The two-pass solution is intentionally straightforward. A transactional update could also create a transformed copy and replace the original only after success, trading additional storage and copying for another implementation shape. Both approaches need a clear failure contract. Choose based on the type's operations and workload rather than assuming that one generic pattern fits every collection.

Appendix C — A worked solution for construction failure and cleanup

Requirement and design

Chapter 18 distinguishes a completed object's exception unwinding from failure during an enclosing object's construction. The companion's `construction_unwind` program demonstrates both. In the first path, a completed local probe is destroyed when an ordinary exception leaves its scope. In the second path, a completed member probe is destroyed when the enclosing constructor body throws.

The borrowed live counter is declared outside both construction attempts. It therefore remains valid during every cleanup operation. The program tests an explicit caught flag and the final zero count, avoiding the false conclusion that a never-executed construction path demonstrated cleanup.

source/labs/construction_unwind.cpp

```
#include <iostream>
#include <stdexcept>

class Probe {
public:
    explicit Probe(int& live) : live_(live) { ++live_; }
    ~Probe() { --live_; }
    Probe(const Probe&) = delete;
    Probe& operator=(const Probe&) = delete;
private:
    int& live_;
};

class FailsAfterMember {
public:
    explicit FailsAfterMember(int& live) : member_(live) {
        throw std::runtime_error("intentional construction rejection");
    }
private:
    Probe member_; // Completed member is cleaned up if the enclosing body throws.
};

int main() {
    int live{};
    bool caught{};
    try {
        Probe first{live};
        if (live != 1) return 1;
        throw std::runtime_error("intentional scope exit");
    } catch (const std::runtime_error&) {
        caught = true;
    }
    if (!caught || live != 0) return 2;

    caught = false;
    try {
        FailsAfterMember rejected{live};
    } catch (const std::runtime_error&) {
```

```
        caught = true;
    }
    if (!caught || live != 0) return 3;
    std::cout << "ordinary unwind and failed enclosing construction PASS\n";
}
```

Interpret the result

The first path is ordinary stack unwinding for a completed local owner. The second path is cleanup of already-constructed subobjects when the full construction fails. The example does not invoke an enclosing destructor for an object whose construction never completed, and it does not rely on abrupt process termination running C++ cleanup.

The test is small enough to trace with a counter, but the ownership lesson is broader. A resource acquired as an owning member can remain protected during later construction failure. A raw acquisition whose release exists only in the enclosing destructor may not have that protection. Book II develops this relationship into general RAII design.

Appendix D — Capstone acceptance worksheet

Use this worksheet after completing Chapter 20. Fill it with observed evidence from your own build, not the publishing environment's results copied without qualification.

Build identity. Record the source revision or archive checksum, operating system, architecture, compiler and standard-library versions, language mode, CMake version, build configuration, and exact commands. Identify the produced executable path.

Input contract. Write every supported command and its fields. Explain quoting, unexpected trailing tokens, empty lines, malformed numeric input, ordinary end-of-file, and the difference between command rejection and process failure.

State contract. Record identifier uniqueness, name policy, quantity bounds, maximum record count, insertion-order behavior, and aggregate total bounds. Identify the object that owns accepted records and the lifetime of any exposed borrow.

Failure evidence. Name a concrete fixture for each rejected input category. Show that domain rejection preserves the complete prior records and total, and that malformed lines do not prevent a following valid line from being processed.

Change review. Describe one implemented extension, the requirement it satisfies, the regression that would catch a broken implementation, and the effect on ownership or lifetime. Separate behavior changes from refactoring.

Operational limits. State plainly that the reference capstone does not implement persistence, authentication, concurrent access, or unbounded-input protection. Identify what would need to change before valuable real data depended on it.

Handoff result. A second person should be able to build, run, test, and explain the program from the package alone. Any instruction supplied only from memory is a documentation defect to repair. A complete acceptance review describes both what works and the boundary beyond which no claim has been established.

Glossary

Aggregate. A class or array meeting the language's aggregate criteria and supporting aggregate initialization.

Assignment. Replacing or modifying the value of an object whose lifetime already exists.

Borrow. Access to an object without taking responsibility for its lifetime; validity depends on the owner.

Capacity. Storage available to a container before reallocation is needed; not the number of live elements.

Const correctness. Using const-qualified interfaces to express and enforce intended nonmutation through those interfaces.

Constructor. The class operation that participates in establishing a valid object during initialization.

Dangling access. An attempted access through a pointer or reference after the required target lifetime has ended.

Declaration. Introducing a name and information about an entity so it can be used and checked.

Definition. A declaration that provides the entity's defining body or storage as appropriate.

Destructor. The class operation associated with ending an object's lifetime and releasing its owned responsibilities.

Domain validation. Checking whether a syntactically representable candidate satisfies the application's rules.

Encapsulation. Controlling permitted operations so callers do not casually bypass the representation's rules.

Fixture. A controlled test input and expected outcome, often accompanied by a defined execution environment.

Half-open range. A range including its beginning and excluding its end, such as indices zero through size minus one.

Initialization. Establishing an object's initial state; distinct from assignment to an existing object.

Invariant. A condition that an object or algorithm maintains at specified boundaries.

Linker. The tool that resolves compatible object-file and library references into an executable or library.

Narrowing. A conversion restricted by list-initialization rules because it may lose representational information.

Null pointer. A pointer value that does not designate an object or function; it must not be dereferenced.

Object. A region of storage with a type and a lifetime; an object is not the same thing as a source-level name.

Ownership. Responsibility for ending a resource lifetime and releasing its associated storage or handle.

Parsing. Interpreting input according to a grammar to form values or candidate operations.

Pointer. An object whose value can designate another object or function, a permitted boundary, or no object; validity requires more than non-nullness.

Postcondition. A promised property after an operation completes under its documented conditions.

Precondition. A requirement that must hold before an operation is used under its contract.

Preprocessing. Source transformation involving directives, includes, and macros before later compilation stages.

RAII. Resource Acquisition Is Initialization: tying resource management to object lifetime, developed fully in Book II.

Refactoring. Changing internal organization while preserving the intended observable behavior.

Reference. An alias bound to an existing object or function; a reference does not by itself own the referent.

Regression test. A retained check intended to detect a behavior defect returning after a change.

Sanitizer. Compiler/runtime instrumentation that detects selected kinds of defects along executed paths.

Scope. The region in which a name can be found by the language's name-lookup rules.

Short-circuit evaluation. Conditional evaluation of the right operand of built-in logical AND or OR based on the left operand.

Storage duration. The category governing how storage is provided and retained, such as automatic, static, or dynamic.

Translation unit. A source file after preprocessing, including the declarations contributed by its headers.

Undefined behavior. Behavior for which the language imposes no requirements after a relevant rule is violated.

Technical reference directory

Primary references for the mechanisms discussed in this volume. The linked working draft evolves and is not presented as the published C++20 standard. Compiler-specific instructions must be checked against the installed release. Original examples, contract choices, and exercises are teaching material, not claims made by these projects. Reference URLs were consulted for this revision or retained from the source edition; changing online pages can describe features beyond this book's baseline.

ARRAY — Fixed-size array container

[Fixed-size array container](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

CHARCONV — Primitive numeric conversions

[Primitive numeric conversions](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

CLANG — Clang C++ language support

[Clang C++ language support](#)

Compiler feature status; check the standard library separately.

CLASS — Classes and special member functions

[Classes and special member functions](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

CMAKE — CMake tutorial

[CMake tutorial](#)

Target-oriented configure, build, test, and install concepts. Latest documentation may describe newer CMake features than the 3.24 project minimum.

EXPR — Expressions and evaluation

[Expressions and evaluation](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

FUNC — Functions and declarations

[Functions and declarations](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

GCC — GCC C++ language support

[GCC C++ language support](#)

Compiler support varies by version, language mode, and library.

GUIDELINES — C++ Core Guidelines

[C++ Core Guidelines](#)

Design guidance on ownership, interfaces, safety, and concurrency; recommendations are not language rules.

IO — Input/output library

[Input/output library](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

LANG — C++ working draft

[C++ working draft](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

LIFETIME — Object lifetime

[Object lifetime](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

MSVCINSTALL — Install Microsoft C++ tools

[Install Microsoft C++ tools](#)

Windows installation procedure; Windows execution was not validated for this package.

MSVCSTD — Microsoft C++ /std options

[Microsoft C++ /std options](#)

Language-mode options and implementation boundaries.

NUMERIC — Fundamental types and numeric limits

[Fundamental types and numeric limits](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

ODR — One-definition rule

[One-definition rule](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

STMT — Statements and control flow

[Statements and control flow](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

STRING — Basic string

[Basic string](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

VECTOR — Vector container

[Vector container](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

GCC: Overall Options

[GCC: Overall Options](#)

Driver stages and stage-selection flags.

Microsoft: Compile a native C++ program from the command line

[Microsoft: Compile a native C++ program from the command line](#)

Developer command prompt and C++ build tools.

GCC: Warning Options

[GCC: Warning Options](#)

Warning groups and diagnostic controls.

C++ working draft: vector capacity

[C++ working draft: vector capacity](#)

Size, reserve, capacity, and invalidation. Living draft; use the C++20 baseline of this book.

C++ working draft: initializing bases and members

[C++ working draft: initializing bases and members](#)

Member order and constructor delegation. Living draft.

C++ working draft: destructors

[C++ working draft: destructors](#)

Destruction and subobject lifetime. Living draft.

Clang: AddressSanitizer

[Clang: AddressSanitizer](#)

Instrumentation, use, and limitations.

Clang: UndefinedBehaviorSanitizer

[Clang: UndefinedBehaviorSanitizer](#)

Executed undefined-behavior checks and runtime options.